

Data Engineering

From Foundations to Production

Modules 0–10 · Hands-on Practitioner's Guide
2026 Edition

Table of Contents

00 — Module 0: Welcome & Setup

01 — Module 1: Data Modeling Foundations

02 — Module 2: SQL for Data Engineers

03 — Module 3: Python for Data Engineers

04 — Module 4: Apache Airflow — Orchestration

05 — Module 5: Apache Spark & Big Data Processing

06 — Module 6: Apache Kafka & Streaming

07 — Module 7: Data Quality & Testing

08 — Module 8: Cloud Infrastructure for Data Engineers

09 — Module 9: Data Engineering for AI

10 — Module 10: Capstone Project — Real-Time E-Commerce Analytics

11 — Appendix A: Career Resources & Cost Optimization

12 — Appendix B: Real-World Case Studies

13 — Appendix C: Interview Preparation

Module 0: Welcome & Setup

Who This Course Is For

If you've ever stared at a job posting for "Data Engineer" and wondered whether you're ready, this course is for you. We built it for three kinds of people, and if you see yourself in any of them, you're in the right place.

The analyst leveling up. You know SQL. You can build dashboards in Tableau or Looker. You've probably written some Python scripts to clean data. But you've hit a ceiling. You want to build the pipelines that *feed* those dashboards – the infrastructure that makes analytics possible at scale. You're tired of waiting for someone else to give you clean data and you want to be the person who builds the system that delivers it.

The backend developer pivoting. You can build APIs, deploy services, and write production code. You understand databases and have probably worked with Docker. But you've noticed that data engineering roles pay well, the problems are interesting, and the field is growing fast. What you need is the domain knowledge – how data warehouses work, what orchestration means in practice, why "data quality" is a job unto itself. Your engineering skills will translate directly; you just need the data-specific concepts.

The junior DE who needs depth. Maybe you've already landed a data engineering role, or you're in an adjacent position where you touch pipelines. You can follow tutorials, but when something breaks at 2 AM, you're not confident you understand the system well enough to fix it. You need a mental model of how all the pieces fit together – not just how to use Airflow, but *why* Airflow exists and what problem it solved that cron jobs couldn't.

No matter which camp you fall into, we assume you have basic Python skills, some SQL knowledge, and familiarity with the command line. That's it. Everything else, we'll build from the ground up.

The 2026 Data Engineering Landscape

Before we touch a single tool, let's zoom out. What does a modern data engineering system actually look like? When an interviewer asks you to "describe the modern data stack," or when your manager asks you to architect a data platform from scratch, this is the mental model you need.

Every data platform in production today – whether it's at a three-person startup or at Netflix – follows the same fundamental flow:

Let's walk through each stage, because understanding this pipeline end-to-end is the single most important thing you can learn in this course.

Sources: Where Data Comes From

Data doesn't appear out of thin air. It comes from somewhere, and understanding the "somewhere" matters more than most people realize.

Transactional databases are the most common source. Your company's PostgreSQL or MySQL database backing the web application – the one that stores users, orders, products, payments. This is typically the first data source any DE connects to. At your first job, someone will say, "We need analytics on our orders table," and you'll need to figure out how to get that data out without killing the production database.

APIs and SaaS tools are the second major category. Stripe for payments, Salesforce for CRM, Google Analytics for web traffic, Shopify for e-commerce. Every business runs on dozens of SaaS tools, and each one has data locked inside it that the business needs for reporting. You'll spend a surprising amount of your career writing API extractors or configuring tools that pull data from these sources.

Event streams come from user behavior – clicks, page views, add-to-cart events, search queries. These arrive in real time, often at massive volume (think millions of events per hour for a mid-size e-commerce site), and they're the foundation of real-time analytics and personalization.

Files and flat data still matter more than you'd think. CSVs from partners, Excel files from finance teams, JSON dumps from third-party vendors. A huge part of real-world data engineering is dealing with messy, inconsistent file-based data that arrives on unpredictable schedules.

Ingestion: Getting Data In

Ingestion is the process of moving data from sources into your data platform. This sounds simple – and conceptually it is – but it's where a shocking number of pipelines break.

The two fundamental patterns are **batch** and **streaming**. Batch ingestion runs on a schedule: every hour, every day, pull the latest data. Streaming ingestion is continuous: as events happen, they flow into your platform in near-real-time. Most companies use both. Your order history might be batch-ingested nightly, while clickstream data flows in continuously via Kafka.

The key challenge in ingestion is **change data capture** (CDC) – figuring out what changed since your last pull. If you have a table with 100 million rows and 500 changed today, you don't want to re-copy all 100 million. You want to capture just the 500 changes. Tools like Debezium, Fivetran, and

Airbyte exist specifically to solve this problem, and understanding CDC will come up in virtually every DE interview.

Storage: Where Data Lives

Once data is ingested, it needs to live somewhere. The choice of storage system is one of the most consequential architectural decisions you'll make, because it affects everything downstream.

Data warehouses (Snowflake, BigQuery, Redshift) are optimized for analytical queries – the kind of SQL that aggregates millions of rows across joins. They're columnar, meaning they store data by column rather than by row, which makes aggregations blazingly fast.

Data lakes (S3, GCS, ADLS) store raw data in its original format – Parquet files, JSON, CSVs, images, whatever. They're cheap, scalable, and flexible. The trade-off is that they don't have a query engine built in; you need to bring your own (Spark, DuckDB, Trino).

Lakehouses (Delta Lake, Apache Iceberg) are the modern hybrid: lake-like storage with warehouse-like features (ACID transactions, schema enforcement, time travel). In 2026, this is where the industry is headed, and we'll cover it in depth in Modules 6-7.

Operational databases (PostgreSQL, MySQL) store the transactional data your application uses. As a DE, you'll read from these but rarely write to them. Your job is to get data *out* of the operational database and into the analytical layer without impacting production performance.

Processing: Transforming Data

Raw data is almost never in the shape your business needs. Orders might be in one table, customers in another, products in a third, and the business wants a single report that joins all three, calculates revenue by category, and filters to the last 90 days.

Batch processing (Spark, dbt, pandas, Polars, DuckDB) handles transformations that run on a schedule. "Every morning at 6 AM, rebuild the revenue dashboard." This is the bread and butter of most DE work.

Stream processing (Kafka Streams, Flink, Spark Streaming) handles transformations that must happen in real time. "When a user places an order, update the inventory count and trigger a shipping notification within 5 seconds." This is more complex and less common, but increasingly important.

At your first DE job, you'll almost certainly start with batch processing. A senior engineer will ask you to write a transformation in SQL or Python that takes raw data and produces a clean, aggregated table for the analytics team. This is where you prove your value early.

Orchestration: Making It All Run

Orchestration is the layer that ties everything together. It answers the question: "In what order do things run, and what happens when something fails?"

Apache Airflow is the dominant orchestration tool in 2026, and it's what we'll use in this course. Airflow lets you define DAGs (Directed Acyclic Graphs) – workflows where each step depends on the completion of previous steps. “First, extract data from Postgres. Then, load it into the warehouse. Then, run the dbt transformations. Then, update the dashboard. If any step fails, alert the on-call engineer.”

Other tools exist (Dagster, Prefect, Mage), and each has trade-offs we'll discuss. But Airflow has the largest community, the most job postings, and the deepest integration ecosystem. When you learn Airflow, you learn the concepts that transfer to any orchestrator.

Quality: Trusting Your Data

Data quality is the difference between a data platform that people trust and one they ignore. If your revenue dashboard shows \$0 on a Monday morning because a pipeline silently failed, the business loses trust in the entire data team. Rebuilding that trust takes months.

Data quality tools (dbt tests, Great Expectations, Soda, Monte Carlo) let you define expectations about your data and get alerted when reality doesn't match. “This table should have between 10,000 and 100,000 rows. Revenue should never be negative. Every order should have a valid customer ID.” These checks are the safety net that catches problems before they reach stakeholders.

Serving: Delivering Value

All of the above exists to serve one purpose: getting the right data to the right people in the right format at the right time. Serving is the final mile.

BI dashboards (Tableau, Looker, Metabase, Power BI) are the most common serving layer. Analysts and business users query pre-built dashboards or write ad-hoc SQL against the warehouse.

Reverse ETL pushes data back into operational tools. “Take the customer lifetime value we calculated in the warehouse and push it back into Salesforce so the sales team can see it.” Tools like Census and Hightouch specialize in this.

ML features serve processed data to machine learning models. “Every time a user loads the home page, fetch their real-time engagement score from the feature store so the recommendation model can personalize what they see.”

APIs expose data to applications. “The mobile app needs to show the user's order history, pulled from the data warehouse.”

Key Takeaway: Every data engineering role you'll ever have exists somewhere in this pipeline. Understanding the full picture – Sources through Serving – is what separates a junior DE who can follow instructions from a senior DE who can architect solutions.

Career Paths: What Data Engineering Roles Actually Look Like

Let's talk about what you're actually signing up for. Data engineering roles vary dramatically depending on the size and stage of the company.

Startup DE (Seed to Series B, 1-3 DEs)

At a startup, you *are* the data team. You'll set up the entire data stack from scratch – choose the warehouse, configure ingestion, write the first pipelines, build the first dashboards. The upside is enormous learning: you'll touch every part of the stack in your first month. The downside is that there's no one to learn from, no code review, and no on-call rotation – it's just you.

Typical salary range: \$120k-\$180k base + equity (equity can be worth a lot or nothing)

What your day looks like: Monday you're debugging a broken Fivetran connector. Tuesday you're writing a dbt model for the sales team. Wednesday you're setting up Airflow. Thursday the CEO needs a one-off analysis. Friday you're figuring out why Snowflake costs went up 300%.

Skills that matter most: Breadth over depth. SQL fluency, basic Python, one cloud platform (AWS or GCP), comfort with ambiguity.

Mid-Size Company DE (Series C+, 5-20 DEs)

This is where many DEs find their sweet spot. The data platform exists but needs to scale and mature. There's a team to collaborate with, established processes, and enough complexity to be intellectually challenging without the chaos of a startup.

Typical salary range: \$150k-\$250k total compensation

What your day looks like: You own a specific domain (payments pipeline, user analytics, ML feature engineering). You write code, review PRs, participate in on-call rotations, and work with stakeholders to understand their data needs. There are sprint planning meetings and architecture discussions.

Skills that matter most: Deep expertise in 2-3 areas (e.g., Spark + Airflow + dbt), ability to design and own systems end-to-end, strong communication with non-technical stakeholders.

Big Tech DE (FAANG/MANGA, 50+ DEs)

Big tech data engineering is a different world. The scale is enormous (petabytes, millions of events per second), the tooling is often internal (you might use Google's proprietary orchestrator instead of Airflow), and the role is more specialized.

Typical salary range: \$200k-\$350k+ total compensation (base + bonus + RSUs)

What your day looks like: You own a specific pipeline or set of pipelines. You spend significant time on system design, code review, and cross-team coordination. You might not write a SQL query for weeks because you're designing the infrastructure that runs SQL queries for 500 other people.

Skills that matter most: Distributed systems knowledge, ability to work at massive scale, system design interview skills, comfort with internal tools and proprietary systems.

The Interview Landscape

Data engineering interviews in 2026 typically involve:

1. **SQL** – Always. Often medium-to-hard LeetCode-style problems involving window functions, CTEs, and self-joins.
2. **Python coding** – Data manipulation, API interaction, basic algorithms.
3. **System design** – “Design a real-time analytics pipeline for an e-commerce platform.” This is where this course pays for itself.
4. **Domain knowledge** – Questions about data modeling, warehouse design, pipeline architecture, and data quality.
5. **Behavioral** – “Tell me about a time a pipeline broke in production and how you fixed it.”

The system design round is where most candidates fail and where the salary negotiation leverage lives. Companies pay a premium for engineers who can think architecturally.

The Course Project: Real-Time E-Commerce Analytics Platform

Theory without practice is just trivia. Throughout this course, you'll build a complete data platform for a fictional e-commerce company called **ShopFast**. This isn't a toy project – it's designed to mirror what you'd actually build at a Series B e-commerce startup.

The Business Context

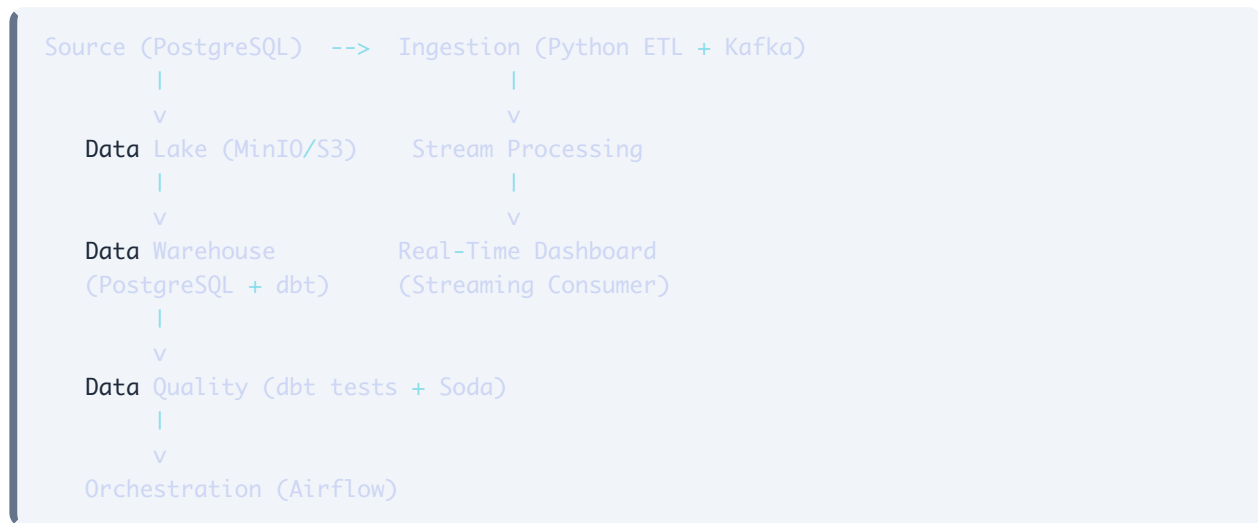
ShopFast is a mid-size online retailer. The dataset you'll work with includes:

- **~1 million orders** spanning two years
- **~50,000 customers** with demographic and behavioral data
- **~10,000 products** across multiple categories

The business needs answers to questions like: - What's our revenue by category, trending over the last 90 days? - Which customer segments have the highest lifetime value? - What products are frequently purchased together? - What's our real-time order volume, and are we seeing anomalies?

The Architecture You'll Build

By the end of this course, you'll have a working platform that includes:



Each module adds a new layer to this architecture. By Module 10, you'll have a fully functioning, monitored, tested data platform running locally on your machine.

Why These Specific Tools?

Every technology choice in this course is deliberate. Let me explain the reasoning, because “why this tool?” is one of the most important questions a data engineer can ask.

PostgreSQL as the source and warehouse database. Postgres is the most popular relational database in the data engineering world. It's open source, it's incredibly capable (supporting JSON, arrays, window functions, CTEs, materialized views), and it runs everywhere. We use it as both the source (simulating the production app database) and the analytical warehouse. In a real production setting, you'd likely use Snowflake or BigQuery for the warehouse, but Postgres lets us run everything locally without cloud costs. More importantly, everything you learn about writing analytical SQL in Postgres transfers directly to any warehouse.

Why Postgres over MySQL? Postgres has superior support for analytical workloads – better window functions, richer data types, native JSON support, and more standards-compliant SQL. MySQL is great for web application backends, but Postgres is the DE's database.

Apache Airflow for orchestration. Airflow is the most widely adopted workflow orchestrator in data engineering. Created at Airbnb in 2014 and open-sourced in 2015, it defines the category. When you look at DE job postings, Airflow appears more than any other orchestration tool. Its DAG-based model (define workflows as Python code, with tasks that have dependencies) has become the standard mental model for pipeline orchestration.

The competitors are real – Dagster has better local development, Prefect has a cleaner API, Mage has a better UI. But Airflow has the ecosystem: thousands of pre-built connectors, a massive community, managed offerings from every cloud provider (MWAA on AWS, Cloud Composer on GCP),

and deep integration with virtually every data tool. Learning Airflow first gives you the strongest foundation.

Apache Kafka for streaming. Kafka is the industry standard for event streaming. Created at LinkedIn to handle their massive event volume (trillions of events per day), it's now used by virtually every company that does real-time data processing. Kafka's core concept – a distributed, durable, append-only log – is one of the most important ideas in modern data architecture.

In this course, we use Kafka to stream e-commerce events (orders, page views, inventory changes) in real time. The patterns you'll learn – producers, consumers, topics, consumer groups – apply to every streaming platform (Amazon Kinesis, Google Pub/Sub, Azure Event Hubs).

MinIO as an S3 replacement. Amazon S3 is the de facto standard for object storage and data lake storage. Every cloud provider has an S3-compatible service (GCS, ADLS). MinIO is an open-source, S3-compatible object store that runs locally. This means you can learn S3 patterns – bucket creation, object upload/download, lifecycle policies – without spending a cent on AWS.

When you get to your real job and someone says “put the raw files in S3,” you'll already know the API and the mental model. The only difference is that the endpoint URL changes from `localhost:9000` to `s3.amazonaws.com`.

Docker for environment reproducibility. Here's a painful truth about data engineering: “it works on my machine” is the root cause of an embarrassing number of production incidents. Docker solves this by packaging your code *and its entire environment* (OS, libraries, configuration) into a container that runs identically everywhere.

In this course, Docker serves a practical purpose: with a single `docker compose up` command, you get PostgreSQL, Airflow, Kafka, and MinIO all running on your laptop, configured correctly and talking to each other. No manual installation, no version conflicts, no “I'm on Windows and the instructions are for Mac” problems.

At your first DE job, Docker will be one of the first things you encounter. Your pipelines will likely run in Docker containers, your local development environment will use Docker Compose, and your deployment pipeline will build and push Docker images. Getting comfortable with Docker now pays dividends immediately.

Setting Up Your Development Environment

Let's get your hands dirty. By the end of this section, you'll have a fully functional data engineering environment running on your machine.

Prerequisites

Before we start, make sure you have the following installed:

- **Python 3.11+** – Check with `python --version` or `python3 --version`
- **Docker Desktop** – [Download here](#). Make sure Docker is running (you should see the whale icon in your system tray/menu bar). Allocate at least 4GB of RAM to Docker in Settings > Resources.
- **Git** – Check with `git --version`
- **A code editor** – VS Code is recommended, but anything works

Step 1: Clone the Course Repository

```
git clone https://github.com/gyatesofficial/de-fast-track.git
cd de-fast-track
```

The repository is structured by module:

```
de-fast-track/
  docker-compose.yml      # The entire dev environment
  requirements.txt        # Python dependencies
  .env.example            # Environment variables template
  init-scripts/
    01_init.sql           # Database initialization
  modules/
    module-0/
      test_setup.py       # Verify your setup
    module-1/
      starter/            # Your starting point
      solution/           # Reference solution
    module-2/
      starter/
      solution/
    ...
```

Each module has a `starter/` directory with scaffolded code and a `solution/` directory with the complete implementation. Try to work through the starter files before looking at solutions.

Step 2: Start the Infrastructure

This is the moment Docker earns its keep. One command brings up the entire stack:

```
docker compose up -d
```

This starts five services defined in the `docker-compose.yml`:

```

services:
  postgres:
    image: postgres:16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dataeng
      POSTGRES_PASSWORD: dataeng
      POSTGRES_DB: warehouse
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dataeng"]
      interval: 10s
      timeout: 5s
      retries: 5

  airflow-init:
    image: apache/airflow:2.9.0
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
dataeng:dataeng@postgres:5432/warehouse
      AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    entrypoint: >
      bash -c "
        airflow db init &&
        airflow users create
          --username admin
          --password admin
          --firstname Admin
          --lastname User
          --role Admin
          --email admin@example.com || true
      "

  airflow-webserver:
    image: apache/airflow:2.9.0
    depends_on:
      airflow-init:
        condition: service_completed_successfully
    ports:
      - "8080:8080"
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
dataeng:dataeng@postgres:5432/warehouse
      AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    volumes:
      - ./airflow/dags:/opt/airflow/dags

```

```

- ./modules:/opt/airflow/modules
command: webserver

airflow-scheduler:
  image: apache/airflow:2.9.0
  depends_on:
    airflow-init:
      condition: service_completed_successfully
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
dataeng:dataeng@postgres:5432/warehouse
    AIRFLOW__CORE__LOAD_EXAMPLES: "false"
  volumes:
    - ./airflow/dags:/opt/airflow/dags
    - ./modules:/opt/airflow/modules
  command: scheduler

zookeeper:
  image: confluentinc/cp-zookeeper:7.6.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
  ports:
    - "2181:2181"

kafka:
  image: confluentinc/cp-kafka:7.6.0
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://
localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
T
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

minio:
  image: minio/minio:latest
  ports:
    - "9000:9000"
    - "9001:9001"
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  command: server /data --console-address ":9001"
  volumes:
    - minio_data:/data

volumes:
  postgres_data:
  minio_data:

```

Let's break down what's happening here, because understanding Docker Compose is a skill you'll use every single day as a DE:

Postgres is our database. The `healthcheck` block is important – it tells Docker to ping the database every 10 seconds and only consider it “healthy” when `pg_isready` succeeds. Other services use `depends_on: postgres: condition: service_healthy` to wait for Postgres to actually be ready before starting. Without this, Airflow would try to connect before Postgres is accepting connections, and you'd get cryptic connection errors.

The `volumes` section mounts `./init-scripts` into the container at `/docker-entrypoint-initdb.d`. Postgres automatically executes any `.sql` files in that directory on first startup. Our `01_init.sql` creates the schemas we'll use throughout the course:

```
-- Initialize warehouse schemas
CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS staging;
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE SCHEMA IF NOT EXISTS quality;

GRANT ALL ON SCHEMA raw TO dataeng;
GRANT ALL ON SCHEMA staging TO dataeng;
GRANT ALL ON SCHEMA analytics TO dataeng;
GRANT ALL ON SCHEMA quality TO dataeng;
```

These four schemas follow a common data warehouse pattern you'll see at most companies: - **raw** – Data as it arrived from the source, untransformed. The “source of truth.” - **staging** – Data that's been cleaned, typed, and deduplicated, but not yet modeled for analytics. - **analytics** – The final, business-ready tables that stakeholders query. Dimensional models, aggregations, metrics. - **quality** – Tables and logs related to data quality checks and monitoring.

Airflow has three services: `airflow-init` (runs once to initialize the database and create an admin user), `airflow-webserver` (the UI you'll access at `localhost:8080`), and `airflow-scheduler` (the process that actually triggers your DAGs on schedule). The `LocalExecutor` configuration means tasks run in the scheduler process itself – simple for development, though production deployments typically use `CeleryExecutor` or `KubernetesExecutor` for parallel execution.

Kafka requires Zookeeper for cluster coordination (managing broker metadata, leader election, etc.). In production, newer versions of Kafka can run without Zookeeper using KRaft mode, but the Confluent images still use the traditional setup. The `KAFKA_ADVERTISED_LISTENERS` configuration is the most common source of Kafka connection issues – it tells clients how to connect to the broker. We configure two listeners: one for internal Docker network communication (`kafka:29092`) and one for your local machine (`localhost:9092`).

MinIO provides an S3-compatible API on port 9000 and a web console on port 9001. The web console is genuinely useful – you can browse buckets, upload files, and manage access policies through a clean UI.

Step 3: Install Python Dependencies

Create a virtual environment and install the required packages:

```
python -m venv .venv
source .venv/bin/activate # On Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

The `requirements.txt` includes the core libraries you'll use throughout the course:

```
# Core
psycopg2-binary==2.9.9
sqlalchemy==2.0.30
pandas==2.2.2
polars==0.20.25
duckdb==0.10.3
pydantic==2.7.1
# Testing
pytest==8.2.1
pytest-cov==5.0.0
# API & HTTP
requests==2.32.3
httpx==0.27.0
# File formats
pyarrow==16.1.0
fastparquet==2024.5.0
# Utilities
python-dotenv==1.0.1
rich==13.7.1
```

A few things worth noting about these choices:

- **psycopg2-binary** is the PostgreSQL adapter. The `-binary` variant includes pre-compiled C extensions so you don't need `libpq-dev` installed. In production, you'd use `psycopg2` (non-binary) for better performance, but binary is simpler for development.
- **pandas and Polars** are both included intentionally. Pandas is the incumbent – you'll encounter it at every company, in every legacy codebase. Polars is the modern alternative, significantly faster for large datasets and with a more expressive API. We'll use both so you can compare.
- **DuckDB** is an embedded analytical database – think “SQLite for analytics.” It can query Parquet files, CSV files, and pandas DataFrames directly using SQL. It's become incredibly popular for local data exploration and is a tool you should have in your toolkit.
- **PyArrow** is the Python implementation of Apache Arrow, a columnar memory format. It's the bridge between file formats (Parquet), compute engines (Spark, DuckDB), and Python (pandas, Polars). You'll encounter Arrow everywhere in modern data engineering.
- **Pydantic** provides data validation using Python type hints. We'll use it to define schemas for our data models, ensuring that data conforms to expected shapes before we process it.

Step 4: Configure Environment Variables

Copy the example environment file:

```
cp .env.example .env
```

The defaults work out of the box with our Docker Compose setup:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=dataeng
DB_PASSWORD=dataeng
DB_NAME=warehouse
API_BASE_URL=http://localhost:5000
API_KEY=dev-key-change-me
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=raw-data
LOG_LEVEL=INFO
AIRFLOW_UID=50000
```

A note on `AIRFLOW_UID`: this sets the user ID inside the Airflow container. On Linux, this should match your host user ID (run `id -u` to check). On Mac and Windows, `50000` works fine. Getting this wrong causes file permission issues where Airflow can't read your DAG files – a common stumbling block.

Step 5: Verify Everything Works

Now the moment of truth. Run the verification script:

```
python modules/module-0/test_setup.py
```

Here's what the script does:


```

"""Module 0: Environment Verification Script
Run this to verify your development environment is set up correctly.
"""

import psycopg2
import pandas as pd
import polars as pl
import duckdb
import pyarrow
from pydantic import BaseModel

print(f"pandas: {pd.__version__}")
print(f"polars: {pl.__version__}")
print(f"duckdb: {duckdb.__version__}")
print(f"pyarrow: {pyarrow.__version__}")

conn = psycopg2.connect(
    host="localhost",
    database="warehouse",
    user="dataeng",
    password="dataeng"
)
cur = conn.cursor()
cur.execute("SELECT version();")
print(f"PostgreSQL: {cur.fetchone()[0]}")
conn.close()

print("\n✅ All good! You're ready for the course.")

```

If everything is set up correctly, you'll see version numbers for each library and a successful PostgreSQL connection. If you see errors, here are the most common fixes:

“connection refused” from PostgreSQL – Docker isn't running, or Postgres hasn't finished starting. Run `docker compose ps` to check service status. Wait 30 seconds and try again.

Import errors for Python packages – You're not in the virtual environment. Run `source .venv/bin/activate` and try again.

Docker Compose errors – Make sure Docker Desktop is running and you've allocated enough RAM (4GB minimum). On Mac, check Docker Desktop > Settings > Resources.

Step 6: Verify Airflow and MinIO

Open your browser and check the UIs:

- **Airflow:** Navigate to `http://localhost:8080`. Log in with username `admin`, password `admin`. You should see an empty DAGs list – we'll populate it in Module 4.
- **MinIO:** Navigate to `http://localhost:9001`. Log in with `minioadmin` / `minioadmin`. You should see the MinIO console with no buckets yet.

If both UIs load, your environment is fully operational.

How This Course Is Structured

Each module in this course follows a consistent pattern:

1. **Concept introduction** – We explain the *what* and the *why* before we touch any code.
Understanding the problem a tool solves is more important than memorizing its syntax.
2. **Guided walkthrough** – We build something together, with detailed explanations at each step.
3. **Hands-on exercise** – You take the `starter/` code and complete it. This is where the real learning happens.
4. **Solution review** – Compare your approach to the `solution/` directory. There's often more than one right answer; the goal is to understand the trade-offs.

Here's the module map:

MODULE	TOPIC	WHAT YOU'LL BUILD
0	Welcome & Setup	Dev environment
1	Data Modeling Foundations	Dimensional model for ShopFast
2	Incremental Loading	SCD2 dimensions + incremental facts
3	Python ETL Pipelines	Production-grade extraction pipeline
4	Orchestration with Airflow	DAGs for the ShopFast pipeline
5	Modern File Formats & DataFrames	Parquet, Polars, DuckDB exploration
6	Real-Time Streaming with Kafka	Event producer + consumer
7	Data Quality & Contracts	dbt tests, Soda checks, schema contracts
8	Infrastructure & Deployment	Terraform, Docker, CI/CD
9	Semantic Search & AI Pipelines	Embeddings + vector search
10	Monitoring & Observability	Pipeline metrics + alerting

The best way to learn: Don't just read. Type the code. Break it. Fix it. Change parameters and see what happens. Add a print statement to understand what a variable contains. The gap between "I understand this conceptually" and "I can build this" is only bridged by doing.

A Note from Your Instructor

Data engineering is one of the most rewarding careers in tech. You build the infrastructure that powers every data-driven decision a company makes. When the CEO quotes a revenue number in an all-hands meeting, that number came from a pipeline you built. When the ML team deploys a model that increases conversion by 3%, it was trained on data you delivered. When an analyst discovers a trend that changes the company's strategy, they found it in a table you created.

The work is often invisible. Nobody toasts the data engineer at the company party. But when the data is wrong, or late, or missing – everyone notices. There's a quiet satisfaction in building systems that just work, day after day, processing millions of records while you sleep.

This course will give you the skills to build those systems. Let's get started.

What Comes Next

In Module 1, we dive into data modeling – the art and science of structuring data for analytical workloads. You'll learn about dimensional modeling (star schemas, fact tables, dimension tables), why it matters, and you'll build your first dimensional model for the ShopFast dataset. If SQL is your strongest skill, this is where you'll shine. If it's not, this is where you'll build it.

Let's go.

Module 1: Data Modeling Foundations

Data modeling is where data engineering begins. Before you write a single pipeline, before you spin up Airflow, before you touch Spark – you need to understand how to structure data so that it actually serves the people who use it. This module covers the theory, patterns, and practical decisions that separate a well-designed data warehouse from a dumpster fire of duplicated, contradictory tables.

1.1 Why Data Modeling Matters

If you take one thing from this module, let it be this: **bad models produce bad data**. No amount of clever ETL, expensive tooling, or dashboard polish can fix a data model that was designed poorly from the start.

You have seen the anti-pattern even if you do not have the vocabulary for it yet. Somewhere in every poorly-governed data environment there is a table called `orders_final_FINAL_v2` sitting alongside `orders_backup_USE_THIS_ONE`. Nobody knows which is correct. Reports from marketing do not match reports from finance. Analysts spend 80% of their time wrangling data and 20% actually analyzing it. That is what happens when nobody invests in modeling.

A good data model gives you:

- **Clean, intuitive queries** – analysts can self-serve without needing to understand 47 joins
- **A single source of truth** – one agreed-upon definition of “revenue,” “active customer,” or “churn”
- **Performance** – the data is physically organized for the questions people actually ask
- **Auditability** – you can trace any number back to its source

When You’ll Use This at Work

Data modeling is not a one-time design exercise you do on day one and forget. You will revisit your models every time the business adds a new product line, enters a new market, acquires a company, or changes how it defines a metric. At mature companies, data modeling review is a formal process with stakeholder sign-off. At startups, it is often you and a whiteboard.

Interview Prep

Data modeling is the single most common topic in data engineering interviews. Interviewers will hand you a business scenario and ask you to design a schema on the spot. They are testing wheth-

er you can identify the grain of a fact table, choose appropriate dimension attributes, and reason about trade-offs like normalization versus denormalization. The concepts in this module will come up in virtually every technical interview loop you encounter.

1.2 OLTP vs OLAP

Before you can model data for analytics, you need to understand the two fundamentally different worlds that data lives in.

OLTP: Online Transaction Processing

OLTP databases are the databases behind your applications. When a customer places an order on Amazon, that order gets written to an OLTP database. When you update your profile on LinkedIn, that write hits an OLTP database.

Characteristics of OLTP systems:

- **Small, fast queries:** `SELECT * FROM orders WHERE id = 12345` – look up one row by primary key
- **High concurrency:** thousands of users reading and writing simultaneously
- **Normalized schemas:** data is split across many tables to eliminate redundancy (3rd normal form)
- **Optimized for writes:** inserts and updates need to be fast so the application feels responsive
- **Row-oriented storage:** entire rows are stored together on disk, which is great for fetching a single complete record

Real-world examples: the PostgreSQL database behind a Rails app, the MySQL database behind a WordPress site, the DynamoDB table storing user sessions for a mobile app.

OLAP: Online Analytical Processing

OLAP databases are where analytics happens. When a VP of Sales asks “What was our revenue by region for Q3, compared to last year?” that question gets answered by an OLAP system.

Characteristics of OLAP systems:

- **Large scan queries:** `SELECT region, SUM(revenue) FROM fact_orders WHERE year = 2025 GROUP BY region` – aggregate millions of rows
- **Low concurrency:** maybe a few dozen analysts and dashboards, not thousands of app users
- **Denormalized schemas:** data is pre-joined and flattened so queries are simpler and faster
- **Optimized for reads:** complex aggregations need to be fast; nobody is doing single-row inserts

- **Columnar storage:** columns are stored together on disk, so scanning one column across a billion rows is extremely efficient

Real-world examples: Snowflake, BigQuery, Redshift, ClickHouse, DuckDB.

Your Job as a Data Engineer

Here is the core loop of data engineering, distilled to one sentence: **you move data FROM OLTP systems TO OLAP systems, transforming it along the way so it is useful for analysis.**

The application team owns the OLTP database. The analytics team consumes the OLAP database. You own the space in between – the extraction, transformation, loading, quality checks, and modeling that makes raw operational data into something an analyst can query with confidence.

```
[OLTP: App Database] --> [Your ETL/ELT Pipeline] --> [OLAP: Data Warehouse]
(normalized)           (transform & model)           (denormalized)
(row-oriented)         (columnar)
(small fast queries)   (big scan queries)
```

The Postgres Exception

Here is a practical nuance the textbooks often skip: **PostgreSQL can serve as both your OLTP and OLAP system** if you are a small-to-medium company. Postgres is remarkably versatile. If you have fewer than 50 million rows in your fact tables, a well-indexed Postgres instance with some materialized views can handle both your application workload and your analytics workload.

This is not a permanent architecture – eventually you will outgrow it. But for a Series A startup with 10 engineers, running a separate Snowflake instance that costs \$30k/month makes no sense when Postgres on a beefy RDS instance can do the job for \$500/month.

At Spotify's scale (hundreds of billions of stream events), you absolutely need a dedicated OLAP system. At a 20-person startup tracking thousands of orders per day, Postgres is fine.

Common Mistakes

- **Querying the production OLTP database directly for analytics.** This is the number one mistake junior data engineers make. Your heavy analytical queries will compete with the application's transactional queries, causing the app to slow down or time out. Always use a read replica at minimum, or better yet, extract data into a separate analytical store.
- **Over-normalizing your OLAP schema.** Normalization is great for OLTP. For OLAP, it means analysts need 12 joins to answer a simple question. Denormalize aggressively in your warehouse.

Interview Prep

If an interviewer asks “What is the difference between OLTP and OLAP?” they are not looking for a textbook definition. They want you to demonstrate that you understand the implications for schema design, storage format, query patterns, and how data flows between the two. Mention that your job as a DE is to bridge the gap.

1.3 Dimensional Modeling

Dimensional modeling is the dominant approach for designing OLAP schemas. It was formalized by Ralph Kimball in the 1990s, and it remains the most practical and widely-used methodology in the industry. The reason it has persisted for 30 years is simple: it works.

The core idea is to separate your data into two types of tables: **facts** and **dimensions**.

Facts: The Business Events

A fact table records something that happened. Each row represents a measurable business event at a specific grain (level of detail).

Examples of facts: - **E-commerce**: an order line item (one product within one order) - **Streaming**: a stream event (one user listened to one track) - **Ad tech**: an impression (one ad was shown to one user) - **SaaS**: a subscription event (one user started, renewed, or cancelled a plan) - **Finance**: a transaction (one debit or credit to one account)

At Spotify, the central fact table might be `fact_streams` where each row represents one stream event: a user played a track, on a specific date, from a specific playlist, on a specific device. The grain is “one stream.”

At Uber, the central fact table might be `fact_trips` where each row represents one completed trip: a rider, a driver, a pickup location, a dropoff location, a fare amount, a date.

Facts should be lean. A fact row contains: 1. Foreign keys pointing to dimension tables (who, what, when, where) 2. Numeric measures (how much, how many) 3. Degenerate dimensions (order ID, invoice number – identifiers that do not warrant their own dimension table)

That is it. No descriptive text. No attributes that change. Keep fact tables skinny and long (many rows, few columns).

Why keep facts lean? Because fact tables are by far the largest tables in your warehouse. If your fact table has 500 million rows, every unnecessary column you add wastes storage and slows down every query that scans the table. A VARCHAR(200) product name sitting in every fact row is repeated 500 million times. That same product name stored once in a dimension table with 10,000 rows and joined at query time is negligible.

Dimensions: The Context

A dimension table provides the descriptive context around a fact. Dimensions answer the “who, what, when, where, how” questions.

Examples of dimensions: - **dim_customer**: name, email, city, state, segment, signup date - **dim_product**: product name, category, subcategory, brand, price - **dim_date**: calendar date, day of week, month, quarter, year, is_weekend, is_holiday - **dim_store**: store name, address, region, square footage - **dim_campaign**: campaign name, channel, start date, budget

Dimensions are typically wide (many columns) and short (relatively few rows compared to facts). A retailer might have 50,000 products but 500 million order line items.

Star Schema

When you put facts and dimensions together, you get a **star schema** – so named because the diagram looks like a star with the fact table at the center and dimension tables radiating outward.



The star schema is the default starting point for virtually every analytical data model. It is intuitive for analysts (they understand “orders” joined with “customers” and “products”), it performs well (dimension lookups via foreign keys are fast), and it is easy to extend (adding a new dimension just means adding a new foreign key to the fact table).

Surrogate Keys vs Natural Keys

This is a decision you will make for every dimension table, and getting it wrong causes real pain downstream.

A **natural key** is the identifier from the source system: `customer_id = 12345` from your application database. A **surrogate key** is a synthetic key you generate in your warehouse: `customer_key = 1` (typically a serial/auto-increment integer).

Why use surrogate keys?

1. **Source systems change.** If Marketing migrates from Salesforce to HubSpot, the customer IDs change. If your fact table uses the natural key, you have to update millions of rows. If it uses a surrogate key, you update one row in the dimension table.
2. **SCD Type 2 requires them.** When a customer moves from New York to San Francisco, you create a new dimension row for the same customer (more on this in section 1.4). That new row

needs its own unique key. The natural key (`customer_id = 12345`) now appears in two rows – so it cannot be the primary key.

3. **Cross-system integration.** When you combine customers from two different source systems, natural keys collide. Customer 12345 in the billing system is not the same as customer 12345 in the CRM. Surrogate keys eliminate this problem.

4. **Performance.** Integer surrogate keys are smaller and faster to join on than VARCHAR natural keys or composite keys.

The rule: dimension tables have a surrogate key as their primary key and also store the natural key for traceability. Fact tables reference dimensions via surrogate keys.

Pre-Calculated Measures

One of the most practical decisions in dimensional modeling is what to pre-calculate in the fact table. Consider an order line item:

```
quantity = 3, unit_price = 29.99, discount = 5.00
```

You could store just these raw values and calculate revenue at query time: `quantity * unit_price - discount`. But if every single dashboard query needs to do this multiplication across 500 million rows, you are wasting compute on every query.

Instead, pre-calculate at load time:

```
revenue = quantity * unit_price - discount -- 84.97
cost = quantity * unit_cost -- 45.00
profit = revenue - cost -- 39.97
```

Now every query that needs revenue just does `SUM(revenue)` instead of `SUM(quantity * unit_price - discount)`. This is faster, simpler, and – critically – it ensures every report calculates revenue the same way. No more arguments about whether the discount should be subtracted before or after tax.

Pre-calculate measures that have an agreed-upon business definition. Revenue, cost, profit, margin – these should be computed once, correctly, at load time. Do not make every analyst re-derive them.

The Date Dimension

Every dimensional model has a date dimension. It deserves special attention because dates are used in virtually every analytical query and because the date dimension has unique properties.

Unlike other dimensions, the date dimension is **fully pre-populated**. You generate all rows upfront (typically 10-20 years of dates) before any facts are loaded. Every calendar date gets a row, whether or not any business events occurred on that date.

The date key is typically an integer in `YYYYMMDD` format (e.g., `20250315` for March 15, 2025). This is both human-readable and efficient for range queries.

```
CREATE TABLE dim_date (  
    date_key          INT PRIMARY KEY,          -- YYYYMMDD format  
    calendar_date     DATE NOT NULL UNIQUE,  
    day_of_week       VARCHAR(10) NOT NULL,  
    day_of_month      INT NOT NULL,  
    month             INT NOT NULL,  
    month_name        VARCHAR(10) NOT NULL,  
    quarter           INT NOT NULL,  
    year              INT NOT NULL,  
    is_weekend        BOOLEAN NOT NULL,  
    is_holiday        BOOLEAN NOT NULL DEFAULT FALSE  
);
```

Why not just use a `DATE` column in the fact table? Because the date dimension gives you pre-built attributes for grouping and filtering. `WHERE is_weekend = TRUE` is cleaner than `WHERE EXTRACT(ISODOW FROM order_date) IN (6, 7)`. `GROUP BY quarter` is cleaner than `GROUP BY EXTRACT(QUARTER FROM order_date)`. And you can add business-specific attributes like fiscal quarter, holiday flags, or promotion periods that do not exist in a raw `DATE` type.

Here is how to populate a date dimension in PostgreSQL:

```
INSERT INTO dim_date (date_key, calendar_date, day_of_week, day_of_month,  
                     month, month_name, quarter, year, is_weekend, is_holiday)  
SELECT  
    TO_CHAR(d, 'YYYYMMDD')::INT AS date_key,  
    d AS calendar_date,  
    TO_CHAR(d, 'Day') AS day_of_week,  
    EXTRACT(DAY FROM d) AS day_of_month,  
    EXTRACT(MONTH FROM d) AS month,  
    TO_CHAR(d, 'Month') AS month_name,  
    EXTRACT(QUARTER FROM d) AS quarter,  
    EXTRACT(YEAR FROM d) AS year,  
    EXTRACT(ISODOW FROM d) IN (6, 7) AS is_weekend,  
    FALSE AS is_holiday  
FROM generate_series('2024-01-01'::date, '2026-12-31'::date, '1 day') AS d;
```

When You'll Use This at Work

Dimensional modeling is the bread and butter of data warehouse design. Every time you build a new analytical dataset – whether in Snowflake, BigQuery, Redshift, or plain Postgres – you will be thinking in terms of facts and dimensions. Even if your team uses dbt models that produce flat tables, the mental framework of “what is the grain, what are the measures, what are the dimensions” guides every design decision.

Common Mistakes

- **Making the grain too coarse.** If your grain is “one row per order” but an order can have multiple products, you lose the ability to analyze at the product level. Always define the finest useful grain and aggregate upward.
- **Putting descriptive attributes in the fact table.** Product name, customer name, category – these belong in dimensions. The fact table should only have keys and measures.
- **Forgetting to index dimension foreign keys in the fact table.** Without indexes on `customer_key`, `product_key`, and `date_key`, your star schema joins will be full table scans.
- **Not pre-populating the date dimension.** If you generate date rows on the fly as facts arrive, you will have gaps for dates with no events, which breaks time-series reporting.

1.4 Slowly Changing Dimensions (SCDs)

Here is the problem: dimensions change. A customer moves from New York to San Francisco. A product gets reclassified from “Electronics” to “Smart Home.” A salesperson transfers to a different region.

When that change happens, what do you do with the historical data? If Customer 12345 placed an order when they lived in New York and now they live in San Francisco, which city should show up in a report analyzing orders by customer location?

The answer depends on your business requirements, and Slowly Changing Dimension (SCD) types give you a vocabulary for the options.

SCD Type 1: Overwrite

The simplest approach: just overwrite the old value with the new one. The customer moved to San Francisco? Update the row:

```
Before: customer_id=12345, name="Alice", city="New York"
After:  customer_id=12345, name="Alice", city="San Francisco"
```

Pros: Simple. No complexity. Dimension table stays small.

Cons: You lose history. Every historical order for this customer now shows “San Francisco” even if they were in New York when they placed it. If the VP of Sales asks “What was our NYC revenue last year?” the numbers are wrong because Alice’s old NYC orders now count toward San Francisco.

When to use Type 1: For attributes where history does not matter. A customer’s email address, a product’s internal SKU code, a typo correction. If nobody will ever ask “what was the old value?” then Type 1 is fine.

SCD Type 2: Add a New Row

The industry standard for important attributes. When a value changes, you do NOT update the existing row. Instead, you expire the old row and insert a new one:

```
customer_key=1001, customer_id=12345, name="Alice", city="New York",  
    effective_date='2023-01-15', expiry_date='2025-03-01', is_current=FALSE  
  
customer_key=1002, customer_id=12345, name="Alice", city="San Francisco",  
    effective_date='2025-03-01', expiry_date='9999-12-31', is_current=TRUE
```

Now the same natural key (`customer_id = 12345`) has two rows in the dimension table, each with its own surrogate key. Orders placed before March 2025 reference `customer_key = 1001` (New York). Orders placed after reference `customer_key = 1002` (San Francisco). Historical queries are accurate.

This is why surrogate keys exist. The natural key is no longer unique in the dimension table, so you need a separate primary key.

The 9999-12-31 convention: The current row always has an expiry date of `9999-12-31` (a far-future sentinel value). This makes range queries simple: `WHERE '2025-06-15' BETWEEN effective_date AND expiry_date` works for both current and historical rows.

Pros: Full history preserved. Historical reports remain accurate. This is the gold standard.

Cons: Dimension tables grow over time (one new row per change per entity). Queries for “current” data need a `WHERE is_current = TRUE` filter.

When to use Type 2: For any attribute where historical accuracy matters. Customer location (for geographic analysis), product category (for category-level reporting), product price (for margin calculations), customer segment (for cohort analysis).

Type 2 is the default. If you are not sure which type to use, use Type 2. It is better to preserve history you do not need than to lose history you do need. You cannot go back in time and recreate overwritten data.

SCD Type 3: Add a Column

Instead of adding rows, add columns for the old and new values:

```
customer_id=12345, name="Alice", current_city="San Francisco", previous_city="New York"
```

Pros: Simple to query both current and previous values. No row explosion.

Cons: Only tracks one level of history (current vs previous, not the full history). What happens when the customer moves a third time? You lose the original value.

When to use Type 3: Rarely. It is useful when you specifically need to compare “before and after” for a known change event (like a company reorganization that reassigns all accounts to new regions). In practice, Type 3 is uncommon.

The Real-World Approach: Mix and Match

In practice, you do not pick one SCD type for an entire dimension table. You pick a type per attribute:

- **Customer dimension:**

- `city`, `state`, `segment` → Type 2 (historical accuracy matters for geographic and cohort analysis)
- `email` → Type 1 (nobody analyzes by historical email addresses)
- `name` → Type 1 (name corrections are not analytically relevant)

- **Product dimension:**

- `category`, `subcategory` → Type 2 (category-level reporting needs historical accuracy)
- `unit_price`, `unit_cost` → Type 2 (margin calculations need the price at time of sale)
- `product_name` → Type 1 (name tweaks are cosmetic)

Here is a complete SCD Type 2 dimension table definition:

```
CREATE TABLE dim_customer (  
  customer_key SERIAL PRIMARY KEY,      -- surrogate key  
  customer_id INT NOT NULL,             -- natural key  
  name VARCHAR(200) NOT NULL,  
  email VARCHAR(200),  
  city VARCHAR(100),  
  state VARCHAR(50),  
  country VARCHAR(50) DEFAULT 'US',  
  segment VARCHAR(50),  
  created_date DATE,  
  effective_date DATE NOT NULL,  
  expiry_date DATE NOT NULL DEFAULT '9999-12-31',  
  is_current BOOLEAN NOT NULL DEFAULT TRUE  
);  
  
CREATE INDEX idx_dim_customer_natural ON dim_customer(customer_id, is_current);
```

And the matching product dimension:

```
CREATE TABLE dim_product (
  product_key      SERIAL PRIMARY KEY,
  product_id       INT NOT NULL,
  product_name     VARCHAR(300) NOT NULL,
  category         VARCHAR(100),
  subcategory      VARCHAR(100),
  brand            VARCHAR(100),
  unit_cost        NUMERIC(10,2),
  unit_price       NUMERIC(10,2),
  effective_date    DATE NOT NULL,
  expiry_date       DATE NOT NULL DEFAULT '9999-12-31',
  is_current        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_dim_product_natural ON dim_product(product_id, is_current);
```

When You'll Use This at Work

Every warehouse you touch will have SCD decisions baked into it. When a stakeholder says “our geographic revenue numbers shifted overnight and nobody changed the reports,” the first thing you check is whether a dimension attribute was overwritten (Type 1) when it should have been versioned (Type 2). Getting SCD right is the difference between a trustworthy warehouse and one that produces mysterious data drift.

Interview Prep

Interviewers love SCD questions. The standard question is: “A customer changes their address. How do you handle this in your data warehouse?” The strong answer walks through all three types, explains the trade-offs, and states that Type 2 is the default for analytically important attributes. Bonus points if you mention the `effective_date` / `expiry_date` / `is_current` pattern and explain why surrogate keys are necessary for Type 2.

Common Mistakes

- **Using Type 1 for everything because it is simpler.** You will not realize the damage until a stakeholder asks a historical question and the numbers are wrong. By then, the history is gone.
- **Forgetting the `is_current` flag.** Without it, every query that wants “current customer data” needs a `WHERE expiry_date = '9999-12-31'` filter, which is error-prone and non-obvious to analysts.
- **Not indexing the natural key + `is_current` combo.** The most common lookup pattern is “give me the current row for customer_id = 12345.” Without an index, this scans the entire dimension table.

1.5 Modern Approaches

The Kimball dimensional model has been the standard for decades, and it still works well. But modern columnar warehouses (Snowflake, BigQuery, Redshift, Databricks) have changed some of the performance assumptions that drove traditional star schema design. Two modern patterns are worth knowing.

One Big Table (OBT)

The One Big Table approach says: skip the star schema entirely. Pre-join all your dimensions into a single wide, denormalized table.

Instead of `fact_orders` joining to `dim_customer`, `dim_product`, and `dim_date`, you create a single table called `orders_wide` that has every column from every dimension baked in:

```
order_id | customer_name | customer_city | customer_segment | product_name | category |  
brand | order_date | day_of_week | quantity | revenue | profit
```

Why this works in modern warehouses:

- **Columnar storage makes wide tables cheap.** If your query only needs `customer_city` and `revenue`, the engine only reads those two columns from disk, even if the table has 200 columns. The other 198 columns cost nothing.
- **No joins means simpler queries and faster execution.** `SELECT city, SUM(revenue) FROM orders_wide GROUP BY city` is faster than joining three tables.
- **Compression.** Columnar warehouses compress repeated values extremely efficiently. The string “Electronics” repeated 50 million times in a category column compresses down to almost nothing.

When OBT makes sense:

- Your fact table has fewer than ~100 million rows (beyond that, the duplication of dimension data can become meaningful even with compression)
- You are using a columnar warehouse (Snowflake, BigQuery, Redshift)
- Your analysts are writing SQL directly and prefer simple queries
- You do not need SCD Type 2 history (OBT is awkward with versioned dimensions)

When OBT does not make sense:

- You need historical dimension tracking (SCD Type 2)
- You have very large fact tables (billions of rows) where dimension data duplication adds up
- Multiple fact tables share the same dimensions (you would duplicate dimension data across every OBT)

Wide Event Tables with JSONB

Another modern pattern: store events as wide rows with a JSONB (or VARIANT in Snowflake, or STRUCT in BigQuery) column for flexible attributes.

```
CREATE TABLE events (  
  event_id      UUID PRIMARY KEY,  
  event_type    VARCHAR(50),  
  event_time    TIMESTAMP,  
  user_id       INT,  
  properties    JSONB      -- flexible key-value pairs  
);
```

This is common for product analytics (tracking user clicks, page views, feature usage) where the set of attributes varies by event type. A “page_view” event has a URL; a “purchase” event has an amount; a “signup” event has a referral source. JSONB lets you store all of these in the same table without needing 300 nullable columns.

Modern warehouses handle semi-structured data well. In BigQuery, you can query into STRUCT fields. In Snowflake, you can query into VARIANT columns. The performance penalty compared to native columns is small and shrinking.

The Practical Approach: Use Both

At most companies, the answer is not “star schema OR OBT OR event tables.” It is all of them, layered:

1. **Raw layer:** event tables and source system replicas (messy, complete, append-only)
2. **Modeled layer:** star schema with proper dimensions and facts (the “single source of truth”)
3. **Mart layer:** OBT-style wide tables pre-joined for specific teams or dashboards

This layered approach (often called “medallion architecture” – bronze/silver/gold) gives you the best of all worlds. The star schema preserves history and enforces consistency. The OBT marts give analysts the simplicity they want.

When You’ll Use This at Work

If you join a modern data team using dbt, you will almost certainly see this layered pattern. Your staging models clean the raw data, your intermediate models implement dimensional modeling, and your mart models denormalize into wide tables for specific use cases. Understanding when to use each pattern – and why – is what makes you effective.

1.6 Data Vault Basics

Data Vault is an enterprise data modeling methodology designed for large organizations with complex, evolving source systems. You do not need to master Data Vault for most jobs, but you need to know it exists and understand the core concepts – because it comes up in interviews and you may encounter it at large enterprises.

The Core Components

Hubs store the unique business keys for a business concept. A `hub_customer` table might have just `customer_id` and a hash key and a load timestamp. That is it – no descriptive attributes.

Links store the relationships between hubs. An `link_order_customer` table connects orders to customers. Links are where many-to-many relationships live.

Satellites store the descriptive attributes and their change history. A `sat_customer_details` table has name, email, city, segment – with effective dates for tracking changes.

Why Data Vault Exists

Data Vault solves a specific problem: **what happens when your source systems are messy, constantly changing, and you need to integrate data from dozens of them?** The hub/link/satellite separation means you can load data from any source system without redesigning your model. New sources just add new satellites.

It is also fully auditable and supports parallel loading (hubs, links, and satellites can be loaded independently).

Why You Probably Won't Use It

Data Vault is complex. The number of tables explodes (a simple star schema with 4 tables becomes 10+ tables in Data Vault). Queries require many joins. It makes sense for Fortune 500 companies with 50 source systems and dedicated data architecture teams. It does not make sense for a 5-person data team at a startup.

Interview Prep

If an interviewer asks about Data Vault, they are testing whether you know the landscape of modeling methodologies beyond Kimball. A strong answer is: “Data Vault uses hubs for business keys, links for relationships, and satellites for descriptive attributes with history tracking. It is designed for enterprise environments with many heterogeneous source systems. I would use it when I need to integrate many sources at scale, but for most analytical use cases I would start with dimensional modeling because it is simpler and more accessible for analysts.”

1.7 Hands-On: Design a Schema for ShopFast E-Commerce

Now we put it all together. ShopFast is a mid-size e-commerce company with approximately 50,000 customers, 10,000 products in 20 categories, and 1,000,000 orders over the past 2 years.

The Business Questions

The analytics team needs to answer:

1. **Revenue by category and month** – Which product categories are growing? Which are declining?
2. **Customer lifetime value** – Who are our best customers? What segments spend the most?
3. **Product performance** – Which products have the highest margins? Which are being returned most?
4. **Geographic trends** – Which cities and states generate the most revenue?

Step 1: Identify the Grain

The grain is the most fundamental decision. Ask: “What does one row in my fact table represent?”

For ShopFast, the grain is **one row per order line item** – one product within one order. Not one row per order (too coarse – you lose product-level detail). Not one row per shipment or per payment (those could be separate fact tables if needed).

Why this grain? Because every business question above can be answered at the line item level and aggregated upward. Revenue by category requires knowing which products were in each order. Customer lifetime value requires summing line-item revenue per customer. Product margins require knowing cost and revenue per product per order.

Step 2: Identify the Dimensions

Look at the business questions and ask “by what?” and “for which?”

- Revenue **by category** and **by month** → `dim_product` (has category), `dim_date` (has month)
- Customer lifetime value **by customer** and **by segment** → `dim_customer` (has segment)
- Product performance **by product** → `dim_product`
- Geographic trends **by city/state** → `dim_customer` (has city, state)

Three dimensions: `dim_date`, `dim_customer`, `dim_product`.

Step 3: Identify the Measures

What numeric values do we need to aggregate?

- `quantity` – how many units

- `unit_price` – the price charged (at time of sale)
- `discount_amount` – any discount applied
- `revenue` – pre-calculated: `quantity * unit_price - discount_amount`
- `cost` – pre-calculated: `quantity * unit_cost` (unit_cost comes from dim_product)
- `profit` – pre-calculated: `revenue - cost`

Step 4: SCD Decisions

What changes over time and does historical accuracy matter?

- **Customer city/state** → SCD Type 2. Geographic analysis needs to reflect where the customer was when they ordered, not where they are now.
- **Customer segment** → SCD Type 2. Cohort analysis (comparing enterprise vs SMB revenue) needs historical accuracy.
- **Customer email/name** → SCD Type 1. Not analytically relevant.
- **Product category/subcategory** → SCD Type 2. Category-level reporting needs historical accuracy.
- **Product price** → SCD Type 2. Margin calculations need the price at time of sale.
- **Product name** → SCD Type 1. Name changes are cosmetic.

The Complete Solution

Here is the full dimensional model. This is what you would implement in a real warehouse:

```

-- =====
-- DIMENSION: Date
-- =====
CREATE TABLE dim_date (
    date_key          INT PRIMARY KEY,          -- YYYYMMDD format
    calendar_date     DATE NOT NULL UNIQUE,
    day_of_week       VARCHAR(10) NOT NULL,
    day_of_month      INT NOT NULL,
    month             INT NOT NULL,
    month_name        VARCHAR(10) NOT NULL,
    quarter           INT NOT NULL,
    year              INT NOT NULL,
    is_weekend        BOOLEAN NOT NULL,
    is_holiday        BOOLEAN NOT NULL DEFAULT FALSE
);

-- Populate date dimension (2024-01-01 through 2026-12-31)
INSERT INTO dim_date (date_key, calendar_date, day_of_week, day_of_month,
                     month, month_name, quarter, year, is_weekend, is_holiday)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT AS date_key,
    d AS calendar_date,
    TO_CHAR(d, 'Day') AS day_of_week,
    EXTRACT(DAY FROM d) AS day_of_month,
    EXTRACT(MONTH FROM d) AS month,
    TO_CHAR(d, 'Month') AS month_name,
    EXTRACT(QUARTER FROM d) AS quarter,
    EXTRACT(YEAR FROM d) AS year,
    EXTRACT(ISODOW FROM d) IN (6, 7) AS is_weekend,
    FALSE AS is_holiday
FROM generate_series('2024-01-01'::date, '2026-12-31'::date, '1 day') AS d;

-- =====
-- DIMENSION: Customer (SCD Type 2)
-- =====
CREATE TABLE dim_customer (
    customer_key      SERIAL PRIMARY KEY,      -- surrogate key
    customer_id       INT NOT NULL,            -- natural key from source
    name              VARCHAR(200) NOT NULL,
    email             VARCHAR(200),
    city              VARCHAR(100),
    state             VARCHAR(50),
    country           VARCHAR(50) DEFAULT 'US',
    segment           VARCHAR(50),
    created_date      DATE,
    effective_date    DATE NOT NULL,
    expiry_date       DATE NOT NULL DEFAULT '9999-12-31',
    is_current        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_dim_customer_natural ON dim_customer(customer_id, is_current);

-- =====
-- DIMENSION: Product (SCD Type 2)

```

```

-- =====
CREATE TABLE dim_product (
    product_key    SERIAL PRIMARY KEY,
    product_id     INT NOT NULL,
    product_name   VARCHAR(300) NOT NULL,
    category       VARCHAR(100),
    subcategory    VARCHAR(100),
    brand          VARCHAR(100),
    unit_cost      NUMERIC(10,2),
    unit_price     NUMERIC(10,2),
    effective_date DATE NOT NULL,
    expiry_date    DATE NOT NULL DEFAULT '9999-12-31',
    is_current     BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_dim_product_natural ON dim_product(product_id, is_current);

-- =====
-- FACT: Orders (grain = one row per order line item)
-- =====
CREATE TABLE fact_orders (
    order_key      SERIAL PRIMARY KEY,
    order_id       INT NOT NULL,           -- degenerate dimension
    customer_key   INT NOT NULL REFERENCES dim_customer(customer_key),
    product_key    INT NOT NULL REFERENCES dim_product(product_key),
    date_key       INT NOT NULL REFERENCES dim_date(date_key),
    quantity       INT NOT NULL,
    unit_price     NUMERIC(10,2) NOT NULL,
    discount_amount NUMERIC(10,2) DEFAULT 0,
    revenue        NUMERIC(12,2) NOT NULL, -- pre-calculated
    cost           NUMERIC(12,2),          -- pre-calculated
    profit         NUMERIC(12,2),          -- pre-calculated
    order_status   VARCHAR(20)
);

CREATE INDEX idx_fact_orders_date ON fact_orders(date_key);
CREATE INDEX idx_fact_orders_customer ON fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product ON fact_orders(product_key);

```

Loading the Model from Source Data

The source system has a typical OLTP schema: `src_customers`, `src_products`, `src_orders`, and `src_order_items`. Here is how you load the dimensional model from these sources:

```

-- Load dim_customer (initial load -- all customers are "current")
INSERT INTO dim_customer (customer_id, name, email, city, state, country,
                        segment, created_date, effective_date)
SELECT id, name, email, city, state, country, segment,
       created_at::date, created_at::date
FROM src_customers;

-- Load dim_product (initial load)
INSERT INTO dim_product (product_id, product_name, category, subcategory,
                        brand, unit_cost, unit_price, effective_date)
SELECT id, product_name, category, subcategory, brand,
       unit_cost, unit_price, created_at::date
FROM src_products;

-- Load fact_orders (join source tables and pre-calculate measures)
INSERT INTO fact_orders (order_id, customer_key, product_key, date_key,
                        quantity, unit_price, discount_amount,
                        revenue, cost, profit, order_status)

SELECT
    o.id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    (oi.quantity * oi.unit_price - oi.discount),           -- revenue
    (oi.quantity * dp.unit_cost),                         -- cost
    (oi.quantity * oi.unit_price - oi.discount) - (oi.quantity * dp.unit_cost), --
profit
    o.status
FROM src_orders o
JOIN src_order_items oi ON o.id = oi.order_id
JOIN dim_customer dc ON o.customer_id = dc.customer_id AND dc.is_current = TRUE
JOIN dim_product dp ON oi.product_id = dp.product_id AND dp.is_current = TRUE;

```

Notice what happens in the fact load:

1. We join `src_orders` to `src_order_items` to get the line-item grain
2. We look up the surrogate keys from `dim_customer` and `dim_product` (joining on the natural key with `is_current = TRUE` for the initial load)
3. We convert the order date to the `YYYYMMDD` integer format to match `dim_date`
4. We pre-calculate `revenue`, `cost`, and `profit` at load time

Querying the Star Schema

With the model in place, the business questions become straightforward SQL:

Revenue by category and month:

```

SELECT
    dp.category,
    dd.year,
    dd.month,
    dd.month_name,
    SUM(fo.revenue) AS total_revenue,
    COUNT(DISTINCT fo.order_id) AS num_orders
FROM fact_orders fo
JOIN dim_product dp ON fo.product_key = dp.product_key
JOIN dim_date dd ON fo.date_key = dd.date_key
GROUP BY dp.category, dd.year, dd.month, dd.month_name
ORDER BY dd.year, dd.month, dp.category;

```

Top customers by lifetime value:

```

SELECT
    dc.customer_id,
    dc.name,
    dc.segment,
    SUM(fo.revenue) AS lifetime_revenue,
    SUM(fo.profit) AS lifetime_profit,
    COUNT(DISTINCT fo.order_id) AS num_orders
FROM fact_orders fo
JOIN dim_customer dc ON fo.customer_key = dc.customer_key
GROUP BY dc.customer_id, dc.name, dc.segment
ORDER BY lifetime_revenue DESC
LIMIT 20;

```

Geographic revenue trends:

```

SELECT
    dc.state,
    dc.city,
    dd.quarter,
    dd.year,
    SUM(fo.revenue) AS total_revenue,
    SUM(fo.profit) AS total_profit
FROM fact_orders fo
JOIN dim_customer dc ON fo.customer_key = dc.customer_key
JOIN dim_date dd ON fo.date_key = dd.date_key
GROUP BY dc.state, dc.city, dd.quarter, dd.year
ORDER BY total_revenue DESC;

```

These queries are clean, readable, and performant. An analyst who has never seen this schema before can understand what each query does. That is the power of dimensional modeling.

Module 1 Project

Build the complete ShopFast dimensional model from scratch.

Starter files: - `modules/module-1/starter/requirements.md` – the full business requirements - `modules/module-1/starter/source_data.sql` – OLTP source tables with seed data (100 customers, 50 products, 200 orders, 500 order line items) - `modules/module-1/starter/template_ddl.sql` – skeleton DDL with hints for each table

Your deliverables: 1. Complete the DDL in `template_ddl.sql` with all column definitions, data types, and constraints 2. Add appropriate indexes for common query patterns 3. Write the INSERT statements to load data from source tables into your dimensional model 4. Write at least two analytical queries that answer the business questions from the requirements

What “good” looks like: - Every dimension has a surrogate key and a natural key - SCD Type 2 metadata (`effective_date`, `expiry_date`, `is_current`) on customer and product dimensions - Fact table has pre-calculated `revenue`, `cost`, and `profit` - Grain is clearly one row per order line item - Indexes on all foreign keys in the fact table - The date dimension is fully populated for the relevant date range

Solution: `modules/module-1/solution/dimensional_model.sql`

What's Next

With a solid understanding of data modeling – facts, dimensions, star schemas, SCDs, and modern alternatives – you have the foundation for everything that follows. In Module 2, we will tackle the tools and infrastructure that move data from source systems into these models: ingestion pipelines, orchestration, and the mechanics of ETL and ELT.

Module 2: SQL for Data Engineers

SQL is the lingua franca of data engineering. Not Python, not Spark, not Terraform – SQL. Every data warehouse, every transformation layer, every quality check runs on SQL at the core. This module takes you from “I can write a SELECT” to “I can build a production incremental ETL pipeline in pure SQL.” These are the exact patterns you will write every single day on the job.

Why this module matters: In data engineering interviews, SQL is the great equalizer. You can talk about Spark architecture and Kubernetes orchestration all day long, but if you cannot write a window function on a whiteboard, you will not get the offer. Window functions and CTEs are the number one SQL skill gap interviewers test for. Master this module and you close that gap permanently.

2.1 CTEs and Recursive CTEs

Common Table Expressions: Named Steps for Readable SQL

A CTE (Common Table Expression) is a named temporary result set that exists only for the duration of a single query. It replaces nested subqueries with readable, named steps. You will write CTEs like this every day – every dbt model is basically a chain of CTEs.

Here is a simple CTE that replaces a nested subquery:

```

-- WITHOUT CTEs: nested subqueries become unreadable fast
SELECT *
FROM (
    SELECT customer_id, SUM(amount) AS total
    FROM (
        SELECT * FROM orders WHERE status = 'completed'
    ) completed_orders
    GROUP BY customer_id
) customer_totals
WHERE total > 1000;

-- WITH CTEs: each step has a name and a purpose
WITH completed_orders AS (
    SELECT *
    FROM orders
    WHERE status = 'completed'
),
customer_totals AS (
    SELECT customer_id, SUM(amount) AS total
    FROM completed_orders
    GROUP BY customer_id
)
SELECT *
FROM customer_totals
WHERE total > 1000;

```

The second version does the exact same thing, but six months from now when you or a teammate revisits this code, every step is self-documenting.

The Pipeline Pattern: staging -> dedup -> enrich -> validate -> output

In production data engineering, CTEs follow a standard pipeline pattern. This is the skeleton of every serious transformation you will write:

```

WITH staging AS (
  -- Raw data from source, minimal transformation
  SELECT
    id,
    name,
    email,
    city,
    state,
    updated_at,
    ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) AS rn
  FROM raw_customers
),
deduped AS (
  -- Remove duplicates: keep only the most recent row per customer
  SELECT * FROM staging WHERE rn = 1
),
enriched AS (
  -- Add business logic: segment classification, derived fields
  SELECT
    d.*,
    CASE
      WHEN state IN ('CA', 'NY', 'TX') THEN 'Tier 1'
      WHEN state IN ('FL', 'IL', 'PA') THEN 'Tier 2'
      ELSE 'Tier 3'
    END AS market_tier,
    CURRENT_DATE AS load_date
  FROM deduped d
),
validated AS (
  -- Filter out bad data before loading
  SELECT *
  FROM enriched
  WHERE email IS NOT NULL
    AND email LIKE '%%.%%'
    AND name IS NOT NULL
)
SELECT * FROM validated;

```

When you will use this at work: Every dbt model you write. Every staging-to-warehouse transformation. Every data pipeline that reads from a source, cleans it, enriches it, and loads it. The staging-dedup-enrich-validate-output pattern is the backbone of data transformation. When you open a dbt project at a new company on your first day, you will see exactly this structure in every model.

Recursive CTEs: Hierarchies in SQL

Recursive CTEs solve the class of problems where data references itself – org charts (who reports to whom), category trees (Electronics > Phones > Smartphones), bill-of-materials (a car contains an engine, which contains pistons...).

```

-- Org chart: find all reports under a given manager
WITH RECURSIVE org_tree AS (
  -- Anchor: start with the top-level manager
  SELECT
    employee_id,
    name,
    manager_id,
    1 AS depth
  FROM employees
  WHERE manager_id IS NULL -- CEO / top of tree

  UNION ALL

  -- Recursive step: find all direct reports of people already in our result
  SELECT
    e.employee_id,
    e.name,
    e.manager_id,
    ot.depth + 1
  FROM employees e
  JOIN org_tree ot ON e.manager_id = ot.employee_id
  WHERE ot.depth < 10 -- ALWAYS add a depth limit
)
SELECT * FROM org_tree ORDER BY depth, name;

```

Always add a depth limit. Without one, a single circular reference in your data (employee A reports to B, B reports to A) will cause an infinite loop that consumes memory until Postgres kills the query – or your DBA kills you. In production, set the depth limit to the maximum reasonable level for your hierarchy plus a safety margin. If your org chart is never more than 8 levels deep, set the limit to 15.

Another common use – exploding category trees for a product catalog:

```

-- Category breadcrumb: Electronics > Audio > Headphones
WITH RECURSIVE category_path AS (
  SELECT
    category_id,
    name,
    parent_id,
    name AS full_path,
    1 AS depth
  FROM categories
  WHERE parent_id IS NULL -- root categories

  UNION ALL

  SELECT
    c.category_id,
    c.name,
    c.parent_id,
    cp.full_path || ' > ' || c.name,
    cp.depth + 1
  FROM categories c
  JOIN category_path cp ON c.parent_id = cp.category_id
  WHERE cp.depth < 10
)
SELECT * FROM category_path ORDER BY full_path;

```

When you will use this at work: E-commerce category trees, organizational reporting structures, permission hierarchies (role A inherits from role B), network topology (which switches connect to which routers). Any time your data has a parent-child relationship, recursive CTEs are the answer.

Key Takeaway: CTEs turn unreadable nested SQL into a pipeline of named steps. The staging-dedup-enrich-validate-output pattern is the universal skeleton for data transformations. Recursive CTEs handle hierarchies, but always add a depth limit to prevent infinite loops.

2.2 Window Functions Deep Dive

Window functions are the most powerful feature in SQL and the single biggest skill gap in data engineering interviews. If you can write window functions fluently, you are ahead of 80% of candidates.

A window function performs a calculation across a set of rows that are *related to the current row* – without collapsing those rows into a single output like GROUP BY does. Think of it as: GROUP BY gives you one row per group, but a window function gives you one result per row with awareness of the group.

Anatomy of a Window Function

```
SELECT
  employee_id,
  department,
  salary,
  -- This is a window function
  AVG(salary) OVER (
    PARTITION BY department -- which rows form the "window"
    ORDER BY hire_date      -- how they are ordered within the window
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- the "frame"
  ) AS running_avg_salary
FROM employees;
```

The three parts: 1. **PARTITION BY** – Divides rows into groups (like GROUP BY, but without collapsing). Each partition is processed independently. 2. **ORDER BY** – Defines the order of rows within each partition. Required for ranking functions and running aggregations. 3. **Frame clause** (ROWS BETWEEN...) – Defines exactly which rows relative to the current row are included in the calculation. This is the part most people get wrong.

ROW_NUMBER for Dedup: THE Dedup Technique

This is the single most important SQL pattern in data engineering. When source systems send you duplicate records, CDC streams replay events, or batch loads overlap, you need to deduplicate. ROW_NUMBER is how you do it:

```
-- THE dedup pattern: keep only the most recent record per customer
WITH ranked AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY customer_id
      ORDER BY updated_at DESC
    ) AS rn
  FROM raw_customers
)
SELECT * FROM ranked WHERE rn = 1;
```

How it works: PARTITION BY customer_id groups all rows for the same customer together. ORDER BY updated_at DESC sorts them newest-first within each group. ROW_NUMBER() assigns 1 to the newest, 2 to the second-newest, and so on. Filtering to rn = 1 keeps only the latest version of each customer.

Why ROW_NUMBER and not DISTINCT or GROUP BY? DISTINCT removes exact duplicates but cannot pick “the most recent” among near-duplicates. GROUP BY collapses rows so you lose the non-aggregated columns. ROW_NUMBER gives you full control: keep the latest by timestamp, the

highest-priority by status, or the first-arrived by ingestion time – whatever your business logic requires.

When you will use this at work: Every. Single. Day. CDC streams often replay events, leaving you with multiple versions of the same record. Batch loads can overlap, giving you two copies of yesterday's data. Source systems have bugs that create true duplicates. The ROW_NUMBER dedup pattern is the universal fix.

RANK vs DENSE_RANK vs ROW_NUMBER

These three ranking functions differ only in how they handle ties:

```
SELECT
  student_name,
  score,
  ROW_NUMBER() OVER (ORDER BY score DESC) AS row_num,    -- No ties: 1,2,3,4,5
  RANK()      OVER (ORDER BY score DESC) AS rank_val,    -- Ties skip: 1,2,2,4,5
  DENSE_RANK() OVER (ORDER BY score DESC) AS dense_val   -- Ties don't skip:
1,2,2,3,4
FROM exam_scores;
```

STUDENT	SCORE	ROW_NUMBER	RANK	DENSE_RANK
Alice	95	1	1	1
Bob	90	2	2	2
Carol	90	3	2	2
David	85	4	4	3
Eve	80	5	5	4

- **ROW_NUMBER:** Always unique. Use for dedup (you want exactly one row per group, no ties).
- **RANK:** Gaps after ties. Use when gaps matter (“3rd place, no 2nd place” like Olympic medals).
- **DENSE_RANK:** No gaps. Use for “top N categories” where you want exactly N distinct values.

LAG/LEAD: Time-Series Analysis

LAG looks at the previous row, LEAD looks at the next row. These are indispensable for time-series analysis – month-over-month growth, session boundary detection, and change detection.

```

-- Month-over-month revenue growth
WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', order_date) AS month,
        SUM(revenue) AS revenue
    FROM fact_orders f
    JOIN dim_date d ON f.date_key = d.date_key
    GROUP BY DATE_TRUNC('month', order_date)
)
SELECT
    month,
    revenue,
    LAG(revenue) OVER (ORDER BY month) AS prev_month_revenue,
    ROUND(
        (revenue - LAG(revenue) OVER (ORDER BY month))
        / LAG(revenue) OVER (ORDER BY month) * 100, 1
    ) AS growth_pct
FROM monthly_revenue
ORDER BY month;

```

When you will use this at work: Every executive dashboard has a “vs. last month” or “vs. last year” metric. LAG computes it in one line. You will also use it for detecting anomalies – if today’s row count is 50% lower than yesterday’s, something is broken.

Running Aggregations: Cumulative Sum and Moving Average

```

-- Cumulative revenue: "how much total revenue have we earned through this date"
SELECT
    order_date,
    daily_revenue,
    SUM(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS cumulative_revenue
FROM daily_revenue_summary;

-- 7-day moving average: smooths out daily noise
SELECT
    order_date,
    daily_revenue,
    AVG(daily_revenue) OVER (
        ORDER BY order_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) AS moving_avg_7d
FROM daily_revenue_summary;

```

Understanding window frames – the part most people get wrong:

- **ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW** – All rows from the start of the partition up to the current row. This is for cumulative/running totals.

- **ROWS BETWEEN 6 PRECEDING AND CURRENT ROW** – The current row plus the 6 before it (7 total). This is for moving averages.
- **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING** – The entire partition. Same as no frame at all.

The critical gotcha: if you use ORDER BY in a window without specifying a frame, Postgres defaults to **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**. RANGE and ROWS behave differently with ties. ROWS is almost always what you want – it is deterministic and row-based. Use ROWS explicitly to avoid surprises.

FIRST_VALUE / LAST_VALUE

```
-- For each order, show the first and most recent order date per customer
SELECT
  customer_id,
  order_date,
  order_id,
  FIRST_VALUE(order_date) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS first_order_date,
  LAST_VALUE(order_date) OVER (
    PARTITION BY customer_id ORDER BY order_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS last_order_date
FROM orders;
```

Critical: LAST_VALUE requires **ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING**. Without it, the default frame ends at the current row, so LAST_VALUE just returns the current row's value – completely useless. This is one of the most common window function bugs.

The WINDOW Clause: DRY Principle for Windows

When you reuse the same window specification, define it once:

```
SELECT
  customer_id,
  order_date,
  revenue,
  SUM(revenue) OVER w AS cumulative_revenue,
  AVG(revenue) OVER w AS running_avg,
  COUNT(*) OVER w AS order_number
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date
              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
ORDER BY customer_id, order_date;
```

The Sessionization Pattern

Sessionization groups a stream of events (clickstream, app events, IoT readings) into logical sessions. This is a classic interview question and a real production need.

The logic: if more than 30 minutes pass between two consecutive events from the same user, start a new session.

```
WITH events_with_gap AS (
  SELECT
    user_id,
    event_time,
    event_type,
    -- Time since the previous event for this user
    EXTRACT(EPOCH FROM (
      event_time - LAG(event_time) OVER (PARTITION BY user_id ORDER BY
event_time)
    )) / 60.0 AS minutes_since_last
  FROM clickstream_events
),
session_boundaries AS (
  SELECT
    *,
    -- Flag: is this the start of a new session?
    CASE
      WHEN minutes_since_last IS NULL THEN 1 -- first event ever
      WHEN minutes_since_last > 30 THEN 1    -- gap > 30 min
      ELSE 0
    END AS is_new_session
  FROM events_with_gap
),
sessionized AS (
  SELECT
    *,
    -- Running sum of session boundaries = session ID
    SUM(is_new_session) OVER (
      PARTITION BY user_id ORDER BY event_time
      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS session_id
  FROM session_boundaries
)
SELECT
  user_id,
  session_id,
  MIN(event_time) AS session_start,
  MAX(event_time) AS session_end,
  COUNT(*) AS events_in_session,
  EXTRACT(EPOCH FROM (MAX(event_time) - MIN(event_time))) / 60.0 AS session_duration_
min
FROM sessionized
GROUP BY user_id, session_id;
```

How it works, step by step: 1. LAG computes the gap between consecutive events per user. 2. Any gap over 30 minutes (or the first event) is flagged as a session boundary. 3. A running SUM of boundary flags creates a monotonically increasing session ID. 4. GROUP BY session_id gives you session-level metrics.

When you will use this at work: Product analytics teams need session data for every analysis – average session duration, sessions per user, conversion funnels within sessions. Google Analytics, Amplitude, and Mixpanel all do exactly this internally. If your company builds its own product analytics on a warehouse, you will write this pattern.

Key Takeaway: Window functions are the #1 SQL skill gap in interviews. ROW_NUMBER is THE dedup technique. LAG/LEAD handle time-series comparison. Running SUM handles cumulative metrics. Always specify your window frame explicitly. The sessionization pattern is a classic interview question that combines LAG, CASE, and running SUM.

2.3 MERGE/UPSERT Patterns

In real data pipelines, you rarely load data into empty tables. Data arrives continuously – new records appear, existing records change, sometimes records need to be deleted. You need SQL patterns that handle “insert if new, update if changed” – an upsert.

INSERT ON CONFLICT: Postgres UPSERT (SCD Type 1)

Postgres provides `INSERT ... ON CONFLICT ... DO UPDATE` as its upsert mechanism. This is the most common pattern for SCD Type 1 (overwrite) dimensions:

```
-- SCD Type 1: if the customer exists, overwrite with new values
-- If they don't exist, insert a new row
INSERT INTO dim_customer (customer_id, name, email, city, state, segment)
VALUES (1, 'Alice Johnson', 'alice@newjob.com', 'Boston', 'MA', 'Enterprise')
ON CONFLICT (customer_id) DO UPDATE SET
    name      = EXCLUDED.name,
    email     = EXCLUDED.email,
    city      = EXCLUDED.city,
    state     = EXCLUDED.state,
    segment   = EXCLUDED.segment;
```

`EXCLUDED` is a special reference to the row that was proposed for insertion but conflicted. It is the “incoming” row. This pattern is atomic – no race condition between checking existence and inserting/updating.

Bulk upsert from a staging table:

```
-- Upsert all customers from staging into dimension
INSERT INTO dim_customer (customer_id, name, email, city, state, segment)
SELECT id, name, email, city, state, segment
FROM stg_customers
ON CONFLICT (customer_id) DO UPDATE SET
    name      = EXCLUDED.name,
    email     = EXCLUDED.email,
    city      = EXCLUDED.city,
    state     = EXCLUDED.state,
    segment   = EXCLUDED.segment;
```

When you will use this at work: Any dimension where you do not need to track history. Lookup tables, reference data, configuration tables. “The customer changed their email – just overwrite the old one.” This is the default for Type 1 slowly changing dimensions.

SCD Type 2 Incremental Load: The Full Pattern

SCD Type 2 is harder. When a tracked attribute changes, you do not overwrite – you expire the old row and insert a new one, preserving full history. This is a multi-step process:

Step 1: Find what changed and expire the old rows

```
-- Expire current rows where tracked attributes changed
UPDATE dim_customer dc
SET expiry_date = src.updated_at::date - 1,
    is_current = FALSE
FROM src_customers src
WHERE dc.customer_id = src.id
    AND dc.is_current = TRUE
    AND src.updated_at > '2025-01-10' -- since last load
    AND (dc.city != src.city
        OR dc.state != src.state
        OR dc.segment != src.segment);
```

Step 2: Insert new current rows for changed records AND brand new records

```

INSERT INTO dim_customer (customer_id, name, email, city, state, country,
                          segment, created_date, effective_date)
SELECT src.id, src.name, src.email, src.city, src.state, 'US',
       src.segment, src.created_at::date, src.updated_at::date
FROM src_customers src
WHERE src.updated_at > '2025-01-10'
AND (
  -- Changed tracked attributes (we just expired the old row)
  EXISTS (
    SELECT 1 FROM dim_customer dc
    WHERE dc.customer_id = src.id AND dc.is_current = FALSE
          AND dc.expiry_date = src.updated_at::date - 1
  )
  -- Or brand new customer (no row exists at all)
  OR NOT EXISTS (
    SELECT 1 FROM dim_customer dc WHERE dc.customer_id = src.id
  )
);

```

Step 3: Type 1 update for non-tracked attributes

Some attributes do not warrant history tracking – email addresses, phone numbers. For these, just overwrite (Type 1) even within an SCD Type 2 dimension:

```

-- Type 1 update for email (no need to track email history)
UPDATE dim_customer dc
SET email = src.email
FROM src_customers src
WHERE dc.customer_id = src.id
      AND dc.is_current = TRUE
      AND dc.email != src.email;

```

Production scenario: Your Airflow DAG runs this incremental load nightly. When customer Alice Johnson moves from New York to Boston, here is exactly what happens in the SCD Type 2 process:

1. The source system updates `src_customers` row for Alice: city changes from 'New York' to 'Boston', `updated_at` becomes '2025-01-11'.
2. Step 1 finds Alice's current row in `dim_customer` where city='New York'. It sets `expiry_date = '2025-01-10'` and `is_current = FALSE`.
3. Step 2 inserts a brand new row for Alice with city='Boston', `effective_date = '2025-01-11'`, `expiry_date = '9999-12-31'`, `is_current = TRUE`.
4. Now Alice has TWO rows in `dim_customer` – the historical row (New York, effective 2024-06-15 to 2025-01-10) and the current row (Boston, effective 2025-01-11 to 9999-12-31).
5. When you join `fact_orders` to `dim_customer` for Alice's old orders, the join on `is_current = TRUE` gives you Boston. But if you need to know where she lived when she placed that January 5th order, you join on `order_date BETWEEN effective_date AND expiry_date` and you get New York. That is the power of SCD Type 2.

MERGE: The SQL Standard (Postgres 15+)

MERGE arrived in Postgres 15 and is the SQL standard way to do conditional insert/update/delete in a single statement:

```
-- MERGE: one statement handles insert, update, and delete
MERGE INTO dim_product AS target
USING stg_products AS source
ON target.product_id = source.product_id AND target.is_current = TRUE
WHEN MATCHED AND (target.category != source.category
                  OR target.unit_price != source.unit_price) THEN
    UPDATE SET
        expiry_date = CURRENT_DATE - 1,
        is_current = FALSE
WHEN NOT MATCHED THEN
    INSERT (product_id, product_name, category, subcategory,
           brand, unit_cost, unit_price, effective_date)
    VALUES (source.product_id, source.product_name, source.category,
            source.subcategory, source.brand, source.unit_cost,
            source.unit_price, CURRENT_DATE);
```

MERGE is elegant but has limitations – the SCD Type 2 pattern still requires multiple statements because you need to both expire the old row and insert a new one, and MERGE cannot insert *and* update for the same matched row. In practice, the UPDATE/INSERT two-step pattern from above remains the standard approach for SCD Type 2.

When you will use this at work: MERGE is great for simple upserts (insert-or-update). For SCD Type 2, you will use the multi-step UPDATE + INSERT pattern. Both are bread and butter in any ETL pipeline that loads dimensional models.

Key Takeaway: INSERT ON CONFLICT handles Type 1 (overwrite) upserts atomically. SCD Type 2 requires a multi-step process: find changes, expire old rows, insert new rows. MERGE is the SQL standard but cannot fully handle SCD Type 2 in a single statement.

2.4 Surrogate Keys and SCD SQL Mechanics

Why Surrogate Keys?

A surrogate key is a meaningless auto-generated identifier assigned by the warehouse, as opposed to the natural/business key from the source system. In our schema, `customer_key` (SERIAL) is the surrogate key, while `customer_id` is the natural key from the source.

Why not just use the natural key? Because SCD Type 2 creates multiple rows for the same customer. Customer Alice (`customer_id` = 1) has two rows – one for her New York era and one for her

Boston era. If the fact table joined on `customer_id`, it could not distinguish which Alice row to use. The surrogate `customer_key` is unique per row, so the fact table can point to exactly the right version.

Surrogate Key Generation Strategies

SERIAL / IDENTITY (auto-increment):

```
CREATE TABLE dim_customer (  
    customer_key SERIAL PRIMARY KEY, -- auto-increment: 1, 2, 3, ...  
    customer_id INT NOT NULL,        -- natural key from source  
    ...  
);  
  
-- Or the SQL-standard equivalent:  
CREATE TABLE dim_customer (  
    customer_key INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id INT NOT NULL,  
    ...  
);
```

Simple, fast, database-managed. The downside: not portable across databases and not deterministic (reloading data produces different keys).

Hash-based keys:

```
-- Deterministic: same input always produces the same key  
SELECT MD5(customer_id::text || '|' || effective_date::text) AS customer_key;
```

Deterministic and portable. The same source record always gets the same surrogate key regardless of load order. dbt uses this approach (via `dbt_utils.generate_surrogate_key`). The downside: hash collisions (astronomically unlikely but theoretically possible with MD5), and less human-readable.

UUID:

```
SELECT gen_random_uuid() AS customer_key;  
-- Output: 'a81bc81b-dead-4e5d-abff-90865d1e13b1'
```

Globally unique and no collisions. But UUIDs are large (16 bytes vs 4 for INT), bad for index performance, and completely unreadable when debugging. Use them when you need globally unique identifiers across distributed systems, not for warehouse surrogate keys.

When you will use this at work: SERIAL for simple warehouses. Hash-based keys for dbt projects (idempotent loads, deterministic keys). UUID when you absolutely need global uniqueness. For the module project, we use SERIAL because it is simple and fits the single-database model.

IS DISTINCT FROM: NULL-Safe Comparison

Standard SQL comparison with `!=` returns NULL when either side is NULL. That means `NULL != 'Boston'` is NULL (not TRUE), and your change detection silently misses records where old or new values are NULL.

```
-- BROKEN: misses changes involving NULLs
WHERE dc.city != src.city -- NULL != 'Boston' is NULL, not TRUE

-- FIXED: IS DISTINCT FROM treats NULL like a value
WHERE dc.city IS DISTINCT FROM src.city -- NULL IS DISTINCT FROM 'Boston' is TRUE
```

`IS DISTINCT FROM` treats NULL as a regular value: NULL IS DISTINCT FROM NULL is FALSE, NULL IS DISTINCT FROM 'anything' is TRUE. Use it everywhere in change detection logic. Missing this is a silent data quality bug that can go undetected for months.

Complete SCD Type 2 Stored Procedure

In production, the SCD Type 2 logic lives in a stored procedure that your scheduler (Airflow, dbt, cron) calls:


```

CREATE OR REPLACE PROCEDURE load_dim_customer(p_last_load_ts TIMESTAMP)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Step 1: Expire changed rows
    UPDATE dim_customer dc
    SET expiry_date = src.updated_at::date - 1,
        is_current = FALSE
    FROM src_customers src
    WHERE dc.customer_id = src.id
        AND dc.is_current = TRUE
        AND src.updated_at > p_last_load_ts
        AND (dc.city IS DISTINCT FROM src.city
            OR dc.state IS DISTINCT FROM src.state
            OR dc.segment IS DISTINCT FROM src.segment);

    -- Step 2: Insert new current rows for changed + new
    INSERT INTO dim_customer (customer_id, name, email, city, state, country,
                             segment, created_date, effective_date)
    SELECT src.id, src.name, src.email, src.city, src.state, 'US',
           src.segment, src.created_at::date, src.updated_at::date
    FROM src_customers src
    WHERE src.updated_at > p_last_load_ts
        AND (
            EXISTS (
                SELECT 1 FROM dim_customer dc
                WHERE dc.customer_id = src.id AND dc.is_current = FALSE
                    AND dc.expiry_date = src.updated_at::date - 1
            )
            OR NOT EXISTS (
                SELECT 1 FROM dim_customer dc WHERE dc.customer_id = src.id
            )
        );

    -- Step 3: Type 1 update for non-tracked attributes
    UPDATE dim_customer dc
    SET email = src.email
    FROM src_customers src
    WHERE dc.customer_id = src.id
        AND dc.is_current = TRUE
        AND dc.email IS DISTINCT FROM src.email;
END;
$$;

-- Called by Airflow:
CALL load_dim_customer('2025-01-10 00:00:00');

```

Key Takeaway: Surrogate keys enable SCD Type 2 by giving each version of a dimension record a unique identifier. IS DISTINCT FROM is essential for NULL-safe change detection. Wrap the SCD logic in a stored procedure so your scheduler can call it with the last load timestamp.

2.5 Performance

You can write the most elegant SQL in the world, but if it takes 45 minutes to run on a nightly schedule with a 30-minute window, it is useless. Performance tuning is not optional – it is a core data engineering skill.

EXPLAIN ANALYZE: See What the Database Actually Does

EXPLAIN ANALYZE runs the query and shows the actual execution plan with real timings:

```
EXPLAIN ANALYZE
SELECT dc.name, SUM(f.revenue)
FROM fact_orders f
JOIN dim_customer dc ON f.customer_key = dc.customer_key
WHERE dc.is_current = TRUE
GROUP BY dc.name;
```

Output (simplified):

```
HashAggregate (actual time=12.3..12.5 rows=10 loops=1)
  -> Hash Join (actual time=0.08..11.2 rows=15 loops=1)
    Hash Cond: (f.customer_key = dc.customer_key)
    -> Seq Scan on fact_orders f (actual time=0.01..5.3 rows=20 loops=1)
    -> Hash (actual time=0.05..0.05 rows=10 loops=1)
      -> Seq Scan on dim_customer dc (actual time=0.01..0.03 rows=10 loops=1)
        Filter: is_current
    Planning Time: 0.3 ms
    Execution Time: 12.8 ms
```

What to look for: - **Seq Scan** on large tables: a full table scan. Fine for small tables, deadly for millions of rows. You need an index. - **actual time vs estimated rows**: if the planner thinks there are 100 rows but there are 10 million, statistics are stale. Run `ANALYZE table_name;` to refresh. - **Nested Loop with high loops count**: If the inner side runs millions of times, the join strategy is wrong. Usually means a missing index.

Indexing: The Biggest Performance Lever

Indexes are the difference between a query scanning 50 million rows and one that reads exactly the 200 rows it needs.

Composite index (multi-column):

```
-- This index covers the most common fact table join pattern
CREATE INDEX idx_fact_orders_date ON fact_orders(date_key);
CREATE INDEX idx_fact_orders_customer ON fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product ON fact_orders(product_key);
```

Partial index – the secret weapon:

```
-- Only index current dimension rows (is_current = TRUE)
-- This index is tiny compared to indexing the whole table
CREATE INDEX idx_dim_customer_current
ON dim_customer(customer_id)
WHERE is_current = TRUE;

CREATE INDEX idx_dim_product_current
ON dim_product(product_id)
WHERE is_current = TRUE;
```

Production story: I once saw a query go from 45 minutes to 2 seconds just by adding a partial index. The dimension table had 50 million historical rows but only 2 million current rows. Every SCD Type 2 join was filtering on `is_current = TRUE`, but without the partial index, Postgres was scanning all 50 million rows to find the 2 million current ones. A partial index `WHERE is_current = TRUE` reduced the index size by 96% and turned sequential scans into instant index lookups.

Composite index for common filter patterns:

```
-- If you frequently query by customer_id and is_current together
CREATE INDEX idx_dim_customer_lookup
ON dim_customer(customer_id, is_current)
WHERE is_current = TRUE;
```

Column order in composite indexes matters. The left-most column is the primary filter. Put the most selective column first.

Table Partitioning by Date

For fact tables that grow continuously, partitioning by date is essential. Instead of one massive table, you split it into smaller pieces by time period:

```

-- Create a partitioned fact table
CREATE TABLE fact_orders (
  order_key      SERIAL,
  order_id       INT NOT NULL,
  customer_key   INT NOT NULL,
  product_key    INT NOT NULL,
  date_key       INT NOT NULL,
  quantity       INT NOT NULL,
  unit_price     NUMERIC(10,2) NOT NULL,
  revenue        NUMERIC(12,2) NOT NULL,
  order_status   VARCHAR(20)
) PARTITION BY RANGE (date_key);

-- Create partitions for each month
CREATE TABLE fact_orders_2025_01 PARTITION OF fact_orders
  FOR VALUES FROM (20250101) TO (20250201);
CREATE TABLE fact_orders_2025_02 PARTITION OF fact_orders
  FOR VALUES FROM (20250201) TO (20250301);
-- ... and so on

```

Why partition? When a query filters on `WHERE date_key BETWEEN 20250101 AND 20250131`, Postgres only scans the January partition and completely ignores the other 11 months. This is called partition pruning and it can reduce I/O by 90%+.

Partitioning also makes maintenance easier: - Drop an old partition instantly instead of DELETE-ing millions of rows. - VACUUM and ANALYZE run faster on smaller partitions. - Backfilling a specific month means replacing one partition, not scanning the entire table.

When you will use this at work: Every fact table over ~100 million rows should be partitioned. Most data warehouses partition by date because nearly every query has a date filter. Some also partition by a high-cardinality dimension (customer region, product category) if queries always filter on it.

Key Takeaway: EXPLAIN ANALYZE shows you what is actually slow. Indexes – especially partial indexes on `is_current = TRUE` – are the biggest performance lever. Partition large fact tables by date so queries only scan the time range they need.

2.6 SQL for Data Quality

Data quality checks are not optional nice-to-haves. They are production infrastructure. Every pipeline should include automated quality checks that run after every load. When they fail, you catch the problem before your CFO sees wrong revenue numbers on a dashboard.

The Six Essential Checks

1. Completeness – Are required fields populated?

```
-- Check for NULL values in columns that should never be NULL
SELECT
    'NULL customer names' AS check_name,
    COUNT(*) AS violations
FROM dim_customer
WHERE name IS NULL AND is_current = TRUE;
```

When you will use this at work: Source systems change. A developer removes a NOT NULL constraint, an API starts sending empty strings, a CSV import maps columns wrong. Completeness checks catch it on the first load.

2. Freshness – Is the data up to date?

```
-- Check that the most recent data is from today (or yesterday for nightly loads)
SELECT
    'Stale data' AS check_name,
    MAX(calendar_date) AS latest_date,
    CURRENT_DATE - MAX(calendar_date) AS days_behind,
    CASE
        WHEN CURRENT_DATE - MAX(calendar_date) > 1 THEN 'FAIL'
        ELSE 'PASS'
    END AS status
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key;
```

When you will use this at work: Your Airflow DAG ran, completed “successfully,” but actually loaded zero rows because the source table was empty. Without a freshness check, nobody notices for days.

3. Uniqueness – Are there duplicate dimension records?

```
-- No customer_id should have more than one current row
SELECT 'Duplicate current customers' AS check_name,
    COUNT(*) AS violations
FROM (
    SELECT customer_id, COUNT(*) AS cnt
    FROM dim_customer WHERE is_current = TRUE
    GROUP BY customer_id HAVING COUNT(*) > 1
) dupes
UNION ALL
SELECT 'Duplicate current products',
    COUNT(*)
FROM (
    SELECT product_id, COUNT(*) AS cnt
    FROM dim_product WHERE is_current = TRUE
    GROUP BY product_id HAVING COUNT(*) > 1
) dupes;
```

When you will use this at work: The SCD Type 2 logic has a bug, or two pipeline runs overlap, and now customer Alice has two current rows. Every downstream join doubles her revenue. This check catches it immediately.

4. Volume Anomaly Detection (Z-Score)

```
-- Alert if today's load volume is statistically unusual
WITH daily_counts AS (
  SELECT
    d.calendar_date,
    COUNT(*) AS row_count
  FROM fact_orders f
  JOIN dim_date d ON f.date_key = d.date_key
  WHERE d.calendar_date > CURRENT_DATE - INTERVAL '30 days'
  GROUP BY d.calendar_date
),
stats AS (
  SELECT
    AVG(row_count) AS avg_count,
    STDDEV(row_count) AS stddev_count
  FROM daily_counts
)
SELECT
  dc.calendar_date,
  dc.row_count,
  ROUND((dc.row_count - s.avg_count) / NULLIF(s.stddev_count, 0), 2) AS z_score,
  CASE
    WHEN ABS((dc.row_count - s.avg_count) / NULLIF(s.stddev_count, 0)) > 2 THEN 'ANOMALY'
    ELSE 'NORMAL'
  END AS status
FROM daily_counts dc
CROSS JOIN stats s
ORDER BY dc.calendar_date DESC
LIMIT 1;
```

A z-score above 2 or below -2 means today's volume is more than 2 standard deviations from the 30-day average. That is unusual enough to warrant investigation. Maybe a source system went down (too few rows) or replayed data (too many rows).

When you will use this at work: This catches the failures that do not produce errors. The pipeline completes successfully, but loads 10 rows instead of the usual 10,000. Without a volume check, you do not notice until someone complains about a blank dashboard.

5. Referential Integrity – Do all foreign keys resolve?

```

-- Orphaned fact records: fact rows pointing to non-existent dimension keys
SELECT 'Orphaned customer keys' AS check_name,
       COUNT(*) AS violations
FROM fact_orders f
LEFT JOIN dim_customer dc ON f.customer_key = dc.customer_key
WHERE dc.customer_key IS NULL
UNION ALL
SELECT 'Orphaned product keys',
       COUNT(*)
FROM fact_orders f
LEFT JOIN dim_product dp ON f.product_key = dp.product_key
WHERE dp.product_key IS NULL
UNION ALL
SELECT 'Orphaned date keys',
       COUNT(*)
FROM fact_orders f
LEFT JOIN dim_date dd ON f.date_key = dd.date_key
WHERE dd.date_key IS NULL;

```

When you will use this at work: A late-arriving fact references a customer who has not been loaded into the dimension yet. Or a dimension cleanup accidentally deletes rows that the fact table still points to. Referential integrity checks catch broken joins before they turn into NULL values in dashboards.

6. Business Rule Checks – Do the numbers make sense?

```

-- Revenue reconciliation: source vs warehouse
SELECT
    'Revenue reconciliation' AS check_name,
    ABS(source_total - warehouse_total) AS difference,
    CASE WHEN ABS(source_total - warehouse_total) < 0.01 THEN 'PASS' ELSE 'FAIL' END
AS status
FROM (
    SELECT SUM(quantity * unit_price - discount) AS source_total FROM src_order_items
) src,
(
    SELECT SUM(revenue) AS warehouse_total FROM fact_orders
) wh;

-- Negative revenue check
SELECT 'Negative revenue' AS check_name,
    COUNT(*) AS violations
FROM fact_orders WHERE revenue < 0;

-- Future date check
SELECT 'Future order dates' AS check_name,
    COUNT(*) AS violations
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.calendar_date > CURRENT_DATE;

-- Null measure check
SELECT 'Null revenues' AS check_name, COUNT(*) AS violations
FROM fact_orders WHERE revenue IS NULL
UNION ALL
SELECT 'Null quantities', COUNT(*)
FROM fact_orders WHERE quantity IS NULL;

```

When you will use this at work: Your pipeline computes revenue as `quantity * unit_price - discount`. A bug in the discount logic produces negative revenue for some rows. The revenue reconciliation catches it when the warehouse total does not match the source. Without this check, the finance team gets a report showing the company lost money last Tuesday.

Reusable Quality Check Template

In production, combine all checks into a single quality report that runs after every load:


```

-- Master quality check: one query, all checks, clear pass/fail
WITH checks AS (
  -- Completeness
  SELECT 'NULL customer names' AS check_name, COUNT(*) AS violations
  FROM dim_customer WHERE name IS NULL AND is_current = TRUE
  UNION ALL
  -- Uniqueness
  SELECT 'Duplicate current customers', COUNT(*)
  FROM (
    SELECT customer_id FROM dim_customer
    WHERE is_current = TRUE
    GROUP BY customer_id HAVING COUNT(*) > 1
  ) d
  UNION ALL
  -- Referential integrity
  SELECT 'Orphaned customer keys', COUNT(*)
  FROM fact_orders f
  LEFT JOIN dim_customer dc ON f.customer_key = dc.customer_key
  WHERE dc.customer_key IS NULL
  UNION ALL
  -- Business rules
  SELECT 'Negative revenue', COUNT(*)
  FROM fact_orders WHERE revenue < 0
  UNION ALL
  SELECT 'Future dates', COUNT(*)
  FROM fact_orders f
  JOIN dim_date d ON f.date_key = d.date_key
  WHERE d.calendar_date > CURRENT_DATE
)
SELECT
  check_name,
  violations,
  CASE WHEN violations = 0 THEN 'PASS' ELSE 'FAIL' END AS status
FROM checks
ORDER BY violations DESC;

```

Your Airflow DAG runs this after every load. If any check returns FAIL, the DAG sends a Slack alert and optionally marks the load as failed so downstream tasks do not consume bad data.

Key Takeaway: Data quality checks are production infrastructure, not optional polish. Run completeness, freshness, uniqueness, volume anomaly, referential integrity, and business rule checks after every load. A single quality report query with PASS/FAIL status makes monitoring straightforward.

Module 2 Project: Incremental ETL Pipeline in Pure SQL

This project ties together every concept in the module. You will build a complete incremental ETL pipeline that loads a dimensional warehouse from a source system – handling initial loads, SCD Type 2 changes, incremental fact loading, late-arriving data, and data quality checks. All in pure SQL.

Project Setup

The companion code lives in: - **Starter files:** `modules/module-2/starter/` – schema definitions, source data for 3 simulated “days” - **Solution files:** `modules/module-2/solution/` – complete working code

Run the files in this order:

```
# Set up source tables and Day 1 data
psql -f starter/sample_data_day1.sql

# Set up the warehouse schema (dimensional model)
psql -f starter/warehouse_schema.sql
```

The Warehouse Schema

The warehouse uses a classic star schema with SCD Type 2 dimensions:

```

CREATE TABLE dim_date (
    date_key          INT PRIMARY KEY,
    calendar_date     DATE NOT NULL UNIQUE,
    day_of_week       VARCHAR(10) NOT NULL,
    day_of_month      INT NOT NULL,
    month             INT NOT NULL,
    month_name        VARCHAR(10) NOT NULL,
    quarter           INT NOT NULL,
    year              INT NOT NULL,
    is_weekend        BOOLEAN NOT NULL,
    is_holiday        BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE dim_customer (
    customer_key      SERIAL PRIMARY KEY,          -- surrogate key
    customer_id       INT NOT NULL,                -- natural key from source
    name              VARCHAR(200) NOT NULL,
    email             VARCHAR(200),
    city              VARCHAR(100),
    state             VARCHAR(50),
    country           VARCHAR(50) DEFAULT 'US',
    segment           VARCHAR(50),
    created_date      DATE,
    effective_date    DATE NOT NULL,                -- SCD Type 2: when this version became
active
    expiry_date       DATE NOT NULL DEFAULT '9999-12-31', -- SCD Type 2: when this
version expired
    is_current        BOOLEAN NOT NULL DEFAULT TRUE    -- convenience flag
);

CREATE TABLE dim_product (
    product_key       SERIAL PRIMARY KEY,
    product_id        INT NOT NULL,
    product_name      VARCHAR(300) NOT NULL,
    category          VARCHAR(100),
    subcategory       VARCHAR(100),
    brand             VARCHAR(100),
    unit_cost         NUMERIC(10,2),
    unit_price        NUMERIC(10,2),
    effective_date    DATE NOT NULL,
    expiry_date       DATE NOT NULL DEFAULT '9999-12-31',
    is_current        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE fact_orders (
    order_key         SERIAL PRIMARY KEY,
    order_id          INT NOT NULL,
    customer_key      INT NOT NULL REFERENCES dim_customer(customer_key),
    product_key       INT NOT NULL REFERENCES dim_product(product_key),
    date_key          INT NOT NULL REFERENCES dim_date(date_key),
    quantity          INT NOT NULL,
    unit_price        NUMERIC(10,2) NOT NULL,
    discount_amount   NUMERIC(10,2) DEFAULT 0,
    revenue           NUMERIC(12,2) NOT NULL,

```

```
cost          NUMERIC(12,2),
profit        NUMERIC(12,2),
order_status   VARCHAR(20)
);

CREATE INDEX idx_fact_orders_date ON fact_orders(date_key);
CREATE INDEX idx_fact_orders_customer ON fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product ON fact_orders(product_key);
```

Notice the SCD Type 2 fields: `effective_date`, `expiry_date`, `is_current`. The `expiry_date` defaults to '9999-12-31' (far future) for current rows. The surrogate `customer_key` / `product_key` is SERIAL, auto-assigned by Postgres for each version of the record.

Step 1: Initial Load (01_initial_load.sql)

Populate all dimensions and facts from the Day 1 snapshot:

```

-- 1. Populate dim_date (generate a full calendar)
INSERT INTO dim_date (date_key, calendar_date, day_of_week, day_of_month,
                     month, month_name, quarter, year, is_weekend, is_holiday)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d,
    TRIM(TO_CHAR(d, 'Day')),
    EXTRACT(DAY FROM d),
    EXTRACT(MONTH FROM d),
    TRIM(TO_CHAR(d, 'Month')),
    EXTRACT(QUARTER FROM d),
    EXTRACT(YEAR FROM d),
    EXTRACT(ISODOW FROM d) IN (6, 7),
    FALSE
FROM generate_series('2024-01-01'::date, '2026-12-31'::date, '1 day') AS d;

-- 2. Initial load dim_customer
INSERT INTO dim_customer (customer_id, name, email, city, state, country,
                        segment, created_date, effective_date)
SELECT id, name, email, city, state, 'US', segment,
       created_at::date, created_at::date
FROM src_customers;

-- 3. Initial load dim_product
INSERT INTO dim_product (product_id, product_name, category, subcategory,
                       brand, unit_cost, unit_price, effective_date)
SELECT id, product_name, category, subcategory, brand,
       unit_cost, unit_price, created_at::date
FROM src_products;

-- 4. Initial load fact_orders
INSERT INTO fact_orders (order_id, customer_key, product_key, date_key,
                      quantity, unit_price, discount_amount,
                      revenue, cost, profit, order_status)
SELECT
    o.id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    (oi.quantity * oi.unit_price - oi.discount),
    (oi.quantity * dp.unit_cost),
    (oi.quantity * oi.unit_price - oi.discount) - (oi.quantity * dp.unit_cost),
    o.status
FROM src_orders o
JOIN src_order_items oi ON o.id = oi.order_id
JOIN dim_customer dc ON o.customer_id = dc.customer_id AND dc.is_current = TRUE
JOIN dim_product dp ON oi.product_id = dp.product_id AND dp.is_current = TRUE;

```

The key insight in the fact load: we join to `dim_customer` and `dim_product` with `is_current = TRUE` to get the correct surrogate keys. The fact table stores surrogate keys, not natural keys, so it can point to the right version of each dimension record.

Step 2: Simulate Day 2 Changes, Then Run Incremental Dimensions (02_incremental_dimensions.sql)

```
# Load Day 2 changes into the source tables
psql -f starter/sample_data_day2.sql
```

Day 2 brings: - Customer 1 (Alice) moved from New York to Boston - Customer 5 (Eve) changed segment from SMB to Enterprise - Two new customers (Karen, Leo) - Product 3 price increased from \$19.99 to \$24.99 - Two new products (Bluetooth Speaker, Winter Jacket) - New orders, order status updates

Now run the SCD Type 2 incremental load:

```

-- SCD Type 2: Customer Dimension
-- Step 1: Expire current rows where tracked attributes changed
UPDATE dim_customer dc
SET expiry_date = src.updated_at::date - 1,
    is_current = FALSE
FROM src_customers src
WHERE dc.customer_id = src.id
    AND dc.is_current = TRUE
    AND src.updated_at > '2025-01-10' -- since last load
    AND (dc.city != src.city OR dc.state != src.state OR dc.segment != src.segment);

-- Step 2: Insert new current rows for changed customers
INSERT INTO dim_customer (customer_id, name, email, city, state, country,
                          segment, created_date, effective_date)
SELECT src.id, src.name, src.email, src.city, src.state, 'US',
       src.segment, src.created_at::date, src.updated_at::date
FROM src_customers src
WHERE src.updated_at > '2025-01-10'
    AND (
        EXISTS (
            SELECT 1 FROM dim_customer dc
            WHERE dc.customer_id = src.id AND dc.is_current = FALSE
                AND dc.expiry_date = src.updated_at::date - 1
        )
        OR NOT EXISTS (
            SELECT 1 FROM dim_customer dc WHERE dc.customer_id = src.id
        )
    );

-- Step 3: Type 1 update for non-tracked attributes (email)
UPDATE dim_customer dc
SET email = src.email
FROM src_customers src
WHERE dc.customer_id = src.id
    AND dc.is_current = TRUE
    AND dc.email != src.email;

```

After this runs, `dim_customer` will have: - Alice: 2 rows (New York expired, Boston current) - Eve: 2 rows (SMB expired, Enterprise current) - Karen and Leo: 1 row each (new customers) - Everyone else: 1 row, unchanged

The same pattern applies to products (Product 3 gets a new row for the price change, new products get inserted).

Step 3: Incremental Fact Loading (03_incremental_facts.sql)

```
-- Load new orders, looking up current dimension keys
INSERT INTO fact_orders (order_id, customer_key, product_key, date_key,
                        quantity, unit_price, discount_amount,
                        revenue, cost, profit, order_status)

SELECT
    o.id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    (oi.quantity * oi.unit_price - oi.discount),
    (oi.quantity * dp.unit_cost),
    (oi.quantity * oi.unit_price - oi.discount) - (oi.quantity * dp.unit_cost),
    o.status
FROM src_orders o
JOIN src_order_items oi ON o.id = oi.order_id
JOIN dim_customer dc ON o.customer_id = dc.customer_id AND dc.is_current = TRUE
JOIN dim_product dp ON oi.product_id = dp.product_id AND dp.is_current = TRUE
-- Only new orders (not already loaded)
WHERE NOT EXISTS (
    SELECT 1 FROM fact_orders f WHERE f.order_id = o.id
);

-- Update status for orders that changed (e.g., pending -> completed)
UPDATE fact_orders f
SET order_status = o.status
FROM src_orders o
WHERE f.order_id = o.id
    AND f.order_status != o.status;
```

The `NOT EXISTS` clause is the incremental logic – it only loads orders that are not already in the fact table. This is idempotent: you can safely run it twice and it will not create duplicates.

The status UPDATE handles order lifecycle changes. When order 5 went from ‘pending’ to ‘completed’, we update the existing fact row rather than creating a new one. Order status is a Type 1 attribute on the fact table.

Step 4: Late-Arriving Facts (04_late_arriving.sql)

Late-arriving facts are orders whose `order_date` is in the past but that arrive in the source system today. The critical question: which dimension version was active when the order was placed?


```

-- Late-arriving fact load with point-in-time dimension lookup
INSERT INTO fact_orders (order_id, customer_key, product_key, date_key,
                        quantity, unit_price, discount_amount,
                        revenue, cost, profit, order_status)

SELECT
    o.id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    (oi.quantity * oi.unit_price - oi.discount),
    (oi.quantity * dp.unit_cost),
    (oi.quantity * oi.unit_price - oi.discount) - (oi.quantity * dp.unit_cost),
    o.status
FROM src_orders o
JOIN src_order_items oi ON o.id = oi.order_id
-- Point-in-time lookup: which customer row was active on the order date?
JOIN dim_customer dc ON o.customer_id = dc.customer_id
    AND o.order_date::date BETWEEN dc.effective_date AND dc.expiry_date
-- Point-in-time lookup: which product row was active on the order date?
JOIN dim_product dp ON oi.product_id = dp.product_id
    AND o.order_date::date BETWEEN dp.effective_date AND dp.expiry_date
WHERE NOT EXISTS (
    SELECT 1 FROM fact_orders f WHERE f.order_id = o.id
);

```

The key difference: Instead of `dc.is_current = TRUE`, we join on `o.order_date BETWEEN dc.effective_date AND dc.expiry_date`. This finds the dimension version that was active at the time the order was placed.

Production scenario: Order 11 was placed on January 6 but arrives in the source system on January 12 (Day 3). By January 12, Alice has already moved from New York to Boston (Day 2 change). If we joined on `is_current = TRUE`, we would attribute this January 6 order to Alice-in-Boston. But Alice was still in New York on January 6. The point-in-time join correctly assigns Alice-in-New-York's surrogate key.

Fallback for missing dimension rows:

Sometimes the dimension record for the order date does not exist yet (the dimension arrived *after* the fact). The fallback uses `JOIN LATERAL` to find the earliest known dimension record:

```

-- Fallback: use the earliest known dimension record
INSERT INTO fact_orders (order_id, customer_key, product_key, date_key,
                        quantity, unit_price, discount_amount,
                        revenue, cost, profit, order_status)

SELECT
    o.id,
    dc.customer_key,
    dp.product_key,
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT,
    oi.quantity,
    oi.unit_price,
    oi.discount,
    (oi.quantity * oi.unit_price - oi.discount),
    (oi.quantity * dp.unit_cost),
    (oi.quantity * oi.unit_price - oi.discount) - (oi.quantity * dp.unit_cost),
    o.status
FROM src_orders o
JOIN src_order_items oi ON o.id = oi.order_id
JOIN LATERAL (
    SELECT customer_key FROM dim_customer
    WHERE customer_id = o.customer_id
    ORDER BY effective_date LIMIT 1
) dc ON TRUE
JOIN LATERAL (
    SELECT product_key, unit_cost FROM dim_product
    WHERE product_id = oi.product_id
    ORDER BY effective_date LIMIT 1
) dp ON TRUE
WHERE NOT EXISTS (
    SELECT 1 FROM fact_orders f WHERE f.order_id = o.id
);

```

Step 5: Data Quality Checks (05_quality_checks.sql)

After every load, run the full quality suite:

```

-- 1. Orphaned fact records (missing dimension keys)
SELECT 'Orphaned customer keys' AS check_name,
       COUNT(*) AS violations
FROM fact_orders f
LEFT JOIN dim_customer dc ON f.customer_key = dc.customer_key
WHERE dc.customer_key IS NULL
UNION ALL
SELECT 'Orphaned product keys',
       COUNT(*)
FROM fact_orders f
LEFT JOIN dim_product dp ON f.product_key = dp.product_key
WHERE dp.product_key IS NULL
UNION ALL
SELECT 'Orphaned date keys',
       COUNT(*)
FROM fact_orders f
LEFT JOIN dim_date dd ON f.date_key = dd.date_key
WHERE dd.date_key IS NULL;

-- 2. Duplicate dimension entries (same natural key marked as current)
SELECT 'Duplicate current customers' AS check_name,
       COUNT(*) AS violations
FROM (
  SELECT customer_id, COUNT(*) AS cnt
  FROM dim_customer WHERE is_current = TRUE
  GROUP BY customer_id HAVING COUNT(*) > 1
) dupes
UNION ALL
SELECT 'Duplicate current products',
       COUNT(*)
FROM (
  SELECT product_id, COUNT(*) AS cnt
  FROM dim_product WHERE is_current = TRUE
  GROUP BY product_id HAVING COUNT(*) > 1
) dupes;

-- 3. Revenue reconciliation: source vs warehouse
SELECT
  'Revenue reconciliation' AS check_name,
  ABS(source_total - warehouse_total) AS difference,
  CASE WHEN ABS(source_total - warehouse_total) < 0.01 THEN 'PASS' ELSE 'FAIL' END
AS status
FROM (
  SELECT SUM(quantity * unit_price - discount) AS source_total FROM src_order_items
) src,
(
  SELECT SUM(revenue) AS warehouse_total FROM fact_orders
) wh;

-- 4. Negative revenue check
SELECT 'Negative revenue' AS check_name,
       COUNT(*) AS violations
FROM fact_orders WHERE revenue < 0;

```

```
-- 5. Future date check
SELECT 'Future order dates' AS check_name,
       COUNT(*) AS violations
FROM fact_orders f
JOIN dim_date d ON f.date_key = d.date_key
WHERE d.calendar_date > CURRENT_DATE;

-- 6. Null measure check
SELECT 'Null revenues' AS check_name, COUNT(*) AS violations
FROM fact_orders WHERE revenue IS NULL
UNION ALL
SELECT 'Null quantities', COUNT(*)
FROM fact_orders WHERE quantity IS NULL;
```

All violations should be zero. If any are not, investigate before signing off on the load.

Running the Full Pipeline

```
# Day 1: Initial load
psql -f starter/sample_data_day1.sql
psql -f starter/warehouse_schema.sql
psql -f solution/01_initial_load.sql
psql -f solution/05_quality_checks.sql # verify

# Day 2: Incremental load with SCD changes
psql -f starter/sample_data_day2.sql
psql -f solution/02_incremental_dimensions.sql
psql -f solution/03_incremental_facts.sql
psql -f solution/05_quality_checks.sql # verify

# Day 3: Late-arriving data
psql -f starter/sample_data_day3.sql
psql -f solution/02_incremental_dimensions.sql # handle new dimension changes
psql -f solution/04_late_arriving.sql         # handle late-arriving facts
psql -f solution/03_incremental_facts.sql      # handle normal new facts
psql -f solution/05_quality_checks.sql         # verify
```

This is the exact sequence your Airflow DAG would orchestrate in production. Each step is idempotent (safe to re-run). The quality checks gate downstream consumption.

Key Takeaway: This project is a complete, production-style incremental ETL pipeline written in pure SQL. It handles initial loads, SCD Type 2 dimension changes, incremental fact loading, late-arriving data with point-in-time dimension lookups, and automated data quality checks. These are the exact patterns you will build and maintain on the job.

What's Next

You now have the SQL toolkit that data engineers use daily – CTEs for readable transformations, window functions for analytics and dedup, UPSERT/MERGE for incremental loading, SCD Type 2 for dimension history, performance tuning with indexes and partitioning, and automated data quality checks. In Module 3, we move from *how to write the SQL* to *where to store the data* – storage engines, columnar formats, table formats like Delta Lake and Apache Iceberg, and the architecture decisions that determine whether your warehouse costs \$500/month or \$50,000/month.

Module 3: Python for Data Engineers

The Reality of Data Engineering in Python

About half of data engineering is just getting data out of APIs. The other half is cleaning what you got, validating it, converting it to a format that does not make everyone downstream want to cry, and loading it into a warehouse. The APIs are never as clean as the docs suggest. Fields that the documentation calls “required” will be null. Dates will arrive in three different formats across the same endpoint. Pagination will behave differently under load than it does in the sandbox. Rate limits will kick in at the worst possible moment.

This module teaches you the Python patterns that production data pipelines actually use. Not toy scripts that fetch one page of results and print them – real, structured projects with proper configuration, retry logic, data validation, and testing. By the end, you will have built a complete extract-validate-transform-load pipeline against a realistic API, using the same tools and patterns you will encounter on the job.

What we are building: A pipeline called “ShopFast” that extracts product and order data from a vendor API, validates every record with Pydantic, transforms it with Polars, and loads it into a PostgreSQL warehouse. The companion code lives in `de-fast-track/modules/module-3/` with both `starter/` (TODO stubs for you to complete) and `solution/` (reference implementation).

3.1 Project Structure for Data Engineering Projects

Most data engineering code you encounter in the wild is a mess. Single scripts with hundreds of lines, hardcoded credentials, no tests, and `import sys; sys.path.append("../../..")` hacks scattered everywhere. Your first contribution to any team should be setting up a proper project structure. It costs almost nothing and prevents an enormous class of problems.

The src/ Layout

The modern Python standard is the `src/` layout. Here is what the ShopFast project looks like:

```

shopfast-pipeline/
  pyproject.toml      # Project metadata and dependencies
  .env.example        # Template showing required env vars (committed to git)
  .env                # Actual secrets (NEVER committed to git)
  .gitignore
  mock_api/
    app.py            # Fake API server for local development
  src/
    shopfast/
      __init__.py
      config.py        # Pydantic-settings configuration
      models.py        # Pydantic data models
      pipeline.py      # Main ETL orchestration
      extractors/
        __init__.py
        api_client.py  # API extraction logic
      transformers/
        __init__.py
      loaders/
        __init__.py
      utils/
        __init__.py
    tests/
      conftest.py      # Shared pytest fixtures
      test_api_client.py
      test_models.py

```

Why this layout matters. The `src/` directory prevents a subtle but nasty bug: without it, Python might import your local package source instead of the installed version, leading to tests that pass locally but fail in CI. With the `src/` layout, you *must* install the package (even in editable mode with `pip install -e .`) before imports work. This guarantees that what you test is what you ship.

The `extractors/`, `transformers/`, `loaders/` split mirrors the ETL stages. When your pipeline grows from one API to five, you add new files under `extractors/` rather than inflating a single script. When a new team member joins, they can look at the directory structure and immediately understand where things live.

pyproject.toml: The Modern Standard

The `pyproject.toml` file replaces the old `setup.py` and `requirements.txt` approach. Here is the one from the companion repo:

```

[project]
name = "shopfast-pipeline"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "httpx>=0.27",
    "pydantic>=2.7",
    "pydantic-settings>=2.2",
    "polars>=0.20",
    "duckdb>=0.10",
    "pyarrow>=16",
    "tenacity>=8.3",
    "python-dotenv>=1.0",
    "rich>=13.7",
    "psycopg2-binary>=2.9",
]

[project.optional-dependencies]
dev = ["pytest>=8.2", "ruff>=0.4"]

[tool.ruff]
line-length = 100

```

Notice the dependency choices. Each one is deliberate, and we will cover why throughout this module. The `dev` extras keep test and lint tools separate from production dependencies – your Docker image does not need `pytest`.

Configuration with pydantic-settings

I have seen data engineers hardcode API keys in scripts checked into GitHub. I have seen database passwords in plain text in Jupyter notebooks that got shared in Slack. I have seen AWS credentials committed to public repos and exploited within minutes by bots that scan GitHub for exactly this. Here is why `pydantic-settings` exists.


```

from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    db_host: str = "localhost"
    db_port: int = 5432
    db_user: str = "dataeng"
    db_password: str = "dataeng"
    db_name: str = "warehouse"
    api_base_url: str = "http://localhost:5000"
    api_key: str = "dev-key-change-me"
    s3_endpoint: str = "http://localhost:9000"
    s3_access_key: str = "minioadmin"
    s3_secret_key: str = "minioadmin"
    s3_bucket: str = "raw-data"
    log_level: str = "INFO"

    @property
    def database_url(self) -> str:
        return f"postgresql://{self.db_user}:{self.db_password}@{self.db_host}:{self.db_port}/{self.db_name}"

    class Config:
        env_file = ".env"

settings = Settings()

```

This does several things at once:

1. **Type safety.** If someone sets `DB_PORT=not_a_number` in their environment, Pydantic raises a clear error at startup, not a cryptic `psycopg2` connection failure five minutes into a pipeline run.
2. **Environment variable loading.** `pydantic-settings` automatically reads environment variables matching the field names (case-insensitive). In production, you set `DB_PASSWORD` as a Kubernetes secret or a CI/CD variable. In development, you put it in a `.env` file.
3. **Sensible defaults.** The defaults point at local development services (localhost, default ports, dev credentials). A new developer clones the repo, runs `docker compose up`, and everything connects without configuration.
4. **Single source of truth.** Every part of the pipeline imports from `config.py`. There is exactly one place to look when you need to understand what configuration exists.

The `.env` / `.env.example` pattern is critical:

```
# .env.example -- committed to git, shows what variables are needed
DB_HOST=localhost
DB_PORT=5432
DB_USER=dataeng
DB_PASSWORD=changeme
API_BASE_URL=http://localhost:5000
API_KEY=your-api-key-here
```

```
# .env -- NEVER committed to git, contains actual secrets
DB_PASSWORD=real-production-password
API_KEY=sk-prod-abc123def456
```

Your `.gitignore` must include `.env` but NOT `.env.example`. The example file serves as documentation for what environment variables the project expects. When a new engineer joins, they copy `.env.example` to `.env` and fill in their values. No Slack messages asking “what environment variables do I need?”

At your job: Your first day on a data engineering team, you will clone a repo. If there is a `.env.example`, you know exactly what to configure. If there is not, you will spend an hour reading code and asking teammates what secrets you need. Be the person who adds the `.env.example`.

3.2 Working with APIs

At your job, one of your first tasks will probably sound like this: “We signed a contract with VendorX. They have a REST API. Pull their data into our warehouse daily.” This sounds simple until you discover that the API paginates with cursor tokens instead of offsets, returns dates as Unix timestamps in some endpoints and ISO strings in others, rate-limits you to 100 requests per minute, and occasionally returns 500 errors for no discernible reason.

Here is how you build an API client that handles all of that.

Why httpx Over requests

The `requests` library is everywhere. It has been the default HTTP library in Python for over a decade. So why use `httpx`?

```
import httpx

# httpx has an almost identical API to requests
response = httpx.get("https://api.example.com/data")
data = response.json()
```

The API is nearly identical, so switching is painless. But `httpx` gives you three things that matter for data engineering:

1. **Async support built in.** When you eventually need to hit 10 API endpoints in parallel (and you will), `httpx` supports `async/await` natively. With `requests`, you would have to rewrite your entire client using `aiohttp`, which has a completely different API. With `httpx`, you change `httpx.Client` to `httpx.AsyncClient` and add `async/await` keywords. The rest stays the same.
2. **HTTP/2 support.** Some modern APIs only support HTTP/2. More importantly, HTTP/2 multiplexing lets you send multiple requests over a single connection, which is faster when you are paginating through thousands of pages.
3. **Better timeout handling.** `httpx` has separate connect, read, write, and pool timeouts. `requests` has a single timeout that covers everything, which means you either set it too low (and timeout on large responses) or too high (and hang forever on connection issues).

The Production API Client

Here is the complete API client from the companion repo. Study it carefully – this pattern will serve you for years:

```

import logging
from datetime import datetime

import httpx
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type

logger = logging.getLogger(__name__)

class ShopFastClient:
    def __init__(self, base_url: str, api_key: str):
        self.base_url = base_url.rstrip("/")
        self.client = httpx.Client(
            base_url=self.base_url,
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=30.0,
        )

    @retry(
        stop=stop_after_attempt(3),
        wait=wait_exponential(multiplier=1, min=2, max=30),
        retry=retry_if_exception_type((httpx.HTTPStatusError, httpx.ConnectError)),
    )
    def _get(self, path: str, params: dict | None = None) -> dict | list:
        response = self.client.get(path, params=params)
        response.raise_for_status()
        return response.json()

    def fetch_products(self, limit: int = 100, offset: int = 0) -> list[dict]:
        all_products = []
        while True:
            batch = self._get("/products", params={"limit": limit, "offset": offset})
            if not batch:
                break
            all_products.extend(batch)
            if len(batch) < limit:
                break
            offset += limit
            logger.info(f"Fetches {len(all_products)} products so far...")
        logger.info(f"Total products fetched: {len(all_products)}")
        return all_products

    def fetch_orders(self, since: datetime | None = None) -> list[dict]:
        params = {}
        if since:
            params["since"] = since.isoformat()
        orders = self._get("/orders", params=params)
        logger.info(f"Fetches {len(orders)} orders")
        return orders

    def health_check(self) -> bool:
        try:
            result = self._get("/health")

```

```

        return result.get("status") == "ok"
    except Exception:
        return False

    def close(self):
        self.client.close()

```

Let us break down every design decision.

The Client Pattern

```

self.client = httpx.Client(
    base_url=self.base_url,
    headers={"Authorization": f"Bearer {api_key}"},
    timeout=30.0,
)

```

We create a **persistent client** rather than making individual `httpx.get()` calls. This matters because:

- **Connection reuse.** The client maintains a connection pool. Instead of opening a new TCP connection (and doing a TLS handshake) for every request, it reuses existing connections. When you are making hundreds of paginated requests, this is dramatically faster.
- **Consistent headers.** The auth header is set once. You cannot accidentally forget it on one request.
- **Resource management.** The client has a `close()` method. In a production pipeline, you want to cleanly shut down connections when you are done, not leave them hanging.

A more robust version would use a context manager:

```

class ShopFastClient:
    def __enter__(self):
        return self

    def __exit__(self, *args):
        self.close()

# Usage:
with ShopFastClient(base_url, api_key) as client:
    products = client.fetch_products()
# Connection is guaranteed to close, even if an exception occurs

```

Retry Logic with tenacity

```
@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=30),
    retry=retry_if_exception_type((httpx.HTTPStatusError, httpx.ConnectError)),
)
def _get(self, path: str, params: dict | None = None) -> dict | list:
    response = self.client.get(path, params=params)
    response.raise_for_status()
    return response.json()
```

This is where `tenacity` earns its place in every data engineer's toolkit. The decorator says: "If this function raises an `HTTPStatusError` or `ConnectError`, wait and try again. Try up to 3 times. Wait exponentially between attempts (2 seconds, then 4, then 8, capped at 30)."

Why exponential backoff matters. Imagine 100 pipeline instances all hitting an API that just returned a 500 error. If they all retry immediately, they slam the API with 100 simultaneous requests again. The API, already struggling, gets worse. All 100 retry again. This is the **thundering herd problem**.

Exponential backoff spreads the retries over time. The first retry waits 2 seconds. The second waits 4 seconds. The third waits 8 seconds. With some randomized jitter (which `tenacity` supports), the 100 instances end up retrying at different times, giving the API breathing room to recover.

```
Without backoff: [100 requests] -> fail -> [100 requests] -> fail -> [100 requests]
With exp backoff: [100 requests] -> fail -> [scattered retries over 2-8s] ->
[scattered retries over 4-16s]
```

Why we only retry specific exceptions. A `ConnectError` means the server is temporarily unreachable – retrying makes sense. An `HTTPStatusError` from a 500 means the server had an internal error – retrying often works. But a 400 Bad Request means our request is malformed – retrying the same bad request will never work. The `retry_if_exception_type` filter prevents wasting time on unrecoverable errors.

In a production API client, you would also handle HTTP 429 (Too Many Requests) explicitly:

```

from tenacity import retry_if_exception

def is_retryable(exc):
    if isinstance(exc, httpx.HTTPStatusError):
        return exc.response.status_code in (429, 500, 502, 503, 504)
    if isinstance(exc, httpx.ConnectError):
        return True
    return False

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=60),
    retry=retry_if_exception(is_retryable),
)
def _get(self, path, params=None):
    response = self.client.get(path, params=params)
    if response.status_code == 429:
        retry_after = int(response.headers.get("Retry-After", 30))
        raise httpx.HTTPStatusError(
            f"Rate limited, retry after {retry_after}s",
            request=response.request,
            response=response,
        )
    response.raise_for_status()
    return response.json()

```

Pagination

The `fetch_products` method demonstrates offset-based pagination:

```

def fetch_products(self, limit: int = 100, offset: int = 0) -> list[dict]:
    all_products = []
    while True:
        batch = self._get("/products", params={"limit": limit, "offset": offset})
        if not batch:
            break
        all_products.extend(batch)
        if len(batch) < limit:
            break
        offset += limit
        logger.info(f"Fetched {len(all_products)} products so far...")
    logger.info(f"Total products fetched: {len(all_products)}")
    return all_products

```

This works fine for small datasets (hundreds or thousands of records). But what happens when an API returns 10 million records? You cannot hold them all in memory. This is where **generators** come in.

Generators: The Memory-Efficient Alternative

A generator function uses `yield` instead of `return`. Instead of building a complete list in memory and returning it all at once, it produces one item (or one batch) at a time:

```
def fetch_products_streaming(self, limit: int = 100) -> Iterator[list[dict]]:
    """Yield one page of products at a time, never holding all pages in memory."""
    offset = 0
    while True:
        batch = self._get("/products", params={"limit": limit, "offset": offset})
        if not batch:
            break
        yield batch # Hand this batch to the caller, then pause here
        if len(batch) < limit:
            break
        offset += limit
```

What `yield` actually does under the hood. When Python encounters `yield`, it transforms the function into a generator. Calling the function does not execute it – it returns a generator object. Each time you call `next()` on that generator (or iterate with `for`), the function runs until it hits `yield`, hands back the value, and *freezes its entire state* – local variables, instruction pointer, everything. The next call to `next()` resumes from exactly where it left off.

```
# Without generators: all 10M records in memory at once
all_products = client.fetch_products() # Might use 8 GB of RAM
for product in all_products:
    process(product)

# With generators: only one page (100 records) in memory at a time
for page in client.fetch_products_streaming():
    for product in page:
        process(product)
# When we loop back, the previous page can be garbage collected
```

This is not premature optimization. I have seen pipelines OOM-killed in production because someone called `.fetchall()` on a query returning 50 million rows or loaded every page of an API response into a list. Generators are how you handle data that might be larger than memory.

At your job: When you see `all_results = []` followed by a loop that appends to it, ask yourself: “Could this list grow unbounded?” If the answer involves the words “depends on how much data” or “it should be fine,” that is a generator waiting to happen.

3.3 File Formats: CSV, JSON, and Parquet

Data arrives in many formats. Understanding which format to use where – and why – is fundamental to data engineering.

CSV: The Legacy Format

CSV is the cockroach of data formats. It has survived for decades because every tool can read it. But it is terrible for data engineering:

- **No type information.** Is `"123"` an integer, a float, or a string? Is `"2025-04-01"` a date or a string? The reader has to guess, and it will guess wrong.
- **No compression.** A 1 GB CSV file is 1 GB on disk and 1 GB to transfer.
- **Ambiguous parsing.** Does a comma in a value mean a delimiter or part of the data? What about newlines inside quoted fields? Every CSV parser handles edge cases differently.
- **Slow to read.** To find data in row 1,000,000, you must read through the previous 999,999 rows.
- **No schema.** If a column is renamed or removed between two CSV exports, you discover this at parse time, not at write time.

You will still encounter CSV constantly – legacy systems export it, business users create it in Excel, vendors email it as attachments. The rule is: **receive as CSV, immediately convert to something better.**

JSON and JSONL: The API Format

JSON is the native language of APIs. When you call a REST endpoint, you almost always get JSON back. It supports nested structures, which makes it natural for representing complex data like orders with line items.

JSONL (JSON Lines) puts one JSON object per line. This is better for data engineering than regular JSON because:

```
{"order_id": 1, "customer_id": 42, "status": "completed", "total": 79.97}
{"order_id": 2, "customer_id": 17, "status": "pending", "total": 149.50}
{"order_id": 3, "customer_id": 42, "status": "returned", "total": 33.00}
```

- You can process it line by line without loading the entire file into memory
- You can append new records by appending lines (no need to parse the whole file)
- Splittable – distributed systems can split the file at newline boundaries

But JSON/JSONL still lacks types (everything is a string, number, boolean, or null – no dates, no decimals), does not compress well, and is slow to parse at scale.

Parquet: The Data Engineering Format

Parquet is where you want your data to end up. It is a **columnar, compressed, typed** binary format that was designed specifically for analytical workloads. Here is why it matters:

Columnar storage. In a row-based format (CSV, JSON), data is stored row by row:

```
Row 1: [order_id=1, customer_id=42, status="completed", amount=79.97, ...]
Row 2: [order_id=2, customer_id=17, status="pending", amount=149.50, ...]
```

To answer “what is the total amount across all orders?”, you must read every field of every row, even though you only need the `amount` column. With a 50-column table, that is 49 columns of wasted I/O.

Parquet stores data column by column:

```
order_id column:  [1, 2, 3, ...]
customer_id column: [42, 17, 42, ...]
status column:    ["completed", "pending", "returned", ...]
amount column:    [79.97, 149.50, 33.00, ...]
```

Now the same query reads only the `amount` column – a fraction of the data. **This is why Parquet queries are 5-20x faster than the same queries on CSV.** It is not a minor optimization; it is the difference between a query taking 30 seconds and taking 2 seconds.

Why columnar compression works so well. Adjacent values in a column tend to be similar. A `status` column with 10 million rows might have only 4 unique values (pending, completed, returned, cancelled). Parquet uses dictionary encoding to store those 10 million values as 10 million tiny integers plus a 4-entry lookup table. A `date` column in a daily-partitioned file might have the same date for every row – run-length encoding compresses that to essentially zero overhead. A typical Parquet file is 5-10x smaller than the equivalent CSV.

Built-in types and schema. Every Parquet file embeds its schema in the file footer. When you open a Parquet file, you know immediately that `order_date` is a timestamp, `amount` is a decimal, and `quantity` is a 32-bit integer. No guessing, no coercion errors downstream.

Predicate pushdown. Parquet files contain metadata about each chunk of data: min/max values, null counts, etc. A query filtering `WHERE order_date = '2025-04-01'` can skip entire chunks where the max date is before April 2025 – without reading the data itself.

The Conversion Rule

Here is the rule that should be drilled into your muscle memory:

Receive as CSV or JSON. Immediately convert to Parquet. Store and process only Parquet from that point forward.

```
import polars as pl

# CSV arrives from a vendor
df = pl.read_csv("vendor_export.csv")

# Immediately convert and save as Parquet
df.write_parquet("vendor_export.parquet")

# From here on, only work with the Parquet file
df = pl.read_parquet("vendor_export.parquet") # 5-20x faster to read back
```

Every minute you spend working with CSV at scale is wasted time. Convert once, benefit forever.

3.4 Data Validation with Pydantic

Raw API data is dirty. Fields will be missing. Types will be wrong. Values will violate business rules that nobody documented. If you load this data into your warehouse unchecked, you will spend your weekends debugging why revenue numbers do not match.

Pydantic models define what your data *should* look like and reject anything that does not match:

```

from pydantic import BaseModel, field_validator, computed_field
from datetime import datetime

class OrderItem(BaseModel):
    product_id: int
    product_name: str
    quantity: int
    unit_price: float

    @field_validator("quantity")
    @classmethod
    def quantity_must_be_positive(cls, v):
        if v <= 0:
            raise ValueError("quantity must be greater than 0")
        return v

    @field_validator("unit_price")
    @classmethod
    def price_must_be_non_negative(cls, v):
        if v < 0:
            raise ValueError("unit_price must be >= 0")
        return v

    @computed_field
    @property
    def line_total(self) -> float:
        return round(self.quantity * self.unit_price, 2)

class Order(BaseModel):
    order_id: int
    customer_id: int
    order_date: datetime
    status: str
    items: list[OrderItem]

    @field_validator("status")
    @classmethod
    def status_must_be_valid(cls, v):
        allowed = {"pending", "completed", "returned", "cancelled"}
        if v not in allowed:
            raise ValueError(f"status must be one of {allowed}, got '{v}'")
        return v

    @computed_field
    @property
    def total_amount(self) -> float:
        return round(sum(item.line_total for item in self.items), 2)

```

What This Buys You

Type coercion. If the API returns `"order_date": "2025-06-15T10:30:00"` (a string), Pydantic automatically converts it to a `datetime` object. If it returns `"quantity": "3"` (a string), Pydantic converts it to `int`. If it returns `"quantity": "abc"`, Pydantic raises a clear validation error telling you exactly which field on which record failed.

Business rule enforcement. The `@field_validator` decorators encode rules that live in someone's head or in a Confluence page nobody reads: quantities must be positive, prices cannot be negative, status must be one of a known set. When the API starts returning `"status": "refunded"` because the vendor added a new status without telling you, your pipeline catches it immediately instead of silently loading garbage.

Computed fields. The `@computed_field` decorator on `line_total` and `total_amount` means these values are always derived from the source data, never from whatever the API claims they are. If the API says `line_total: 100` but `quantity: 2` and `unit_price: 29.99`, your computed field correctly produces `59.98`. You trust your math, not theirs.

Validation in the pipeline. Here is how the pipeline uses these models:

```
def validate(self, raw_data: dict) -> list[Order]:
    logger.info("Validating orders...")
    valid_orders = []
    errors = []
    for raw_order in raw_data["orders"]:
        try:
            order = Order(**raw_order)
            valid_orders.append(order)
        except Exception as e:
            errors.append({"order_id": raw_order.get("order_id"), "error": str(e)})
    logger.info(f"Validated: {len(valid_orders)} valid, {len(errors)} errors")
    if errors:
        for err in errors[:5]:
            logger.warning(f"Validation error: {err}")
    return valid_orders
```

Notice the pattern: do not fail the entire pipeline because one record is bad. Validate each record, collect the good ones, log the bad ones. In production, you would also write the bad records to a “dead letter” table or file for later investigation. The ratio of good to bad records tells you a lot – if 0.1% of records fail validation, that is normal API messiness. If 30% fail, something changed upstream and you need to investigate.

At your job: The first time a stakeholder asks “why are the numbers different from yesterday?”, the answer is almost always bad data that was not validated. Pydantic models are your first line of defense. When you add a new data source, writing the Pydantic model should be the *first* thing you do, not an afterthought.

3.5 The Complete Pipeline

Here is the full pipeline that ties everything together – extract, validate, transform, load:

```

import logging

import polars as pl
import psycopg2

from shopfast.config import settings
from shopfast.extractors.api_client import ShopFastClient
from shopfast.models import Order

logging.basicConfig(level=settings.log_level)
logger = logging.getLogger(__name__)

class ShopFastPipeline:
    def __init__(self):
        self.api = ShopFastClient(settings.api_base_url, settings.api_key)
        self.conn = psycopg2.connect(settings.database_url)

    def extract(self) -> dict:
        logger.info("Extracting data from API...")
        products = self.api.fetch_products()
        orders = self.api.fetch_orders()
        logger.info(f"Extracted {len(products)} products, {len(orders)} orders")
        return {"products": products, "orders": orders}

    def validate(self, raw_data: dict) -> list[Order]:
        logger.info("Validating orders...")
        valid_orders = []
        errors = []
        for raw_order in raw_data["orders"]:
            try:
                order = Order(**raw_order)
                valid_orders.append(order)
            except Exception as e:
                errors.append({"order_id": raw_order.get("order_id"), "error": str(e)})
        logger.info(f"Validated: {len(valid_orders)} valid, {len(errors)} errors")
        if errors:
            for err in errors[:5]:
                logger.warning(f" Validation error: {err}")
        return valid_orders

    def transform(self, orders: list[Order]) -> pl.DataFrame:
        logger.info("Transforming data with Polars...")
        rows = []
        for order in orders:
            for item in order.items:
                rows.append({
                    "order_id": order.order_id,
                    "customer_id": order.customer_id,
                    "order_date": order.order_date,
                    "status": order.status,
                    "product_id": item.product_id,
                    "product_name": item.product_name,
                    "quantity": item.quantity,
                })

```

```

        "unit_price": item.unit_price,
        "line_total": item.line_total,
    })

    df = pl.DataFrame(rows)
    df = df.with_columns([
        pl.col("order_date").cast(pl.Date).alias("order_date_only"),
        pl.col("status").str.to_lowercase().alias("status_clean"),
    ])
    logger.info(f"Transformed DataFrame: {df.shape[0]} rows, {df.shape[1]}
columns")
    return df

def load(self, df: pl.DataFrame):
    logger.info(f"Loading {df.shape[0]} rows to warehouse...")
    cur = self.conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS staging.order_items (
            order_id INT, customer_id INT, order_date TIMESTAMP,
            status VARCHAR(20), product_id INT, product_name VARCHAR(300),
            quantity INT, unit_price NUMERIC(10,2), line_total NUMERIC(12,2)
        )
    """)
    cur.execute("TRUNCATE staging.order_items")

    for row in df.iter_rows(named=True):
        cur.execute("""
            INSERT INTO staging.order_items
            (order_id, customer_id, order_date, status, product_id,
             product_name, quantity, unit_price, line_total)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        """, (
            row["order_id"], row["customer_id"], row["order_date"],
            row["status"], row["product_id"], row["product_name"],
            row["quantity"], row["unit_price"], row["line_total"],
        ))
    self.conn.commit()
    logger.info("Load complete.")

def run(self):
    try:
        raw = self.extract()
        valid = self.validate(raw)
        transformed = self.transform(valid)
        self.load(transformed)
        logger.info("Pipeline finished successfully.")
    finally:
        self.api.close()
        self.conn.close()

if __name__ == "__main__":
    pipeline = ShopFastPipeline()
    pipeline.run()

```


Design Decisions Worth Noting

The `try/finally` in `run()`. This guarantees that the API client and database connection are closed even if the pipeline fails halfway through. Without this, a failed pipeline leaves open connections that eventually exhaust the database connection pool. I have seen production databases brought down because a retry loop kept opening connections without closing the failed ones.

Staging table with TRUNCATE. The load step writes to a `staging.order_items` table and truncates it first. This is the **full-refresh** pattern: every run replaces the entire staging table. It is simple and correct – you never end up with duplicate records from a pipeline that ran twice. For incremental loads, you would use a merge/upsert pattern instead.

Row-by-row inserts. The current `load()` method inserts one row at a time. This works for small datasets but is slow for large ones. In production, you would use one of these approaches:

```
# Option 1: COPY command (fastest for PostgreSQL)
import io
buffer = io.StringIO()
df.write_csv(buffer)
buffer.seek(0)
cur.copy_expert("COPY staging.order_items FROM STDIN WITH CSV HEADER", buffer)

# Option 2: executemany with page_size
from psycopg2.extras import execute_batch
execute_batch(cur, insert_sql, rows, page_size=1000)

# Option 3: Write Parquet, then use database's bulk import
df.write_parquet("/tmp/order_items.parquet")
# Then use COPY ... FROM ... (FORMAT PARQUET) if your DB supports it
```

Polars over pandas. The transform step uses Polars, not pandas. Polars is faster (it is written in Rust and uses all CPU cores), more memory-efficient, and has a more consistent API. For data engineering workloads – especially transforming millions of rows – the performance difference is significant. Polars is the new default for new data engineering projects.

3.6 Testing Data Pipelines

Data pipelines that are not tested break in production. Testing pipelines is different from testing web applications because the primary failure mode is not “the code crashes” but “the code runs fine and produces wrong numbers.” Here are the patterns from the companion repo.

Test Fixtures

Shared test data lives in `conftest.py`:

```

import pytest

@pytest.fixture
def sample_order_data():
    return {
        "order_id": 1,
        "customer_id": 42,
        "order_date": "2025-06-15T10:30:00",
        "status": "completed",
        "items": [
            {"product_id": 1, "product_name": "Widget Pro", "quantity": 2,
 "unit_price": 29.99},
            {"product_id": 3, "product_name": "Gadget Lite", "quantity": 1, "unit_price
": 19.99},
        ],
    }

@pytest.fixture
def sample_product_data():
    return {
        "id": 1,
        "product_name": "Widget Pro",
        "category": "Electronics",
        "subcategory": "Peripherals",
        "brand": "TechBrand",
        "unit_cost": 12.50,
        "unit_price": 29.99,
    }

@pytest.fixture
def sample_invalid_order():
    return {
        "order_id": 99,
        "customer_id": 1,
        "order_date": "2025-06-15T10:30:00",
        "status": "invalid_status",
        "items": [
            {"product_id": 1, "product_name": "Widget", "quantity": -1, "unit_price": 1
0.00},
        ],
    }

```

Notice the `sample_invalid_order` fixture: it has both an invalid status and a negative quantity. Testing the unhappy path is more important than testing the happy path for data engineering. You know your pipeline works when the data is clean. The question is what happens when the data is dirty.

Testing Models

```
import pytest
from shopfast.models import Order, OrderItem

def test_valid_order_item():
    item = OrderItem(product_id=1, product_name="Widget", quantity=2, unit_price=29.99)
    assert item.line_total == 59.98

def test_invalid_quantity():
    with pytest.raises(ValueError, match="quantity must be greater than 0"):
        OrderItem(product_id=1, product_name="Widget", quantity=0, unit_price=10.0)

def test_negative_price():
    with pytest.raises(ValueError, match="unit_price must be >= 0"):
        OrderItem(product_id=1, product_name="Widget", quantity=1, unit_price=-5.0)

def test_valid_order(sample_order_data):
    order = Order(**sample_order_data)
    assert order.total_amount == 79.97
    assert order.status == "completed"

def test_invalid_status():
    with pytest.raises(ValueError, match="status must be one of"):
        Order(
            order_id=1, customer_id=1, order_date="2025-01-01T00:00:00",
            status="bogus",
            items=[{"product_id": 1, "product_name": "X", "quantity": 1, "unit_price":
10}],
        )
```

These tests verify both the happy path (valid data produces correct computed fields) and the sad path (invalid data raises clear errors). The `match` parameter on `pytest.raises` ensures you are catching the *right* `ValueError`, not just any `ValueError`.

Testing the API Client

```
import pytest
from unittest.mock import patch
from shopfast.extractors.api_client import ShopFastClient

@pytest.fixture
def client():
    return ShopFastClient(base_url="http://localhost:5000", api_key="test-key")

def test_health_check(client):
    with patch.object(client, "_get", return_value={"status": "ok"}):
        assert client.health_check() is True

def test_health_check_failure(client):
    with patch.object(client, "_get", side_effect=Exception("connection refused")):
        assert client.health_check() is False

def test_fetch_products(client):
    mock_products = [{"id": i, "product_name": f"Product {i}"} for i in range(5)]
    with patch.object(client, "_get", return_value=mock_products):
        products = client.fetch_products()
        assert len(products) == 5

def test_fetch_orders(client):
    mock_orders = [{"order_id": i} for i in range(10)]
    with patch.object(client, "_get", return_value=mock_orders):
        orders = client.fetch_orders()
        assert len(orders) == 10
```

Key pattern here: **mock at the HTTP boundary, not higher**. We patch `_get` (the method that makes HTTP calls) rather than mocking the entire client. This lets us test the pagination logic, error handling, and data assembly without actually hitting a network. The tests run in milliseconds and never flake due to network issues.

What Else to Test in Production

The companion tests cover the basics. In a production pipeline, you would also add:

- **Data quality assertions.** After transformation, check that row counts are within expected ranges, that no critical columns are entirely null, that dates fall within reasonable bounds.
- **Schema tests.** Verify that the output DataFrame has the expected columns and types before loading.
- **Integration tests.** Use the mock API server (`mock_api/app.py` in the starter code) to test the full extract-validate-transform flow end-to-end without touching the real API or database.

```
# Example data quality test
def test_transform_output_quality():
    # ... set up pipeline and run transform ...
    assert df.shape[0] > 0, "Transform produced no rows"
    assert df["order_id"].null_count() == 0, "order_id has nulls"
    assert df["line_total"].min() >= 0, "Negative line totals found"
    assert df["order_date"].max() <= datetime.now(), "Future dates found"
```

3.7 The Mock API Server

The starter code includes a mock API server for local development. This is worth studying because it shows a pattern you will use constantly: building a fake version of an external dependency so you can develop and test without relying on the real thing.

```

"""Mock ShopFast API for local development.
Run with: python -m mock_api.app
Serves fake product and order data on localhost:5000.
"""

from http.server import HTTPServer, BaseHTTPRequestHandler
import json
from datetime import datetime, timedelta
import random

PRODUCTS = [
    {"id": i, "product_name": f"Product {i}",
     "category": ["Electronics", "Clothing", "Home", "Sports", "Books"][i % 5],
     "unit_price": round(10 + random.random() * 190, 2)}
    for i in range(1, 51)
]

def generate_orders(count=100):
    orders = []
    for i in range(1, count + 1):
        num_items = random.randint(1, 4)
        items = [
            {"product_id": random.randint(1, 50),
             "product_name": f"Product {random.randint(1, 50)}",
             "quantity": random.randint(1, 5),
             "unit_price": round(10 + random.random() * 190, 2)}
            for _ in range(num_items)
        ]
        orders.append({
            "order_id": i,
            "customer_id": random.randint(1, 100),
            "order_date": (datetime.now() - timedelta(days=random.randint(0, 365))).isoformat(),
            "status": random.choice(["completed", "completed", "completed", "pending", "returned"]),
            "items": items,
        })
    return orders

ORDERS = generate_orders()

```

Notice how the mock data is not perfectly clean – the status distribution is weighted toward “completed” (like real order data), prices are random floats (not round numbers), and dates span the last year. Good mock data mimics the messiness of real data, not the tidiness of documentation examples.

Putting It All Together

Here is the mental model for this entire module. Every data engineering project you work on will follow this pattern, whether it takes a week or a year:

1. PROJECT STRUCTURE	Set up <code>src/</code> layout, config, <code>.env</code> , <code>.gitignore</code>
2. EXTRACT	Build API client with <code>httpx + tenacity</code> Handle pagination, retries, rate limits
3. VALIDATE	Define Pydantic models for every entity Separate valid records from errors
4. TRANSFORM	Convert to Parquet / Polars DataFrame Add computed columns, clean data
5. LOAD	Write to warehouse (staging first)
6. TEST	Unit tests for models, mocked API tests, data quality assertions

At your job: Your first task might be: “Pull data from this vendor’s API into our warehouse.” Here is exactly how you would structure that. Start with the project layout and config (30 minutes). Write the Pydantic models based on the API docs (1 hour – this forces you to understand the data before you write any pipeline code). Build the API client with retry logic (2 hours). Wire up the pipeline (1 hour). Write tests (1 hour). The entire thing takes a day, and you have a production-grade pipeline, not a script that works-on-my-machine.

Companion Code

The full implementation lives in the course repository:

- **Starter code** (with TODOs for you to complete): `de-fast-track/modules/module-3/starter/`
- **Reference solution:** `de-fast-track/modules/module-3/solution/`

Work through the starter code in this order: 1. `src/shopfast/config.py` – Fill in the Settings fields 2. `src/shopfast/models.py` – Add validators and computed fields 3. `src/shopfast/extractors/api_client.py` – Implement the HTTP client 4. `src/shopfast/pipeline.py` – Wire up the full ETL pipeline 5. `tests/conftest.py` – Create test fixtures matching the API format 6. Run `python -m mock_api.app` in one terminal, then `python -m shopfast.pipeline` in another

What's Next

Now that you can extract data from APIs, validate it, and load it into a warehouse, Module 4 covers how data is *processed at scale*. We will move from single-machine pipelines to distributed batch processing, workflow orchestration, and the architectural patterns that determine how your pipelines run in production.

Module 4: Apache Airflow – Orchestration

What you'll learn: Build production-grade DAGs, not toy examples. Airflow is the most in-demand orchestration tool in data engineering, and by the end of this module you'll know how to use it like a practitioner.

Airflow is the most in-demand orchestration tool in data engineering. Roughly 70% of DE job postings mention it. When you interview, they WILL ask about DAGs – what they are, how you structure them, how you handle failures. This module gives you those answers with real code, not abstract theory.

4.1 Why Orchestration Matters (Cron Is Not Enough)

TL;DR - Cron jobs can't handle dependencies, retries, visibility, or backfills - Orchestrators manage the *flow* of your pipeline, not just the timing - Airflow is the industry standard (~70% of DE job postings mention it) - Don't wait for a production incident to migrate off cron – start with Airflow

The 3 AM Failure Story

You've got a Python script. It pulls data from an API, transforms it, dumps it into Postgres. Works great. So you throw it on a cron job – `0 6 * * *` – runs every morning at 6 AM. Life is good.

Then one morning, the API is down at 6 AM. Your script fails silently. Nobody knows until 2 PM when the VP of Sales asks why the dashboard is empty. You SSH into the server, check the logs, re-run it manually. Crisis averted.

But then next week, your script runs fine, but the *downstream* dbt model that depends on it didn't wait – it ran at 6:15 AM like it always does, but your script took 45 minutes instead of 10 because the API was slow. Now your dashboard has yesterday's data and nobody knows.

That's why cron is not enough.

The Four Problems Orchestration Solves

Orchestration addresses four problems that cron simply can't:

1. **Dependencies** – “Don’t run step 3 until steps 1 and 2 are done.” Cron doesn’t know about dependencies. It just fires at a time.
2. **Retries and error handling** – If something fails, retry it 3 times with a 5-minute delay. If it still fails, alert someone. Cron? It either ran or it didn’t.
3. **Visibility** – A UI where you can see: what ran, when, did it succeed, how long did it take, what’s running right now. Cron gives you... log files. If you remembered to set up logging.
4. **Backfills** – “The API was returning wrong data for the last 3 days. Re-run everything for those dates.” With cron, you’re writing bash scripts. With orchestration, it’s a button click.

Key Concept: Orchestration vs. Scheduling

Cron is a *scheduler* – it fires jobs at specific times. An orchestrator like Airflow is a *workflow manager* – it manages the flow of tasks, their dependencies, retries, and state. Think of the difference between a kitchen timer and a head chef who coordinates every dish to arrive at the table together.

The Orchestration Landscape (2025-2026)

The most common orchestrators you’ll encounter:

- **Apache Airflow** – the industry standard, and what we’re learning
- **Prefect** – newer, more Pythonic, gaining traction
- **Dagster** – asset-centric, great developer experience
- **Mage** – newer, notebook-style, gaining popularity

We’re teaching Airflow because ~70% of job postings mention it and it’s what you’ll most likely encounter in the real world.

Common Mistake

People build a pipeline with cron, it works for 3 months, then they spend 2 weeks migrating to Airflow in a panic after a major incident. Just start with Airflow. The learning curve is worth it.

At your job: When you join a team, one of your first questions should be “What orchestrator do we use?” If the answer is “cron jobs on an EC2 instance,” that’s a red flag – and an opportunity for you to propose a migration to Airflow.

Checkpoint

1. Name two problems that cron can’t solve but Airflow can.
2. What does “backfill” mean in the context of orchestration?

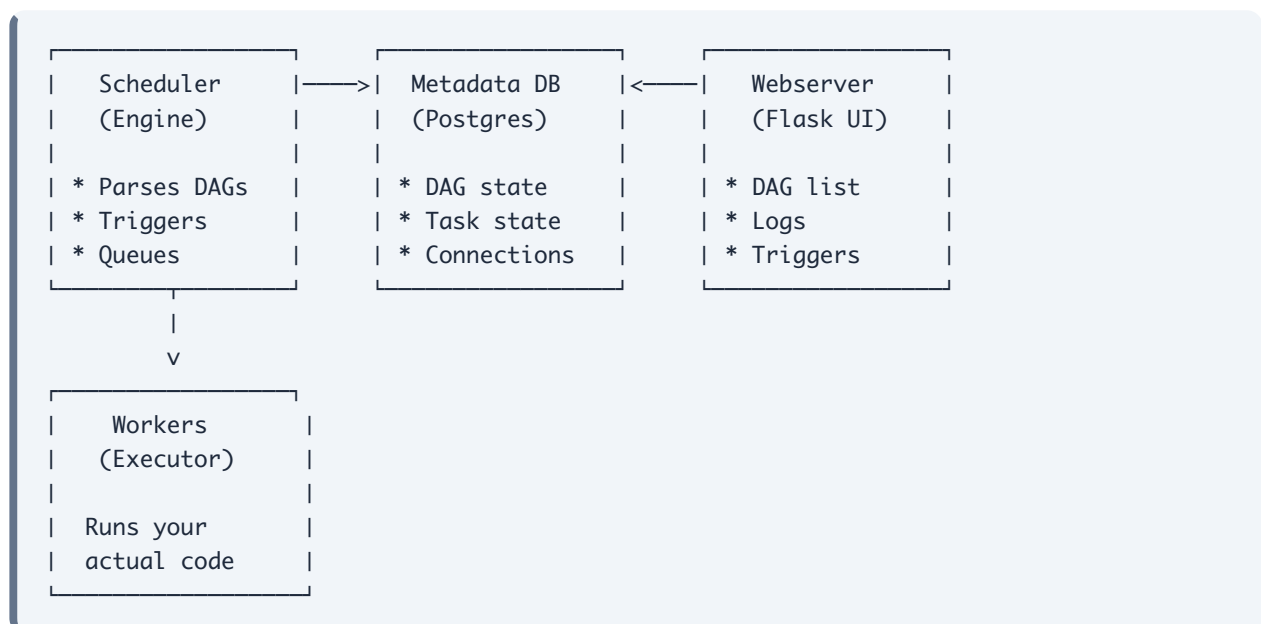
3. Why would you choose Airflow over Prefect or Dagster for a new job?

4.2 Airflow Architecture – Scheduler, Webserver, Workers, Metadata DB

TL;DR - Airflow has four components: Metadata DB, Scheduler, Webserver, and Workers - The Scheduler is the engine – it decides what runs and when - The Webserver is just a UI; it doesn't execute tasks - Use LocalExecutor for learning and small-to-medium production workloads

Before you write a single line of DAG code, you need a mental model of what's actually happening when Airflow runs your pipeline.

The Four Components



1. The Metadata Database (Postgres)

This is Airflow's brain. It stores everything: what DAGs exist, when they last ran, what state each task is in, connections, variables, user info. It's just a Postgres database (or MySQL, but use Postgres).

Every other component reads from and writes to this database. If the metadata DB goes down, Airflow is dead.

2. The Scheduler

This is the engine. The scheduler does three things in a loop: - Scans your DAG files to find new/updated DAGs - Checks if any DAG runs need to be triggered (based on schedule) - Checks if any tasks are ready to run (dependencies met) and queues them

The scheduler is the *only* component that decides what runs and when. It's the most critical piece.

3. The Webserver

This is the UI – the thing you open in your browser. It reads from the metadata DB and shows you DAG status, task logs, Gantt charts, and trigger buttons.

Important: the webserver does NOT run your tasks. It's just a Flask app showing you what's happening. You could turn it off and Airflow would still run pipelines. You just couldn't see them.

4. The Workers (Executor)

This is where your actual code runs. When the scheduler says “run this task,” a worker picks it up and executes it. How workers behave depends on your *executor*:

EXECUTOR	DESCRIPTION	WHEN TO USE
SequentialExecutor	One task at a time	Development only. Never production.
LocalExecutor	Multiple tasks as separate processes	Small-to-medium scale. What we'll use.
CeleryExecutor	Distributes across machines via Redis/RabbitMQ	Production at scale.
KubernetesExecutor	Spins up a K8s pod per task	Modern cloud-native deployments.

At your job: Ask what executor is in use. If someone says SequentialExecutor in production, that's a problem – it runs one task at a time. LocalExecutor is fine for most teams. CeleryExecutor or KubernetesExecutor for large-scale operations.

Setting Up Airflow with Docker Compose

Here's a complete Docker Compose file for a local Airflow environment:

```

# docker-compose.yml
# Airflow with LocalExecutor -- suitable for learning and small-to-medium workloads
version: '3.8'

x-airflow-common: &airflow-common
  image: apache/airflow:2.9.1-python3.11
  environment: &airflow-common-env
  AIRFLOW__CORE__EXECUTOR: LocalExecutor # Multiple tasks in
parallel
  AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
airflow:airflow@postgres/airflow
  AIRFLOW__CORE__FERNET_KEY: '' # Encryption key (set in
prod)
  AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true' # New DAGs start paused
  AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
  AIRFLOW__API__AUTH_BACKENDS:
    'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
  _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-}
volumes:
  - ./dags:/opt/airflow/dags # Your DAG files
  - ./logs:/opt/airflow/logs # Task logs
  - ./plugins:/opt/airflow/plugins # Custom plugins
  - ./data:/opt/airflow/data # Data staging area
user: "${AIRFLOW_UID:-50000}:0"
depends_on:
  postgres:
    condition: service_healthy

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s
    ports:
      - "5432:5432"

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        airflow db migrate
        airflow users create \
          --username admin \

```

```

        --firstname Admin \
        --lastname User \
        --role Admin \
        --email admin@example.com \
        --password admin
depends_on:
  postgres:
    condition: service_healthy

airflow-webserver:
  <<: *airflow-common
  command: webserver
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    airflow-init:
      condition: service_completed_successfully

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  healthcheck:
    test: ["CMD-SHELL", "airflow jobs check --job-type SchedulerJob --hostname $(hostname)"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  depends_on:
    airflow-init:
      condition: service_completed_successfully

volumes:
  postgres-data:

```

Start it up:

```

# Set the Airflow UID (avoids permission issues on mounted volumes)
echo -e "AIRFLOW_UID=$(id -u)" > .env

# Create the required directories
mkdir -p dags logs plugins data

# Start everything
docker compose up -d

# Check that everything is running
docker compose ps

```

Expected output:

NAME	SERVICE	STATUS
airflow-init	airflow-init	exited (0)
airflow-scheduler	airflow-scheduler	running (healthy)
airflow-webserver	airflow-webserver	running (healthy)
postgres	postgres	running (healthy)

Open `http://localhost:8080` – username `admin`, password `admin`. You'll see an empty DAGs page.

Common Mistake

Not setting AIRFLOW_UID – you'll get permission errors on the mounted volumes. Always create the `.env` file first.

Using SequentialExecutor in production – it runs one task at a time. Fine for dev, disaster for prod.

Forgetting `LOAD_EXAMPLES: 'false'` – you'll get 30+ example DAGs cluttering your UI.

Checkpoint

1. Which component decides what tasks to run and when?
2. What happens if the metadata database goes down?
3. Why should you never use SequentialExecutor in production?

4.3 Your First DAG – Tasks, Dependencies, Operators

TL;DR - A DAG is a Directed Acyclic Graph – your pipeline definition - Tasks are units of work; Operators define the *type* of task - The `>>` operator defines dependencies between tasks - Always set `catchup=False` during development

Time to write some code. By the end of this section, you'll have a working DAG in Airflow – not a “hello world,” but an actual pipeline that does something useful.

What a DAG Actually Is

Before you see the code, let's understand the concept. DAG stands for **Directed Acyclic Graph**. Let's break that down:

- **Graph** – a collection of nodes (tasks) connected by edges (dependencies)

- **Directed** – the edges have a direction. Task A runs *before* Task B, not the other way around. (A -> B)
- **Acyclic** – there are no loops. You can't have A -> B -> C -> A. Why? Because if A depends on C, and C depends on B, and B depends on A, nothing can ever start. It's a deadlock.

Why “acyclic” matters in interviews: Interviewers love asking “what does DAG stand for and why does ‘acyclic’ matter?” The answer is: cycles create deadlocks. If Task A waits for Task C, and Task C waits for Task B, and Task B waits for Task A, your pipeline never runs. Airflow validates this at parse time and rejects any DAG with cycles.

Key Concept: DAG Terminology

- **DAG** = Directed Acyclic Graph. Your pipeline definition – what tasks to run and in what order.
- **Task** = A single unit of work. “Extract data from API,” “Transform the data,” “Load into Postgres.”
- **Operator** = The *type* of task. `PythonOperator` runs a Python function. `BashOperator` runs a bash command. `PostgresOperator` runs SQL.
- **Task Instance** = A specific run of a task at a specific time. Your “extract” task on February 19th at 6 AM is one task instance.

Building an ETL DAG

This DAG will: 1. Create a JSON file with sample data (simulating an API extraction) 2. Validate the data quality 3. Transform it to CSV with computed fields 4. Create a Postgres table 5. Send a notification

Save this as `dags/my_first_dag.py`:


```

"""
My First Airflow DAG
Demonstrates: tasks, dependencies, operators, and basic patterns.
"""

from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash import BashOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
import csv
import os

# Default arguments applied to all tasks in this DAG
default_args = {
    "owner": "george",                # Who owns this DAG
    "depends_on_past": False,          # Don't wait for previous runs to succeed
    "email_on_failure": False,        # We'll use callbacks instead
    "email_on_retry": False,
    "retries": 2,                     # Retry failed tasks twice
    "retry_delay": timedelta(minutes=2), # Wait 2 min between retries
}

DATA_PATH = "/opt/airflow/data"

def extract_data(**kwargs):
    """Simulate extracting data from an API.
    In production, this would be requests.get("https://api.example.com/orders")
    """
    import json

    orders = [
        {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"},
        {"order_id": 2, "customer": "Bob", "amount": 149.50, "date": "2026-02-19"},
        {"order_id": 3, "customer": "Charlie", "amount": 29.99, "date": "2026-02-19"},
        {"order_id": 4, "customer": "Diana", "amount": 199.00, "date": "2026-02-19"},
        {"order_id": 5, "customer": "Eve", "amount": 75.25, "date": "2026-02-19"},
    ]

    filepath = os.path.join(DATA_PATH, "raw_orders.json")
    with open(filepath, "w") as f:
        json.dump(orders, f)

    print(f"Extracted {len(orders)} orders to {filepath}")
    return filepath

def validate_data(**kwargs):
    """Check data quality before loading."""
    import json

    filepath = os.path.join(DATA_PATH, "raw_orders.json")
    with open(filepath, "r") as f:
        orders = json.load(f)

```

```

# Quality checks
assert len(orders) > 0, "No orders found!"
for order in orders:
    assert "order_id" in order, f"Missing order_id in {order}"
    assert "amount" in order, f"Missing amount in {order}"
    assert order["amount"] > 0, f"Invalid amount: {order['amount']}"

print(f"Validation passed: {len(orders)} orders, all valid")
return len(orders)

def transform_data(**kwargs):
    """Transform raw data into a clean CSV for loading."""
    import json

    filepath = os.path.join(DATA_PATH, "raw_orders.json")
    with open(filepath, "r") as f:
        orders = json.load(f)

    # Add computed fields
    for order in orders:
        order["amount_with_tax"] = round(order["amount"] * 1.08, 2)
        order["processed_at"] = datetime.now().isoformat()

    # Write as CSV for Postgres COPY
    csv_path = os.path.join(DATA_PATH, "clean_orders.csv")
    with open(csv_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=orders[0].keys())
        writer.writeheader()
        writer.writerows(orders)

    print(f"Transformed {len(orders)} orders -> {csv_path}")
    return csv_path

# Define the DAG
with DAG(
    dag_id="my_first_pipeline",
    default_args=default_args,
    description="Extract, validate, transform, and load order data",
    schedule="@daily",  # Runs once per day
    start_date=datetime(2026, 2, 1),
    catchup=False,  # Don't backfill past dates
    tags=["tutorial", "etl"],
) as dag:

    extract = PythonOperator(
        task_id="extract_data",
        python_callable=extract_data,
    )

    validate = PythonOperator(
        task_id="validate_data",
        python_callable=validate_data,

```

```

    )

    transform = PythonOperator(
        task_id="transform_data",
        python_callable=transform_data,
    )

    create_table = PostgresOperator(
        task_id="create_table",
        postgres_conn_id="postgres_default",
        sql="""
            CREATE TABLE IF NOT EXISTS orders (
                order_id INTEGER PRIMARY KEY,
                customer VARCHAR(100),
                amount DECIMAL(10,2),
                date DATE,
                amount_with_tax DECIMAL(10,2),
                processed_at TIMESTAMP
            );
        """,
    )

    notify = BashOperator(
        task_id="notify_complete",
        bash_command='echo "Pipeline complete at $(date). Orders loaded successfully!"',
    )

    # Define dependencies -- this is the "graph" part of DAG
    extract >> validate >> transform >> create_table >> notify

```

Companion code: See `modules/module-4/starter/dags/my_first_dag.py` for the starter skeleton with TODOs, and `modules/module-4/solution/dags/my_first_dag.py` for the complete solution.

Expected output (in Airflow task logs):

```

[extract_data] Extracted 5 orders to /opt/airflow/data/raw_orders.json
[validate_data] Validation passed: 5 orders, all valid
[transform_data] Transformed 5 orders -> /opt/airflow/data/clean_orders.csv
[notify_complete] Pipeline complete at Thu Feb 19 06:00:00 UTC 2026. Orders loaded successfully!

```

Key Concepts Breakdown

`default_args` – Applied to every task. `retries: 2` means if a task fails, Airflow tries again twice.

`retry_delay` is the wait between retries.

`schedule="@daily"` – Runs once per day. You can also use cron syntax: `"0 6 * * *"` for 6 AM daily.

`catchup=False` – If your `start_date` is in the past and `catchup` is `True`, Airflow tries to run the DAG for every day since `start_date`. That’s usually not what you want during development.

`>>` – The bitshift operator defines dependencies. `extract >> validate` means “run `extract` first, then `validate`.”

Verify Your DAG

```
# Check that Airflow can parse your DAG (catches syntax errors)
docker compose exec airflow-scheduler airflow dags list
```

Expected output:

dag_id	filepath	owner	paused
my_first_pipeline	my_first_dag.py	george	True

Then go to the Airflow UI, find `my_first_pipeline`, unpause it, and trigger it manually. Watch the tasks go green one by one in the Graph view.

Common Mistake

Import errors at the top level – If your DAG file has an import error, the ENTIRE file is silently skipped. Always verify with `airflow dags list`.

Heavy logic at module level – Don’t do API calls or DB queries outside your task functions. The scheduler reads your DAG file every 30 seconds. A `requests.get()` at the top level hits the API every 30 seconds.

Forgetting `catchup=False` – You’ll trigger hundreds of backfill runs and wonder why your Airflow is on fire.

At your job: When you see a DAG file for the first time, look at three things: (1) the schedule, (2) the dependencies (the `>>` chain), and (3) the `default_args` (especially retries and timeout). Those tell you 80% of what you need to know.

Try It Yourself

Modify the DAG to add a sixth task that counts the rows in the CSV file and prints the total revenue. Chain it after `transform` but before `create_table`.

Checkpoint

1. What does “acyclic” mean in DAG, and why does it matter?
2. What happens if you forget to set `catchup=False` with a `start_date` in the past?

3. Where should heavy logic (API calls, DB queries) live – at the module level or inside task functions?
-

4.4 TaskFlow API – The Modern Way to Write DAGs

TL;DR - The TaskFlow API uses `@task` decorators instead of explicit Operator objects - Return values automatically become XComs (cross-task communication) - Dependencies are inferred from function calls – no `>>` needed - XCom is for small data only (configs, paths, counts) – never Data-Frames

The DAG we just wrote works, but it's kind of verbose. Defining `PythonOperator` separately, passing `python_callable`, managing data between tasks with file paths... There's a cleaner way. Airflow 2.0 introduced the TaskFlow API, and once you use it, you'll never go back.

The TaskFlow Rewrite

```

"""
TaskFlow API version of our pipeline.
Cleaner, more Pythonic, and handles data passing automatically.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task
import json
import os

DATA_PATH = "/opt/airflow/data"

@dag(
    dag_id="taskflow_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 2, 1),
    catchup=False,
    default_args={
        "owner": "george",
        "retries": 2,
        "retry_delay": timedelta(minutes=2),
    },
    tags=["tutorial", "taskflow"],
)
def my_taskflow_pipeline():
    """An ETL pipeline using the TaskFlow API."""

    @task()
    def extract() -> list[dict]:
        """Extract orders from source (simulated)."""
        orders = [
            {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"},
            {"order_id": 2, "customer": "Bob", "amount": 149.50, "date": "2026-02-19"},
            {"order_id": 3, "customer": "Charlie", "amount": 29.99, "date": "2026-02-19"},
            {"order_id": 4, "customer": "Diana", "amount": 199.00, "date": "2026-02-19"},
            {"order_id": 5, "customer": "Eve", "amount": 75.25, "date": "2026-02-19"},
        ]
        print(f"Extracted {len(orders)} orders")
        return orders # Automatically stored as XCom

    @task()
    def validate(orders: list[dict]) -> list[dict]:
        """Validate data quality."""
        assert len(orders) > 0, "No orders!"
        for order in orders:
            assert order["amount"] > 0, f"Bad amount: {order['amount']}"
        print(f"Validated {len(orders)} orders -- all clean")
        return orders # Pass along to next task

    @task()
    def transform(orders: list[dict]) -> list[dict]:

```

```

        """Add computed fields."""
        for order in orders:
            order["amount_with_tax"] = round(order["amount"] * 1.08, 2)
            order["processed_at"] = datetime.now().isoformat()
        print(f"Transformed {len(orders)} orders")
        return orders

    @task()
    def load(orders: list[dict]):
        """Load into destination (simulated)."""
        filepath = os.path.join(DATA_PATH, "final_orders.json")
        with open(filepath, "w") as f:
            json.dump(orders, f, indent=2)
        print(f"Loaded {len(orders)} orders to {filepath}")

    # Just call functions -- Airflow infers dependencies from data flow
    raw_data = extract()
    validated_data = validate(raw_data)
    transformed_data = transform(validated_data)
    load(transformed_data)

    # Don't forget this line! Instantiates the DAG.
    my_taskflow_pipeline()

```

Companion code: See [modules/module-4/solution/dags/taskflow_pipeline.py](#) for the complete solution.

Expected output (task logs):

```

[extract] Extracted 5 orders
[validate] Validated 5 orders -- all clean
[transform] Transformed 5 orders
[load] Loaded 5 orders to /opt/airflow/data/final_orders.json

```

Old Style vs. TaskFlow – Side by Side

OLD STYLE	TASKFLOW
Define function separately	Decorate function with <code>@task</code>
Create <code>PythonOperator(python_callable=func)</code>	Just call the function
Manual XCom push/pull	Return values = automatic XCom
<code>extract >> validate >> transform</code>	<code>validate(extract())</code> – implicit

Understanding XCom (Cross-Communication)

Key Concept: XCom

XCom is how tasks pass data to each other. With TaskFlow, return values are automatically serialized to JSON and stored in the metadata database. The next task receives them as function arguments. Simple.

Size limits matter: XCom stores data in Postgres. Small data (configs, file paths, row counts, small lists) is perfect. Large data (DataFrames, millions of rows) will either crash your metadata DB or make it enormous. For large datasets, write to a file and pass the *path* via XCom.

Common Mistake

Passing DataFrames through XCom – A 1GB DataFrame serialized to JSON in Postgres is a recipe for disaster. Pass file paths instead.

Forgetting to instantiate the DAG – See that last line `my_taskflow_pipeline()` ? Without it, the DAG won't appear. Easy to miss.

At your job: Most modern Airflow codebases use TaskFlow for Python-based tasks. If you see a legacy codebase with the old `PythonOperator` style, it works fine – but new DAGs should use TaskFlow.

Checkpoint

1. What does `@task()` replace in the old-style DAG definition?
2. Why shouldn't you pass a Pandas DataFrame through XCom?
3. What happens if you forget the `my_taskflow_pipeline()` call at the bottom?

4.5 Connections, Variables, and Secrets Management

TL;DR - Use **Connections** for external credentials (databases, APIs, cloud) - Use **Variables** for non-sensitive config (thresholds, paths, flags) - Never call `Variable.get()` at the module level – put it inside tasks - In production, use a secrets backend (AWS Secrets Manager, Vault, etc.)

Pop quiz: what's the fastest way to get fired as a data engineer? Hardcode your database password in a DAG file and push it to GitHub. Let's talk about how to manage credentials properly.

Connections – External System Credentials

```
# Set via CLI
docker compose exec airflow-scheduler airflow connections add 'postgres_warehouse' \
  --conn-type 'postgres' \
  --conn-host 'postgres' \
  --conn-schema 'warehouse' \
  --conn-login 'warehouse_user' \
  --conn-password 'warehouse_pass' \
  --conn-port 5432

# Or via environment variable (useful for Docker/K8s)
export AIRFLOW_CONN_POSTGRES_WAREHOUSE='postgresql://warehouse_user:warehouse_pass@postgres:5432/warehouse'
```

Then use it in your DAG:

```
from airflow.providers.postgres.hooks.postgres import PostgresHook

@task()
def query_warehouse():
    # The Hook looks up credentials from the connection by ID
    hook = PostgresHook(postgres_conn_id="postgres_warehouse")
    df = hook.get_pandas_df("SELECT COUNT(*) as cnt FROM orders")
    print(f"Row count: {df['cnt'].iloc[0]}")
    return df['cnt'].iloc[0]
```

Variables – Non-Sensitive Configuration

```
from airflow.models import Variable

@task()
def check_threshold():
    # Variable.get() queries the metadata DB
    threshold = int(Variable.get("my_threshold", default_var=50))
    print(f"Using threshold: {threshold}")
```

Set variables via CLI: `airflow variables set my_threshold 100`

Common Mistake: `Variable.get()` at Module Level

The scheduler parses DAG files every 30 seconds. If `Variable.get()` is at the top level, that's a database query every 30 seconds per DAG.

```
``python
```

BAD – runs every time scheduler parses the file (every 30 seconds)

```
threshold = Variable.get("my_threshold")
@task() def my_task(): # GOOD – runs only when the task executes threshold =
Variable.get("my_threshold") ``
```

Secrets Backend – For Production

In production, use an external secrets manager instead of storing credentials in the Airflow metadata DB:

```
# In airflow.cfg or environment variables:
AIRFLOW__SECRETS__BACKEND=airflow.providers.amazon.aws.secrets.secrets_manager.SecretsM
anagerBackend
AIRFLOW__SECRETS__BACKEND_KWARGS={"connections_prefix": "airflow/connections", "variabl
es_prefix": "airflow/variables"}
```

This pulls secrets from AWS Secrets Manager, HashiCorp Vault, or GCP Secret Manager transparently. Your DAG code doesn't change – Airflow just looks up the credentials from a different place.

Pro Tip: Use the “Test” button in the Airflow UI when setting up connections. It validates the connection immediately and saves you debugging time later.

At your job: On day one, ask where credentials are stored. If they're in environment variables on the Airflow server, that's okay for small teams. If they're hardcoded in DAG files, that's a security incident waiting to happen. Push for a secrets backend.

Checkpoint

1. What's the difference between a Connection and a Variable in Airflow?
 2. Why is `Variable.get()` at the module level dangerous?
 3. What's a secrets backend, and when should you use one?
-

4.6 Error Handling – Retries, SLAs, Alerting

TL;DR - Always configure retries with exponential backoff for external calls - Set `execution_timeout` on every task to prevent stuck jobs - Use callbacks for Slack/PagerDuty alerts on final failure - SLA monitoring alerts you if a DAG isn't done by a deadline

Data pipelines break. APIs go down, databases timeout, files are malformed, services hit rate limits. The question isn't IF your pipeline will fail, it's HOW it handles failure.

Retries with Exponential Backoff

```
@task(
    retries=3,                                # Try 3 more times after first failure
    retry_delay=timedelta(minutes=5),         # Start with 5 min delay
    retry_exponential_backoff=True,           # 5 min -> 10 min -> 20 min
    max_retry_delay=timedelta(minutes=30),
)
def flaky_api_call():
    """Call an API that sometimes fails."""
    import requests
    response = requests.get("https://api.example.com/data", timeout=30)
    response.raise_for_status() # Raises on 4xx/5xx
    return response.json()
```

Pro Tip: `retry_exponential_backoff=True` is essential for API calls. If an API is overloaded, hammering it every 5 minutes makes things worse. Exponential backoff gives it time to recover.

Timeouts

```
@task(
    execution_timeout=timedelta(minutes=30), # Kill task if it runs > 30 min
)
def long_running_transform():
    """Transform that should complete in 30 minutes."""
    # ... processing code ...
    pass
```

Without a timeout, a stuck task can block your entire pipeline for hours.

Alerting with Callbacks

This is how you actually get notified when things go wrong:

```

from airflow.decorators import dag, task
from airflow.models import Variable
from datetime import datetime, timedelta
import requests

def send_slack_alert(context):
    """Send a Slack notification on task failure."""
    task_instance = context.get("task_instance")
    dag_id = context.get("dag").dag_id
    task_id = task_instance.task_id
    log_url = task_instance.log_url

    message = (
        f"*Task Failed*\n"
        f"DAG: `{dag_id}`\n"
        f"Task: `{task_id}`\n"
        f"Execution: {context.get('execution_date')}\n"
        f"<{log_url}|View Logs>"
    )

    webhook_url = Variable.get("slack_webhook_url")
    requests.post(webhook_url, json={"text": message})

def send_success_notification(context):
    """Notify on DAG success."""
    dag_id = context.get("dag").dag_id
    webhook_url = Variable.get("slack_webhook_url")
    requests.post(webhook_url, json={
        "text": f"DAG `{dag_id}` completed successfully!"
    })

@dag(
    dag_id="alerting_pipeline",
    schedule="@daily",
    start_date=datetime(2026, 2, 1),
    catchup=False,
    default_args={
        "owner": "george",
        "retries": 3,
        "retry_delay": timedelta(minutes=5),
        "on_failure_callback": send_slack_alert,      # Per-task failure alert
    },
    on_success_callback=send_success_notification,  # DAG-level success
)
def alerting_pipeline():

    @task()
    def risky_extraction():
        """This might fail -- and that's okay, we have retries and alerts."""
        import random
        if random.random() < 0.3:

```

```

        raise Exception("API rate limited! (simulated)")
    return {"status": "success", "rows": 1000}

@task()
def process(data: dict):
    print(f"Processing {data['rows']} rows")

    data = risky_extraction()
    process(data)

alerting_pipeline()

```

SLA Monitoring

SLAs alert you when a DAG isn't done by a certain time – critical for the CFO's morning report that better be ready by 7 AM:

```

@task(sla=timedelta(hours=1)) # Must complete within 1 hour of DAG start
def critical_task():
    pass

```

Common Mistake

No retries on API calls – Always add retries. External APIs are unreliable.

No execution_timeout – A stuck task can silently block everything for hours.

Alerting on every retry – Don't spam Slack on retries. The `on_failure_callback` fires only on *final* failure (after all retries), which is the right behavior.

At your job: The first thing to set up on any production DAG is alerting. A pipeline that fails silently is worse than one that fails loudly. Slack alerts with log URLs are the minimum. PagerDuty for critical pipelines.

Checkpoint

1. What does `retry_exponential_backoff=True` do, and why is it useful for API calls?
2. What's the difference between `on_failure_callback` on `default_args` vs. on the `@dag` decorator?
3. Why should you always set `execution_timeout`?

4.7 Dynamic DAGs and DAG Factories

TL;DR - DAG factories generate multiple DAGs from a config – no copy-pasting - Dynamic task mapping (`.expand()`) creates parallel tasks at runtime - Use factory functions (not bare loops) to avoid Python closure issues - Don't generate more than ~200 DAGs from a factory or you'll slow the scheduler

If you work at a company with 50 clients and each one gets the same ETL pipeline with different configs, you're not going to copy-paste a DAG file 50 times. You're going to use a DAG factory.

Pattern 1: Config-Driven DAGs

```

"""
dags/dag_factory.py
Generate one DAG per client from a configuration list.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task

# Config could come from a file, database, or API
CLIENTS = [
    {
        "client_id": "acme_corp",
        "api_url": "https://api.acme.com/v1/orders",
        "schedule": "0 6 * * *",          # 6 AM daily
        "owner": "george",
    },
    {
        "client_id": "globex",
        "api_url": "https://api.globex.com/v2/data",
        "schedule": "0 8 * * *",          # 8 AM daily
        "owner": "george",
    },
    {
        "client_id": "initech",
        "api_url": "https://initech.io/api/reports",
        "schedule": "0 7 * * 1-5",        # 7 AM weekdays only
        "owner": "george",
    },
]

def create_client_dag(client_config: dict):
    """Factory function that creates a DAG for a single client."""

    client_id = client_config["client_id"]

    @dag(
        dag_id=f"client_pipeline_{client_id}",
        schedule=client_config["schedule"],
        start_date=datetime(2026, 2, 1),
        catchup=False,
        default_args={
            "owner": client_config["owner"],
            "retries": 3,
            "retry_delay": timedelta(minutes=5),
        },
        tags=["client", client_id],
    )
    def client_pipeline():

        @task()
        def extract(api_url: str) -> dict:
            print(f"Extracting from {api_url}")
            return {"client": client_id, "rows": 100} # Simulated

```

```

@task()
def transform(data: dict) -> dict:
    data["transformed"] = True
    data["processed_at"] = datetime.now().isoformat()
    return data

@task()
def load(data: dict):
    print(f"Loading {data['rows']} rows for {data['client']}")

    raw = extract(api_url=client_config["api_url"])
    clean = transform(raw)
    load(clean)

return client_pipeline()

# Generate a DAG for each client
for client in CLIENTS:
    create_client_dag(client)

```

When Airflow parses this file, it creates three DAGs: `client_pipeline_acme_corp`, `client_pipeline_globex`, and `client_pipeline_initech`. Each with its own schedule and config.

Companion code: See `modules/module-4/solution/dags/dag_factory.py` for the complete solution.

Pattern 2: Dynamic Task Mapping (Airflow 2.3+)

This is even more powerful – expand tasks dynamically at runtime:

```

@dag(
    dag_id="dynamic_mapping_example",
    schedule="@daily",
    start_date=datetime(2026, 2, 1),
    catchup=False,
)
def dynamic_pipeline():

    @task()
    def get_files() -> list[str]:
        """Discover files to process -- could be from S3, API, etc."""
        return ["file_a.csv", "file_b.csv", "file_c.csv"]

    @task()
    def process_file(filename: str) -> dict:
        """Process a single file."""
        print(f"Processing {filename}")
        return {"file": filename, "rows": 1000}

    @task()
    def summarize(results: list[dict]):
        """Aggregate results."""
        total = sum(r["rows"] for r in results)
        print(f"Total rows processed: {total}")

    files = get_files()
    # .expand() creates one task instance per item in the list -- in parallel!
    results = process_file.expand(filename=files)
    summarize(results)

dynamic_pipeline()

```

The `.expand()` call is the magic. If `get_files()` returns 3 files, Airflow creates 3 parallel `process_file` tasks. If tomorrow it returns 10 files, you get 10 tasks. No code changes needed.

Common Mistake

Creating DAGs in a loop with mutable variables – Classic Python closure issue. Always use a factory function (like `create_client_dag` above) to capture the config properly.

Too many dynamic DAGs – 500 DAGs from a factory will slow down the scheduler. If you have that many, consider grouping them.

At your job: DAG factories are extremely common at companies with multi-tenant data – one client per DAG, one data source per DAG, etc. If you see 50 nearly-identical DAG files, suggest a factory pattern.

Checkpoint

1. Why do you need a factory *function* instead of creating DAGs directly in a loop?

2. What does `.expand()` do in dynamic task mapping?
 3. At what point should you worry about having too many factory-generated DAGs?
-

4.8 Sensors and Triggers – Waiting for Data and Events

TL;DR - Sensors wait for a condition to be true before allowing downstream tasks to run - Always use `mode="reschedule"` or `deferrable=True` to avoid hogging worker slots - Always set a `timeout` - sensors without timeouts wait forever - `soft_fail=True` marks as skipped instead of failed on timeout

What if your pipeline shouldn't run at 6 AM – it should run when the data is *ready*? Maybe a vendor uploads a file to S3 sometime between 6 AM and 9 AM. A schedule won't work. You need a sensor.

Key Concept: Sensors

A sensor is a special type of operator that periodically checks ("pokes") for a condition. When the condition is met, it succeeds and allows downstream tasks to proceed. Think of it as a bouncer checking IDs – nobody gets in until the condition is verified.

```

from airflow.sensors.filesystem import FileSensor
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import datetime, timedelta
from airflow.decorators import dag, task

@dag(
    dag_id="sensor_pipeline",
    schedule="0 6 * * *",
    start_date=datetime(2026, 2, 1),
    catchup=False,
)
def sensor_pipeline():

    # Wait for a local file to appear
    wait_for_file = FileSensor(
        task_id="wait_for_file",
        filepath="/opt/airflow/data/incoming/daily_feed.csv",
        poke_interval=60,                # Check every 60 seconds
        timeout=3 * 60 * 60,            # Give up after 3 hours
        mode="reschedule",              # Free up the worker slot while waiting
        soft_fail=True,                 # Mark as skipped (not failed) on timeout
    )

    # Wait for a file in S3
    wait_for_s3 = S3KeySensor(
        task_id="wait_for_s3_file",
        bucket_name="my-data-lake",
        bucket_key="raw/{{ ds }}/orders.parquet", # Jinja template for date
        aws_conn_id="aws_default",
        poke_interval=120,
        timeout=4 * 60 * 60,
        mode="reschedule",
    )

    # Wait for another DAG to finish
    wait_for_upstream = ExternalTaskSensor(
        task_id="wait_for_ingestion_dag",
        external_dag_id="data_ingestion_pipeline",
        external_task_id=None,           # None = wait for the entire DAG
        timeout=2 * 60 * 60,
        mode="reschedule",
    )

    @task()
    def process():
        print("All conditions met -- processing!")

    [wait_for_file, wait_for_s3, wait_for_upstream] >> process()

sensor_pipeline()

```

Key Parameters

- `poke_interval` – How often to check (seconds). Don't set too low or you'll hammer the external system.
- `timeout` – Maximum wait time. After this, the sensor fails (or skips if `soft_fail=True`).
- `mode="reschedule"` – Frees the worker slot between checks. The default `poke` mode keeps the slot occupied the entire time. Always use `reschedule` unless you have a specific reason not to.

Deferrable Operators (Airflow 2.6+)

Even better than `reschedule` – deferrable operators use almost zero resources while waiting:

```
wait_for_s3 = S3KeySensor(  
    task_id="wait_for_s3",  
    bucket_name="my-bucket",  
    bucket_key="data/{{ ds }}/file.parquet",  
    deferrable=True, # Uses the Triggerer -- minimal resource usage  
)
```

Common Mistake

`mode="poke"` **on long-running sensors** – Occupies a worker slot the entire time. A sensor waiting 3 hours in poke mode is a wasted worker for 3 hours.

No timeout – A sensor without a timeout waits forever. Always set one.

Poke interval too short – Checking S3 every 5 seconds is unnecessary and costly (S3 LIST operations aren't free at scale).

At your job: Sensors are how you coordinate pipelines that depend on external events. The most common pattern: an `ExternalTaskSensor` that waits for an upstream ingestion DAG to complete before running transformations.

Checkpoint

1. What's the difference between `mode="poke"` and `mode="reschedule"` ?
2. Why should you always set a `timeout` on sensors?
3. What does `deferrable=True` do that `reschedule` mode doesn't?

4.9 Testing DAGs Locally

TL;DR - DAG integrity tests (does it parse?) should run in CI/CD – catches 80% of deployment issues - Separate business logic from DAG files so you can unit test normally - Use `airflow dags test` for free integration testing - Don't test Airflow internals – test YOUR logic

Would you deploy application code without tests? Then why do so many people deploy DAGs without tests? It's easier than you think.

Level 1: DAG Integrity Tests

Does the DAG even parse? Run this in CI/CD:

```
# tests/test_dag_integrity.py
import pytest
from airflow.models import DagBag

@pytest.fixture()
def dagbag():
    return DagBag(dag_folder="dags/", include_examples=False)

def test_no_import_errors(dagbag):
    """Verify no DAG import errors."""
    assert len(dagbag.import_errors) == 0, f"Import errors: {dagbag.import_errors}"

def test_dag_count(dagbag):
    """Verify expected number of DAGs."""
    assert len(dagbag.dags) >= 1, "No DAGs found!"

@pytest.mark.parametrize("dag_id", [
    "my_first_pipeline",
    "taskflow_pipeline",
])
def test_dag_exists(dagbag, dag_id):
    """Check that expected DAGs are loaded."""
    assert dag_id in dagbag.dags, f"DAG '{dag_id}' not found"

def test_no_cycles(dagbag):
    """Verify no DAGs have circular dependencies."""
    for dag_id, dag in dagbag.dags.items():
        assert dag.topological_sort() # Raises if there's a cycle
```


Companion code: See `modules/module-4/solution/tests/test_dag_integrity.py` for the complete test suite.

Level 2: Task Logic Tests

Pro Tip: Separate Logic from DAGs

The biggest testing tip: **don't put business logic inside your DAG file.** Put it in separate Python modules, test those normally, and have your DAG tasks just call the functions.

```
project/
|-- dags/
|   |-- my_pipeline.py           # DAG definition -- thin, just wiring
|-- src/
|   |-- extractors.py           # API calls, file reads
|   |-- transforms.py           # Business logic
|   |-- validators.py           # Data quality checks
|   |-- loaders.py              # Database writes
|-- tests/
|   |-- test_dag_integrity.py    # DAGs parse correctly
|   |-- test_transforms.py       # Unit tests for logic
|   |-- test_validators.py       # Unit tests for validation
```

```
# src/transforms.py
def calculate_tax(amount: float, rate: float = 0.08) -> float:
    """Calculate amount with tax."""
    if amount < 0:
        raise ValueError(f"Amount cannot be negative: {amount}")
    return round(amount * (1 + rate), 2)

def validate_order(order: dict) -> bool:
    """Validate a single order record."""
    required_fields = ["order_id", "customer", "amount", "date"]
    return all(field in order for field in required_fields) and order["amount"] > 0
```

```
# tests/test_transforms.py
import pytest
from src.transforms import calculate_tax, validate_order

def test_calculate_tax():
    assert calculate_tax(100.00) == 108.00
    assert calculate_tax(0) == 0.00

def test_calculate_tax_negative():
    with pytest.raises(ValueError):
        calculate_tax(-50.00)

def test_validate_order_valid():
    order = {"order_id": 1, "customer": "Alice", "amount": 99.99, "date": "2026-02-19"}
    assert validate_order(order) is True

def test_validate_order_missing_field():
    order = {"order_id": 1, "customer": "Alice"}
    assert validate_order(order) is False

def test_validate_order_negative_amount():
    order = {"order_id": 1, "customer": "Alice", "amount": -10, "date": "2026-02-19"}
    assert validate_order(order) is False
```

Level 3: Integration Tests

```
# Test a specific task
docker compose exec airflow-scheduler \
    airflow tasks test my_first_pipeline extract_data 2026-02-19

# Test the whole DAG (one-off run without the scheduler)
docker compose exec airflow-scheduler \
    airflow dags test my_first_pipeline 2026-02-19
```

Common Mistake

No tests at all – At minimum, run DAG integrity tests in CI. Takes 30 seconds, catches 80% of deployment issues.

Testing Airflow internals – Don't test that `PythonOperator` works. Airflow already tests that. Test YOUR business logic.

At your job: Push for DAG integrity tests in CI from day one. It's the single highest-value, lowest-effort testing investment you can make. One import error in a DAG file can silently break an entire pipeline.

Checkpoint

1. What do DAG integrity tests verify?
 2. Why should business logic live in separate modules, not inside DAG files?
 3. What's the difference between `airflow tasks test` and `airflow dags test`?
-

4.10 Airflow in Production – Best Practices at Scale

TL;DR - Deploy DAGs via Git + CI/CD – never edit files directly on the server - Every task must be idempotent (safe to run twice) - Use pools to prevent resource exhaustion on shared systems - Monitor the scheduler heartbeat – if it dies, nothing runs

Running Airflow on your laptop is easy. Running it in production where hundreds of DAGs execute reliably every day – that's where the real lessons are.

1. Git-Based DAG Deployment

Never edit DAG files directly on the Airflow server:

```
# .github/workflows/deploy-dags.yml
name: DAG CI/CD

on:
  push:
    branches: [main]
    paths: ["dags/**", "src/**", "tests/**"]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install apache-airflow==2.9.0 pytest
      - run: python -m pytest tests/ -v

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - run: echo "Deploy DAGs to production (replace with your deploy command)"
```

Companion code: See `modules/module-4/solution/.github/workflows/deploy-dags.yml` for the complete CI/CD workflow.

2. Resource Management with Pools

```
# Set pool slots to prevent resource exhaustion
@task(pool="api_calls", pool_slots=1)
def call_external_api():
    """Only N concurrent API calls across all DAGs."""
    pass

# Set priority for important DAGs
@task(priority_weight=10) # Higher = runs first when resources are scarce
def critical_transform():
    pass
```

Pools are essential in production. If you have 20 DAGs that all call the same API, you'll rate-limit yourself instantly without pools.

3. Idempotency – The Golden Rule

Key Concept: Idempotency

Every task should be **idempotent** – running it twice with the same inputs produces the same output. Airflow WILL retry tasks. If your task inserts rows without checking for duplicates, you'll get double data.

```
@task()
def idempotent_load(data: list[dict], execution_date: str):
    """Use MERGE/upsert, not INSERT, for idempotency."""
    hook = PostgresHook(postgres_conn_id="warehouse")

    # Delete-then-insert (atomic replace for a partition)
    hook.run(f"DELETE FROM orders WHERE date = '{execution_date}'")
    # ... insert new data ...

    # Or use upsert
    # INSERT INTO orders ... ON CONFLICT (order_id) DO UPDATE SET ...
```

Why this matters: Imagine your extract task succeeds but the transform fails. Airflow retries the entire chain. If your extract task appends data instead of replacing it, you now have duplicate rows. Idempotency prevents this.

4. Templating with Jinja

Airflow passes execution context as Jinja templates. Use them for date partitioning:

```
from airflow.operators.bash import BashOperator

# {{ ds }} = execution date (YYYY-MM-DD)
# {{ ds_nodash }} = YYYYMMDD
# {{ data_interval_start }} / {{ data_interval_end }}
extract = BashOperator(
    task_id="extract",
    bash_command="python extract.py --date {{ ds }} --output /data/raw/"
    "{{ ds_nodash }}/",
)
```

5. Monitoring

Set up StatsD + Grafana for Airflow metrics: - DAG/task success/failure rates - Task duration trends (is something getting slower?) - Scheduler heartbeat (is it alive?) - Queue depth (are tasks piling up?)

Production Checklist

- [] LocalExecutor **or** CeleryExecutor (never Sequential)
- [] External Postgres **for** metadata (**not** SQLite)
- [] Secrets backend (AWS SM, Vault, etc.)
- [] Git-based DAG deployment with CI/CD
- [] DAG integrity tests in CI
- [] All tasks are idempotent
- [] Retries + timeouts on every **task**
- [] Pools **for** shared resources
- [] Alerting on failure (Slack/PagerDuty)
- [] Monitoring (StatsD + Grafana)
- [] Log rotation configured
- [] Airflow upgraded regularly (security patches)

Common Mistake

Not making tasks idempotent – The #1 cause of duplicate data in production.

Not monitoring the scheduler – If the scheduler dies, nothing runs. Monitor its heartbeat.

Massive DAGs – If a DAG has 100+ tasks, break it into smaller DAGs connected by sensors.

At your job: When you inherit an Airflow deployment, walk through this checklist. Most teams have 3-4 items missing. Fixing them before the next incident is how you build credibility fast.

Checkpoint

1. Why must every Airflow task be idempotent?
2. What's the purpose of pools in Airflow?
3. What happens if the Airflow scheduler goes down?

Common Airflow Gotchas – A Reference

These are the mistakes that bite everyone at least once. Keep this list handy.

GOTCHA	WHAT HAPPENS	FIX
Top-level code in DAGs	<code>requests.get()</code> called every 30s by scheduler	Move all logic inside task functions
<code>Variable.get()</code> at module level	DB query every 30s per DAG file	Move inside <code>@task</code> functions
Forgetting <code>catchup=False</code>	Hundreds of backfill runs on deploy	Set <code>catchup=False</code> during development
No <code>execution_timeout</code>	Stuck task blocks pipeline for hours	Set timeout on every task
DataFrames in XCom	Metadata DB bloats, performance degrades	Write to file, pass path via XCom
Import errors in DAG file	DAG silently disappears	Run <code>airflow dags list</code> after changes
DAGs in a loop without factory function	Python closure captures last iteration's value	Use a factory function
<code>mode="poke"</code> on sensors	Worker slot occupied for hours	Use <code>mode="reschedule"</code> or <code>deferrable=True</code>
INSERT without idempotency	Duplicate rows on retry	Use UPSERT or DELETE-then-INSERT
Missing <code>my_pipeline()</code> call	TaskFlow DAG doesn't appear	Always call the decorated function

Module 4 Project: Multi-Step Production ETL Pipeline

Build a production-grade e-commerce ETL pipeline with extraction, validation, transformation, loading, and alerting.

Project Overview

Your pipeline will: 1. Check for new data via a sensor 2. Extract data from a simulated API 3. Validate data quality 4. Transform using computed fields 5. Load to a warehouse table (with upserts for idempotency) 6. Run post-load quality checks 7. Send notification on success/failure

Step 1: Set Up the Project Structure

```
module-4-project/  
|-- docker-compose.yaml      # Airflow + Postgres (from Section 4.2)  
|-- dags/  
|   |-- ecommerce_pipeline.py # Your main DAG  
|-- src/  
|   |-- __init__.py  
|   |-- extractors.py         # API extraction logic  
|   |-- validators.py         # Data quality checks  
|   |-- transforms.py         # Transform functions  
|-- sql/  
|   |-- create_tables.sql  
|-- tests/  
|   |-- test_dag_integrity.py  
|   |-- test_validators.py  
|   |-- test_transforms.py  
|-- data/                     # Mounted volume for file staging  
|-- .env
```


Step 2: Create the Warehouse Tables

```
-- sql/create_tables.sql
CREATE TABLE IF NOT EXISTS raw_orders (
  order_id INTEGER,
  customer_id INTEGER,
  product_id INTEGER,
  quantity INTEGER,
  unit_price DECIMAL(10,2),
  order_date DATE,
  status VARCHAR(20),
  ingested_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS dim_customers (
  customer_id INTEGER PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  segment VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS fact_orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER REFERENCES dim_customers(customer_id),
  product_id INTEGER,
  quantity INTEGER,
  unit_price DECIMAL(10,2),
  total_amount DECIMAL(10,2),
  tax_amount DECIMAL(10,2),
  order_date DATE,
  status VARCHAR(20),
  processed_at TIMESTAMP DEFAULT NOW()
);
```

Companion code: See `modules/module-4/starter/sql/create_tables.sql` for the starter SQL.

Step 3: Build the Extraction Logic

```
# src/extractors.py
"""Simulated API extraction -- in production, this would call an actual API."""

import random
from datetime import datetime

def extract_orders(execution_date: str) -> list[dict]:
    """Simulate extracting orders for a given date."""
    customers = list(range(1, 21))
    products = list(range(100, 120))
    statuses = ["completed", "pending", "shipped", "cancelled"]

    num_orders = random.randint(10, 50)
    orders = []

    for i in range(num_orders):
        orders.append({
            "order_id": int(datetime.now().timestamp() * 1000) + i,
            "customer_id": random.choice(customers),
            "product_id": random.choice(products),
            "quantity": random.randint(1, 5),
            "unit_price": round(random.uniform(9.99, 199.99), 2),
            "order_date": execution_date,
            "status": random.choice(statuses),
        })

    return orders

def extract_customers() -> list[dict]:
    """Simulate customer dimension data."""
    segments = ["premium", "standard", "basic"]
    return [
        {
            "customer_id": i,
            "name": f"Customer_{i}",
            "email": f"customer_{i}@example.com",
            "segment": segments[i % 3],
        }
        for i in range(1, 21)
    ]
```

Step 4: Build the Validation Logic

```

# src/validators.py
"""Data quality validation functions."""

class DataQualityError(Exception):
    """Raised when data quality checks fail."""
    pass

def validate_orders(orders: list[dict]) -> list[dict]:
    """Validate order data. Returns valid orders, raises on critical issues."""
    if not orders:
        raise DataQualityError("No orders received -- extraction may have failed")

    required_fields = ["order_id", "customer_id", "product_id",
                       "quantity", "unit_price", "order_date"]
    valid_statuses = {"completed", "pending", "shipped", "cancelled"}

    valid_orders = []
    issues = []

    for order in orders:
        # Check required fields
        missing = [f for f in required_fields if f not in order]
        if missing:
            issues.append(f"Order {order.get('order_id', '?')}: missing {missing}")
            continue

        # Check value ranges
        if order["quantity"] <= 0:
            issues.append(f"Order {order['order_id']}: invalid quantity {order['quantity']}")
            continue

        if order["unit_price"] <= 0:
            issues.append(f"Order {order['order_id']}: invalid price {order['unit_price']}")
            continue

        if order.get("status") not in valid_statuses:
            issues.append(f"Order {order['order_id']}: unknown status {order.get('status')}")
            continue

        valid_orders.append(order)

    # Log issues
    if issues:
        print(f"Found {len(issues)} data quality issues:")
        for issue in issues[:10]:
            print(f"  - {issue}")

    # Fail if more than 20% of records are bad
    error_rate = len(issues) / len(orders) if orders else 0

```

```
if error_rate > 0.2:
    raise DataQualityError(
        f"Error rate too high: {error_rate:.1%} ({len(issues)}/{len(orders)}"
        f"records)"
    )

print(f"Validation passed: {len(valid_orders)}/{len(orders)} orders valid")
return valid_orders
```

Step 5: Build the DAG

```

# dags/ecommerce_pipeline.py
"""
E-Commerce ETL Pipeline
=====
Production-grade pipeline: extract, validate, transform, load
with error handling and post-load quality checks.
"""

from datetime import datetime, timedelta
from airflow.decorators import dag, task
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.providers.postgres.operators.postgres import PostgresOperator
from airflow.sensors.filesystem import FileSensor
import os
import sys

# Add src to path so we can import our modules
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "src"))

def on_failure(context):
    """Called when any task fails (after retries exhausted)."""
    ti = context["task_instance"]
    print(f"ALERT: Task {ti.task_id} in DAG {ti.dag_id} failed!")
    print(f"  Execution date: {context['ds']}")
    print(f"  Log URL: {ti.log_url}")
    # In production: send to Slack, PagerDuty, etc.

def on_success(context):
    """Called when the entire DAG succeeds."""
    print(f"DAG {context['dag'].dag_id} completed for {context['ds']}")

@dag(
    dag_id="ecommerce_etl_pipeline",
    description="Extract, validate, transform, and load e-commerce order data",
    schedule="0 6 * * *",
    start_date=datetime(2026, 2, 1),
    catchup=False,
    default_args={
        "owner": "george",
        "retries": 3,
        "retry_delay": timedelta(minutes=2),
        "retry_exponential_backoff": True,
        "max_retry_delay": timedelta(minutes=15),
        "execution_timeout": timedelta(minutes=30),
        "on_failure_callback": on_failure,
    },
    on_success_callback=on_success,
    tags=["ecommerce", "etl", "production"],
)
def ecommerce_pipeline():
    @task()

```

```

def extract_orders(execution_date: str = "{{ ds }}") -> list[dict]:
    from extractors import extract_orders as _extract
    orders = _extract(execution_date)
    print(f"Extracted {len(orders)} orders for {execution_date}")
    return orders

@task()
def extract_customers() -> list[dict]:
    from extractors import extract_customers as _extract
    customers = _extract()
    print(f"Extracted {len(customers)} customers")
    return customers

@task()
def validate(orders: list[dict]) -> list[dict]:
    from validators import validate_orders
    return validate_orders(orders)

@task()
def transform(orders: list[dict]) -> list[dict]:
    for order in orders:
        order["total_amount"] = round(order["quantity"] * order["unit_price"], 2)
        order["tax_amount"] = round(order["total_amount"] * 0.08, 2)
        order["processed_at"] = datetime.now().isoformat()
    print(f"Transformed {len(orders)} orders")
    return orders

@task()
def load_orders(orders: list[dict], execution_date: str = "{{ ds }}"):
    hook = PostgresHook(postgres_conn_id="postgres_default")
    # Idempotent: delete-then-insert for this date partition
    hook.run(f"DELETE FROM fact_orders WHERE order_date = '{execution_date}'")
    for order in orders:
        hook.run(
            """INSERT INTO fact_orders
            (order_id, customer_id, product_id, quantity, unit_price,
             total_amount, tax_amount, order_date, status)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)"""
            ,
            parameters=(
                order["order_id"], order["customer_id"], order["product_id"],
                order["quantity"], order["unit_price"], order["total_amount"],
                order["tax_amount"], order["order_date"], order["status"],
            )
        )
    print(f"Loaded {len(orders)} orders for {execution_date}")

@task()
def post_load_checks(execution_date: str = "{{ ds }}"):
    hook = PostgresHook(postgres_conn_id="postgres_default")
    df = hook.get_pandas_df(
        f"SELECT COUNT(*) as cnt FROM fact_orders WHERE order_date = '{execution_date}'"
    )
    count = df['cnt'].iloc[0]
    print(f"Post-load check: {count} rows for {execution_date}")

```



```
    assert count > 0, f"No rows loaded for {execution_date}!"

# Wire it up
raw_orders = extract_orders()
customers = extract_customers()
valid_orders = validate(raw_orders)
transformed = transform(valid_orders)
load_orders(transformed) >> post_load_checks()

ecommerce_pipeline()
```

Companion code: See `modules/module-4/solution/dags/ecommerce_pipeline.py` for the complete production DAG.

Evaluation Criteria

- All tasks have retries and `execution_timeout`
- Load task is idempotent (safe to re-run)
- Validation rejects bad records with clear error messages
- Business logic lives in `src/`, not in the DAG file
- DAG integrity tests pass
- Alerting callbacks are configured
- Code follows the project structure from Section 4.9

What's Next

You now know how to orchestrate data pipelines with Airflow – from writing your first DAG to running production-grade workflows with error handling, testing, and CI/CD. Module 5 introduces stream processing – how to handle data that arrives continuously and needs to be acted on immediately, using tools like Apache Kafka.

Module 5: Apache Spark & Big Data Processing

What you'll learn: When your data gets too big for a single machine, Spark is the industry standard. By the end of this module you'll know PySpark the way it's actually used in production – and just as importantly, when NOT to use it.

DuckDB has changed everything. You don't always need Spark anymore. But when your data hits 100GB+, Spark is still king. At a mid-to-large company, your daily batch jobs are probably Spark. Even if you never write a Spark job from scratch, you will inherit one, debug one, or optimize one. This module prepares you for all three.

5.1 When You Need Spark (And When You Don't)

TL;DR - Most data engineers don't need Spark for their day-to-day work - DuckDB/Polars handle up to ~100GB on a single machine, often faster than Spark - Spark shines at 100GB+ or when you need distributed compute across a cluster - Learn it because job postings require it and you WILL eventually hit big data

Here's something that might be controversial: most data engineers don't need Spark. If you're processing 500MB of data daily, Spark is like renting a school bus to drive yourself to work. It works, but you're paying for 40 seats you're not using and it takes 10 minutes just to start the engine.

So why learn it? Because at some point in your career, you WILL hit data that's too big for a single machine. And when that happens, Spark is the industry standard.

The Decision Framework

DATA SIZE	RECOMMENDED TOOL	WHY
< 1 GB	Pandas, Polars	Fits in memory, simple
1–10 GB	DuckDB, Polars	Single machine, crazy fast
10–100 GB	DuckDB (maybe Spark)	DuckDB handles this surprisingly well
100 GB+	Spark	You actually need distributed compute
1 TB+	Spark (cluster)	No question

The twist: DuckDB and Polars have gotten ridiculously good. A modern laptop with 32GB RAM can process 50–100GB files with DuckDB. That wasn't true 3 years ago.

Quick Comparison: DuckDB vs. Spark on 1GB

```
# DuckDB -- processes 1GB CSV in seconds
import duckdb

con = duckdb.connect()
result = con.sql("""
    SELECT
        vendor_id,
        COUNT(*) as trips,
        AVG(total_amount) as avg_amount
    FROM read_csv('nyc_taxi_2024.csv')
    GROUP BY vendor_id
""").fetchdf()
# Time: ~3 seconds on a laptop
```

```
# Spark on the same data -- significant overhead
from pyspark.sql import SparkSession
from pyspark.sql.functions import count, avg

spark = SparkSession.builder.appName("test").getOrCreate()
df = spark.read.csv("nyc_taxi_2024.csv", header=True, inferSchema=True)
result = df.groupBy("vendor_id").agg(
    count("*").alias("trips"),
    avg("total_amount").alias("avg_amount"),
)
result.show()
# Time: ~15-30 seconds (most of it is JVM startup)
```

DuckDB: 3 seconds. Spark: 20 seconds. On the same data.

When You Actually Need Spark

1. **Data literally doesn't fit on one machine** – 500GB+ with no single server large enough
2. **You need cluster-level processing** – EMR, Databricks, Dataproc
3. **Your company already uses Spark** – ecosystem is there, team knows it, infra exists
4. **Streaming at scale** – Spark Structured Streaming for real-time processing
5. **Job postings require it** – this is a valid reason

Pro Tip: In production, companies sometimes spin up a 20-node Spark cluster to process 5GB daily. That's \$3,000/month in cloud costs for something DuckDB could do on a \$20/month server. Know your data size. Pick the right tool.

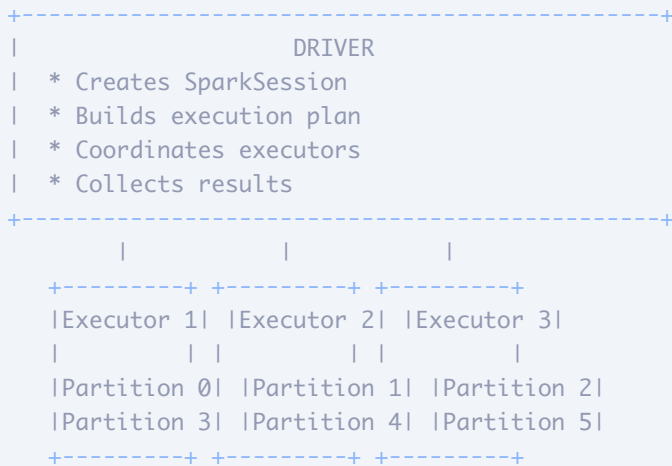
At your job: When someone proposes using Spark for a new project, the first question you should ask is “how big is the data?” If the answer is under 50GB, push back and suggest DuckDB or Polars first. You'll save your team thousands in cloud costs and hours in development time. If the data is 500GB+, Spark is the obvious choice – don't fight it.

5.2 Spark Architecture – Drivers, Executors, Partitions

TL;DR - Spark splits data into partitions and processes them in parallel across executors - The Driver coordinates everything; Executors do the actual work - Transformations are lazy (build a plan); Actions trigger execution - Shuffles (data exchange between executors) are the most expensive operation

If you don't understand Spark's architecture, you'll write code that's 10x slower than it should be. PySpark jobs that take 4 hours can often be optimized to 15 minutes just by understanding how data moves through the system.

The Three Components



1. The Driver – The boss. Your main program runs here. It creates the SparkSession, reads your code, builds an execution plan, coordinates executors, and collects results. The driver runs on ONE machine. If you accidentally pull all your data to the driver (with `.collect()`), you'll crash it.

2. Executors – The workers. They run on separate machines (or separate cores in local mode), store data partitions in memory, execute tasks, and report results back to the driver.

3. Partitions – The data chunks. This is THE concept in Spark: - Your data is split into partitions (chunks) - Each partition is processed independently and in parallel - More partitions = more parallelism (up to a point) - Default: 200 partitions for shuffles, but configurable

Think of it like counting words in a library of 10,000 books. Without Spark, you read every book yourself. With Spark, you hire 10 workers, split the books into 10 piles, and each worker counts their pile simultaneously. 10x faster.

But if you need to SORT all the words alphabetically, the workers need to exchange data – Worker A has some “Z” words that Worker B needs. This data exchange is called a **shuffle**, and it's the most expensive operation in Spark.

Lazy Evaluation: Why It Matters

Key Concept: Transformations vs. Actions

Spark is lazy. It doesn't do anything until it absolutely has to.

Transformations = instructions that build a plan (lazy): `filter()`, `select()`, `groupBy()`, `join()`, `withColumn()`

Actions = triggers that execute the plan: `show()`, `count()`, `collect()`, `write()`

This matters because Spark optimizes the entire chain before running it – reordering operations, skipping unnecessary columns, combining steps. This is called the **Catalyst optimizer**.

This is not just a trivia fact for interviews. Lazy evaluation is what makes Spark fast at scale. Here's why:

Imagine you write this chain: read a 100GB file, add 5 columns, filter to 1% of rows, then group by category. A naive engine would read 100GB, compute 5 new columns on every row, THEN filter. But Spark's optimizer sees the whole plan and says: "Wait, I can push that filter earlier, read only the columns I need, and skip computing columns on rows I'm going to throw away." That single optimization can turn a 2-hour job into a 10-minute job.

```
# This does NOTHING yet (just builds a plan)
df = spark.read.parquet("data/")
filtered = df.filter(df.amount > 100)
grouped = filtered.groupBy("category").count()

# THIS triggers execution of the entire chain
grouped.show()
```

At your job: When your Spark job is slow, the first thing to check is the execution plan. Run `.explain(mode="formatted")` on your DataFrame before calling an action. Look for `BroadcastHashJoin` (good) vs `SortMergeJoin` (potentially slow), `Exchange` (shuffle – expensive), and `FileScan` (are you reading all columns or just what you need?).

Getting Started with PySpark

```
# Start PySpark with Docker
docker run -it --rm \
  -p 4040:4040 \
  -v $(pwd)/data:/data \
  apache/spark:3.5.1-python3 \
  /opt/spark/bin/pyspark
```

Or use the companion Docker Compose setup:

Companion code: `de-fast-track/modules/module-5/starter/docker-compose.yml`

```

services:
  spark-master:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=master
    ports:
      - "8081:8080"
      - "7077:7077"
    volumes:
      - ./scripts:/opt/spark-scripts
      - ./data:/opt/spark-data

  spark-worker:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
    depends_on:
      - spark-master

```

```

# In the PySpark shell -- SparkSession is pre-created as `spark`
data = [(i, f"product_{i % 10}", i * 9.99) for i in range(1000000)]
df = spark.createDataFrame(data, ["id", "product", "amount"])

# Check partitions
print(f"Number of partitions: {df.rdd.getNumPartitions()}")

# This is lazy -- nothing happens yet
filtered = df.filter(df.amount > 5000)
print("Filtered defined -- nothing executed yet")

# This triggers execution
count = filtered.count()
print(f"Count: {count}")

# Now check the Spark UI at http://localhost:4040 -- you'll see the completed job

```

Expected output:

```

Number of partitions: 8
Filtered defined -- nothing executed yet
Count: 499500

```

Common Mistake:

Calling `.collect()` on large data – Pulls everything to the driver. 50GB of data = dead driver.
Use `.show()` for previews, `.write()` for output.

Too few partitions – 100 cores but 10 partitions means 90 cores sit idle.

Too many partitions – 100,000 partitions for 1GB creates scheduling overhead. Rule of thumb: target ~128MB per partition.

5.3 DataFrames API – Reading, Transforming, Writing Data

TL;DR - The DataFrame API is where you spend 90% of your time in Spark - Always define schemas explicitly in production (don't use `inferSchema`) - Use Parquet for output – it's 10x smaller and faster than CSV - `F.col()` is your best friend for column references

The DataFrame API is similar to Pandas, but distributed. If you know Pandas, you'll pick this up fast – just some syntax differences.

Reading Data

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("DataFrames Tutorial") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# CSV (quick and dirty)
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("data/orders.csv")

# Parquet (preferred -- columnar, compressed, schema included)
df_parquet = spark.read.parquet("data/orders.parquet")

# JSON
df_json = spark.read.json("data/events.json")

# With explicit schema (faster and more reliable than inferSchema)
from pyspark.sql.types import (
    StructType, StructField, StringType,
    IntegerType, DoubleType, DateType
)

order_schema = StructType([
    StructField("order_id", IntegerType(), False),      # Not nullable
    StructField("customer_id", IntegerType(), False),
    StructField("product", StringType(), True),        # Nullable
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("order_date", DateType(), True),
])

df = spark.read.schema(order_schema).csv("data/orders.csv", header=True)

# Always check your data after loading
df.printSchema()
df.show(5)
print(f"Rows: {df.count()}, Partitions: {df.rdd.getNumPartitions()}")
```

Pro Tip: Always define your schema explicitly in production. `inferSchema=True` reads the entire file twice – once to figure out types, once to actually load. On large files, that doubles your read time.

At your job: The first time you load a new dataset, use `inferSchema` to see what Spark guesses. Then hardcode that schema. Your coworkers will thank you when the pipeline doesn't break because a column that was always integers suddenly has a string value in row 50 million.

Transformations

Here's the companion code for exploring data with Spark (see [de-fast-track/modules/module-5/solution/scripts/01_explore.py](#)):

```
from pyspark.sql import functions as F

# Select columns
df.select("order_id", "customer_id", "price").show()

# Filter rows
expensive = df.filter(F.col("price") > 100)
# Equivalently:
expensive = df.where(df.price > 100)

# Add computed columns
df_enriched = df.withColumn(
    "total", F.col("quantity") * F.col("price")
).withColumn(
    "tax", F.col("quantity") * F.col("price") * 0.08
).withColumn(
    "order_year", F.year("order_date")
)

# Rename columns
df_renamed = df.withColumnRenamed("price", "unit_price")

# Drop columns
df_slim = df.drop("some_unnecessary_column")

# Handle nulls
df_clean = df.fillna({"quantity": 0, "product": "unknown"})
df_no_nulls = df.dropna(subset=["order_id", "customer_id"])

# Type casting
df_typed = df.withColumn("price", F.col("price").cast("decimal(10,2)"))

# String operations
df_strings = df.withColumn(
    "product_upper", F.upper(F.col("product"))
).withColumn(
    "product_trimmed", F.trim(F.col("product"))
)

# Date operations
df_dates = df.withColumn(
    "order_month", F.date_format("order_date", "yyyy-MM")
).withColumn(
    "days_ago", F.datediff(F.current_date(), F.col("order_date"))
)
```

Aggregations

```
# Basic groupby
summary = df.groupBy("product").agg(
    F.count("*").alias("order_count"),
    F.sum("price").alias("total_revenue"),
    F.avg("price").alias("avg_price"),
    F.min("order_date").alias("first_order"),
    F.max("order_date").alias("last_order"),
)
summary.show()

# Multiple groupby columns with ordering
monthly = df.withColumn("month", F.date_format("order_date", "yyyy-MM")) \
    .groupBy("month", "product") \
    .agg(F.sum(F.col("quantity") * F.col("price")).alias("revenue")) \
    .orderBy("month", F.desc("revenue"))

monthly.show(20)
```

Here's the enriched exploration from the companion code ([01_explore.py](#)):

```

"""Module 5 Solution: Spark Data Exploration."""
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder.appName("Module5-Explore").getOrCreate()

# Load data
orders = spark.read.csv("/opt/spark-data/sample_orders.csv", header=True, inferSchema=True)
customers = spark.read.csv("/opt/spark-data/sample_customers.csv", header=True, inferSchema=True)
products = spark.read.csv("/opt/spark-data/sample_products.csv", header=True, inferSchema=True)

# Schema inspection
print("=== Orders Schema ===")
orders.printSchema()
print(f"Row count: {orders.count()}")
orders.show(10)

# Basic statistics
print("=== Summary Statistics ===")
orders.describe("quantity", "unit_price", "discount").show()

# Revenue by product (join orders to products)
enriched = orders.join(products, orders.product_id == products.product_id, "left") \
    .withColumn("revenue", F.col("quantity") * F.col("unit_price") - F.col("discount")) \
    .withColumn("cost", F.col("quantity") * F.col("unit_cost")) \
    .withColumn("profit", F.col("revenue") - F.col("cost"))

print("=== Revenue by Category ===")
enriched.groupBy("category") \
    .agg(
        F.sum("revenue").alias("total_revenue"),
        F.count("*").alias("order_count"),
        F.avg("revenue").alias("avg_order_value"),
    ) \
    .orderBy(F.desc("total_revenue")) \
    .show()

# Window functions: rank customers by total spend
customer_spend = enriched.groupBy("customer_id") \
    .agg(F.sum("revenue").alias("total_spend"))

window = Window.orderBy(F.desc("total_spend"))
ranked = customer_spend.withColumn("rank", F.rank().over(window))

print("=== Customer Ranking by Spend ===")
ranked.join(customers, "customer_id", "left") \
    .select("rank", "name", "city", "total_spend") \
    .show()

```

```
# Daily trends
print("=== Daily Order Trends ===")
orders.withColumn("order_day", F.to_date("order_date")) \
    .groupBy("order_day") \
    .agg(F.count("*").alias("orders"), F.sum(F.col("quantity") * F.col("unit_price")).a
    lias("revenue")) \
    .orderBy("order_day") \
    .show()

spark.stop()
```

Writing Data

```
# Parquet -- the go-to for data lakes
df_enriched.write \
    .mode("overwrite") \
    .parquet("output/orders_enriched")

# Partitioned by date -- standard for data lakes
df_enriched.write \
    .mode("overwrite") \
    .partitionBy("order_year") \
    .parquet("output/orders_by_year")

# CSV (for humans or legacy systems)
summary.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv("output/summary")

# To a database via JDBC
df_enriched.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql://localhost:5432/warehouse") \
    .option("dbtable", "public.orders_enriched") \
    .option("user", "postgres") \
    .option("password", "postgres") \
    .mode("overwrite") \
    .save()
```

Write modes: - `overwrite` – replace everything - `append` – add to existing data - `ignore` – do nothing if data exists - `error` / `errorifexists` – fail if data exists (default)

Common Mistake:

`inferSchema=True` on large files – Define your schema. It's faster and more reliable.

Writing CSV when you should write Parquet – Parquet is 10x smaller, 10x faster to read, and includes the schema. Always use Parquet unless someone specifically needs CSV.

`.coalesce(1)` on large data – Merges all partitions to 1 file. Fine for small summary tables. Do NOT coalesce 10GB of data into one file.

5.4 Spark SQL – Running SQL on Distributed Data

TL;DR - Register DataFrames as views, then query with standard SQL - Spark SQL supports window functions, CTEs, and everything you'd expect - You can query Parquet files directly without registering views - Mix DataFrame API and SQL freely – use whatever's clearest

You can use Spark and barely write any Python. Spark SQL lets you run standard SQL on distributed datasets.

Basic Usage

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Spark SQL").getOrCreate()

# Read data
orders = spark.read.parquet("data/orders.parquet")
customers = spark.read.parquet("data/customers.parquet")

# Register as SQL views
orders.createOrReplaceTempView("orders")
customers.createOrReplaceTempView("customers")

# Now write standard SQL!
result = spark.sql("""
    SELECT
        c.name,
        c.segment,
        COUNT(o.order_id) AS total_orders,
        SUM(o.quantity * o.price) AS total_revenue,
        AVG(o.quantity * o.price) AS avg_order_value
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.order_date >= '2026-01-01'
    GROUP BY c.name, c.segment
    HAVING total_revenue > 1000
    ORDER BY total_revenue DESC
""")

result.show(20)
```

It's the exact same SQL you'd write in Postgres. But it runs distributed across your Spark cluster.

Window Functions in SQL

```
spark.sql("""
    SELECT
        order_id,
        customer_id,
        price,
        order_date,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id ORDER BY order_date DESC
        ) as rn,
        SUM(price) OVER (
            PARTITION BY customer_id ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
        ) as running_total,
        LAG(price) OVER (
            PARTITION BY customer_id ORDER BY order_date
        ) as prev_order_amount
    FROM orders
""").show()
```

Querying Files Directly

You don't even need to register views for ad-hoc analysis:

```
# Query Parquet files directly
spark.sql("""
    SELECT * FROM parquet.`data/orders.parquet`
    WHERE price > 100
    LIMIT 10
""").show()

# Query CSV files directly
spark.sql("SELECT * FROM csv.`data/orders.csv`").show()
```

When to Use DataFrame API vs. SQL

USE CASE	PREFERENCE
Complex multi-step transformations	DataFrame API (chaining)
Ad-hoc analysis / exploration	SQL
Analysts maintaining the code	SQL
Dynamic column operations	DataFrame API
Joins with complex logic	SQL (often easier to read)

In practice, use both. Many production jobs mix DataFrame API and SQL in the same script.

At your job: If you're building a pipeline that analysts will eventually maintain, lean toward Spark SQL. Most analysts are fluent in SQL but less comfortable with PySpark's DataFrame API. If you're building complex transformation logic with lots of conditional branching, the DataFrame API gives you more programmatic control.

5.5 Joins, Aggregations, and Window Functions

TL;DR - Broadcast small tables in joins to avoid expensive shuffles - Anti joins are great for data quality checks (finding orphaned records) - Window functions without `partitionBy` send all data to one executor – very slow - Check for data skew on join keys before running large joins

Joins are where most Spark performance problems live. Understanding how Spark joins data will make or break your job execution times.

Join Types

```
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder.appName("Joins").getOrCreate()

# Sample data
orders = spark.createDataFrame([
    (1, 101, 29.99), (2, 102, 49.99), (3, 101, 19.99),
    (4, 103, 99.99), (5, 999, 14.99), # 999 = no matching customer
], ["order_id", "customer_id", "amount"])

customers = spark.createDataFrame([
    (101, "Alice", "premium"), (102, "Bob", "standard"),
    (103, "Charlie", "basic"), (104, "Diana", "premium"), # 104 = no orders
], ["customer_id", "name", "segment"])

# INNER JOIN -- only matching rows
inner = orders.join(customers, "customer_id", "inner")
inner.show()
# Result: 4 rows (order 5 excluded, customer 104 excluded)

# LEFT JOIN -- all orders, even without customer match
left = orders.join(customers, "customer_id", "left")
left.show()
# Result: 5 rows (order 5 has null name/segment)

# ANTI JOIN -- orders WITHOUT matching customers (data quality check!)
orphans = orders.join(customers, "customer_id", "left_anti")
orphans.show()
# Result: just order 5 -- great for finding data quality issues

# SEMI JOIN -- orders that HAVE matching customers (no customer columns added)
matched = orders.join(customers, "customer_id", "left_semi")
matched.show()
# Result: 4 order rows, no customer columns
```

At your job: Anti joins are your secret weapon for data quality. Before loading data into your warehouse, run an anti join against your dimension tables to find orphaned foreign keys. This catches data issues before they corrupt your reports. The companion code in `02_quality_checks.py` shows this pattern in action.

Broadcast Joins – The Key Optimization

When you join two DataFrames, Spark shuffles data across the network so matching keys land on the same executor. This is slow. If one side is small, you can avoid the shuffle entirely:

```
from pyspark.sql.functions import broadcast

# Send the entire small table to every executor -- no shuffle needed
result = orders.join(broadcast(customers), "customer_id", "inner")
```

Key Concept: Broadcast Joins

Spark auto-broadcasts tables under 10MB. For tables up to ~100MB, manually broadcast them with `broadcast()`. If one table has 100MB and the other has 100GB, broadcasting saves massive shuffle time.

Window Functions with DataFrame API

```
from pyspark.sql.window import Window

orders_data = spark.createDataFrame([
    (1, 101, 100.0, "2026-01-01"),
    (2, 101, 200.0, "2026-01-15"),
    (3, 101, 150.0, "2026-02-01"),
    (4, 102, 300.0, "2026-01-10"),
    (5, 102, 250.0, "2026-01-20"),
    (6, 102, 175.0, "2026-02-05"),
], ["order_id", "customer_id", "amount", "order_date"])

# Define window specs
customer_window = Window.partitionBy("customer_id").orderBy("order_date")
customer_all = Window.partitionBy("customer_id")

result = orders_data.select(
    "*",
    F.row_number().over(customer_window).alias("order_number"),

    F.sum("amount").over(
        customer_window.rowsBetween(Window.unboundedPreceding, Window.currentRow)
    ).alias("running_total"),

    F.lag("amount", 1).over(customer_window).alias("prev_amount"),
    F.lead("amount", 1).over(customer_window).alias("next_amount"),

    F.sum("amount").over(customer_all).alias("customer_total"),

    (F.col("amount") / F.sum("amount").over(customer_all) * 100)
    .cast("decimal(5,2)").alias("pct_of_total"),
)

result.show()
```

Expected output:

```

+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|order_id|customer_id|amount|order_date|order_number|running_total|prev_amount|
next_amount|customer_total|pct_of_total|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      1|      101| 100.0|2026-01-01|      1|      100.0|      null|
200.0|      450.0|      22.22|
|      2|      101| 200.0|2026-01-15|      2|      300.0|     100.0|
150.0|      450.0|      44.44|
|      3|      101| 150.0|2026-02-01|      3|      450.0|     200.0|
null|      450.0|      33.33|
|      4|      102| 300.0|2026-01-10|      1|      300.0|      null|
250.0|      725.0|      41.38|
|      5|      102| 250.0|2026-01-20|      2|      550.0|     300.0|
175.0|      725.0|      34.48|
|      6|      102| 175.0|2026-02-05|      3|      725.0|     250.0|
null|      725.0|      24.14|
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+

```

Common Mistake:

Joining two huge tables without checking for data skew – If 90% of your data has `customer_id = NULL`, one partition processes 90% of the work. Filter nulls before joining.

Window functions without `partitionBy` – `Window.orderBy(...)` without `partitionBy(...)` creates a single partition for ALL data. Everything goes to one executor. Very slow.

5.6 Data Quality Checks in Spark

At your job: Data quality checks are not optional. Every production Spark pipeline should validate its inputs before transforming and its outputs before publishing. Bad data in, bad data out – except now it's bad data at scale.

Here's the companion quality checks script ([de-fast-track/modules/module-5/solution/scripts/02_quality_checks.py](#)):

```

"""Module 5 Solution: Spark Data Quality Checks."""
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("Module5-Quality").getOrCreate()

orders = spark.read.csv("/opt/spark-data/sample_orders.csv", header=True, inferSchema=True)
customers = spark.read.csv("/opt/spark-data/sample_customers.csv", header=True, inferSchema=True)
products = spark.read.csv("/opt/spark-data/sample_products.csv", header=True, inferSchema=True)

print("=" * 60)
print("DATA QUALITY REPORT")
print("=" * 60)

# 1. Null value analysis
print("\n--- Null Values ---")
for col_name in orders.columns:
    null_count = orders.filter(F.col(col_name).isNull()).count()
    if null_count > 0:
        print(f" {col_name}: {null_count} nulls")

# 2. Duplicate detection
print("\n--- Duplicates ---")
total = orders.count()
distinct = orders.select("order_id").distinct().count()
dupes = total - distinct
print(f" Total rows: {total}, Distinct order_ids: {distinct}, Duplicates: {dupes}")

# 3. Range validation
print("\n--- Range Validation ---")
neg_prices = orders.filter(F.col("unit_price") < 0).count()
neg_qty = orders.filter(F.col("quantity") <= 0).count()
print(f" Negative prices: {neg_prices}")
print(f" Non-positive quantities: {neg_qty}")

# 4. Referential integrity
print("\n--- Referential Integrity ---")
orphan_customers = orders.join(customers, orders.customer_id == customers.customer_id, "left_anti").count()
orphan_products = orders.join(products, orders.product_id == products.product_id, "left_anti").count()
print(f" Orders with unknown customer_id: {orphan_customers}")
print(f" Orders with unknown product_id: {orphan_products}")

# 5. Quality report as DataFrame
quality_checks = spark.createDataFrame([
    ("null_check", "No nulls in key columns", "PASS" if dupes == 0 else "FAIL"),
    ("duplicate_check", f"{dupes} duplicate order_ids", "PASS" if dupes == 0 else "FAIL"),
    ("negative_prices", f"{neg_prices} negative prices", "PASS" if neg_prices == 0 else "FAIL"),
])

```

```

        ("orphan_customers", f"{orphan_customers} orphaned", "PASS" if orphan_customers ==
0 else "WARN"),
        ("orphan_products", f"{orphan_products} orphaned", "PASS" if orphan_products == 0 e
lse "WARN"),
    ], ["check_name", "details", "result"])

print("\n--- Quality Summary ---")
quality_checks.show(truncate=False)

spark.stop()

```

At your job: Build a reusable quality check framework. Every time you load data into a new table, run nulls, duplicates, range checks, and referential integrity. Write the results to a quality report table. When the CFO asks “can we trust this data?”, you point to the quality report instead of shrugging.

5.7 Transformation Pipelines

The real power of Spark is chaining transformations into pipelines. Here’s the companion transformation script ([de-fast-track/modules/module-5/solution/scripts/03_transform.py](#)):

```

"""Module 5 Solution: Spark Transformation Pipeline."""
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("Module5-Transform").getOrCreate()

# Read raw data
orders = spark.read.csv("/opt/spark-data/sample_orders.csv", header=True, inferSchema=True)
customers = spark.read.csv("/opt/spark-data/sample_customers.csv", header=True, inferSchema=True)
products = spark.read.csv("/opt/spark-data/sample_products.csv", header=True, inferSchema=True)

# Enrich orders with customer and product info
enriched = orders \
    .join(customers, "customer_id", "left") \
    .join(products, "product_id", "left") \
    .withColumn("revenue", F.col("quantity") * orders["unit_price"] - F.col("discount")) \
    .withColumn("cost", F.col("quantity") * F.col("unit_cost")) \
    .withColumn("profit", F.col("revenue") - F.col("cost")) \
    .withColumn("order_month", F.date_format("order_date", "yyyy-MM")) \
    .withColumn("order_day_of_week", F.dayofweek("order_date"))

print("=== Enriched Orders ===")
enriched.select(
    "order_id", "name", "category", "quantity", "revenue", "profit", "status"
).show(10)

# Aggregate: revenue by category and month
revenue_by_category = enriched \
    .groupBy("category", "order_month") \
    .agg(
        F.sum("revenue").alias("total_revenue"),
        F.sum("profit").alias("total_profit"),
        F.count("*").alias("order_count"),
        F.countDistinct("customer_id").alias("unique_customers"),
    ) \
    .orderBy("order_month", F.desc("total_revenue"))

print("=== Revenue by Category & Month ===")
revenue_by_category.show()

# Aggregate: customer summary
customer_summary = enriched \
    .groupBy("customer_id", "name", "city", "state") \
    .agg(
        F.sum("revenue").alias("lifetime_value"),
        F.count("*").alias("total_orders"),
        F.avg("revenue").alias("avg_order_value"),
        F.min("order_date").alias("first_order"),
        F.max("order_date").alias("last_order"),
    ) \

```

```

.orderBy(F.desc("lifetime_value"))

print("=== Customer Summary ===")
customer_summary.show()

# Write results as Parquet
enriched.write.mode("overwrite").parquet("/opt/spark-data/output/enriched_orders")
revenue_by_category.write.mode("overwrite").parquet("/opt/spark-data/output/revenue_by_category")
customer_summary.write.mode("overwrite").parquet("/opt/spark-data/output/customer_summary")

print("Parquet files written to /opt/spark-data/output/")
spark.stop()

```

At your job: This is the pattern you'll see everywhere: read raw data, join with dimension tables, compute business metrics, write enriched output. The specifics change (taxi trips instead of orders, ad impressions instead of products), but the skeleton is always the same. Master this pattern once and you can build any Spark pipeline.

5.8 UDFs – When to Use Them (Rarely) and Alternatives

TL;DR - UDFs are 10–100x slower than built-in functions due to serialization overhead - Always try built-in functions first (`F.when` , `F.regexp_extract` , etc.) - If you must use UDFs, use Pandas UDFs (vectorized) for 10–100x speedup over regular UDFs - Regular UDFs are a last resort

UDFs (User Defined Functions) let you run custom Python code on Spark data. The problem: they're slow. Like, 10–100x slower than built-in functions.

Why UDFs Are Slow

Built-in Spark functions run in the JVM – compiled, optimized, fast. UDFs run in Python. For every row, Spark serializes data from JVM to Python (via a socket), runs your function, then serializes back. That per-row overhead kills performance.

Don't Use a UDF When Built-In Functions Work

```
from pyspark.sql import functions as F
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

# BAD -- UDF for something built-in functions handle perfectly
@udf(returnType=StringType())
def categorize_amount_udf(amount):
    if amount > 100: return "high"
    elif amount > 50: return "medium"
    else: return "low"

df.withColumn("tier", categorize_amount_udf(df.amount)) # Slow!

# GOOD -- Built-in when/otherwise (runs in JVM, no serialization)
df.withColumn("tier",
    F.when(F.col("amount") > 100, "high")
    .when(F.col("amount") > 50, "medium")
    .otherwise("low")
)
```

When You Genuinely Need a UDF

```
import re
from pyspark.sql.functions import udf
from pyspark.sql.types import ArrayType, StringType

# Complex regex that can't be expressed with built-in functions
@udf(returnType=ArrayType(StringType()))
def extract_emails(text):
    """Extract all email addresses from text."""
    if text is None:
        return []
    pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
    return re.findall(pattern, text)

df.withColumn("emails", extract_emails(df.description)).show()
```

Better Alternative: Pandas UDFs (Vectorized)

If you must use UDFs, Pandas UDFs process batches of rows instead of one at a time – 10–100x faster:


```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")
def calculate_discount(amounts: pd.Series, quantities: pd.Series) -> pd.Series:
    """Vectorized UDF -- processes batches, not individual rows."""
    base_discount = 0.05
    volume_discount = (quantities > 10).astype(float) * 0.1
    return amounts * (1 - base_discount - volume_discount)

df.withColumn("discounted", calculate_discount(df.amount, df.quantity)).show()
```

The UDF Decision Ladder

1. First, try **built-in functions** (`F.when`, `F.regexp_extract`, `F.split`, etc.)
2. If that fails, try **Spark SQL expressions**
3. If that fails, use a **Pandas UDF**
4. **Last resort:** regular UDF

5.9 Performance Tuning – Partitioning, Caching, Broadcast Joins, AQE

TL;DR - The four performance killers: data skew, wrong partition count, no caching, unnecessary shuffles - Enable AQE (Adaptive Query Execution) – it's a free performance boost - Cache Data-Frames you use multiple times; unpersist when done - Read execution plans to understand what Spark is actually doing

This is the section that'll make you look like a Spark wizard. Most PySpark jobs are slow because of a handful of common issues.

Performance Killer #1: Data Skew

Data skew means some partitions have way more data than others. If partition 1 has 10 rows and partition 2 has 10 million rows, your cluster sits idle while one executor does all the work.

```
# Detect skew -- check key distribution
df.groupBy("join_key").count().orderBy(F.desc("count")).show(10)
# If one key has 10M rows and others have 1K, you have skew

# Fix: salt the skewed key
num_salts = 10
skewed_df = skewed_df.withColumn(
    "salt", (F.rand() * num_salts).cast("int")
)
# Explode salts on the other table to match
other_df = other_df.withColumn(
    "salt", F.explode(F.array([F.lit(i) for i in range(num_salts)]))
)
# Join on key + salt distributes the work evenly
result = skewed_df.join(other_df, ["join_key", "salt"]).drop("salt")
```

At your job: Data skew is the #1 reason Spark jobs take forever. The classic scenario: you're joining on `customer_id` and 40% of your orders have `customer_id = NULL` (guest checkouts). One executor gets 40% of all data while the others finish in seconds. Always filter nulls before joining on nullable columns, or salt the keys.

Performance Killer #2: Wrong Partition Count

```
# Check current partition count
print(f"Partitions: {df.rdd.getNumPartitions()}")

# Repartition -- more partitions (triggers shuffle)
df_repartitioned = df.repartition(200)

# Coalesce -- fewer partitions (no shuffle, faster)
df_small = df.coalesce(10)

# Rule of thumb: target 128MB per partition
# 10GB of data: 10,000MB / 128MB = ~80 partitions
```

Performance Killer #3: Not Caching

If you read from the same DataFrame multiple times, cache it:

```
# Without cache -- reads from disk TWICE
count = df.filter(df.status == "active").count()
total = df.filter(df.status == "active").agg(F.sum("amount")).collect()

# With cache -- reads once, stores in memory
active = df.filter(df.status == "active").cache()
count = active.count()           # First call: reads + caches
total = active.agg(F.sum("amount")).collect() # Second call: from memory

# IMPORTANT: free memory when done
active.unpersist()
```

Cache when: you use the same DataFrame in multiple actions, or it's expensive to compute.

Don't cache when: you only use it once, or it's too big to fit in memory.

Performance Killer #4: Unnecessary Shuffles

```
# BAD: Repartition then immediately coalesce -- useless shuffle
df.repartition(200).coalesce(50)

# BAD: Distinct before a join -- two shuffles when one might do
df1.distinct().join(df2, "key")

# If possible, deduplicate AFTER the join
```

Adaptive Query Execution (AQE) – Free Performance

AQE (Spark 3.0+) optimizes your job DURING execution based on actual data statistics:

```
# Enable AQE (default in Spark 3.2+)
spark.conf.set("spark.sql.adaptive.enabled", "true")

# Auto-coalesce: combines small shuffle partitions
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")

# Auto-broadcast: detects small join sides at runtime
spark.conf.set("spark.sql.adaptive.autoBroadcastJoinThreshold", "10MB")

# Skew optimization: splits skewed partitions automatically
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Reading Execution Plans

```
df.filter(df.amount > 100) \
    .groupBy("category") \
    .agg(F.sum("amount")) \
    .explain(mode="formatted")

# Look for:
# - "BroadcastHashJoin" (good) vs "SortMergeJoin" (potentially slow)
# - "Exchange" = shuffle (expensive)
# - "FileScan" -- are you reading all columns or just what you need?
```

Quick Wins Checklist

- Enable AQE: `spark.sql.adaptive.enabled = true`
- Select only columns you need (don't `SELECT *`)
- Filter early (push filters before joins)
- Use Parquet (column pruning + predicate pushdown)
- Broadcast small tables in joins
- Cache DataFrames used multiple times
- Partition writes by date for downstream efficiency
- Check for data skew on join keys

Debugging OOM Errors

At your job: When your Spark job dies with `OutOfMemoryError`, here's the debugging checklist:

1. **Driver OOM:** Did you call `.collect()` or `.toPandas()` on a huge DataFrame? Use `.show()` or `.write()` instead.
2. **Executor OOM:** Is one executor getting all the data? Check for data skew. Run `df.groupBy("join_key").count().orderBy(F.desc("count")).show(10)`.
3. **Too many partitions cached:** Are you caching everything and never calling `.unpersist()`? Memory is finite.
4. **Increase memory:** Sometimes you just need more RAM. `spark.executor.memory=8g` in your config. But fix the root cause first – throwing memory at a skew problem just delays the crash.

5.10 Delta Lake / Apache Iceberg – Modern Table Formats

TL;DR - Plain Parquet files lack transactions, updates, deletes, and time travel - Delta Lake adds ACID transactions, MERGE/upsert, time travel, and schema evolution - Delta Lake (Databricks) vs. Iceberg (Netflix) – both solve the same problems - Use Delta if on Databricks; use Iceberg for multi-engine flexibility

Here's a real scenario: you have a data lake with millions of Parquet files in S3. A pipeline writes new data while a Spark job reads. The read job picks up a half-written file. Corrupt data. Dashboard breaks.

Or: you need to delete all data for a GDPR request. Good luck finding and rewriting every Parquet file containing that user.

The Problems with Plain Parquet

1. **No ACID transactions** – reads can see partially written data
2. **No updates/deletes** – Parquet files are immutable
3. **No schema evolution** – adding a column breaks old files
4. **No time travel** – can't roll back mistakes
5. **Small file problem** – streaming creates thousands of tiny files

Delta Lake in Action

Here's the companion Delta Lake code ([de-fast-track/modules/module-5/solution/scripts/04_delta_lake.py](#)):

```

from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip

# Setup Spark with Delta Lake
builder = SparkSession.builder \
    .appName("DeltaLake") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog")

spark = configure_spark_with_delta_pip(builder).getOrCreate()

# Write a Delta table
orders = spark.createDataFrame([
    (1, "Alice", 99.99, "2026-02-19"),
    (2, "Bob", 149.50, "2026-02-19"),
    (3, "Charlie", 29.99, "2026-02-19"),
], ["order_id", "customer", "amount", "date"])

orders.write.format("delta").mode("overwrite").save("data/delta/orders")

# Read it back
df = spark.read.format("delta").load("data/delta/orders")
df.show()

```

Updates, Deletes, and MERGE

```

from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, "data/delta/orders")

# UPDATE -- change Alice's amount
delta_table.update(
    condition="customer = 'Alice'",
    set={"amount": "109.99"}
)

# DELETE -- remove Charlie
delta_table.delete("customer = 'Charlie'")

# MERGE (upsert) -- the most powerful operation
new_data = spark.createDataFrame([
    (1, "Alice", 119.99, "2026-02-20"), # Update existing
    (4, "Diana", 199.00, "2026-02-20"), # Insert new
], ["order_id", "customer", "amount", "date"])

delta_table.alias("target").merge(
    new_data.alias("source"),
    "target.order_id = source.order_id"
).whenMatchedUpdateAll() \
.whenNotMatchedInsertAll() \
.execute()

```

At your job: MERGE is the operation you'll use most in production pipelines. It handles the “upsert” pattern: if the record exists, update it; if not, insert it. This makes your pipelines idempotent – you can re-run them safely without creating duplicates.

Time Travel

```
# Read a previous version
df_v0 = spark.read.format("delta") \
    .option("versionAsOf", 0) \
    .load("data/delta/orders")

# Read as of a specific timestamp
df_yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2026-02-18") \
    .load("data/delta/orders")

# View history
delta_table.history().show()

# Restore to a previous version (undo a mistake!)
delta_table.restoreToVersion(0)
```

Schema Evolution

```
# Add a new column -- existing data gets nulls
new_with_status = spark.createDataFrame([
    (5, "Eve", 75.25, "2026-02-20", "completed"),
], ["order_id", "customer", "amount", "date", "status"])

new_with_status.write.format("delta") \
    .mode("append") \
    .option("mergeSchema", "true") \
    .save("data/delta/orders")
```

OPTIMIZE – Fix the Small Files Problem

```
# Compact small files into larger ones
delta_table.optimize().executeCompaction()

# Z-ORDER -- co-locate related data for faster queries
delta_table.optimize().executeZOrderBy("date", "customer")
```

Delta Lake vs. Iceberg

FEATURE	DELTA LAKE	APACHE ICEBERG
Creator	Databricks	Netflix
Ecosystem	Tight Spark integration	Multi-engine (Spark, Trino, Flink)
Industry adoption	Dominant in Databricks shops	Growing fast, especially multi-cloud
Community	Open source + Databricks extras	Fully open source

If you're on Databricks, use Delta Lake. If you want engine flexibility, Iceberg is the better bet. Both solve the same problems.

5.11 Running Spark on AWS EMR and Databricks

TL;DR - In production, Spark runs on managed clusters: EMR, Databricks, Dataproc - EMR Serverless is the simplest AWS option – no cluster management - Databricks is the most popular Spark platform overall - Your PySpark code is identical everywhere – only deployment differs

Running Spark on your laptop is great for learning. In production, you need managed clusters.

Option 1: AWS EMR

```
# Submit a PySpark job to EMR
aws emr add-steps \
  --cluster-id j-XXXXXXXXXXXX \
  --steps Type=Spark,Name="Daily ETL",\
  ActionOnFailure=CONTINUE,\
  Args=[--deploy-mode,cluster,s3://my-bucket/scripts/etl.py]
```

EMR Serverless – no cluster management at all:

```
aws emr-serverless start-job-run \
  --application-id 00xxxxxxxxxxxxxxxx \
  --execution-role-arn arn:aws:iam::123456789:role/emr-role \
  --job-driver '{
    "sparkSubmit": {
      "entryPoint": "s3://my-bucket/scripts/etl.py",
      "sparkSubmitParameters": "--conf spark.sql.adaptive.enabled=true"
    }
  }'
```


Option 2: Databricks

The most popular Spark platform. Founded by the creators of Spark. Features: managed notebooks, built-in Delta Lake, Unity Catalog for governance, auto-scaling clusters, and built-in job scheduling.

Option 3: Google Dataproc / Azure Synapse

Same concept, different cloud: Dataproc for GCP, Synapse for Azure.

Option 4: Kubernetes

```
spark-submit \
  --master k8s://https://k8s-apiserver:443 \
  --deploy-mode cluster \
  --conf spark.kubernetes.container.image=my-spark:latest \
  --conf spark.executor.instances=5 \
  s3://my-bucket/etl.py
```

Pro Tip: For job hunting: learn EMR and Databricks terminology. Most job postings mention one or both. The actual PySpark code is identical everywhere – it’s just the deployment wrapper that differs.

At your job: Your first week, ask: “Where does our Spark run?” The answer tells you everything about the team’s maturity. If it’s “we ssh into a server and run spark-submit,” there’s work to do. If it’s “Databricks with scheduled jobs,” you’re in a good spot.

Module 5 Project: Processing NYC Taxi Data with PySpark

Process the NYC Taxi dataset using PySpark – exploration, data quality checks, transformations, analytics with window functions, and Delta Lake output.

Step 1: Set Up the Environment

```
# docker-compose.yaml
version: '3.8'

services:
  spark-master:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=master
      - SPARK_MASTER_HOST=spark-master
    ports:
      - "8081:8080"    # Spark Master UI
      - "7077:7077"
      - "4040:4040"    # Spark App UI
    volumes:
      - ./data:/data
      - ./scripts:/scripts
      - ./output:/output

  spark-worker:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_MEMORY=4g
      - SPARK_WORKER_CORES=2
    depends_on:
      - spark-master
    volumes:
      - ./data:/data
      - ./output:/output
```

```
mkdir -p data scripts output

# Download NYC taxi data (3-6 months, ~100MB each)
for month in 01 02 03 04 05 06; do
  wget -P data/ \
    "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2023-$month.parquet"
done

docker compose up -d
```

Step 2: Explore the Data

```
# scripts/01_explore.py
"""Explore and understand the dataset."""
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("NYC Taxi - Explore") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# Read all months at once with glob pattern
df = spark.read.parquet("data/yellow_tripdata_2023-*.parquet")

print(f"Total rows: {df.count():,}")
print(f"Partitions: {df.rdd.getNumPartitions()}")

print("\nSchema:")
df.printSchema()

print("\nSample data:")
df.show(5, truncate=False)

# Check for nulls across all columns
print("\nNull counts per column:")
null_counts = df.select([
    F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in df.columns
])
null_counts.show(truncate=False)

# Check value distributions
print("\nPassenger count distribution:")
df.groupBy("passenger_count").count().orderBy("passenger_count").show()

spark.stop()
```

Step 3: Data Quality Checks

```
# scripts/02_quality_checks.py
"""Identify and handle data quality issues."""
from pyspark.sql import SparkSession, functions as F

spark = SparkSession.builder \
    .appName("NYC Taxi - Quality") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

df = spark.read.parquet("data/yellow_tripdata_2023-*.parquet")
total = df.count()

print("=== DATA QUALITY REPORT ===\n")

neg_fares = df.filter(F.col("fare_amount") < 0).count()
print(f"Negative fares: {neg_fares:,} ({neg_fares/total*100:.2f}%)")

long_trips = df.filter(F.col("trip_distance") > 200).count()
print(f"Trips > 200 miles: {long_trips:,}")

zero_pax = df.filter(
    (F.col("passenger_count") == 0) | F.col("passenger_count").isNull()
).count()
print(f"Zero/null passengers: {zero_pax:,}")

time_travel = df.filter(
    F.col("tpep_dropoff_datetime") < F.col("tpep_pickup_datetime")
).count()
print(f"Dropoff before pickup: {time_travel:,}")

# Apply quality filters
print(f"\n=== CLEANING ===")
print(f"Before: {total:,} rows")

df_clean = df.filter(
    (F.col("fare_amount") >= 0) &
    (F.col("fare_amount") < 1000) &
    (F.col("trip_distance") > 0) &
    (F.col("trip_distance") < 200) &
    (F.col("passenger_count") > 0) &
    (F.col("tpep_dropoff_datetime") > F.col("tpep_pickup_datetime")) &
    (F.col("tpep_pickup_datetime").between("2023-01-01", "2023-12-31"))
)

clean_count = df_clean.count()
print(f"After: {clean_count:,} rows")
print(f"Removed: {total - clean_count:,} rows ({(total-clean_count)/total*100:.2f}%)")

df_clean.write.mode("overwrite").parquet("output/cleaned_taxi_data")

spark.stop()
```

Step 4: Transformations and Analytics

```

# scripts/03_transform.py
"""Transform data and compute analytics with window functions."""
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window

spark = SparkSession.builder \
    .appName("NYC Taxi - Transform") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
    .config("spark.sql.catalog.spark_catalog",
            "org.apache.spark.sql.delta.catalog.DeltaCatalog") \
    .getOrCreate()

df = spark.read.parquet("output/cleaned_taxi_data")

# === ENRICHMENT ===
df_enriched = df.withColumn(
    "trip_duration_minutes",
    (F.unix_timestamp("tpep_dropoff_datetime") -
     F.unix_timestamp("tpep_pickup_datetime")) / 60
).withColumn(
    "avg_speed_mph",
    F.when(
        F.col("trip_duration_minutes") > 0,
        F.col("trip_distance") / (F.col("trip_duration_minutes") / 60)
    ).otherwise(0)
).withColumn(
    "pickup_hour", F.hour("tpep_pickup_datetime")
).withColumn(
    "pickup_date", F.to_date("tpep_pickup_datetime")
).withColumn(
    "pickup_month", F.month("tpep_pickup_datetime")
).withColumn(
    "is_weekend", F.dayofweek("tpep_pickup_datetime").isin([1, 7]).cast("int")
).withColumn(
    "tip_percentage",
    F.when(F.col("fare_amount") > 0,
           (F.col("tip_amount") / F.col("fare_amount") * 100).cast("decimal(5,2)")
    ).otherwise(0)
).withColumn(
    "fare_category",
    F.when(F.col("total_amount") > 100, "premium")
    .when(F.col("total_amount") > 50, "high")
    .when(F.col("total_amount") > 20, "medium")
    .otherwise("budget")
)

# === ANALYTICS ===

# Hourly demand pattern
hourly_demand = df_enriched.groupBy("pickup_hour").agg(
    F.count("*").alias("trip_count"),
    F.avg("fare_amount").alias("avg_fare"),
    F.avg("trip_duration_minutes").alias("avg_duration"),

```

```

        F.avg("tip_percentage").alias("avg_tip_pct"),
    ).orderBy("pickup_hour")

print("=== Hourly Demand ===")
hourly_demand.show(24)

# Daily revenue with window functions (7-day and 30-day moving averages)
daily_revenue = df_enriched.groupBy("pickup_date").agg(
    F.count("*").alias("trips"),
    F.sum("total_amount").alias("revenue"),
    F.avg("total_amount").alias("avg_fare"),
)

window_7d = Window.orderBy("pickup_date").rowsBetween(-6, 0)
window_30d = Window.orderBy("pickup_date").rowsBetween(-29, 0)

daily_with_trends = daily_revenue.select(
    "*",
    F.avg("revenue").over(window_7d).alias("revenue_7d_avg"),
    F.avg("revenue").over(window_30d).alias("revenue_30d_avg"),
    F.avg("trips").over(window_7d).alias("trips_7d_avg"),
).orderBy("pickup_date")

print("=== Daily Revenue with Trends ===")
daily_with_trends.show(10)

# === WRITE TO DELTA LAKE ===
df_enriched.write \
    .format("delta") \
    .mode("overwrite") \
    .partitionBy("pickup_month") \
    .save("output/delta/taxi_enriched")

daily_with_trends.write \
    .format("delta") \
    .mode("overwrite") \
    .save("output/delta/daily_revenue")

hourly_demand.write \
    .format("delta") \
    .mode("overwrite") \
    .save("output/delta/hourly_demand")

print("\nAll outputs written to Delta Lake")
spark.stop()

```

Step 5: Run Everything

```
# Submit jobs to Spark cluster
docker compose exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  /scripts/01_explore.py

docker compose exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  /scripts/02_quality_checks.py

docker compose exec spark-master spark-submit \
  --master spark://spark-master:7077 \
  --packages io.delta:delta-spark_2.12:3.1.0 \
  /scripts/03_transform.py
```

Expected Output

- Data quality report showing issues found and percentage cleaned
- Enriched dataset with trip duration, speed, tip %, fare category
- Hourly demand analysis showing peak hours (typically 6–8 PM)
- Daily revenue with 7-day and 30-day moving averages
- All outputs in Delta Lake format with month partitioning

Stretch Goals

1. **Zone lookup** – Join with the taxi zone shapefile to get pickup/dropoff neighborhood names
2. **Surge pricing indicator** – Flag hours above the 90th percentile of demand
3. **Delta time travel** – Compare this month's data vs. last month's
4. **Anomaly detection** – Flag days where revenue is > 2 standard deviations from the 30-day average

What's Next

Now that you can process data at scale with Spark, Module 6 covers Apache Kafka and streaming – what happens when your data arrives continuously instead of in batches. You'll learn when you actually need real-time processing (hint: less often than you think), how Kafka works under the hood, and how to build streaming pipelines.

Module 6: Apache Kafka & Streaming

What you'll learn: Real-time data is no longer optional for many use cases. By the end of this module you'll understand Kafka's architecture, be able to build producers and consumers in Python, use Kafka Connect for zero-code integrations, and build a complete streaming pipeline.

Every time you add something to cart on Amazon, that event goes through something like Kafka. Every Uber ride update. Every Netflix play event. Every fraud alert on your credit card. Behind all of these is a streaming platform processing millions of events per second, routing them to the right systems, and enabling real-time decisions.

But here is the thing most conference talks won't tell you: **80% of the time, you don't need streaming.** This module will teach you when you do, how to build it, and – just as importantly – when to stick with batch.

6.1 Batch vs. Streaming – When You Actually Need Real-Time

TL;DR - 80% of the time, batch processing is the right choice – simpler and cheaper - Streaming is for fraud detection, real-time recommendations, IoT alerts, live dashboards - Streaming is harder, more expensive, and has more failure modes than batch - Most production systems use both (hot path for speed, cold path for accuracy)

“We need real-time data!” – every startup founder, every VP of Product, every stakeholder who watched a conference talk about streaming. And 80% of the time? They don't.

When Batch Is Fine (And Often Better)

- Daily/hourly dashboards and reports
- Data warehouse loading
- ML model training
- Any use case where “a few hours old” data is acceptable
- Most analytics

If your marketing team looks at a dashboard once a day, processing their data every 5 minutes is a waste of money and engineering effort. A nightly batch job is simpler, cheaper, and easier to debug.

When You Actually Need Streaming

- **Fraud detection** – you can't wait an hour to flag a suspicious transaction
- **Real-time recommendations** – “users viewing this RIGHT NOW also viewed...”
- **IoT monitoring** – factory sensor goes out of range, alert immediately
- **Live dashboards** – stock tickers, live sports, operations monitoring
- **Event-driven microservices** – order placed -> trigger fulfillment -> update inventory

The common thread: **the value of the data degrades rapidly with time**. If a fraudulent transaction happened 2 hours ago, the money is already gone. If a factory sensor hit a dangerous temperature 30 minutes ago, the damage is done.

The Cost of Streaming

Streaming is harder and more expensive than batch:

- More complex infrastructure (Kafka cluster, schema registry, monitoring)
- Harder to debug (try debugging a race condition at 3 AM)
- More failure modes (consumer lag, partition rebalancing, exactly-once delivery)
- Higher cloud costs (always-on infrastructure vs. spin-up-then-down)

Key Concept: The Hybrid Approach

Most production systems use BOTH batch and streaming:

- **Hot path:** Kafka -> real-time consumer -> quick aggregations -> live dashboard
- **Cold path:** Kafka -> S3 -> Spark batch job -> data warehouse -> detailed analytics

The hot path gives you speed. The cold path gives you accuracy and depth.

Pro Tip

At most companies, the journey looks like: (1) start with batch, (2) build Airflow pipelines, (3) one use case genuinely needs real-time, (4) add Kafka for THAT use case, (5) gradually expand. Don't try to stream everything on day one.

At your job: Your manager says “we need real-time everything.” Push back gently. Ask: “What decision changes if we have data in 5 seconds vs. 5 minutes vs. 1 hour?” Most of the time, the honest answer is “nothing changes,” and you just saved your team months of unnecessary complexity.

Checkpoint

1. Name three use cases where streaming is genuinely necessary.
2. Why is streaming more expensive than batch processing?
3. What's the difference between a “hot path” and “cold path” in a hybrid architecture?

6.2 Kafka Architecture – Brokers, Topics, Partitions, Consumer Groups

TL;DR - Kafka is a distributed log, not a message queue – messages persist and multiple consumers read independently - Topics are split into partitions for parallelism; ordering is guaranteed only within a partition - Consumer groups enable independent processing – analytics and fraud detection read the same data without interference - Use KRaft mode (no Zookeeper) for new setups

Kafka isn't a message queue. That's the first misconception to clear up. It's a distributed *log*. Think of an append-only journal that multiple writers can write to and multiple readers can read from, independently, at their own pace.

With a traditional message queue (RabbitMQ, SQS), once a message is consumed, it's gone. With Kafka, messages persist for a configurable retention period (days, weeks, or forever). This is what makes Kafka powerful – the same stream of events can be consumed by your analytics pipeline, your fraud detection system, your recommendation engine, and your data warehouse loader, all independently, all at their own speed.

The Five Core Concepts

```
Producers ==> Topic: "orders" (3 partitions)

|| Partition 0: [msg1][msg4][msg7]... || <= Broker 1
|| Partition 1: [msg2][msg5][msg8]... || <= Broker 2
|| Partition 2: [msg3][msg6][msg9]... || <= Broker 3

Consumer Group "analytics" (3 consumers):
Consumer A -> reads Partition 0
Consumer B -> reads Partition 1
Consumer C -> reads Partition 2

Consumer Group "fraud-detection" (2 consumers):
Consumer X -> reads Partitions 0, 1
Consumer Y -> reads Partition 2

Both groups read independently - no interference.
```

1. Brokers – The servers in a Kafka cluster. Each stores some data. In production, 3-5+ brokers for redundancy. If one goes down, others still have copies. Think of brokers like nodes in a distributed database – each one holds a slice of the overall data and can take over for a failed peer.

2. Topics – Named streams of messages (like tables, but append-only). Examples: `orders`, `user-clicks`, `inventory-updates`. A topic is a logical category. When your e-commerce app processes a purchase, it publishes to the `orders` topic. When a user clicks a product, it goes to `user-clicks`. You organize your data streams the same way you'd organize database tables – by entity or event type.

3. Partitions – Each topic is split into partitions. This is the parallelism mechanism, and understanding it is critical:

- Each partition is an ordered, immutable sequence of messages
- Partitions are distributed across brokers
- Messages within a partition are ordered; across partitions, no ordering guarantee
- More partitions = more parallelism = higher throughput

Why does this matter? If your `orders` topic has 1 partition, only 1 consumer can read it at a time. With 12 partitions, up to 12 consumers can process orders in parallel. Partitions are how Kafka scales horizontally – you don't buy a bigger server, you add more partitions and more consumers.

4. Consumer Groups – A set of consumers that work together. Each partition is assigned to exactly one consumer in the group. Different groups read independently – the analytics team and fraud team consume the same messages without interfering. This is the key architectural insight: you write data once, and N different systems can process it independently, each tracking their own position in the stream.

5. Offsets – Each message has a sequential offset (0, 1, 2, ...). Each consumer tracks where it left off. If a consumer crashes and restarts, it picks up from its last committed offset. This is how Kafka provides durability without deleting messages – each consumer group maintains its own bookmark into the shared log.

Setting Up Kafka with Docker

For development, a single-broker KRaft setup (no Zookeeper needed):

```

# docker-compose.yml
version: '3.8'

services:
  kafka:
    image: bitnami/kafka:3.7
    ports:
      - "9092:9092"
    environment:
      - KAFKA_CFG_NODE_ID=1
      - KAFKA_CFG_PROCESS_ROLES=controller,broker          # KRaft mode (no Zookeeper)
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://localhost:9092
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
      - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=false
  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports:
      - "8080:8080"
    environment:
      - KAFKA_CLUSTERS_0_NAME=local
      - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
    depends_on:
      - kafka

volumes:
  kafka-data:

```

For the hands-on project, the companion code uses a more complete setup with Postgres for persisting processed events. See [de-fast-track/modules/module-6/starter/docker-compose.yml](#):

```

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports:
      - "8082:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: local
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:9092

  postgres:
    image: postgres:16
    ports:
      - "5433:5432"
    environment:
      POSTGRES_USER: dataeng
      POSTGRES_PASSWORD: dataeng
      POSTGRES_DB: streaming
    volumes:
      - ./sql:/docker-entrypoint-initdb.d

```

Start it up and create your first topic:

```

docker compose up -d

# Create a topic with 3 partitions
docker compose exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic orders \
  --partitions 3 \
  --replication-factor 1

# List topics
docker compose exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list

# Describe a topic (shows partition details)
docker compose exec kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe --topic orders

```

Expected output (describe):

```

Topic: orders    TopicId: abc123    PartitionCount: 3    ReplicationFactor: 1
Topic: orders    Partition: 0       Leader: 1           Replicas: 1         Isr: 1
Topic: orders    Partition: 1       Leader: 1           Replicas: 1         Isr: 1
Topic: orders    Partition: 2       Leader: 1           Replicas: 1         Isr: 1

```

Common Mistake

Too few partitions – You can't have more consumers (in a group) than partitions. 3 partitions = max 3 consumers. Plan ahead. You can always add partitions later, but you can never reduce them.

Assuming global ordering – Ordering is only guaranteed within a partition. If you need ordered processing per customer, use `customer_id` as the partition key.

Checkpoint

1. What's the difference between Kafka and a traditional message queue?
2. Why do partitions matter for parallelism?
3. Can two different consumer groups read the same topic independently?

6.3 Producing Messages – Serialization, Partitioning Strategies

TL;DR - Use `acks="all"` and `enable.idempotence=True` for reliable production - Partition keys ensure all messages for the same entity go to the same partition (ordering) - Always call `producer.flush()` before your program exits - Batch messages with `linger.ms` for higher throughput with minimal latency increase

```
pip install confluent-kafka
```

Your First Producer

Here is the producer from the course PDF. Notice the production-ready configuration – this is not a toy example:


```

# producer.py
"""Kafka Producer -- sends e-commerce order events."""

from confluent_kafka import Producer
import json
import time
import random
from datetime import datetime

def delivery_callback(err, msg):
    """Called once per message to indicate delivery result."""
    if err:
        print(f"Delivery failed: {err}")
    else:
        print(f"Delivered to {msg.topic()} [{msg.partition()}] @ offset {msg.offset()}")

def create_producer() -> Producer:
    """Create a Kafka producer with production-ready config."""
    config = {
        "bootstrap.servers": "localhost:9092",

        # Reliability
        "acks": "all",          # Wait for all replicas to acknowledge
        "retries": 5,           # Retry on transient failures
        "retry.backoff.ms": 1000, # 1s between retries

        # Performance
        "linger.ms": 10,        # Batch messages for up to 10ms
        "batch.size": 65536,    # 64KB batch size
        "compression.type": "snappy", # Compress messages

        # Idempotence -- prevents duplicates on retry
        "enable.idempotence": True,
    }
    return Producer(config)

def generate_order_event() -> dict:
    """Generate a realistic e-commerce order event."""
    products = [
        {"id": "PROD-001", "name": "Wireless Headphones", "price": 79.99},
        {"id": "PROD-002", "name": "USB-C Hub", "price": 49.99},
        {"id": "PROD-003", "name": "Mechanical Keyboard", "price": 149.99},
        {"id": "PROD-004", "name": "Monitor Stand", "price": 39.99},
        {"id": "PROD-005", "name": "Webcam HD", "price": 69.99},
    ]

    product = random.choice(products)
    customer_id = f"CUST-{random.randint(1, 100):04d}"
    quantity = random.randint(1, 3)

```

```

return {
    "event_type": "order_placed",
    "order_id": f"ORD-{int(time.time() * 1000)}-{random.randint(100, 999)}",
    "customer_id": customer_id,
    "product_id": product["id"],
    "product_name": product["name"],
    "quantity": quantity,
    "unit_price": product["price"],
    "total_amount": round(product["price"] * quantity, 2),
    "currency": "USD",
    "timestamp": datetime.utcnow().isoformat() + "Z",
}

def main():
    producer = create_producer()
    topic = "orders"

    print(f"Starting producer -- sending events to '{topic}'")
    print("Press Ctrl+C to stop\n")

    try:
        count = 0
        while True:
            event = generate_order_event()

            # key = partition key. Same customer_id -> same partition -> ordered
            producer.produce(
                topic=topic,
                key=event["customer_id"],
                value=json.dumps(event),
                callback=delivery_callback,
            )

            producer.poll(0) # Trigger delivery callbacks

            count += 1
            if count % 10 == 0:
                print(f"Sent {count} events")

            time.sleep(random.uniform(0.1, 0.5))

    except KeyboardInterrupt:
        print(f"\nStopping. Flushing remaining messages...")
        producer.flush(10)
        print(f"Total sent: {count}")

if __name__ == "__main__":
    main()

```

Let's break down every important decision in this code.

Key Producer Concepts

Partition Key: Setting `key=event["customer_id"]` means Kafka hashes the key and always routes it to the same partition. All events for `CUST-0001` go to the same partition, guaranteeing order for that customer. Without a key, Kafka round-robins (good for throughput, bad for ordering).

This is a critical design decision. At your job, when someone says “we need to process events in order,” your first question should be: “In order per *what?*” Per customer? Per device? Per account? That entity becomes your partition key.

`acks="all"`: Waits for ALL replicas to confirm. Safest setting. Alternatives: `acks=0` (fire and forget), `acks=1` (leader only). In production, always use `"all"` unless you have a specific reason not to (like logging where losing a few messages is acceptable).

`enable.idempotence=True`: If the producer retries (network blip), Kafka deduplicates. Without this, retries cause duplicate messages. Always enable in production. Here is why: the producer sends a message, the broker writes it, but the acknowledgment gets lost in the network. The producer retries, and now you have two copies of the same message. Idempotence assigns a sequence number to each message so the broker can detect and discard the duplicate.

`linger.ms` and `batch.size`: Instead of sending one message at a time, the producer batches them. A 10ms linger dramatically improves throughput with a tiny latency increase. This is the classic throughput-vs-latency tradeoff – for most use cases, 10ms of additional latency is invisible but batching can increase throughput 10x.

`producer.flush()`: The producer buffers messages internally. If your program exits without flushing, those buffered messages are lost. Always flush on shutdown. The `10` argument is a timeout in seconds – if messages can’t be delivered in 10 seconds, give up.

`producer.poll(0)`: This triggers delivery callbacks. Without it, your callbacks never fire. The `0` means “don’t wait” – just check if any callbacks are ready and return immediately.

Verify Messages Are Flowing

```
docker compose exec kafka kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning \
  --property print.key=true \
  --property key.separator=": "
```

Common Mistake

Not calling `producer.flush()` – The producer buffers messages. If your program exits without flushing, buffered messages are lost.

Not setting `enable.idempotence` – Network retries cause duplicate messages without it.

Producing massive messages – Kafka's default max is 1MB. Keep messages small; use S3 URLs for large payloads.

The Companion Code Producer

The companion code in `de-fast-track/modules/module-6/solution/producer.py` generates a richer set of e-commerce events (page views, add-to-cart, searches, purchases) with weighted probabilities that mimic real traffic patterns. It uses `customer_id` as the partition key and includes a delivery callback for monitoring. Study both producers – the PDF version shows production config patterns while the companion version shows realistic event modeling.

Checkpoint

1. What does the partition key determine?
2. Why is `enable.idempotence=True` important?
3. What happens if you don't call `flush()` before your program exits?

6.4 Consuming Messages – Offsets, Consumer Groups, Exactly-Once

TL;DR - Disable auto-commit and commit manually AFTER successful processing - At-least-once + idempotent processing is the production sweet spot - Always close the consumer cleanly for proper group rebalancing - If processing is too slow, Kafka kicks you from the group
(`max.poll.interval.ms`)

Producing is the easy part. Consuming is where the real engineering happens – handling crashes, duplicate processing, and offset management.

A Production Consumer

```

# consumer.py
"""Kafka Consumer -- processes e-commerce order events."""

from confluent_kafka import Consumer, KafkaError, KafkaException
import json
from datetime import datetime
import signal
import sys

class OrderProcessor:
    """Processes order events and maintains running aggregations."""

    def __init__(self):
        self.total_revenue = 0.0
        self.order_count = 0
        self.customer_totals = {}

    def process(self, event: dict) -> bool:
        """Process a single order event. Returns True if successful."""
        try:
            order_id = event["order_id"]
            customer_id = event["customer_id"]
            amount = event["total_amount"]

            self.total_revenue += amount
            self.order_count += 1
            self.customer_totals[customer_id] = \
                self.customer_totals.get(customer_id, 0) + amount

            print(
                f"Order {order_id}: ${amount:.2f} from {customer_id} "
                f"| Running total: ${self.total_revenue:.2f} ({self.order_count}
orders)"
            )
            return True

        except (KeyError, TypeError) as e:
            print(f"Bad message format: {e} -- skipping")
            return True # Don't retry malformed messages
        except Exception as e:
            print(f"Processing error: {e} -- will retry")
            return False

    def print_summary(self):
        print(f"\n{' '*50}")
        print("SUMMARY")
        print(f"Total orders: {self.order_count}")
        print(f"Total revenue: ${self.total_revenue:.2f}")
        print(f"Avg order value: ${self.total_revenue/max(self.order_count,1):.2f}")
        print(f"Unique customers: {len(self.customer_totals)}")
        if self.customer_totals:
            top = max(self.customer_totals, key=self.customer_totals.get)
            print(f"Top customer: {top} (${self.customer_totals[top]:.2f}")

```

```

print(f"{' '*50}\n")

def create_consumer(group_id: str) -> Consumer:
    """Create a Kafka consumer with production-ready config."""
    config = {
        "bootstrap.servers": "localhost:9092",
        "group.id": group_id,

        # Offset management
        "auto.offset.reset": "earliest",      # Start from beginning if no offset
        "enable.auto.commit": False,          # Manual commit for safety

        # Performance
        "max.poll.interval.ms": 300000,      # 5 min max between polls
        "session.timeout.ms": 45000,         # 45s heartbeat timeout
        "fetch.min.bytes": 1024,             # Min data per fetch
    }
    return Consumer(config)

def main():
    consumer = create_consumer("order-processing-group")
    processor = OrderProcessor()
    running = True

    def shutdown(signum, frame):
        nonlocal running
        print("\nShutdown signal received...")
        running = False

    signal.signal(signal.SIGINT, shutdown)
    signal.signal(signal.SIGTERM, shutdown)

    consumer.subscribe(["orders"])
    print("Consumer started -- waiting for messages...\n")

    try:
        while running:
            msg = consumer.poll(timeout=1.0)

            if msg is None:
                continue

            if msg.error():
                if msg.error().code() == KafkaError._PARTITION_EOF:
                    continue # End of partition -- not an error
                else:
                    raise KafkaException(msg.error())

            # Deserialize
            try:
                event = json.loads(msg.value().decode("utf-8"))
            except json.JSONDecodeError as e:
                print(f"Invalid JSON: {e} -- skipping")

```

```

        consumer.commit(msg) # Commit to skip this bad message
        continue

    # Process
    success = processor.process(event)

    if success:
        consumer.commit(msg) # Commit AFTER successful processing
    else:
        print("Will retry on next poll")

    finally:
        processor.print_summary()
        consumer.close() # Triggers clean group rebalance
        print("Consumer closed cleanly.")

if __name__ == "__main__":
    main()

```

Let's unpack the critical design decisions.

Why `enable.auto.commit = False`?

With auto-commit enabled (the default), Kafka commits offsets on a timer regardless of whether you actually processed the message. Scenario: Kafka auto-commits offset 42, then your processing crashes on message 42. When the consumer restarts, it picks up at offset 43 – message 42 is silently lost. You'll never know.

With manual commit, you commit *after* successful processing. If processing crashes, the offset hasn't moved, and the message gets reprocessed on restart. You might process it twice, but you'll never lose it.

What Happens When a Consumer Crashes?

Say you have 3 consumers in a group reading 3 partitions. Consumer B dies. Here's the sequence:

1. Kafka notices Consumer B missed its heartbeat (after `session.timeout.ms`)
2. Kafka triggers a **rebalance** – redistributes partitions among surviving consumers
3. Consumer A now reads partitions 0 and 1, Consumer C reads partition 2
4. Processing continues from the last committed offset on each partition

This is why `consumer.close()` matters – it triggers a clean rebalance immediately instead of waiting for the session timeout. Without it, Kafka waits for the full timeout before reassigning partitions, creating a processing gap.

Delivery Semantics

Key Concept: Delivery Guarantees

- **At-most-once:** Commit before processing. If processing fails, the message is lost. Fast but unreliable.
- **At-least-once:** Commit after processing. If the commit fails, the message is reprocessed. Safe but may have duplicates. **This is what we're using.**
- **Exactly-once:** Requires Kafka transactions or idempotent consumers. Complex but possible.

The production sweet spot: **at-least-once delivery + idempotent processing.**

Exactly-once sounds ideal, but it's genuinely hard. The practical approach: accept that duplicates will happen, and make your processing idempotent so duplicates don't matter:

```
# Idempotent processing example -- upsert instead of insert
def process_order_idempotent(event: dict, db_conn):
    """Safe to run multiple times -- uses upsert."""
    db_conn.execute("""
        INSERT INTO processed_orders (order_id, customer_id, amount, processed_at)
        VALUES (%s, %s, %s, NOW())
        ON CONFLICT (order_id) DO NOTHING -- Idempotent!
    """, (event["order_id"], event["customer_id"], event["total_amount"]))
```

The `ON CONFLICT DO NOTHING` clause means processing the same order twice has no effect. This pattern – at-least-once delivery with idempotent writes – is what most production systems use. It's simpler than true exactly-once and just as correct for the end result.

Scaling with Consumer Groups

Run multiple consumers in the same group – Kafka distributes partitions automatically:

```
# Terminal 1 -- gets partitions 0, 1
python consumer.py

# Terminal 2 (same group) -- gets partition 2
python consumer.py

# Kafka automatically rebalances!
```

If you add a third consumer and your topic has 3 partitions, each consumer gets exactly one partition. If you add a fourth consumer, one sits idle – you can't have more consumers than partitions in a group.

The Companion Code Consumer

The companion consumer in `de-fast-track/modules/module-6/solution/consumer.py` extends these patterns by writing events to Postgres. It uses two tables: `raw_events` for all events and `order_events` specifically for purchases. Both use `ON CONFLICT DO NOTHING` for idempotent processing. Study how it handles the database connection alongside the consumer lifecycle – the `finally` block closes both the consumer and the database connection.

Common Mistake

Auto-commit enabled – Leads to silent data loss. Kafka commits offsets on a timer regardless of whether you processed the message. Always commit manually.

Processing too slowly – If `max.poll.interval.ms` expires, Kafka kicks you from the group. Process fast or increase the timeout.

Not closing the consumer – `consumer.close()` triggers a clean rebalance. Without it, Kafka waits for the session timeout before reassigning partitions.

Checkpoint

1. Why should you disable auto-commit?
2. What's the difference between at-most-once and at-least-once delivery?
3. What happens when you start a second consumer with the same group ID?

6.5 Kafka Connect – Plugging Into Databases Without Code

TL;DR - Source connectors stream data FROM external systems INTO Kafka - Sink connectors stream data FROM Kafka TO external systems - Debezium provides CDC (Change Data Capture) – every database change becomes a Kafka message - Manage connectors via REST API; monitor them – they fail silently

Writing a custom producer for every data source gets old fast. Kafka Connect comes with pre-built connectors for Postgres, MySQL, S3, Elasticsearch, and hundreds more.

At your job: This is how most companies actually get data into Kafka. You don't write custom producers for your databases – you deploy a Debezium connector, and every INSERT, UPDATE, and DELETE automatically shows up as a Kafka message. Zero code. This is the “change data capture” pattern that powers most real-time data pipelines in production.

Adding Kafka Connect to Docker Compose

```
kafka-connect:
  image: confluentinc/cp-kafka-connect:7.6.0
  ports:
    - "8083:8083"
  environment:
    CONNECT_BOOTSTRAP_SERVERS: kafka:9092
    CONNECT_REST_PORT: 8083
    CONNECT_GROUP_ID: connect-cluster
    CONNECT_CONFIG_STORAGE_TOPIC: _connect-configs
    CONNECT_OFFSET_STORAGE_TOPIC: _connect-offsets
    CONNECT_STATUS_STORAGE_TOPIC: _connect-status
    CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
    CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
    CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
    CONNECT_KEY_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    CONNECT_VALUE_CONVERTER: org.apache.kafka.connect.json.JsonConverter
    CONNECT_PLUGIN_PATH: /usr/share/java,/usr/share/confluent-hub-components
  command:
    - bash
    - -c
    - |
      confluent-hub install --no-prompt debezium/debezium-connector-postgresql:2.5.0
      confluent-hub install --no-prompt confluentinc/kafka-connect-s3:10.5.7
      /etc/confluent/docker/run
  depends_on:
    - kafka
```

Example: Postgres -> Kafka (CDC with Debezium)

Change Data Capture streams every INSERT, UPDATE, and DELETE from Postgres into Kafka in real-time:

```
# Deploy the connector via REST API
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "postgres-source",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "database.hostname": "postgres",
    "database.port": "5432",
    "database.user": "postgres",
    "database.password": "postgres",
    "database.dbname": "warehouse",
    "database.server.name": "warehouse",
    "table.include.list": "public.orders,public.customers",
    "topic.prefix": "cdc",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_slot"
  }
}'
```

Now every change to `orders` or `customers` automatically appears in Kafka topics `cdc.public.orders` and `cdc.public.customers`. No code required.

Example: Kafka -> S3 (Sink)

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "tasks.max": "1",
    "topics": "orders",
    "s3.bucket.name": "my-data-lake",
    "s3.region": "us-east-1",
    "format.class": "io.confluent.connect.s3.format.parquet.ParquetFormat",
    "flush.size": "1000",
    "rotate.interval.ms": "60000",
    "partitioner.class":
      "io.confluent.connect.storage.partitioners.TimeBasedPartitioner",
    "path.format": "year=YYYY/month=MM/day=dd/hour=HH",
    "timestamp.extractor": "Record"
  }
}'
```

This writes Kafka messages to S3 in Parquet format, partitioned by date/hour. This is how most companies build their data lake ingestion layer. The hot path (Kafka consumers) handles real-time needs, while the S3 sink builds the cold path for batch analytics.

Managing Connectors

```
# List all connectors
curl http://localhost:8083/connectors

# Check status
curl http://localhost:8083/connectors/postgres-source/status

# Pause / Resume / Delete
curl -X PUT http://localhost:8083/connectors/postgres-source/pause
curl -X PUT http://localhost:8083/connectors/postgres-source/resume
curl -X DELETE http://localhost:8083/connectors/postgres-source
```

Common Mistake

Not monitoring connector status – Connectors fail silently. Check status regularly or set up alerts. A connector in a “FAILED” state means data stopped flowing, and nobody tells you unless you check.

No dead letter queue – If a message can’t be processed, it blocks the connector. Configure `errors.deadletterqueue.topic.name` to route bad messages aside. Without a DLQ, one malformed message can halt your entire pipeline.

Checkpoint

1. What’s the difference between a source connector and a sink connector?
2. What is CDC and why is Debezium useful?
3. How do you check if a connector is healthy?

6.6 Schema Registry and Avro – Managing Data Contracts

TL;DR - Without schemas, producer changes break consumers silently at 3 AM - Avro is a compact binary format with embedded schema – smaller and faster than JSON - Schema Registry validates schemas before messages enter Kafka - Schema evolution rules (BACKWARD, FORWARD, FULL) prevent breaking changes

Here’s a production horror story. Team A produces messages to the `orders` topic with a field called `price`. Team B’s consumer reads `price` and does math with it. One day, Team A renames `price` to `unit_price` and deploys. Team B’s consumer crashes on every message. No warning. No review. Just a 3 AM page.

Schema Registry prevents this.

How It Works

Avro – A binary serialization format with a schema. Messages are smaller, faster, and always match a defined structure. While JSON is human-readable, Avro is machine-optimized: field names aren't repeated in every message (they're in the schema), types are enforced, and serialization is faster.

Schema Registry – Stores and validates Avro schemas. When a producer sends a message, the schema is validated. If it doesn't match, the message is rejected BEFORE it hits Kafka. This is a **data contract** – producers and consumers agree on the shape of the data, and the registry enforces it.

Schema Evolution – Rules for how schemas can change: - **BACKWARD** – new schema can read old data (add optional fields, remove fields) - **FORWARD** – old schema can read new data - **FULL** – both directions - **NONE** – no checks (don't do this)

Setup

```
schema-registry:
  image: confluentinc/cp-schema-registry:7.6.0
  ports:
    - "8081:8081"
  environment:
    SCHEMA_REGISTRY_HOST_NAME: schema-registry
    SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:9092
  depends_on:
    - kafka
```

Defining an Avro Schema

```
# Define an Avro schema
ORDER_EVENT_SCHEMA = """
{
  "type": "record",
  "name": "OrderEvent",
  "namespace": "com.ecommerce.events",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "customer_id", "type": "string"},
    {"name": "product_id", "type": "string"},
    {"name": "quantity", "type": "int"},
    {"name": "unit_price", "type": "double"},
    {"name": "total_amount", "type": "double"},
    {"name": "currency", "type": {"type": "string", "default": "USD"}},
    {"name": "timestamp", "type": "string"},
    {"name": "event_type", "type": {
      "type": "enum",
      "name": "EventType",
      "symbols": ["ORDER_PLACED", "ORDER_SHIPPED", "ORDER_CANCELLED"]
    }}
  ]
}
```

Producer with Avro

```
pip install confluent-kafka[avro]
```

```

# avro_producer.py
from confluent_kafka import SerializingProducer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
import time
from datetime import datetime

# Connect to Schema Registry
schema_registry_client = SchemaRegistryClient({"url": "http://localhost:8081"})

# Create Avro serializer -- validates against the schema
avro_serializer = AvroSerializer(
    schema_registry_client,
    schema_str=ORDER_EVENT_SCHEMA,
    to_dict=lambda obj, ctx: obj,
)

producer = SerializingProducer({
    "bootstrap.servers": "localhost:9092",
    "key.serializer": lambda k, ctx: k.encode("utf-8") if k else None,
    "value.serializer": avro_serializer,
})

# This event matches the schema -- it will be accepted
event = {
    "order_id": f"ORD-{int(time.time())}",
    "customer_id": "CUST-0001",
    "product_id": "PROD-001",
    "quantity": 2,
    "unit_price": 79.99,
    "total_amount": 159.98,
    "currency": "USD",
    "timestamp": datetime.utcnow().isoformat() + "Z",
    "event_type": "ORDER_PLACED",
}

producer.produce(topic="orders-avro", key=event["customer_id"], value=event)
producer.flush()
print(f"Sent: {event['order_id']}")

# This would FAIL -- "INVALID_STATUS" isn't in the enum:
# bad_event = {**event, "event_type": "INVALID_STATUS"}
# producer.produce(topic="orders-avro", value=bad_event) # Schema violation!

```


Consumer with Avro

```
# avro_consumer.py
from confluent_kafka import DeserializingConsumer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroDeserializer

schema_registry_client = SchemaRegistryClient({"url": "http://localhost:8081"})
avro_deserializer = AvroDeserializer(schema_registry_client)

consumer = DeserializingConsumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "avro-consumer-group",
    "auto.offset.reset": "earliest",
    "key.deserializer": lambda k, ctx: k.decode("utf-8") if k else None,
    "value.deserializer": avro_deserializer,
})
consumer.subscribe(["orders-avro"])

while True:
    msg = consumer.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        print(f"Error: {msg.error()}")
        continue

    # msg.value() is already a Python dict -- Avro handles deserialization
    event = msg.value()
    print(f"{event['order_id']}: {event['quantity']}x "
          f"{event['product_id']} = ${event['total_amount']}")
```

Schema Evolution

Want to add a `discount_code` field? Add it with a default value:

```
{"name": "discount_code", "type": ["null", "string"], "default": null}
```

This is **backward compatible** – new consumers can read old messages (`discount_code` defaults to null). The Schema Registry validates this automatically. If you try to remove a required field or change a type, the registry rejects the new schema.

This is the data contract in action. Teams can evolve their schemas independently, and the registry ensures nobody breaks the contract.

Common Mistake

Not using schemas at all – JSON in Kafka works but leads to the Team A/B scenario above. Use schemas for anything beyond prototyping.

Setting compatibility to NONE – Defeats the entire purpose. You might as well use JSON.

Checkpoint

1. What problem does Schema Registry solve?
2. What does “backward compatible” mean for schema changes?
3. Why is Avro preferred over JSON for Kafka messages?

6.7 Stream Processing Patterns – Filtering, Enrichment, Aggregation Windows

Understanding common stream processing patterns is just as important as knowing the Kafka API. These patterns show up in every real-time system.

Pattern 1: Filtering

The simplest pattern – read events from one topic, filter based on criteria, write matching events to another topic:

```
# Only forward high-value orders to a separate topic
def filter_high_value(event: dict) -> bool:
    return event.get("total_amount", 0) > 100.0

# In your consumer loop:
if filter_high_value(event):
    producer.produce("high-value-orders", key=event["customer_id"],
                    value=json.dumps(event))
```

At your job: “Your manager says we need real-time fraud detection.” The first step is a filter: flag transactions over \$10,000, or transactions from new accounts, or transactions from unusual locations. You consume from the `transactions` topic, apply your rules, and produce flagged events to a `fraud-alerts` topic that triggers notifications.

Pattern 2: Enrichment

Add context to events by looking up data from external sources:

```
# Enrich order events with customer data from a database or cache
def enrich_event(event: dict, customer_cache: dict) -> dict:
    customer = customer_cache.get(event["customer_id"], {})
    event["customer_name"] = customer.get("name", "Unknown")
    event["customer_tier"] = customer.get("tier", "standard")
    event["is_vip"] = customer.get("tier") == "premium"
    return event
```

The trick with enrichment is keeping lookup data fast. Reading from a database on every event creates a bottleneck. Production systems typically use an in-memory cache (Redis) or a local lookup table that's periodically refreshed from the database.

Pattern 3: Windowed Aggregation

This is the most powerful and most complex pattern. You accumulate events over a time window and compute metrics. The companion code's `stream_processor.py` implements this with a tumbling window:

```

# From de-fast-track/modules/module-6/solution/stream_processor.py
class MetricsAggregator:
    def __init__(self, window_seconds: int = 60):
        self.window_seconds = window_seconds
        self.current_window_start = None
        self.metrics = defaultdict(float)
        self.event_counts = defaultdict(int)
        self.product_counts = defaultdict(int)

    def add_event(self, event: dict) -> dict | None:
        ts = datetime.fromisoformat(event["timestamp"])
        window_start = ts.replace(second=0, microsecond=0)

        if self.current_window_start is None:
            self.current_window_start = window_start

        # Window rolled over -- flush and return metrics
        if window_start > self.current_window_start:
            result = self._flush()
            self.current_window_start = window_start
            return result

        event_type = event["event_type"]
        self.event_counts[event_type] += 1
        self.metrics["total_events"] += 1

        if event_type == "purchase":
            self.metrics["revenue"] += event.get("total_amount", 0)
            self.metrics["purchases"] += 1
            for item in event.get("items", []):
                self.product_counts[item.get("product_id", 0)] += item.get("quantity",
1)

        return None

    def _flush(self) -> dict:
        total = self.metrics["total_events"]
        purchases = self.metrics["purchases"]
        result = {
            "window_start": self.current_window_start,
            "window_end": self.current_window_start + timedelta(seconds=self.window_sec
onds),
            "total_events": int(total),
            "revenue": round(self.metrics["revenue"], 2),
            "purchases": int(purchases),
            "conversion_rate": round(purchases / total * 100, 2) if total > 0 else 0,
            "event_breakdown": dict(self.event_counts),
            "top_products": sorted(self.product_counts.items(), key=lambda x: -x[1])
[:5],
        }
        self.metrics.clear()
        self.event_counts.clear()
        self.product_counts.clear()
        return result

```

This is a **tumbling window** – fixed-size, non-overlapping time windows. Every 60 seconds, the aggregator flushes its metrics and starts fresh. When the window rolls over, you get a summary: total events, revenue, conversion rate, and top products for that minute.

Why windows matter: Without windowing, you’d either aggregate everything ever (useless for “what happened in the last minute?”) or nothing (useless for dashboards). Windows give you bounded aggregations over time – the fundamental building block of real-time analytics.

There are three types of windows you’ll encounter:

- **Tumbling windows** (what we’re using): Fixed size, no overlap. Each event belongs to exactly one window. Simple and predictable.
- **Sliding (hopping) windows**: Fixed size but they overlap. A 5-minute window that slides every 1 minute gives you smoother metrics but uses more memory.
- **Session windows**: Defined by activity gaps. “Group all events from a user until they’re inactive for 30 minutes.” Perfect for web session analytics.

Pattern 4: Dead Letter Queues

In production, some messages will be malformed, corrupt, or otherwise unprocessable. Instead of crashing your consumer or silently dropping them, route bad messages to a dead letter queue:

```
def process_with_dlq(consumer, processor, dlq_producer):
    """Process messages with dead letter queue for failures."""
    msg = consumer.poll(1.0)
    if msg is None or msg.error():
        return

    try:
        event = json.loads(msg.value().decode("utf-8"))
        success = processor.process(event)
        if success:
            consumer.commit(msg)
        else:
            # Transient failure -- will retry on next poll
            pass
    except (json.JSONDecodeError, KeyError) as e:
        # Permanent failure -- send to DLQ, commit to move past it
        dlq_producer.produce(
            "orders-dlq",
            key=msg.key(),
            value=msg.value(),
            headers=[("error", str(e).encode("utf-8"))],
        )
        consumer.commit(msg)
```

The DLQ pattern separates transient failures (retry) from permanent failures (route aside and investigate later). Without it, one bad message can block your entire pipeline.

6.8 Real-Time Dashboards and Materialized Views

The companion code includes a complete dashboard that queries Postgres for the metrics written by the stream processor. The database schema ([de-fast-track/modules/module-6/starter/sql/init.sql](#)) defines three tables:

```
CREATE TABLE raw_events (  
  event_id VARCHAR(50) PRIMARY KEY,  
  event_type VARCHAR(50) NOT NULL,  
  customer_id INT,  
  product_id INT,  
  event_data JSONB,  
  event_timestamp TIMESTAMP NOT NULL,  
  processed_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE order_events (  
  order_id VARCHAR(50) PRIMARY KEY,  
  customer_id INT NOT NULL,  
  items JSONB NOT NULL,  
  total_amount NUMERIC(10,2),  
  status VARCHAR(20) DEFAULT 'pending',  
  created_at TIMESTAMP NOT NULL,  
  processed_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE realtime_metrics (  
  metric_id SERIAL PRIMARY KEY,  
  window_start TIMESTAMP NOT NULL,  
  window_end TIMESTAMP NOT NULL,  
  metric_name VARCHAR(100) NOT NULL,  
  metric_value NUMERIC(12,2) NOT NULL,  
  dimensions JSONB,  
  created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_events_type ON raw_events(event_type);  
CREATE INDEX idx_events_timestamp ON raw_events(event_timestamp);  
CREATE INDEX idx_metrics_window ON realtime_metrics(window_start, metric_name);
```

Notice the design: `raw_events` captures everything (the audit trail), `order_events` captures the business events (what finance cares about), and `realtime_metrics` captures windowed aggregations (what the dashboard displays). The indexes on `event_timestamp` and `window_start` ensure dashboard queries are fast even as data accumulates.

The dashboard ([de-fast-track/modules/module-6/solution/src/dashboard.py](#)) uses the Rich library for a live CLI display:

```

"""Live CLI dashboard for streaming metrics using Rich."""
import time
import psycpg2
from rich.console import Console
from rich.table import Table
from rich.live import Live

DB_URL = "host=localhost port=5433 dbname=streaming user=dataeng password=dataeng"

def get_latest_metrics(conn) -> list[dict]:
    cur = conn.cursor()
    cur.execute("""
        SELECT window_start, metric_name, metric_value
        FROM realtime_metrics
        WHERE window_start >= NOW() - INTERVAL '10 minutes'
        ORDER BY window_start DESC
        LIMIT 20
    """)
    return [{"window": r[0], "name": r[1], "value": r[2]} for r in cur.fetchall()]

def get_recent_events(conn) -> list[dict]:
    cur = conn.cursor()
    cur.execute("""
        SELECT event_type, COUNT(*), MAX(event_timestamp)
        FROM raw_events
        WHERE event_timestamp >= NOW() - INTERVAL '5 minutes'
        GROUP BY event_type
        ORDER BY COUNT(*) DESC
    """)
    return [{"type": r[0], "count": r[1], "latest": r[2]} for r in cur.fetchall()]

def build_dashboard(conn) -> Table:
    metrics = get_latest_metrics(conn)
    events = get_recent_events(conn)

    table = Table(title="Real-Time E-Commerce Dashboard")
    table.add_column("Metric", style="cyan")
    table.add_column("Value", style="green")
    table.add_column("Window", style="dim")

    for m in metrics[:8]:
        table.add_row(m["name"], f"{m['value']:.2f}", str(m["window"])[19])

    if events:
        table.add_section()
        table.add_row("[bold]Event Type[/]", "[bold]Count (5m)[/]", "[bold]Latest[/]")
        for e in events:
            table.add_row(e["type"], str(e["count"]), str(e["latest"])[19])

    return table

```

```
def run():
    conn = psycopg2.connect(DB_URL)
    console = Console()

    console.print("[bold]Starting dashboard. Ctrl+C to stop.[/]")
    try:
        with Live(build_dashboard(conn), refresh_per_second=0.5, console=console) as li
ve:
            while True:
                live.update(build_dashboard(conn))
                time.sleep(2)
    except KeyboardInterrupt:
        pass
    finally:
        conn.close()
```

This is a mini version of what Uber or DoorDash actually runs. The pattern is: events flow through Kafka, a stream processor computes windowed aggregations, a writer persists them to a database, and a dashboard polls the database. Each component is independent and can be scaled separately.

6.9 Building a Streaming Pipeline End-to-End

TL;DR - A complete pipeline: event generator -> Kafka -> stream processor -> metrics topic -> DB writer -> dashboard - The stream processor computes sliding-window aggregations in real-time - The DB writer stores metrics in Postgres for querying - This is a mini version of what Uber or DoorDash actually runs

We've learned the pieces. Now let's assemble them into a real streaming pipeline.

```
Event Generator -> Kafka "raw-events" -> Stream Processor -> Kafka "metrics" -> DB
Writer -> Postgres

|

Dashboard Query
```


The Full Docker Compose Stack

```
version: '3.8'

services:
  kafka:
    image: bitnami/kafka:3.7
    ports:
      - "9092:9092"
    environment:
      - KAFKA_CFG_NODE_ID=1
      - KAFKA_CFG_PROCESS_ROLES=controller,broker
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
      - KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTE
XT
      - KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=1@kafka:9093
      - KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER
      - KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE=true
    volumes:
      - kafka-data:/bitnami/kafka

  postgres:
    image: postgres:16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: streaming
      POSTGRES_USER: streaming
      POSTGRES_PASSWORD: streaming
    volumes:
      - pg-data:/var/lib/postgresql/data

  kafka-ui:
    image: provectuslabs/kafka-ui:latest
    ports:
      - "8080:8080"
    environment:
      - KAFKA_CLUSTERS_0_NAME=local
      - KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS=kafka:9092
    depends_on:
      - kafka

volumes:
  kafka-data:
  pg-data:
```

Component 1: Event Generator

The event generator simulates a busy e-commerce website. It produces weighted events (mostly page views, some add-to-carts, fewer purchases) to mimic real traffic:

```

# event_generator.py
"""Simulates a busy e-commerce website generating click and purchase events."""

from confluent_kafka import Producer
import json
import random
import time
from datetime import datetime

producer = Producer({"bootstrap.servers": "localhost:9092"})

PAGES = ["home", "search", "product/headphones", "product/keyboard",
         "product/monitor", "cart", "checkout"]
# Weighted toward page views (most common event)
EVENT_TYPES = ["page_view", "page_view", "page_view", "page_view", "add_to_cart", "purchase"]

def generate_event():
    user_id = f"user_{random.randint(1, 500)}"
    event_type = random.choice(EVENT_TYPES)

    event = {
        "user_id": user_id,
        "event_type": event_type,
        "page": random.choice(PAGES),
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "session_id": f"sess_{random.randint(1, 1000)}",
    }

    if event_type == "purchase":
        event["amount"] = round(random.uniform(19.99, 299.99), 2)
        event["product_id"] = f"prod_{random.randint(1, 50)}"

    if event_type == "add_to_cart":
        event["product_id"] = f"prod_{random.randint(1, 50)}"

    return user_id, event

def main():
    print("Event generator started -- simulating website traffic")
    count = 0

    try:
        while True:
            key, event = generate_event()
            producer.produce("raw-events", key=key, value=json.dumps(event))
            producer.poll(0)

            count += 1
            if count % 100 == 0:
                print(f"Generated {count} events")
    except KeyboardInterrupt:
        pass

```

```
        time.sleep(random.uniform(0.01, 0.1)) # Simulate bursty traffic

    except KeyboardInterrupt:
        producer.flush()
        print(f"\nGenerated {count} total events")

if __name__ == "__main__":
    main()
```

Component 2: Stream Processor

The stream processor consumes raw events, maintains a sliding window of aggregations, and periodically emits computed metrics to a separate Kafka topic:

```

# stream_processor.py
"""Consumes raw events, computes real-time aggregations, produces metrics."""

from confluent_kafka import Consumer, Producer
import json
from datetime import datetime, timedelta
from collections import defaultdict
import threading
import time

class RealTimeAggregator:
    """Maintains sliding window aggregations."""

    def __init__(self, window_seconds: int = 60):
        self.window_seconds = window_seconds
        self.events = []
        self.lock = threading.Lock()

    def add_event(self, event: dict):
        with self.lock:
            ts = datetime.fromisoformat(event["timestamp"].rstrip("Z"))
            self.events.append((ts, event))
            self._prune()

    def _prune(self):
        """Remove events outside the window."""
        cutoff = datetime.utcnow() - timedelta(seconds=self.window_seconds)
        self.events = [(ts, e) for ts, e in self.events if ts > cutoff]

    def get_metrics(self) -> dict:
        with self.lock:
            self._prune()

            if not self.events:
                return {"window_seconds": self.window_seconds, "events": 0}

            events = [e for _, e in self.events]
            purchases = [e for e in events if e["event_type"] == "purchase"]
            unique_users = len(set(e["user_id"] for e in events))
            revenue = sum(e.get("amount", 0) for e in purchases)

            return {
                "timestamp": datetime.utcnow().isoformat() + "Z",
                "window_seconds": self.window_seconds,
                "total_events": len(events),
                "page_views": sum(1 for e in events if e["event_type"] == "page_view"),
                "cart_adds": sum(1 for e in events if e["event_type"] == "add_to_cart"),
                "purchases": len(purchases),
                "revenue": round(revenue, 2),
                "unique_users": unique_users,
                "conversion_rate": round(
                    len(purchases) / max(unique_users, 1) * 100, 2
                )
            }

```

```

        ),
        "events_per_second": round(
            len(events) / self.window_seconds, 1
        ),
    }

def main():
    consumer = Consumer({
        "bootstrap.servers": "localhost:9092",
        "group.id": "stream-processor",
        "auto.offset.reset": "latest",
        "enable.auto.commit": True,
    })

    metrics_producer = Producer({"bootstrap.servers": "localhost:9092"})
    aggregator = RealTimeAggregator(window_seconds=60)

    consumer.subscribe(["raw-events"])
    print("Stream processor started")

    last_emit = time.time()
    events_processed = 0

    try:
        while True:
            msg = consumer.poll(0.1)

            if msg and not msg.error():
                event = json.loads(msg.value().decode("utf-8"))
                aggregator.add_event(event)
                events_processed += 1

            # Emit metrics every 5 seconds
            if time.time() - last_emit >= 5:
                metrics = aggregator.get_metrics()
                metrics_producer.produce(
                    "metrics", key="realtime", value=json.dumps(metrics)
                )
                metrics_producer.poll(0)

                print(
                    f"[{metrics['timestamp'][:19]}] "
                    f"Events: {metrics['total_events']} | "
                    f"Users: {metrics['unique_users']} | "
                    f"Purchases: {metrics['purchases']} | "
                    f"Revenue: ${metrics['revenue']:.2f} | "
                    f"Conv: {metrics['conversion_rate']}%"
                )
                last_emit = time.time()

    except KeyboardInterrupt:
        print(f"\nProcessed {events_processed} total events")
    finally:
        consumer.close()

```

```
if __name__ == "__main__":  
    main()
```

Component 3: Database Writer

The database writer consumes from the `metrics` topic and persists aggregations to Postgres, where the dashboard can query them. This is the same pattern as the companion code's consumer – it writes to the `realtime_metrics` table with the windowed aggregation data.

Running the Full Pipeline

Open four terminal windows:

```
# Terminal 1: Start infrastructure  
docker compose up -d  
  
# Terminal 2: Start the event generator  
python event_generator.py  
  
# Terminal 3: Start the stream processor  
python stream_processor.py  
  
# Terminal 4: Start the dashboard  
python src/dashboard.py
```

You should see events flowing in real-time: the generator produces events, the stream processor aggregates them into 60-second windows, the metrics get written to Postgres, and the dashboard displays live conversion rates, revenue, and event counts.

When NOT to Use Streaming

Before you build all of this for your project, ask yourself honestly:

Batch is better when: - Your data consumers look at dashboards once a day or less - “Yesterday’s data” is perfectly acceptable for decisions - Your data volumes are manageable with hourly/daily jobs - You don’t have the team to operate streaming infrastructure 24/7 - You’re optimizing for engineering simplicity and cost

Streaming is better when: - Data value degrades rapidly with time (fraud, IoT, recommendations) - You have event-driven architectures where systems react to events - You need sub-minute visibility into operations - You’re building features that are inherently real-time (chat, live tracking)

Most companies need both. Start with batch. Add streaming for the use cases that genuinely require it. The worst outcome is building a complex streaming pipeline for something a cron job could handle.

Module 6 Project: Real-Time E-Commerce Analytics Pipeline

Build a complete streaming pipeline that processes e-commerce events in real-time.

Project Structure

```
module-6/  
  starter/  
    docker-compose.yml      # Kafka + Postgres + Kafka UI  
    sql/init.sql            # Database schema  
    producer_starter.py     # TODO: Implement  
    consumer_starter.py     # TODO: Implement  
  solution/  
    producer.py             # Complete producer with event generation  
    consumer.py             # Complete consumer with Postgres writes  
    stream_processor.py     # Windowed aggregations  
    src/dashboard.py        # Live CLI dashboard
```

Step 1: Start Infrastructure

```
cd de-fast-track/modules/module-6/starter  
docker compose up -d
```

Step 2: Implement the Producer

Open `producer_starter.py` and implement: - Configure the producer with `bootstrap.servers` - Generate realistic e-commerce events (page_view, add_to_cart, purchase, search) - Send events to the `ecommerce-events` topic using `customer_id` as the key - Add a delivery callback for monitoring - Run in a loop with configurable rate

Step 3: Implement the Consumer

Open `consumer_starter.py` and implement: - Configure the consumer with `group.id` and `auto.offset.reset` - Subscribe to `ecommerce-events` - Build an `OrderProcessor` class that writes events to Postgres - Add graceful shutdown with signal handlers - Use `ON CONFLICT DO NOTHING` for idempotent processing

Step 4: Add Stream Processing

Build a stream processor that: - Consumes from `ecommerce-events` - Maintains 60-second tumbling windows - Computes: total events, revenue, conversion rate, top products - Writes window metrics to the `realtime_metrics` table

Step 5: Build the Dashboard

Create a dashboard that queries Postgres and displays live metrics. The solution uses the Rich library, but a simple `while True` loop printing formatted SQL results works too.

Expected Output

- Producer generating 5+ events/second across multiple event types
- Consumer writing all events to `raw_events` and purchases to `order_events`
- Stream processor computing per-minute metrics
- Dashboard showing live conversion rates, revenue, and event breakdowns

Stretch Goals

1. **Multiple consumer groups** – Run an analytics consumer AND a fraud detection consumer on the same topic
 2. **Schema Registry** – Add Avro schemas and validate events before they enter Kafka
 3. **Dead letter queue** – Route malformed events to a DLQ topic instead of dropping them
 4. **Kafka Connect** – Add an S3 sink connector to build the cold path alongside your hot path
 5. **Alerting** – Trigger an alert when conversion rate drops below a threshold
-

End of Module 6

Module 7: Data Quality & Testing

The #1 thing that separates a junior DE from a senior one: the senior one checks their work. I've seen bad data cost companies millions in wrong business decisions. This module is about making sure that never happens on your watch.

7.1 Why Data Quality Is Your Job (Not the Analyst's)

TL;DR: Data quality is the data engineer's responsibility because you control the pipeline. Silent failures – not crashes – are the #1 way data engineers lose credibility. Build quality checks into every stage, not just at the end.

The Silent Killer

Picture this. It's 9:15 AM on a Monday. Your VP of Sales opens their dashboard, sees revenue dropped 40% over the weekend, panics, fires off an email to the CEO. By 9:30 the CEO is pinging your manager. By 10 AM your manager is pinging you. And what happened? A third-party API changed their date format from `YYYY-MM-DD` to `MM/DD/YYYY` and your pipeline silently ingested garbage all weekend.

Nobody's pipeline crashed. No errors in the logs. The data just... went bad. And nobody knew until someone looked at a dashboard two days later.

At your job, this will happen to you. Your pipeline ran successfully. Airflow is green. But the CEO's dashboard shows revenue dropped 90% overnight. What happened? A source API changed its schema, and your pipeline happily loaded the mangled data without a single error. This is the #1 way data engineers lose credibility. Not because of downtime – because of *silent failures*.

Key Concept: Silent Failures A silent failure is when your pipeline runs successfully (no errors, exit code 0) but produces incorrect data. These are far more dangerous than crashes because nobody knows anything is wrong until downstream consumers notice bad numbers.

The Six Categories of Data Quality Issues

Every data quality issue you'll encounter in the real world falls into one of these categories:

1. **Schema drift** – upstream source adds, removes, or renames columns

2. **Completeness** – missing values, nulls where there shouldn't be any. NULL foreign keys = broken JOINS. If `customer_id` is NULL, your entire customer attribution model falls apart.
3. **Freshness** – data is stale; the pipeline didn't run or ran late. Stale data = wrong dashboards. If your dashboard shows yesterday's numbers but doesn't tell anyone, decisions get made on old information.
4. **Accuracy** – values are present but wrong (duplicates, bad calculations)
5. **Consistency** – the same entity has different values in different tables
6. **Volume anomalies** – suddenly getting 10x or 0.1x the normal row count. Sudden drops = broken upstream. If you normally get 10,000 orders a day and suddenly get 50, something is very wrong even if there are no errors.

The nasty part? Most of these don't show up as errors. They show up as *wrong numbers in dashboards* weeks later.

Real-World Horror Stories

These are from real companies:

- **Fintech startup:** Loaded duplicate transactions for 3 weeks because their deduplication key changed upstream. Cost: \$2M in misreported revenue.
- **E-commerce company:** Product dimension table got a column renamed from `category` to `product_category`. Joins silently returned NULLs. Nobody noticed for 5 days.
- **Healthcare company:** Date parsing silently converted European dates (DD/MM/YYYY) to American (MM/DD/YYYY). Patient records got wrong dates for months.

Every single one of these was preventable with basic data quality checks.

The Ownership Model

“Isn't data quality the data team's problem? Like, shouldn't the analysts check their own stuff?”

No. Here's why: you, the data engineer, control the pipeline. You decide what gets loaded, how it gets transformed, and where it lands. If bad data gets through, it's because your pipeline let it through. You're the gatekeeper.

Think of it as defense in depth:

LAYER	OWNER	WHAT IT CHECKS
Layer 1: Pipeline-level	Data Engineer	Schema validation, null checks, freshness, volume
Layer 2: Transformation-level	Data Engineer	Business logic validation, referential integrity
Layer 3: Dashboard-level	Analyst	Metric validation, trend analysis

You own Layers 1 and 2.

Common Mistake: The biggest mistake: adding data quality checks *after* an incident (reactive, not proactive). Build them in from day one. The second biggest: only checking the happy path. Your checks need to catch *unexpected* data, not just validate expected data.

Checkpoint

1. What's the difference between a pipeline failure and a silent failure? Which is more dangerous and why?
2. Name the six categories of data quality issues.
3. Which layers of the data quality defense model are the data engineer's responsibility?

7.2 Types of Data Quality Checks

TL;DR: There are six core categories of data quality checks: schema validation, completeness, freshness, accuracy, consistency, and volume anomalies. You can't check everything – be strategic about which checks you implement for each table.

You're convinced data quality matters. But what do you actually *check*? You can't check everything – that would make your pipeline take 10x longer. You need to be strategic.

Here are the exact checks to add to every pipeline, with real SQL running against our e-commerce tables.

1. Schema Validation

Before you even load data, check that it looks like what you expect.

```
-- Check column count and names match expected schema
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'raw_orders'
ORDER BY ordinal_position;
```

In Python, validate *before* the data touches the database:

```

import pandas as pd

# Define the contract for what the data should look like
EXPECTED_COLUMNS = {
    'order_id': 'int64',
    'customer_id': 'int64',
    'order_date': 'datetime64[ns]',
    'total_amount': 'float64',
    'status': 'object',
}

def validate_schema(df: pd.DataFrame) -> bool:
    """Validate DataFrame matches expected schema."""
    missing = set(EXPECTED_COLUMNS.keys()) - set(df.columns)
    extra = set(df.columns) - set(EXPECTED_COLUMNS.keys())

    if missing:
        raise ValueError(f"Missing columns: {missing}")
    if extra:
        # Don't fail on extra columns -- but log it
        print(f"WARNING: Extra columns detected: {extra}")

    # Check types
    for col, expected_type in EXPECTED_COLUMNS.items():
        actual_type = str(df[col].dtype)
        if actual_type != expected_type:
            raise TypeError(
                f"Column '{col}' expected {expected_type}, got {actual_type}"
            )

    return True

```

Expected output (when validation passes):

```
True
```

Expected output (when a column is missing):

```
ValueError: Missing columns: {'customer_id'}
```

2. Completeness Checks

Are there nulls where there shouldn't be any?

```
-- Null check for critical columns
SELECT
    COUNT(*) AS total_rows,
    COUNT(*) - COUNT(order_id) AS null_order_ids,
    COUNT(*) - COUNT(customer_id) AS null_customer_ids,
    COUNT(*) - COUNT(order_date) AS null_order_dates,
    COUNT(*) - COUNT(total_amount) AS null_amounts
FROM raw_orders
WHERE loaded_at >= CURRENT_DATE; -- Only check today's load
```

Expected output:

total_rows	null_order_ids	null_customer_ids	null_order_dates	null_amounts
1523	0	0	0	12

Key Concept: Not All Nulls Are Bad A `shipping_date` being null is perfectly fine if the order hasn't shipped yet. But an `order_id` being null? That's a hard failure. Define your null-tolerance rules per column.

3. Freshness Checks

Is the data actually current?

```
-- Check that we have data from the expected time window
SELECT
    MAX(order_date) AS most_recent_order,
    NOW() - MAX(order_date) AS data_lag,
    CASE
        WHEN NOW() - MAX(order_date) > INTERVAL '2 hours'
        THEN 'STALE'
        ELSE 'FRESH'
    END AS freshness_status
FROM raw_orders;
```

Expected output:

most_recent_order	data_lag	freshness_status
2026-02-19 14:32:11+00	00:45:22	FRESH

Freshness checks fail when: upstream systems are down, your scheduler skipped a run, or there's a network issue. Freshness is often the *first* signal that something's wrong.

4. Accuracy / Validity Checks

Do the values make sense?

```

-- Range checks
SELECT COUNT(*) AS invalid_amounts
FROM raw_orders
WHERE total_amount < 0
    OR total_amount > 100000; -- No single order should be > $100K

-- Enum/category validation
SELECT status, COUNT(*)
FROM raw_orders
WHERE status NOT IN ('pending', 'processing', 'shipped', 'delivered', 'cancelled')
GROUP BY status;
-- Any rows here = unexpected status values

-- Date sanity
SELECT COUNT(*) AS future_orders
FROM raw_orders
WHERE order_date > NOW(); -- Orders from the future = bad data

-- Referential integrity
SELECT o.order_id
FROM raw_orders o
LEFT JOIN dim_customers c ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;
-- Orphaned orders = customer data missing or mismatched

```

5. Consistency Checks

Does the same thing look the same everywhere?

```

-- Cross-table consistency
SELECT
    (SELECT SUM(total_amount) FROM fact_orders WHERE order_date = '2026-02-18') AS fact_total,
    (SELECT SUM(total_amount) FROM raw_orders WHERE order_date::date = '2026-02-18')
AS raw_total;
-- These should match (or be within acceptable rounding tolerance)

-- Unique constraint validation
SELECT order_id, COUNT(*) AS dupes
FROM fact_orders
GROUP BY order_id
HAVING COUNT(*) > 1;
-- Any results = duplicates in your fact table. Bad.

```

6. Volume Anomaly Detection

Did we get a reasonable amount of data?

```

-- Compare today's volume against a rolling 30-day average
WITH daily_volumes AS (
    SELECT
        order_date::date AS load_date,
        COUNT(*) AS row_count
    FROM raw_orders
    WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
    GROUP BY order_date::date
),
stats AS (
    SELECT
        AVG(row_count) AS avg_count,
        STDDEV(row_count) AS stddev_count
    FROM daily_volumes
    WHERE load_date < CURRENT_DATE
)
SELECT
    dv.load_date,
    dv.row_count,
    s.avg_count,
    CASE
        WHEN dv.row_count < s.avg_count - 2 * s.stddev_count THEN 'ANOMALY_LOW'
        WHEN dv.row_count > s.avg_count + 2 * s.stddev_count THEN 'ANOMALY_HIGH'
        ELSE 'NORMAL'
    END AS volume_status
FROM daily_volumes dv, stats s
WHERE dv.load_date = CURRENT_DATE;

```

Expected output:

load_date	row_count	avg_count	volume_status
2026-02-19	1523	1480.3	NORMAL

This check catches the “pipeline ran but loaded 0 rows” problem. If you’re getting 10,000 orders a day and suddenly get 50, something is wrong even if there are no errors.

Common Mistake: - Checking only for nulls and calling it “data quality.” That’s maybe 20% of what you need. - Hard-coding thresholds instead of using statistical baselines. What’s “normal” changes over time. - Running checks *after* the data hits the warehouse. Run them *before* or *during* loading so you can reject bad batches.

Pro Tip: Start with the checks most likely to catch real problems at your company. For most teams, that’s: null checks on PKs, freshness, and volume anomalies. You can add more as you discover issues.

Checkpoint

1. Write a SQL query that checks for duplicate `order_id` values in a `raw_orders` table.

2. Why is a volume anomaly check better than a simple “row count > 0” check?
3. What’s the difference between an accuracy check and a consistency check?

7.3 Great Expectations & Soda – Automated Data Quality Frameworks

TL;DR: Great Expectations (GX) and Soda let you define data quality checks as config instead of hand-written SQL. GX is the industry standard with powerful features and auto-generated reports. Soda is simpler and YAML-driven. Use whichever fits your team.

Writing SQL checks by hand for every table gets old fast when you have 50 tables and 200 columns. Data quality frameworks let you define checks as config, run them automatically, and generate reports.

Great Expectations Setup

```
pip install great_expectations
great_expectations init
```

This creates a project structure:

```
great_expectations/
  great_expectations.yml    # Main config
  expectations/              # Your expectation suites (the checks)
  checkpoints/              # How and when to run checks
  plugins/                  # Custom expectations
  uncommitted/
    config_variables.yml     # Secrets (don't commit this)
    data_docs/              # Generated HTML reports
```


Connecting to Your Data

```
import great_expectations as gx

context = gx.get_context()

# Add a Postgres datasource
datasource = context.sources.add_postgres(
    name="postgres_db",
    connection_string="postgresql+psycopg2://postgres:postgres@localhost:5432/ecommerce",
)

# Add a data asset (table)
data_asset = datasource.add_table_asset(
    name="raw_orders",
    table_name="raw_orders",
)

# Create a batch request (what data to validate)
batch_request = data_asset.build_batch_request()
```

Writing Expectations

Expectations are the checks themselves – assertions for your data that read like English:

```

# Create an expectation suite
suite = context.add_expectation_suite("raw_orders_suite")

# Get a validator -- connects the suite to actual data
validator = context.get_validator(
    batch_request=batch_request,
    expectation_suite_name="raw_orders_suite",
)

# Schema checks
validator.expect_table_columns_to_match_ordered_list(
    column_list=["order_id", "customer_id", "order_date", "total_amount", "status"]
)

# Null checks
validator.expect_column_values_to_not_be_null(column="order_id")
validator.expect_column_values_to_not_be_null(column="customer_id")
validator.expect_column_values_to_not_be_null(column="order_date")

# Allow some nulls in total_amount (e.g., pending orders)
validator.expect_column_values_to_not_be_null(
    column="total_amount",
    mostly=0.95, # At least 95% non-null
)

# Value ranges
validator.expect_column_values_to_be_between(
    column="total_amount",
    min_value=0,
    max_value=100000,
)

# Valid categories
validator.expect_column_values_to_be_in_set(
    column="status",
    value_set=["pending", "processing", "shipped", "delivered", "cancelled"],
)

# Uniqueness
validator.expect_column_values_to_be_unique(column="order_id")

# Row count within expected range
validator.expect_table_row_count_to_be_between(
    min_value=1000,
    max_value=500000,
)

# No future dates
validator.expect_column_values_to_be_between(
    column="order_date",
    max_value="2026-02-20", # Today + 1 day buffer
)

```

```
# Save the suite
validator.save_expectation_suite(discard_failed_expectations=False)
```

Running Validations via Checkpoints

Expectations don't do anything until you run them. Checkpoints handle execution:

```
checkpoint = context.add_or_update_checkpoint(
    name="raw_orders_checkpoint",
    validations=[
        {
            "batch_request": batch_request,
            "expectation_suite_name": "raw_orders_suite",
        },
    ],
)

# Run it
result = checkpoint.run()

# Check if everything passed
if not result.success:
    for validation_result in result.run_results.values():
        for expectation_result in validation_result["validation_result"]["results"]:
            if not expectation_result["success"]:
                print(f"FAILED: {expectation_result['expectation_config']['expectation_
type']}")
                print(f"  Details: {expectation_result['result']}")
                raise Exception("Data quality checks failed!")
else:
    print("All data quality checks passed!")
```

Expected output (all passing):

```
All data quality checks passed!
```

Data Docs – Auto-Generated Reports

One of GX's killer features:

```
context.build_data_docs()
context.open_data_docs()
```

This opens a browser with a report showing every expectation, whether it passed or failed, with details. When an analyst asks “how do you know the data is good?” you send them the Data Docs link.

Pro Tip: Data Docs are a great way to build trust with stakeholders. Set up automated publishing so the latest quality report is always available at a known URL.

Soda – The Simpler Alternative

Soda is YAML-based and simpler for basic checks:

```
pip install soda-core-postgres
```

Create `checks/raw_orders.yml` :

```
checks for raw_orders:
  - row_count > 0
  - missing_count(order_id) = 0
  - missing_count(customer_id) = 0
  - missing_percent(total_amount) < 5%
  - duplicate_count(order_id) = 0
  - min(total_amount) >= 0
  - max(total_amount) <= 100000
  - invalid_count(status) = 0:
    valid values: ['pending', 'processing', 'shipped', 'delivered', 'cancelled']
  - freshness(order_date) < 2d
  - anomaly detection for row_count
```

Soda configuration (`configuration.yml`):

```
data_source ecommerce:
  type: postgres
  host: localhost
  port: 5432
  username: postgres
  password: postgres
  database: ecommerce
```

Run it:

```
soda scan -d ecommerce -c configuration.yml checks/raw_orders.yml
```

Soda is great for teams that want something quick and YAML-driven. Great Expectations is more powerful and flexible, especially for complex checks and Airflow integration. Use whichever fits your team.

See companion code: The `soda/` directory in the module-7 project contains working Soda check files and configuration.

`de-fast-track/modules/module-7/solution/soda/checks/raw_orders.yml` `de-fast-track/modules/module-7/solution/soda/configuration.yml`

Common Mistake: - Writing 200 expectations for every table. Start with the critical checks and add more as you discover issues. - Not versioning your expectation suites. They're config – they belong in Git. - Running GX checks but not failing the pipeline when they fail. If you don't stop bad data, the checks are pointless.

Checkpoint

1. What's the difference between an expectation suite and a checkpoint in Great Expectations?
2. When would you choose Soda over Great Expectations?
3. What does the `mostly=0.95` parameter do in a null check expectation?

7.4 dbt Tests and Custom Test Macros

TL;DR: Great Expectations checks your raw data. dbt tests validate your *transformation logic* – ensuring your SQL produces correct results and your business rules hold. dbt has generic tests (unique, not_null, relationships) and custom tests (any SQL that returns failing rows).

Great Expectations checks your raw data. But what about the *transformations*? What if your SQL that calculates revenue is wrong? What if your join produces duplicates? That's where dbt tests come in.

The dbt testing philosophy: test your assumptions, not your transformations. You don't need to test that `SELECT` works. You need to test that `order_id` is actually unique, that every order has a valid customer, and that revenue is never negative. You're testing what *should* be true about your data, and catching the cases where reality disagrees.

Interview angle: When they ask “how do you ensure data quality,” here's the framework: “I use a layered approach. Data contracts validate schema at ingestion. Great Expectations or Soda checks raw data quality – nulls, freshness, volume anomalies. dbt tests validate transformation logic – referential integrity, business rules, uniqueness constraints. And I have alerting so I know within minutes when something breaks, not days.”

Generic Tests

In your `schema.yml`:

```

version: 2
models:
  - name: stg_orders
    description: "Staged orders from raw source"
    columns:
      - name: order_id
        description: "Unique order identifier"
        tests:
          - not_null
          - unique
      - name: customer_id
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
      - name: order_date
        tests:
          - not_null
      - name: status
        tests:
          - accepted_values:
              values: ['pending', 'completed', 'returned', 'cancelled']
      - name: total_amount
        tests:
          - not_null
  - name: stg_customers
    description: "Staged customers"
    columns:
      - name: customer_id
        tests:
          - not_null
          - unique
      - name: segment
        tests:
          - accepted_values:
              values: ['Enterprise', 'SMB', 'Consumer']

```

See companion code: The starter and solution `schema.yml` files are in the module-7 project:

`de-fast-track/modules/module-7/starter/dbt/schema.yml` (with TODOs) `de-fast-track/modules/module-7/solution/dbt/schema.yml` (complete)

```
dbt test
```

Expected output:

Completed successfully

Pass: 7 Warn: 0 Error: 0 Skip: 0 Total: 7

Custom Tests

Real business logic needs custom tests. Create files in `tests/` – a failing test returns rows, and zero rows means pass.

`tests/assert_order_total_matches_line_items.sql` :

```
-- Fails if ANY order's total doesn't match its line items sum
SELECT
  o.order_id,
  o.total_amount AS order_total,
  SUM(oi.quantity * oi.unit_price - oi.discount) AS calculated_total,
  ABS(o.total_amount - SUM(oi.quantity * oi.unit_price - oi.discount)) AS difference
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_order_items') }} oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_amount
HAVING ABS(o.total_amount - SUM(oi.quantity * oi.unit_price - oi.discount)) > 0.01
```

`tests/assert_no_negative_revenue.sql` :

```
-- No revenue should be negative after discounts
SELECT order_id, total_amount
FROM {{ ref('stg_orders') }}
WHERE total_amount < 0
```

See companion code:

```
de-fast-track/modules/module-7/solution/dbt/tests/assert_order_total_matches_line_items.sql
de-fast-track/modules/module-7/solution/dbt/tests/assert_no_negative_revenue.sql
```

Custom Generic Tests (Macros)

If you write the same pattern for multiple models, make it a reusable macro.

`macros/test_row_count_within_range.sql` :

```
{% macro test_row_count_within_range(model, min_count=1, max_count=None) %}

WITH row_count AS (
  SELECT COUNT(*) AS cnt FROM {{ model }}
)
SELECT cnt
FROM row_count
WHERE cnt < {{ min_count }}
{% if max_count is not none %}
  OR cnt > {{ max_count }}
{% endif %}

{% endmacro %}
```

Use it in `schema.yml`:

```
models:
  - name: fact_orders
    tests:
      - row_count_within_range:
          min_count: 1000
          max_count: 5000000
```

See companion code:

```
de-fast-track/modules/module-7/solution/dbt/macros/
test_row_count_within_range.sql
```

Freshness Tests

dbt has source freshness built in:

```
sources:
  - name: raw
    database: ecommerce
    schema: raw
    tables:
      - name: orders
        loaded_at_field: loaded_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

```
dbt source freshness
```


Common Mistake: - Only using generic tests. `unique` and `not_null` are just the baseline. Custom tests for business logic are where the real value is. - Not running `dbt test` in CI/CD. Tests should run on every PR and every production deploy.

Pro Tip: Create a `tests/` directory structure that mirrors your models. When a model changes, the associated tests are easy to find and update.

Checkpoint

1. What's the difference between a dbt generic test and a custom test?
2. In a custom dbt test SQL file, what does it mean when the query returns rows?
3. How would you test that a fact table's foreign key always has a matching dimension record?

7.5 Data Contracts – Defining and Enforcing Them

TL;DR: A data contract is a formal agreement between data producers and consumers. It specifies schema, semantics, SLAs, and quality guarantees. Without one, upstream changes silently break your pipeline. Start simple with YAML files in a shared Git repo.

Here's a scenario you'll hit in every data team: the backend team changes a column name. They don't tell you. Your pipeline breaks at 2 AM. You find out at 9 AM when dashboards go red.

Whose fault is it? Neither side's. The fault is that there's no *contract* between the producer and the consumer.

At your job, this is how it goes down. The backend team ships a “small refactor” on Friday afternoon. They rename `user_email` to `email_address` in the API response. Your pipeline ingests the data fine – no errors, because the JSON just has a different key now. But your downstream `dim_customers` table has NULLs in the email column for the entire weekend. Marketing sends a campaign to “Dear NULL” on Monday morning. Nobody's happy.

What a Data Contract Specifies

A data contract covers:

- **Schema** – column names, types, nullable or not
- **Semantics** – what each field means (is `amount` in cents or dollars?)
- **SLAs** – when data will be available, how fresh it'll be
- **Quality guarantees** – max null rate, valid value ranges
- **Change management** – how changes get communicated and versioned

Think of it like a REST API contract. If an API changes its response format without warning, that's a bug. Same principle applies to data.

Key Concept: Data Contract A formal, versioned agreement between a data producer and consumer that defines the expected schema, semantics, quality, and delivery SLAs. A contract without enforcement is just documentation.

Implementing a Basic Contract

```
contracts/raw_orders_contract.yml :
```

```

contract:
  name: raw_orders
  version: "1.0"
  owner: data-engineering
  description: "Raw orders from the ShopFast application database"

  schema:
    fields:
      - name: order_id
        type: integer
        required: true
        unique: true
      - name: customer_id
        type: integer
        required: true
      - name: order_date
        type: timestamp
        required: true
      - name: status
        type: string
        required: true
        allowed_values: [pending, completed, returned, cancelled]
      - name: total_amount
        type: decimal
        required: true
        min_value: 0

  quality:
    freshness:
      field: order_date
      max_age: "48h"
    row_count:
      min: 1
    custom_checks:
      - name: "order_total_positive"
        sql: "SELECT COUNT(*) FROM {table} WHERE total_amount < 0"
        expected: 0

  sla:
    availability: "99.9%"
    update_frequency: "daily"

```

See companion code:

[de-fast-track/modules/module-7/solution/contracts/raw_orders_contract.yml](#) (complete)
 [de-fast-track/modules/module-7/starter/contracts/raw_orders_contract_starter.yml](#) (with TODOs)

Python Contract Validator

This is a production pattern: data contracts as the API between teams. The contract YAML is the interface definition, and the validator is the runtime enforcement.

```

"""Data contract validator -- validates data against YAML contract specs."""
import yaml
import psycopg2
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from pathlib import Path

@dataclass
class ContractViolation:
    check_name: str
    severity: str
    message: str
    details: dict = field(default_factory=dict)

@dataclass
class ValidationResult:
    contract_name: str
    passed: bool
    violations: list[ContractViolation] = field(default_factory=list)
    checked_at: datetime = field(default_factory=datetime.now)

    @property
    def error_count(self) -> int:
        return sum(1 for v in self.violations if v.severity == "error")

    @property
    def warning_count(self) -> int:
        return sum(1 for v in self.violations if v.severity == "warning")

class ContractValidator:
    def __init__(self, connection_string: str):
        self.conn = psycopg2.connect(connection_string)

    def validate(self, contract_path: str, table_name: str) -> ValidationResult:
        contract = self._load_contract(contract_path)
        violations = []
        violations.extend(self._check_schema(contract, table_name))
        if "quality" in contract.get("contract", {}):
            violations.extend(
                self._check_quality(contract["contract"]["quality"], table_name)
            )
        return ValidationResult(
            contract_name=contract["contract"]["name"],
            passed=all(v.severity != "error" for v in violations),
            violations=violations,
        )

    def _load_contract(self, path: str) -> dict:
        with open(path) as f:
            return yaml.safe_load(f)

```

```

def _check_schema(self, contract: dict, table_name: str) -> list[ContractViolation]
:
    violations = []
    cur = self.conn.cursor()
    cur.execute("""
        SELECT column_name, data_type, is_nullable
        FROM information_schema.columns
        WHERE table_name = %s ORDER BY ordinal_position
    """, (table_name,))
    actual = {r[0]: {"type": r[1], "nullable": r[2] == "YES"} for r in
cur.fetchall()}

    for field_spec in contract.get("contract", {}).get("schema", {}).get("fields",
[]):
        name = field_spec["name"]
        if name not in actual:
            violations.append(ContractViolation(
                f"schema.field_exists.{name}", "error",
                f"Required field '{name}' not found in table",
            ))
            continue
        if field_spec.get("required"):
            cur.execute(f"SELECT COUNT(*) FROM {table_name} WHERE {name} IS NULL")
            nulls = cur.fetchone()[0]
            if nulls > 0:
                violations.append(ContractViolation(
                    f"data.not_null.{name}", "error",
                    f"Found {nulls} null values in required field '{name}'",
                    {"null_count": nulls},
                ))
        if field_spec.get("unique"):
            cur.execute(f"SELECT COUNT(*) - COUNT(DISTINCT {name}) FROM
{table_name}")
            dupes = cur.fetchone()[0]
            if dupes > 0:
                violations.append(ContractViolation(
                    f"data.unique.{name}", "error",
                    f"Found {dupes} duplicate values in unique field '{name}'",
                ))
    return violations

def _check_quality(self, quality: dict, table_name: str) -> list[ContractViolation]
:
    violations = []
    cur = self.conn.cursor()

    if "row_count" in quality:
        cur.execute(f"SELECT COUNT(*) FROM {table_name}")
        count = cur.fetchone()[0]
        min_count = quality["row_count"].get("min", 0)
        if count < min_count:
            violations.append(ContractViolation(
                "quality.row_count", "error",
                f"Row count {count} below minimum {min_count}",
            ))

```

```

if "freshness" in quality:
    field_name = quality["freshness"]["field"]
    cur.execute(f"SELECT MAX({field_name}) FROM {table_name}")
    max_date = cur.fetchone()[0]
    if max_date:
        hours = int(quality["freshness"]["max_age"].replace("h", ""))
        if datetime.now() - max_date > timedelta(hours=hours):
            violations.append(ContractViolation(
                "quality.freshness", "error",
                f"Data is stale. Latest record: {max_date}",
            ))

for check in quality.get("custom_checks", []):
    cur.execute(check["sql"].replace("{table}", table_name))
    result = cur.fetchone()[0]
    if result != check["expected"]:
        violations.append(ContractViolation(
            f"quality.custom.{check['name']}", "error",
            f"Check '{check['name']}' failed: got {result}, expected {check['ex
pected']}",
        ))
return violations

def close(self):
    self.conn.close()

```

Expected output (violation detected):

```

[nullable] customer_id: 3 null values in non-nullable column
[allowed_values] status: 7 values not in allowed set
Contract violated: 2 issues found

```

See companion code:

[de-fast-track/modules/module-7/solution/quality/contract_validator.py](#) (complete)

[de-fast-track/modules/module-7/starter/quality/contract_validator_starter.py](#)

(with TODOs)

Pro Tip: At smaller companies, a YAML contract file in a shared Git repo is often enough. The point isn't fancy tooling – it's having an explicit agreement that both sides can reference. Big companies like Google, Airbnb, and Netflix all use data contracts internally.

Common Mistake: - Making contracts too strict too fast. Start with critical fields and expand. - Not versioning contracts. When the schema changes, bump the version and communicate. - Having contracts that nobody checks. A contract without enforcement is just documentation.

Checkpoint

1. What five things does a data contract specify?

2. Why should you version your data contracts?
 3. What's the difference between a data contract and a Great Expectations suite?
-

7.6 Monitoring, Alerting, and Incident Response

TL;DR: Monitor at three levels: pipeline health, data quality, and business metrics. Alert only on actionable failures (avoid alert fatigue). When bad data gets through, follow a structured incident response: assess scope -> stop the bleeding -> communicate -> root cause -> fix & backfill -> post-mortem.

You've built quality checks. They run every day. Everything's green. Until it's not. And when it's not, you need to know *immediately* – not when an analyst Slacks you 3 hours later.

Three Monitoring Layers

1. **Pipeline health** – Did the DAG run? Did it succeed? How long did it take?
2. **Data quality** – Did the quality checks pass? What's the trend over time?
3. **Business metrics** – Are the downstream metrics in expected ranges?

Pipeline Health Monitoring in Airflow

```
from airflow import DAG
from airflow.providers.slack.notifications.slack import send_slack_notification
from datetime import datetime, timedelta

default_args = {
    "owner": "data-engineering",
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "execution_timeout": timedelta(hours=1),
    "sla": timedelta(hours=2), # Must complete within 2 hours
    "on_failure_callback": send_slack_notification(
        slack_conn_id="slack_webhook",
        text="DAG `{{ dag.dag_id }}` failed on task `{{ ti.task_id }}`\n"
        "Execution date: {{ ds }}\n"
        "Log: {{ ti.log_url }}",
        channel="#data-alerts",
    ),
    "on_sla_miss_callback": send_slack_notification(
        slack_conn_id="slack_webhook",
        text="SLA MISS: DAG `{{ dag.dag_id }}` didn't complete in time\n"
        "Expected by: {{ sla }}\n"
        "This affects downstream dashboards.",
        channel="#data-alerts",
    ),
}

dag = DAG(
    "ecommerce_pipeline",
    default_args=default_args,
    schedule_interval="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
)
```

Custom Quality Alerting to Slack

```

import requests

def send_quality_alert(
    webhook_url: str,
    pipeline_name: str,
    check_results: list[dict],
) -> None:
    """Send data quality alert to Slack."""
    failures = [r for r in check_results if not r["success"]]

    if not failures:
        return # No news is good news

    blocks = [
        {
            "type": "header",
            "text": {
                "type": "plain_text",
                "text": f"Data Quality Alert: {pipeline_name}",
            },
        },
        {
            "type": "section",
            "text": {
                "type": "mrkdown",
                "text": f"*{len(failures)} checks failed* out of {len(check_results)} total",
            },
        },
    ]

    for failure in failures[:5]: # Cap at 5 to avoid spam
        blocks.append({
            "type": "section",
            "text": {
                "type": "mrkdown",
                "text": (
                    f"*{failure['check_name']}*\n"
                    f"Column: `{failure.get('column', 'N/A')}`\n"
                    f"Details: {failure['details']}"
                ),
            },
        })

    if len(failures) > 5:
        blocks.append({
            "type": "section",
            "text": {
                "type": "mrkdown",
                "text": f"...and {len(failures) - 5} more failures. Check Data Docs for full report.",
            },
        })

```

```
requests.post(webhook_url, json={"blocks": blocks})
```

Incident Response Playbook

When bad data gets through – and it will eventually – follow this playbook:

Step 1: Assess Scope (5 minutes) - What data is affected? Which tables, which date ranges? - Who's impacted? Which dashboards, reports, or downstream systems? - Is the bad data still flowing, or was it a one-time issue?

Step 2: Stop the Bleeding (10 minutes) - Pause the pipeline (Airflow: pause the DAG) - If bad data is already in the warehouse, mark it – don't delete it yet

```
-- Quarantine bad data (don't delete -- you'll need it for investigation)
ALTER TABLE fact_orders ADD COLUMN IF NOT EXISTS _quarantined BOOLEAN DEFAULT FALSE;

UPDATE fact_orders
SET _quarantined = TRUE
WHERE loaded_at BETWEEN '2026-02-18 00:00:00' AND '2026-02-18 23:59:59'
AND <condition_that_identifies_bad_data>;
```

Step 3: Communicate (immediately) - Post in #data-alerts with what you know so far - Be honest: *"We identified a data quality issue. Dashboard X may show incorrect numbers for [date range]. We're investigating."*

Step 4: Root Cause (30 min - 2 hours) - Check upstream sources: Did the source data change? - Check pipeline logs: Any errors or warnings you missed? - Check transformations: Did a code change introduce a bug?

Step 5: Fix and Backfill (varies) - Fix the root cause - Backfill the affected data - Re-run quality checks to confirm

Step 6: Post-Mortem (next day) - What happened? - Why didn't our checks catch it? - What check do we add to prevent this in the future?

Common Mistake: - Alert fatigue. If you alert on everything, people ignore alerts. Only alert on *actionable* failures. - **No runbook.** When it's 2 AM and you're half asleep, you need a checklist, not a thought process. - **Deleting bad data** instead of quarantining it. You might need it for the investigation.

Checkpoint

1. What are the three levels of monitoring for a data pipeline?
2. Why should you quarantine bad data instead of deleting it?
3. What's the purpose of a post-mortem after a data incident?

7.7 Building a Quality Culture: Shift-Left Testing

This is the production pattern that separates mature data teams from reactive ones: **shift-left testing**. The idea, borrowed from software engineering, is simple: catch problems as early as possible in the pipeline, not at the end.

```
Traditional: Extract -> Load -> Transform -> [CHECK] -> Serve
Shift-left:  Extract -> [CHECK] -> Load -> [CHECK] -> Transform -> [CHECK] -> Serve
```

At each checkpoint, you have a quality gate. If data fails the gate, it doesn't move forward. Bad data gets quarantined, alerts fire, and the pipeline stops – instead of silently corrupting your warehouse.

At your job, this is the difference between: “We caught the schema change at ingestion, quarantined the bad batch, and alerted the backend team within 15 minutes” vs. “The CEO asked why revenue dropped 90% and we spent 4 hours figuring out what happened.”

The Quality Gate Pattern

Every quality gate asks three questions: 1. **Does the data look like what we expect?** (schema, types, columns) 2. **Is the data reasonable?** (nulls, ranges, volumes, freshness) 3. **Is the data consistent?** (cross-table checks, referential integrity)

If any answer is “no” at severity=error, the pipeline stops. Severity=warning logs and continues.

7.8 Module 7 Project – Data Quality Pipeline

TL;DR: Add a comprehensive data quality layer to your Airflow pipeline: contract validation before loading, Great Expectations checks after transformation, Slack alerts on failure, and quarantine for bad data.

Overview

You're going to add comprehensive data quality checks to your Airflow pipeline from Module 4. When complete, your pipeline will:

1. Validate incoming data against a contract before loading
2. Run Great Expectations checks after each transformation step
3. Alert to Slack (or console) when checks fail
4. Quarantine bad data instead of loading it to production tables

Project Structure

```
module-7-project/  
  dags/  
  |   ecommerce_pipeline_with_quality.py  
  contracts/  
  |   raw_orders_contract.yml  
  great_expectations/  
  |   great_expectations.yml  
  |   expectations/  
  |       raw_orders_suite.json  
  |       fact_orders_suite.json  
  quality/  
  |   __init__.py  
  |   contract_validator.py  
  |   alerts.py  
  tests/  
  |   test_contract_validator.py  
  |   test_quality_checks.py  
  docker-compose.yml  
  README.md
```

Step 1: Set Up the Contract

Use the contract YAML from Section 7.5. Customize it for your specific e-commerce schema.

Step 2: Build the Contract Validator

Take the `validate_contract()` function from Section 7.5 and enhance it with severity levels and structured results:

```

# quality/contract_validator.py
import yaml
import pandas as pd
from dataclasses import dataclass, asdict
from datetime import datetime
import json

@dataclass
class ContractViolation:
    field: str
    check: str
    details: str
    severity: str = "error" # "error" or "warning"

@dataclass
class ValidationResult:
    contract_name: str
    contract_version: str
    validated_at: str
    row_count: int
    violations: list[ContractViolation]

    @property
    def passed(self) -> bool:
        return not any(v.severity == "error" for v in self.violations)

    def to_json(self) -> str:
        return json.dumps(asdict(self), indent=2, default=str)

def validate_contract(df: pd.DataFrame, contract_path: str) -> ValidationResult:
    """Full contract validation. Returns a ValidationResult."""
    # YOUR IMPLEMENTATION HERE
    # Use the code from Section 7.5 as your starting point
    # Return a ValidationResult instead of a list
    pass

```

Step 3: Set Up Great Expectations

Create expectation suites for: - `raw_orders_suite` – checks on raw ingested data - `fact_orders_suite` – checks on transformed fact table

Minimum expectations per suite: - Schema validation (columns exist, correct types) - Null checks on primary keys and required fields - Value range checks on numeric columns - Allowed value checks on categorical columns - Uniqueness on primary keys - Row count within expected range

Step 4: Build the Airflow DAG


```

# dags/ecommerce_pipeline_with_quality.py
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["ecommerce", "data-quality"],
)
def ecommerce_pipeline_with_quality():

    @task()
    def extract_orders(**context):
        """Extract orders from source."""
        pass

    @task()
    def validate_contract(**context):
        """Validate raw data against contract BEFORE loading.
        If contract fails, raise exception (pipeline stops)."""
        pass

    @task()
    def load_raw(**context):
        """Load validated data to raw layer."""
        pass

    @task()
    def run_raw_quality_checks(**context):
        """Run Great Expectations on raw data.
        If critical checks fail, raise exception."""
        pass

    @task()
    def transform(**context):
        """Transform raw to fact/dim tables."""
        pass

    @task()
    def run_transformed_quality_checks(**context):
        """Run Great Expectations on transformed data."""
        pass

    @task(trigger_rule="one_failed")
    def quarantine_bad_data(**context):
        """Move bad data to quarantine table instead of deleting."""
        pass

    @task(trigger_rule="all_done")
    def send_quality_report(**context):
        """Send summary of quality results (pass or fail)."""
        pass

```

```
# Define the flow
raw_data = extract_orders()
contract_ok = validate_contract()
loaded = load_raw()
raw_quality = run_raw_quality_checks()
transformed = transform()
final_quality = run_transformed_quality_checks()
quarantine = quarantine_bad_data()
report = send_quality_report()

raw_data >> contract_ok >> loaded >> raw_quality >> transformed >> final_quality
[raw_quality, final_quality] >> quarantine
[final_quality, quarantine] >> report

ecommerce_pipeline_with_quality()
```

Step 5: Write Tests

```
# tests/test_contract_validator.py
import pytest
import pandas as pd
from quality.contract_validator import validate_contract, ValidationResult

def test_valid_data_passes_contract():
    df = pd.DataFrame({
        "order_id": [1, 2, 3],
        "customer_id": [100, 101, 102],
        "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
        "total_amount": [29.99, 49.99, 19.99],
        "status": ["pending", "shipped", "delivered"],
    })
    result = validate_contract(df, "contracts/raw_orders_contract.yml")
    assert result.passed

def test_null_primary_key_fails():
    df = pd.DataFrame({
        "order_id": [1, None, 3],
        "customer_id": [100, 101, 102],
        "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
        "total_amount": [29.99, 49.99, 19.99],
        "status": ["pending", "shipped", "delivered"],
    })
    result = validate_contract(df, "contracts/raw_orders_contract.yml")
    assert not result.passed

def test_invalid_status_fails():
    df = pd.DataFrame({
        "order_id": [1, 2, 3],
        "customer_id": [100, 101, 102],
        "order_date": pd.to_datetime(["2026-01-01", "2026-01-02", "2026-01-03"]),
        "total_amount": [29.99, 49.99, 19.99],
        "status": ["pending", "shipped", "YOLO"], # Invalid status
    })
    result = validate_contract(df, "contracts/raw_orders_contract.yml")
    assert not result.passed
    assert any(v.check == "allowed_values" for v in result.violations)
```

See companion code:

```
de-fast-track/modules/module-7/solution/tests/test_contract_validator.py
```

Try It Yourself

- Add a test for negative `total_amount` values.

- Add a test for duplicate `order_id` values.
- Add a test that verifies extra columns generate a warning but not an error.

Expected Output

When you run the DAG successfully: - All quality checks pass -> green in Airflow, summary report sent - Bad data detected -> pipeline stops at the quality check step, bad data quarantined, alert sent - Quality report shows: total checks run, pass/fail per check, trend vs previous run

Completion Checklist

- ☐ Contract YAML file covers all critical fields
- ☐ Contract validator catches: missing columns, nulls, invalid values, range violations
- ☐ Great Expectations suites for both raw and transformed data
- ☐ Airflow DAG has quality checks at multiple stages
- ☐ Pipeline stops on critical failures (doesn't load bad data)
- ☐ Quarantine mechanism for bad data
- ☐ Alerting function sends notifications on failure
- ☐ At least 5 unit tests for the contract validator
- ☐ README documents the quality checks and how to add new ones

Interview Prep: Data Quality

When they ask “how do you ensure data quality,” here’s the framework that will set you apart:

The Layered Approach: 1. **Data contracts** at the boundary between teams – schema, semantics, SLAs defined in YAML, validated at ingestion 2. **Schema validation** before data enters the pipeline – catch drift immediately 3. **Completeness and freshness checks** on raw data – nulls, staleness, volume anomalies 4. **Business logic tests** on transformed data (dbt tests) – referential integrity, range checks, uniqueness 5. **Automated quality frameworks** (Great Expectations or Soda) for scalable, config-driven checks 6. **Monitoring and alerting** – know within minutes, not days, when something breaks 7. **Incident response** – quarantine, communicate, root cause, backfill, post-mortem

Key phrases interviewers want to hear: - “Shift-left testing” – catch issues early, not at the end - “Quality gates” – data doesn’t move forward until it passes checks - “Data contracts” – formal agreements between producers and consumers - “Defense in depth” – multiple layers of checks, not just one - “Quarantine, don’t delete” – preserve bad data for investigation

The story to tell: “At my previous role, we had a pipeline that loaded orders data daily. Everything looked fine until our CEO’s dashboard showed a 90% revenue drop. Turned out a source API changed its schema silently. After that incident, I implemented data contracts with the backend team, added schema validation at ingestion, and built automated quality checks with Great Expectations. We went from finding issues days later to catching them within minutes.”

What’s Next

With data quality and testing covered, Module 8 shifts to cloud infrastructure for data engineers. You’ll learn the AWS services that matter for DE work (S3, Redshift, Athena, Glue, IAM), how to deploy your pipelines to the cloud, and the patterns that scale.

End of Module 7

Module 8: Cloud Infrastructure for Data Engineers

At some point, your laptop isn't enough. You've built pipelines, modeled data in dbt, added quality checks – all running locally. But nobody is querying your localhost Postgres from a dashboard. To deliver real value, you need to deploy everything you've built to the cloud. This module shows you how.

You don't need to be a DevOps engineer, but you need to deploy your own stuff. AWS-focused with transferable concepts.

Here's a conversation that happens with at least 20 junior data engineers: *"I know Python. I know SQL. I can build pipelines locally. But when it comes to deploying anything to the cloud, I freeze. There are 200+ AWS services and I don't know where to start."*

Good news: you need maybe 10 of them.

8.1 Cloud Fundamentals for DEs – What You Need to Know

TL;DR: Out of 200+ AWS services, you need about 10 for data engineering. The core pattern is: S3 for storage, RDS/Redshift for databases, Athena for ad-hoc queries, ECS for compute, and IAM for security. Learn one cloud well and the others are just different UIs.

The Three Cloud Superpowers

1. **Elasticity** – scale up when you need it, scale down when you don't
2. **Managed services** – someone else handles the patching, backups, and uptime
3. **Pay-per-use** – you pay for what you consume, not what you provision (mostly)

The AWS Data Engineering Stack

For data engineers, the cloud maps to your pipeline stages:

Source -> Ingest -> Store -> Process -> Serve -> Monitor

PIPELINE STAGE	AWS SERVICE	WHAT IT DOES
Store (raw)	S3	Object storage – your data lake
Store (structured)	RDS (Postgres)	Managed relational database
Store (warehouse)	Redshift	Columnar analytics database
Process (batch)	EMR / Glue	Managed Spark
Process (serverless)	Lambda	Run code without servers
Query (ad-hoc)	Athena	SQL on S3 files directly
Orchestrate	MWAA / Step Functions	Managed Airflow / workflow
Stream	Kinesis / MSK	Managed Kafka
Compute	EC2 / ECS / Fargate	Run containers
IAM	IAM	Access control
Secrets	Secrets Manager	Store credentials

That's it. That's the 80/20 of AWS for data engineers.

At Your Job

Your first cloud task will probably be setting up S3 buckets and IAM roles. Maybe provisioning an RDS instance. You won't be designing VPC architectures or managing Kubernetes clusters on day one. Start with storage and permissions – those are the foundation everything else rests on.

The Data Lake Architecture on AWS

Three layers in S3:

- `/raw/` – data as it arrived, never modified (your safety net)
- `/staging/` – cleaned, validated, intermediate transformations
- `/prod/` – final, analytics-ready tables (Parquet/Delta format)

Key Concept: Medallion Architecture This layered approach is called the **medallion architecture** (bronze/silver/gold) or raw/staging/prod. Same concept, different names. The key principle: raw data is immutable. You never modify it. All transformations produce new data in downstream layers.

GCP/Azure Equivalents

Quick reference if you end up at a non-AWS shop:

AWS	GCP	AZURE
S3	Cloud Storage	Blob Storage
RDS	Cloud SQL	Azure SQL
Redshift	BigQuery	Synapse
EMR	Dataproc	HDInsight
Athena	BigQuery	Synapse Serverless
Lambda	Cloud Functions	Azure Functions
ECS/Fargate	Cloud Run	Container Apps
MWAA	Cloud Composer	Data Factory

The concepts transfer completely. Learn one cloud well, and the others are just different UIs.

Common Mistake - Trying to learn all AWS services. Focus on the data stack above. - Using the AWS Console for everything. That's fine for learning, but production uses Infrastructure as Code (Terraform). - Running expensive services 24/7 during learning. Set up billing alerts first (covered in Section 8.6).

Checkpoint

1. What are the three layers of a data lake on S3?
2. What AWS service would you use to run SQL queries directly on files stored in S3?
3. What's the GCP equivalent of S3?

8.2 The AWS Data Stack – S3, RDS, Glue, Athena

TL;DR: S3 is your data lake foundation – cheap, durable, infinitely scalable. Store data as Parquet (not CSV) for 10-100x better query performance. Use Athena to query S3 with standard SQL. Always partition your data to reduce costs. For most startups, S3 + Athena + RDS Postgres covers 90% of use cases.

S3 – Your Data Lake

S3 is the foundation of everything. It's cheap (\$0.023/GB/month for standard storage), durable (99.999999999% – eleven 9s), and infinitely scalable.

Set up a data lake bucket:

```
# Create the bucket
aws s3 mb s3://my-ecommerce-data-lake-2026 --region us-east-1

# Create the layer structure
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key raw/
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key staging/
aws s3api put-object --bucket my-ecommerce-data-lake-2026 --key prod/
```

Upload data with Python and boto3:

```
import boto3
import pandas as pd
from datetime import datetime

s3 = boto3.client("s3")
BUCKET = "my-ecommerce-data-lake-2026"

def upload_to_data_lake(
    df: pd.DataFrame,
    layer: str,          # raw, staging, prod
    dataset: str,        # orders, customers, etc.
    partition_date: str | None = None,
) -> str:
    """Upload a DataFrame to S3 as Parquet with date partitioning."""
    if partition_date is None:
        partition_date = datetime.now().strftime("%Y-%m-%d")

    # Hive-style partitioning: /layer/dataset/date=YYYY-MM-DD/data.parquet
    key = f"{layer}/{dataset}/date={partition_date}/data.parquet"

    # Write to Parquet in memory, then upload
    buffer = df.to_parquet(index=False)
    s3.put_object(Bucket=BUCKET, Key=key, Body=buffer)

    print(f"Uploaded {len(df)} rows to s3://{BUCKET}/{key}")
    return f"s3://{BUCKET}/{key}"

# Example: upload raw orders
orders_df = pd.DataFrame({
    "order_id": range(1, 1001),
    "customer_id": [i % 200 + 1 for i in range(1000)],
    "order_date": pd.date_range("2026-02-01", periods=1000, freq="h"),
    "total_amount": [round(i * 1.5 + 10, 2) for i in range(1000)],
    "status": ["shipped" * 400 + "delivered" * 300 + "pending" * 200 +
["cancelled" * 100,
})

upload_to_data_lake(orders_df, "raw", "orders", "2026-02-18")
```

Expected output:

```
Uploaded 1000 rows to s3://my-ecommerce-data-lake-2026/raw/orders/date=2026-02-18/data.parquet
```

Pro Tip: Why Parquet? Parquet is columnar, compressed, and typed. For analytics workloads, it's 10-100x faster to query than CSV because: (1) query engines only read the columns they need, (2) compression reduces I/O, (3) built-in statistics allow predicate pushdown (skipping rows that don't match your WHERE clause). Never store analytics data as CSV in S3.

Athena – SQL on S3

Athena lets you query S3 files using standard SQL. No server to manage, no data to load into a database. You pay \$5 per TB scanned.

Create a table definition:

```
-- Partitioned table -- critical for cost and performance
CREATE EXTERNAL TABLE IF NOT EXISTS raw_orders_partitioned (
  order_id INT,
  customer_id INT,
  order_date TIMESTAMP,
  total_amount DOUBLE,
  status STRING
)
PARTITIONED BY (date STRING)
STORED AS PARQUET
LOCATION 's3://my-ecommerce-data-lake-2026/raw/orders/'
TBLPROPERTIES ('parquet.compression'='SNAPPY');

-- Tell Athena to discover partitions
MSCK REPAIR TABLE raw_orders_partitioned;
```

Now query it – just SQL:

```
SELECT
  status,
  COUNT(*) AS order_count,
  ROUND(AVG(total_amount), 2) AS avg_amount
FROM raw_orders_partitioned
WHERE date = '2026-02-18' -- Partition filter: only scans this one day
GROUP BY status
ORDER BY order_count DESC;
```

Expected output:

status	order_count	avg_amount
shipped	400	310.25
delivered	300	760.50
pending	200	1210.75
cancelled	100	1511.00

Common Mistake The partition filter (`WHERE date = '2026-02-18'`) is critical – it means Athena only scans that one partition instead of your entire dataset. Without partitions, every query scans everything. That's expensive and slow.

AWS Glue – The Catalog

Glue does two things:

1. **Glue Data Catalog** – a metadata store (think: Hive Metastore). Athena uses this to know about your tables.
2. **Glue ETL** – managed Spark jobs for transformations. (Honestly, most teams use Airflow + Spark directly because Glue ETL has weird limitations and debugging is painful.)

Set up a Glue Crawler to auto-discover your S3 data:

```
aws glue create-crawler \
  --name ecommerce-raw-crawler \
  --role AWSGlueServiceRole \
  --database-name ecommerce_db \
  --targets '{"S3Targets": [{"Path": "s3://my-ecommerce-data-lake-2026/raw/"}]}'

aws glue start-crawler --name ecommerce-raw-crawler
```

The crawler scans your S3 data, infers the schema, and creates table definitions in the Glue Catalog. Athena can then query these tables without you manually writing CREATE TABLE statements.

When to Use What

SCENARIO	BEST CHOICE
Small data (< 10GB), simple queries	Athena on S3 (cheapest, simplest)
Need ACID transactions, complex joins	RDS Postgres (or Aurora)
Large-scale analytics, many users	Redshift (columnar warehouse)
Spark transformations	EMR (or Glue ETL if you must)

For the course project and most startups, **S3 + Athena + RDS Postgres** covers 90% of use cases.

Checkpoint

1. Why is Parquet dramatically better than CSV for Athena queries?
2. What does `MSCK REPAIR TABLE` do?
3. At what scale would you consider Redshift over Postgres + Athena?

8.3 Infrastructure as Code with Terraform

TL;DR: Terraform lets you define your cloud infrastructure as version-controlled code files. You describe the desired state, Terraform figures out the steps. Three commands: `init`, `plan`, `apply`. Always read the plan before applying. Store state remotely in S3 for team collaboration.

Clicking through the AWS console works until you need to recreate your environment. Imagine you built your whole data platform in AWS. It's running great. Then your company wants a staging environment that mirrors production. You open the AWS Console and think, *"How did I set all this up again?"*

That's the problem Infrastructure as Code solves. Terraform makes infrastructure reproducible – you describe what you want, commit it to Git, and anyone on your team can stand up an identical environment with three commands.

How Terraform Works

1. **Write** – describe what you want in `.tf` files (HCL language)
2. **Plan** – Terraform compares your config to what exists and shows what it'll change
3. **Apply** – Terraform makes the changes

It's declarative: you describe the *desired state*, not the steps to get there.

S3 Data Lake in Terraform

Here is the main configuration. See the companion code in `module-8/solution/terraform/main.tf`:

```
# main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  required_version = ">= 1.5.0"
}

provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  default = "us-east-1"
}

variable "environment" {
  default = "dev"
}

variable "project_name" {
  default = "ecommerce-data-platform"
}
```

Now the S3 bucket with all the production guardrails. See [module-8/solution/terraform/s3.tf](#):

```

# s3.tf
resource "aws_s3_bucket" "data_lake" {
    bucket = "${var.project_name}-${var.environment}-data-lake"

    tags = {
        Environment = var.environment
        Project      = var.project_name
        ManagedBy    = "terraform"
    }
}

# Enable versioning (safety net for accidental deletes)
resource "aws_s3_bucket_versioning" "data_lake" {
    bucket = aws_s3_bucket.data_lake.id
    versioning_configuration {
        status = "Enabled"
    }
}

# Block ALL public access (data lakes should NEVER be public)
resource "aws_s3_bucket_public_access_block" "data_lake" {
    bucket = aws_s3_bucket.data_lake.id

    block_public_acls       = true
    block_public_policy     = true
    ignore_public_acls     = true
    restrict_public_buckets = true
}

# Lifecycle rules: move old raw data to cheaper storage tiers
resource "aws_s3_bucket_lifecycle_configuration" "data_lake" {
    bucket = aws_s3_bucket.data_lake.id

    rule {
        id      = "archive-raw-data"
        status  = "Enabled"

        filter {
            prefix = "raw/"
        }

        transition {
            days          = 90
            storage_class = "STANDARD_IA" # Infrequent Access -- 45% cheaper
        }

        transition {
            days          = 365
            storage_class = "GLACIER"     # Long-term archive -- 80% cheaper
        }
    }
}

```

RDS Postgres in Terraform

See [module-8/solution/terraform/rds.tf](#):

```
# rds.tf
resource "aws_db_instance" "postgres" {
  identifier      = "${var.project_name}-${var.environment}-db"
  engine          = "postgres"
  engine_version = "16.1"
  instance_class  = "db.t3.micro" # Cheapest option for dev

  allocated_storage      = 20
  max_allocated_storage = 100 # Auto-scale up to 100GB
  storage_type           = "gp3"

  db_name  = "ecommerce"
  username = "dataengineer"
  password = var.db_password # Never hardcode passwords!

  publicly_accessible = false # Access only from within VPC
  vpc_security_group_ids = [aws_security_group.db.id]
  db_subnet_group_name   = aws_db_subnet_group.db.name

  backup_retention_period = 7
  backup_window           = "03:00-04:00"

  # Safety: don't delete production databases on terraform destroy
  deletion_protection = var.environment == "prod" ? true : false
  skip_final_snapshot = var.environment == "dev" ? true : false

  tags = {
    Environment = var.environment
    Project      = var.project_name
    ManagedBy    = "terraform"
  }
}

variable "db_password" {
  type        = string
  sensitive   = true # Won't show in terraform plan output
}
```

IAM Role for Your Pipeline

This is where the principle of least privilege lives. Your pipeline should only be able to access the S3 bucket and RDS instance it needs – nothing else. See [module-8/solution/terraform/iam.tf](#):

```

# iam.tf -- Role that your Airflow tasks assume
resource "aws_iam_role" "pipeline_role" {
    name = "${var.project_name}-${var.environment}-pipeline-role"

    assume_role_policy = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Action = "sts:AssumeRole"
                Effect = "Allow"
                Principal = {
                    Service = "ecs-tasks.amazonaws.com"
                }
            },
        ]
    })
}

# Policy: pipeline can read/write to S3 data lake -- nothing else
resource "aws_iam_role_policy" "pipeline_s3_access" {
    name = "s3-data-lake-access"
    role = aws_iam_role.pipeline_role.id

    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Effect = "Allow"
                Action = [
                    "s3:GetObject",
                    "s3:PutObject",
                    "s3:ListBucket",
                    "s3:DeleteObject",
                ]
                Resource = [
                    aws_s3_bucket.data_lake.arn,
                    "${aws_s3_bucket.data_lake.arn}/*",
                ]
            },
        ]
    })
}

```


Running Terraform

```
# Initialize -- downloads provider plugins
terraform init

# Plan -- shows what will be created/changed/destroyed
terraform plan -var="db_password=supersecret123"

# Apply -- actually creates the resources
terraform apply -var="db_password=supersecret123"

# When you're done, destroy everything to stop charges
terraform destroy -var="db_password=supersecret123"
```

Expected plan output:

```
Plan: 6 to add, 0 to change, 0 to destroy.
```

Pro Tip: Always read the plan before applying. Especially look for resources being destroyed – that’s usually not what you want.

Remote State Management

In production, store Terraform state in S3 (not locally) so your team can share it:

```
# backend.tf
terraform {
  backend "s3" {
    bucket     = "my-terraform-state-bucket"
    key        = "ecommerce-data-platform/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-locks" # Prevents concurrent applies
  }
}
```

Common Mistake - Hardcoding secrets in `.tf` files. Use variables with `sensitive = true` or AWS Secrets Manager. - Not using remote state from day one. Local state files get lost and cause drift. - Forgetting `terraform destroy` when done experimenting. That’s how you get a surprise AWS bill.

Checkpoint

1. What are the three Terraform commands and what does each do?
2. Why should you never hardcode passwords in Terraform files?
3. What’s the purpose of the DynamoDB table in the remote state backend config?

8.4 Docker for Data Engineers – Containerizing Pipelines

TL;DR: Docker packages your code AND its environment into a container that runs identically everywhere. Key practices: specific version tags (not `latest`), non-root user, multi-stage builds for smaller images, and layer ordering optimized for cache (dependencies first, code last).

“It works on my machine.” Docker eliminates this by packaging your code and its entire environment into a container that runs the same everywhere. For data pipelines, this matters more than you’d think: a pipeline that runs fine with pandas 2.0 on your Mac might silently produce wrong results with pandas 1.5 on a coworker’s Linux machine. Containers make your pipelines reproducible, isolated, and deployable.

Production-Grade Dockerfile

See the companion code in `module-8/solution/Dockerfile`:

```
# Use a specific version -- reproducibility matters
FROM python:3.11-slim AS base

# Don't run as root in production
RUN groupadd -r pipeline && useradd -r -g pipeline pipeline

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

# Copy dependency files first (Docker caches layers -- this rarely changes)
COPY pyproject.toml uv.lock ./

# Install Python dependencies
RUN pip install uv && uv sync --frozen --no-dev

# Copy the actual code LAST (this layer changes most often)
COPY src/ ./src/
COPY configs/ ./configs/

# Switch to non-root user
USER pipeline

ENTRYPOINT ["python", "-m", "src.main"]
```

Key Concept: Layer Ordering Docker caches each layer. If you put `COPY src/` before `pip install`, every code change invalidates the cache and reinstalls all dependencies. Put dependencies first, code last. This makes rebuilds go from minutes to seconds.

Docker Compose for Multi-Container Pipelines

Your pipeline doesn't run alone – it needs Postgres, maybe MinIO for local S3. Docker Compose orchestrates all of it:

```

# docker-compose.yml
version: "3.8"

services:
  pipeline:
    build: .
    environment:
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/ecommerce
      - S3_BUCKET=my-ecommerce-data-lake-2026
      - AWS_REGION=us-east-1
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./data:/app/data # Mount local data directory for dev

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: ecommerce
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

# Local S3-compatible storage (for development)
minio:
  image: minio/minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  ports:
    - "9000:9000"
    - "9001:9001"
  volumes:
    - minio_data:/data

volumes:
  pgdata:
  minio_data:

```

Multi-Stage Builds for Smaller Images

Production images should be small. Multi-stage builds separate the build environment from runtime:

```

# Build stage -- has all the build tools
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev

# Runtime stage -- only what's needed to run
FROM python:3.11-slim AS runtime
RUN groupadd -r pipeline && useradd -r -g pipeline pipeline
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app

# Copy only installed packages from builder
COPY --from=builder /app/.venv /app/.venv
COPY src/ ./src/
COPY configs/ ./configs/

ENV PATH="/app/.venv/bin:$PATH"
USER pipeline
ENTRYPOINT ["python", "-m", "src.main"]

```

This produces images of 200-400MB instead of 1-2GB.

Deploying to AWS ECS

```

# Build the image
docker build -t ecommerce-pipeline .

# Tag for ECR (Elastic Container Registry)
aws ecr get-login-password --region us-east-1 | \
    docker login --username AWS --password-stdin <account-id>.dkr.ecr.us-
east-1.amazonaws.com

docker tag ecommerce-pipeline:latest \
    <account-id>.dkr.ecr.us-east-1.amazonaws.com/ecommerce-pipeline:latest

# Push to ECR
docker push <account-id>.dkr.ecr.us-east-1.amazonaws.com/ecommerce-pipeline:latest

```

Common Mistake - Using `latest` tag in production. Always tag with a specific version or Git SHA. - Running containers as root. Always use a non-root user. - Putting secrets in Dockerfiles or docker-compose. Use environment variables injected at runtime.

Checkpoint

1. Why should you copy dependency files before source code in a Dockerfile?

2. What's the benefit of a multi-stage Docker build?
3. Why should you never use the `latest` tag in production?

8.5 CI/CD for Data Pipelines – GitHub Actions

TL;DR: CI/CD automates testing and deployment triggered by Git pushes. CI runs tests on every push (catches bugs early). CD deploys to production when tests pass on main. Use GitHub Actions with a test Postgres service, and deploy via ECR + ECS. Never deploy from anywhere but your CI pipeline.

You've got Terraform for infrastructure, Docker for packaging. But deploying manually – SSH, pull, restart – is a liability in production. CI/CD ensures that every change is tested before it reaches production, and that deployments are automated and repeatable.

The GitHub Actions Workflow

See the companion code in `module-8/solution/.github/workflows/data-pipeline.yml`:

```

# .github/workflows/data-pipeline.yml
name: Data Pipeline CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: ecommerce-pipeline
  ECS_SERVICE: ecommerce-pipeline-service
  ECS_CLUSTER: data-platform

jobs:
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_ecommerce
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U test"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install uv
          uv sync --frozen

      - name: Lint
        run: |
          uv run ruff check src/ tests/
          uv run ruff format --check src/ tests/

      - name: Type check

```

```

    run: uv run mypy src/

- name: Run unit tests
  env:
    DATABASE_URL: postgresql://test:test@localhost:5432/test_ecommerce
  run: uv run pytest tests/ -v --tb=short

- name: Run dbt tests
  env:
    DATABASE_URL: postgresql://test:test@localhost:5432/test_ecommerce
  run: |
    cd dbt_project
    uv run dbt deps
    uv run dbt build --target test

build-and-deploy:
  name: Build & Deploy
  needs: test # Waits for test job to pass
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' # Only deploy from main branch

  permissions:
    id-token: write # Required for AWS OIDC
    contents: read

  steps:
    - uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ secrets.AWS_DEPLOY_ROLE_ARN }
        aws-region: ${ env.AWS_REGION }

    - name: Login to Amazon ECR
      id: login-ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Build, tag, and push image to ECR
      id: build-image
      env:
        ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
        IMAGE_TAG: ${ github.sha }
      run: |
        docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG .
        docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:latest .
        docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
        docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest
        echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG" >> $GITHUB_OUTPUT

    - name: Deploy to ECS
      run: |
        aws ecs update-service \
          --cluster $ECS_CLUSTER \
          --service $ECS_SERVICE \

```



```

--force-new-deployment

- name: Wait for deployment
  run: |
    aws ecs wait services-stable \
      --cluster $ECS_CLUSTER \
      --services $ECS_SERVICE
    echo "Deployment successful!"

```

How It Works

The workflow has two jobs:

1. `test` – runs on every push and PR. Spins up a test Postgres, runs linting, type checking, unit tests, and dbt tests. If any step fails, the pipeline stops.
2. `build-and-deploy` – only runs on `main` branch, and only if tests pass. Builds the Docker image, pushes to ECR, updates the ECS service.

`needs: test` means deploy waits for tests to succeed. `if: github.ref == 'refs/heads/main'` means PRs get tested but not deployed.

Terraform Plan in PRs

Add infrastructure validation to pull requests:

```

terraform-plan:
  name: Terraform Plan
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'

  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3

    - name: Terraform Init
      working-directory: terraform/
      run: terraform init

    - name: Terraform Plan
      working-directory: terraform/
      run: terraform plan -no-color

```

Pro Tip: Secrets Management Never put credentials in workflow files. Use GitHub Secrets (repo -> Settings -> Secrets -> Actions). Better yet, use OIDC with `role-to-assume` – no static credentials at all.

Common Mistake - Not running tests in CI. “I’ll test locally” doesn’t scale. - Deploying on every push to main without a staging step. - Not pinning action versions. Use `actions/checkout@v4` not `@main`.

Checkpoint

1. What's the difference between CI and CD?
2. Why does the deploy job have `if: github.ref == 'refs/heads/main'`?
3. What's the advantage of OIDC over static AWS access keys for CI/CD?

8.6 Cost Management and Security Basics

TL;DR: Set up billing alerts BEFORE doing anything else. A typical dev data platform costs ~\$47/month on AWS. Key cost savers: Spot instances for Spark, S3 lifecycle rules, right-size RDS, schedule dev resources off at night. Security: least privilege IAM, never hardcode secrets, encrypt everything.

S3 is cheap. Redshift is not. Here's how to estimate your cloud bill – and how to avoid the surprise charges that hit every new cloud user.

Rule #1: Billing Alerts First

```
# In Terraform
resource "aws_cloudwatch_metric_alarm" "billing_alarm" {
  alarm_name          = "monthly-billing-alarm"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "EstimatedCharges"
  namespace           = "AWS/Billing"
  period              = 21600 # 6 hours
  statistic            = "Maximum"
  threshold            = 50    # Alert at $50
  alarm_actions        = [aws_sns_topic.billing_alerts.arn]
  dimensions = {
    Currency = "USD"
  }
}
```

Cost Estimation for a Typical DE Setup

SERVICE	CONFIG	MONTHLY COST
S3	50GB standard	~\$1.15
RDS Postgres	db.t3.micro, 20GB	~\$15
ECS Fargate	2 tasks, 0.5 vCPU, 1GB each	~\$30
Athena	~50 queries/day, 100MB scanned each	~\$0.75
ECR	5 images	~\$0.50
Total		~\$47/month

In production with more data and compute, expect \$200-500/month for a small/medium platform.

Cost-Saving Tips

1. **Use Spot instances for Spark/EMR** – 60-90% cheaper than on-demand
2. **S3 lifecycle rules** – move old data to Glacier (configured in Section 8.3)
3. **Right-size RDS** – start with `db.t3.micro`, scale up only when needed
4. **Schedule dev resources off at night:**

```
# Lambda function to stop dev RDS at night
import boto3

def lambda_handler(event, context):
    rds = boto3.client('rds')
    rds.stop_db_instance(DBInstanceIdentifier='ecommerce-dev-db')
    return {"status": "stopped"}
```

1. **Athena: always use Parquet + partitions** – can reduce query costs by 90%+

Security: Three Rules

1. Principle of Least Privilege

Your pipeline role should only have the permissions it actually needs. Never give it admin access just because it's easier:

```

# BAD -- gives access to everything
resource "aws_iam_role_policy_attachment" "bad" {
  role      = aws_iam_role.pipeline_role.name
  policy_arn = "arn:aws:iam::aws:policy/AdministratorAccess" # NEVER
}

# GOOD -- specific permissions for specific resources
resource "aws_iam_role_policy" "good" {
  name = "pipeline-s3-access"
  role = aws_iam_role.pipeline_role.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = ["s3:GetObject", "s3:PutObject"]
      Resource = ["${aws_s3_bucket.data_lake.arn}/*"]
    }]
  })
}

```

2. Never Hardcode Secrets

Use AWS Secrets Manager:

```

import boto3
import json

def get_secret(secret_name: str, region: str = "us-east-1") -> dict:
    client = boto3.client("secretsmanager", region_name=region)
    response = client.get_secret_value(SecretId=secret_name)
    return json.loads(response["SecretString"])

# Usage
db_creds = get_secret("ecommerce/db-credentials")
conn_string = (
    f"postgresql://{db_creds['username']}:{db_creds['password']}@"
    f"@{db_creds['host']}:5432/{db_creds['dbname']}"
)

```

3. Encrypt Everything

```

resource "aws_s3_bucket_server_side_encryption_configuration" "data_lake" {
  bucket = aws_s3_bucket.data_lake.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
    }
  }
}

```

Common Mistake - Using your root AWS account for daily work. Create an IAM user. - S3 buckets accidentally set to public. Always block public access. - Committing `.env` files with real credentials to Git. Add `.env` to `.gitignore`.

Checkpoint

1. What should you set up BEFORE creating any AWS resources?
2. How much cheaper is S3 Glacier compared to S3 Standard?
3. What does “principle of least privilege” mean for IAM policies?

8.7 Module 8 Project – Deploy Your Pipeline to AWS

TL;DR: Deploy your Airflow pipeline to AWS using Terraform for infrastructure, Docker for packaging, and GitHub Actions for CI/CD. Use MinIO locally as an S3 substitute. The Terraform configs themselves are the primary deliverable – you don’t have to `apply` to a real AWS account if you don’t want to spend money.

Overview

Deploy your Airflow pipeline (from Modules 4 and 7) to AWS using:

- **Terraform** to provision S3, RDS, ECS, IAM
- **Docker** to containerize Airflow and your pipeline code
- **GitHub Actions** for CI/CD
- **MinIO** as a local S3 substitute for development

Project Structure

```
module-8-project/
|-- terraform/
|   |-- main.tf
|   |-- variables.tf
|   |-- s3.tf
|   |-- rds.tf
|   |-- iam.tf
|   |-- ecs.tf           # Bonus
|   |-- outputs.tf
|   |-- terraform.tfvars.example
|-- docker/
|   |-- Dockerfile
|   |-- Dockerfile.airflow
|-- .github/
|   |-- workflows/
|       |-- pipeline.yml
|-- src/
|   |-- pipeline/
|       |-- __init__.py
|       |-- extract.py
|       |-- transform.py
|       |-- load.py
|       |-- quality.py
|-- dags/
|   |-- ecommerce_pipeline.py
|-- tests/
|   |-- test_extract.py
|   |-- test_transform.py
|   |-- test_quality.py
|-- docker-compose.yml
|-- pyproject.toml
|-- README.md
```

Step 1: Terraform Infrastructure

Create the Terraform configs from this module. At minimum:

- S3 bucket with versioning, encryption, lifecycle rules, and public access block
- RDS Postgres instance with security group
- IAM role with least-privilege policy for S3 and RDS access
- CloudWatch billing alarm

```
cd terraform
terraform init
terraform plan -var="db_password=localtest123"
```

Pro Tip: You don't have to actually `terraform apply` to a real AWS account if you don't want to spend money. The configs themselves are the deliverable. But if you have free tier or credits, go for it.

Step 2: Dockerize the Pipeline

Write a production-grade Dockerfile with:

- Multi-stage build
- Non-root user
- Layer ordering optimized for caching
- Health check

```
docker build -t ecommerce-pipeline .
docker run --rm ecommerce-pipeline --help
```

Step 3: Docker Compose for Local Dev

Everything should start with one command:

```
docker-compose up -d
```

Include: your pipeline container, Postgres, MinIO, and Airflow (webserver + scheduler).

Step 4: GitHub Actions CI/CD

Create `.github/workflows/pipeline.yml` that:

1. Runs on push to `main` and on PRs
2. Lints code (ruff)
3. Runs unit tests with a test Postgres service
4. Builds Docker image
5. (Optional) Pushes to ECR and deploys to ECS

Step 5: Environment-Aware Configuration

Your pipeline code should work in both local development and production without code changes.

See `module-8/solution/src/pipeline/config.py`:

```
# src/pipeline/config.py
import os

class Config:
    """Environment-aware configuration."""
    ENVIRONMENT = os.getenv("ENVIRONMENT", "dev")

    # S3 / MinIO
    S3_ENDPOINT = os.getenv("S3_ENDPOINT", "http://localhost:9000")
    S3_BUCKET = os.getenv("S3_BUCKET", "ecommerce-data-lake")
    AWS_ACCESS_KEY_ID = os.getenv("AWS_ACCESS_KEY_ID", "minioadmin")
    AWS_SECRET_ACCESS_KEY = os.getenv("AWS_SECRET_ACCESS_KEY", "minioadmin")

    # Database
    DATABASE_URL = os.getenv(
        "DATABASE_URL",
        "postgresql://postgres:postgres@localhost:5432/ecommerce"
    )

    @classmethod
    def is_local(cls) -> bool:
        return cls.ENVIRONMENT == "dev"
```

In dev, the defaults point to your local Docker Compose services (MinIO, local Postgres). In production, environment variables override them to point at real AWS resources. No code changes, no `if` statements scattered through your pipeline.

Expected Output

- `terraform plan` runs without errors and shows resources to be created
- `docker-compose up` starts all services and the pipeline runs
- GitHub Actions workflow passes on push
- Pipeline reads from and writes to MinIO/S3

Completion Checklist

- ☐ Terraform configs for S3, RDS, IAM, and billing alarm
- ☐ S3 bucket has versioning, encryption, lifecycle rules, public access block
- ☐ RDS has security group restricting access
- ☐ IAM role follows least privilege
- ☐ No secrets hardcoded anywhere
- ☐ Dockerfile uses multi-stage build, non-root user
- ☐ `docker-compose.yml` starts full stack locally
- ☐ GitHub Actions workflow runs tests and builds Docker image
- ☐ Pipeline code uses environment variables for configuration

- [] README explains how to run locally and how to deploy
-

What's Next

Module 9 explores data engineering for AI – embeddings pipelines, feature stores, vector databases, and RAG systems. The infrastructure skills you learned in this module are what make those AI systems production-ready: you'll deploy embedding pipelines in containers, store vectors in managed databases, and use CI/CD to keep everything running reliably.

End of Module 8

Module 9: Data Engineering for AI

The hottest intersection in tech. Learn to build the data infrastructure that powers AI applications.

9.1 How AI Changes the Data Engineer's Job

TL;DR: AI doesn't replace data engineers – it creates MORE work for us. Every AI application needs data pipelines feeding it. The new skills to learn: embeddings pipelines, vector database management, feature stores, RAG data layers, and LLM data preparation. These all build on top of the data engineering skills you already have.

In 2026, every company wants AI. But AI is useless without good data. That's where you come in as a DE.

If you've been watching job postings, you've noticed something: "data engineer" roles increasingly mention AI, LLMs, embeddings, and vector databases. This isn't a fad. AI is fundamentally changing what data engineers build.

But here's what nobody tells you: AI doesn't replace data engineers. It creates MORE work for us. Every AI application needs data pipelines feeding it. Every LLM needs clean, chunked, embedded data. Every ML model needs features computed and served.

Traditional vs AI-Augmented Pipeline

Traditional Pipeline:

```
Source -> Extract -> Transform -> Load -> Dashboard
```

AI-Augmented Pipeline:

```
Source -> Extract -> Transform -> Load -> Dashboard
                        |
                        Chunk -> Embed -> Vector DB -> RAG API
                        |
                        Feature Store -> ML Model -> Predictions
```

Same pipeline, but with new branches. As a data engineer, you're building those branches.

The New Things You'll Build

1. **Embeddings pipelines** – converting text/images into vectors that ML models can use
2. **Vector database management** – storing and indexing embeddings for similarity search
3. **Feature stores** – pre-computing features for ML models
4. **RAG data layers** – preparing and serving data for Retrieval-Augmented Generation
5. **LLM data prep** – cleaning, chunking, and enriching data for fine-tuning or prompting

The AI Data Stack is a layered architecture. The bottom layers – Ingestion, Storage, Data Lake – are the same data engineering you've been learning this whole course. The top layers – Vector DB, Feature Store, Embeddings, Feature Computation, Vector Search, Feature Serving – are new, but they're built ON TOP of your existing skills. In every AI project, the ML/AI engineer builds the model. The data engineer builds everything that feeds it.

Key Concept: The AI Data Stack The bottom layers of the AI stack are the same data engineering you've been learning this whole course. The top layers are new, but they're built ON TOP of your existing skills. In every AI project, the ML/AI engineer builds the model. The data engineer builds everything that feeds it.

Common Mistake - Thinking you need to understand deep learning to do AI data engineering. You don't. You need to understand data formats, pipelines, and APIs. - Ignoring this trend. The DEs who learn these skills now will be the most valuable ones in 2-3 years.

Checkpoint

1. How does an AI-augmented pipeline differ from a traditional data pipeline?
2. Name three new things data engineers build for AI workloads.
3. Why does AI create *more* work for data engineers, not less?

9.2 Feature Stores – What They Are and When You Need One

TL;DR: A feature store is a centralized system for computing, storing, and serving ML-ready features. Most teams don't need a full-blown feature store (like Feast) until they have 5+ ML models. For 1-2 models, well-organized SQL views and a simple API are enough. Build simple first.

An ML engineer says: *"I need the customer's average order value, their total orders in the last 30 days, and their most recent order date. Updated daily for batch predictions, and in real-time for the API."*

You could write a SQL query. But then another ML engineer needs different features. And another. And they all need the *exact same features* computed the *exact same way* so training and serving are consistent. This is the problem feature stores solve.

What a Feature Store Does

1. **Computes features** – running transformations that produce ML-ready values
2. **Stores features** – persisting them for both batch (training) and online (serving) use
3. **Serves features** – making them available with low latency for real-time predictions
4. **Ensures consistency** – same feature definition used in training and serving

Do You Actually Need One?

Honest answer: most teams don't need a full-blown feature store until they have 5+ ML models in production.

SCENARIO	RECOMMENDATION
1-2 models, batch only	SQL views + scheduled materialization. No feature store needed.
1-2 models, real-time	Pre-computed features in Redis. Still no formal feature store.
5+ models, shared features	Feature store (Feast, Tecton, or custom).
Enterprise, 50+ models	Feast, Tecton, or Databricks Feature Store.

Building a Simple Feature Store

Here's what you'd build at a startup before you need Feast – a minimal feature store in Postgres + Python:

```

-- Feature table: one row per customer, updated daily
CREATE TABLE IF NOT EXISTS feature_store.customer_features (
  customer_id INTEGER PRIMARY KEY,

  -- Order features
  total_orders INTEGER,
  total_revenue DECIMAL(12,2),
  avg_order_value DECIMAL(10,2),
  orders_last_30_days INTEGER,
  days_since_last_order INTEGER,

  -- Behavioral features
  favorite_category VARCHAR(100),
  distinct_products_purchased INTEGER,

  -- Metadata
  computed_at TIMESTAMP DEFAULT NOW(),
  feature_version VARCHAR(20) DEFAULT '1.0'
);

-- Materialization query: compute features from raw data
INSERT INTO feature_store.customer_features
SELECT
  c.customer_id,
  COUNT(DISTINCT o.order_id) AS total_orders,
  COALESCE(SUM(o.total_amount), 0) AS total_revenue,
  COALESCE(AVG(o.total_amount), 0) AS avg_order_value,
  COUNT(DISTINCT CASE
    WHEN o.order_date >= CURRENT_DATE - INTERVAL '30 days'
    THEN o.order_id
  END) AS orders_last_30_days,
  EXTRACT(DAY FROM NOW() - MAX(o.order_date))::INTEGER AS days_since_last_order,
  MODE() WITHIN GROUP (ORDER BY p.category) AS favorite_category,
  COUNT(DISTINCT li.product_id) AS distinct_products_purchased,
  NOW() AS computed_at,
  '1.0' AS feature_version
FROM dim_customers c
LEFT JOIN fact_orders o ON c.customer_id = o.customer_id
LEFT JOIN fact_line_items li ON o.order_id = li.order_id
LEFT JOIN dim_products p ON li.product_id = p.product_id
GROUP BY c.customer_id
ON CONFLICT (customer_id) DO UPDATE SET
  total_orders = EXCLUDED.total_orders,
  total_revenue = EXCLUDED.total_revenue,
  avg_order_value = EXCLUDED.avg_order_value,
  orders_last_30_days = EXCLUDED.orders_last_30_days,
  days_since_last_order = EXCLUDED.days_since_last_order,
  favorite_category = EXCLUDED.favorite_category,
  distinct_products_purchased = EXCLUDED.distinct_products_purchased,
  computed_at = EXCLUDED.computed_at;

```

Python class to serve features:

```

# feature_store/store.py
import psycopg2
from dataclasses import dataclass
from typing import Optional

@dataclass
class CustomerFeatures:
    customer_id: int
    total_orders: int
    total_revenue: float
    avg_order_value: float
    orders_last_30_days: int
    days_since_last_order: Optional[int]
    favorite_category: Optional[str]
    distinct_products_purchased: int
    computed_at: str
    feature_version: str

class FeatureStore:
    def __init__(self, connection_string: str):
        self.conn = psycopg2.connect(connection_string)

    def get_customer_features(self, customer_id: int) -> Optional[CustomerFeatures]:
        """Get features for a single customer (online serving)."""
        with self.conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM feature_store.customer_features WHERE customer_id = %s",
                (customer_id,)
            )
            row = cur.fetchone()
            if row is None:
                return None
            return CustomerFeatures(*row)

    def get_batch_features(self, customer_ids: list[int]) -> list[CustomerFeatures]:
        """Get features for multiple customers (batch prediction)."""
        with self.conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM feature_store.customer_features WHERE customer_id = ANY(%s)",
                (customer_ids,)
            )
            return [CustomerFeatures(*row) for row in cur.fetchall()]

    def get_training_dataset(self, feature_version: str = "1.0") -> list[CustomerFeatures]:
        """Get all features for model training."""
        with self.conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM feature_store.customer_features WHERE feature_version = %s",
                (feature_version,)
            )

```

```

    )
    return [CustomerFeatures(*row) for row in cur.fetchall()]

# Usage
store = FeatureStore("postgresql://postgres:postgres@localhost:5432/ecommerce")
features = store.get_customer_features(customer_id=42)
print(f"Customer 42: {features.total_orders} orders, ${features.avg_order_value} avg")

```

Expected output:

```
Customer 42: 17 orders, $84.32 avg
```

Common Mistake - Over-engineering the feature store before you have any ML models in production. Build simple first. - Training-serving skew: computing features differently in training vs serving. Use the SAME SQL/code for both. - Not versioning features. When you change how a feature is computed, old models trained on the old version will break.

Checkpoint

1. What problem does a feature store solve?
2. At what scale should you consider a formal feature store like Feast?
3. What is training-serving skew and why is it dangerous?

9.3 Embeddings Pipelines – Turning Text Into Vectors

TL;DR: An embedding is a list of numbers (vector) representing the “meaning” of text. Similar meanings produce similar vectors. Build pipelines that generate embeddings incrementally (only new/changed items), in batches (API rate limits), and store the original text alongside the vector. OpenAI’s `text-embedding-3-small` is cheap (~\$0.10 per 100K products); open-source `all-MiniLM-L6-v2` is free and runs locally.

How does ChatGPT “understand” that “dog” and “puppy” mean similar things? How does semantic search know that “cheap flights to Paris” should match “budget airline tickets to France”?

Embeddings.

How Embeddings Work

An embedding is a list of numbers (a vector) that represents the “meaning” of a piece of text. Think of it as coordinates in a high-dimensional space, where proximity corresponds to semantic similarity:

```
"I love pizza"          -> [0.023, -0.156, 0.892, ..., 0.044] # 1536 numbers
"Pizza is great"        -> [0.019, -0.148, 0.887, ..., 0.041] # Similar numbers!
"The stock market crashed" -> [-0.445, 0.312, -0.019, ..., 0.678] # Very different
```

Similar meanings -> similar vectors -> similar numbers. Think of embeddings as coordinates on a map. “Pizza” and “pasta” are close together. “Pizza” and “stock market” are far apart.

Key Concept: Embeddings An embedding converts unstructured data (text, images) into a fixed-length numerical vector. The key property: semantically similar inputs produce vectors that are close together in vector space. This enables similarity search, clustering, and classification without keyword matching.

Why Cosine Similarity Works

When you have two embedding vectors, you need a way to measure “how similar are these?” The standard answer is cosine similarity. It measures the angle between two vectors, ignoring their magnitude. A cosine similarity of 1.0 means identical direction (identical meaning), 0.0 means orthogonal (unrelated), and -1.0 means opposite.

Why cosine and not Euclidean distance? Because embeddings can vary in magnitude based on text length and other factors. Cosine similarity normalizes for this – it only cares about the *direction* the vector points, which corresponds to meaning. A short product title and a long product description about the same item will have similar cosine similarity even though their raw vectors have different magnitudes.

In pgvector, the `<=>` operator computes cosine *distance* (1 - cosine_similarity). Lower distance = more similar.

Building an Embeddings Pipeline

A pipeline that generates embeddings for product descriptions – used for “similar products” or semantic search:


```

# embeddings/pipeline.py
import openai
import psycopg2
import time
from dataclasses import dataclass

@dataclass
class ProductEmbedding:
    product_id: int
    product_name: str
    description: str
    embedding: list[float]

class EmbeddingsPipeline:
    """Generate and store embeddings for product catalog."""

    def __init__(
        self,
        openai_api_key: str,
        db_connection_string: str,
        model: str = "text-embedding-3-small", # Cheap and good
        batch_size: int = 100,
    ):
        self.client = openai.OpenAI(api_key=openai_api_key)
        self.conn = psycopg2.connect(db_connection_string)
        self.model = model
        self.batch_size = batch_size

    def generate_embeddings(self, texts: list[str]) -> list[list[float]]:
        """Generate embeddings for a batch of texts using OpenAI API."""
        response = self.client.embeddings.create(
            model=self.model,
            input=texts,
        )
        return [item.embedding for item in response.data]

    def get_products_without_embeddings(self) -> list[dict]:
        """Find products that don't have embeddings yet (incremental)."""
        with self.conn.cursor() as cur:
            cur.execute("""
                SELECT p.product_id, p.product_name, p.description
                FROM dim_products p
                LEFT JOIN product_embeddings pe ON p.product_id = pe.product_id
                WHERE pe.product_id IS NULL
                OR pe.updated_at < p.updated_at -- Re-embed if product changed
                ORDER BY p.product_id
            """)
            columns = [desc[0] for desc in cur.description]
            return [dict(zip(columns, row)) for row in cur.fetchall()]

    def store_embeddings(self, product_embeddings: list[ProductEmbedding]) -> None:
        """Store embeddings in Postgres with pgvector."""

```

```

with self.conn.cursor() as cur:
    for pe in product_embeddings:
        cur.execute("""
            INSERT INTO product_embeddings (product_id, embedding, updated_at)
            VALUES (%s, %s, NOW())
            ON CONFLICT (product_id) DO UPDATE SET
                embedding = EXCLUDED.embedding,
                updated_at = NOW()
        """, (pe.product_id, pe.embedding))
self.conn.commit()

def run(self) -> int:
    """Run the full embeddings pipeline. Returns number processed."""
    products = self.get_products_without_embeddings()

    if not products:
        print("No products need embedding updates.")
        return 0

    print(f"Processing {len(products)} products...")
    total_processed = 0

    for i in range(0, len(products), self.batch_size):
        batch = products[i:i + self.batch_size]

        # Combine name + description for richer embeddings
        texts = [
            f"{p['product_name']}: {p['description']}"
            for p in batch
        ]

        embeddings = self.generate_embeddings(texts)

        product_embeddings = [
            ProductEmbedding(
                product_id=p["product_id"],
                product_name=p["product_name"],
                description=p["description"],
                embedding=emb,
            )
            for p, emb in zip(batch, embeddings)
        ]

        self.store_embeddings(product_embeddings)
        total_processed += len(batch)
        print(f"  Processed {total_processed}/{len(products)} products")

        # Rate limiting
        if i + self.batch_size < len(products):
            time.sleep(1)

    return total_processed

```

Usage

```
if __name__ == "__main__":
    import os
    pipeline = EmbeddingsPipeline(
        openai_api_key=os.environ["OPENAI_API_KEY"],
        db_connection_string=os.environ["DATABASE_URL"],
    )
    processed = pipeline.run()
    print(f"Done! Processed {processed} products.")
```

Expected output:

```
Processing 500 products...
Processed 100/500 products
Processed 200/500 products
Processed 300/500 products
Processed 400/500 products
Processed 500/500 products
Done! Processed 500 products.
```

Using Open-Source Models Instead

OpenAI is easy but costs money. For production at scale, use open-source models:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2") # 384 dimensions, fast

texts = [
    "Wireless Bluetooth Headphones: Premium sound quality with noise cancellation",
    "Running Shoes: Lightweight mesh with responsive cushioning",
]

embeddings = model.encode(texts)
print(f"Embedding shape: {embeddings.shape}")
print(f"Embedding sample: {embeddings[0][:5]}")
```

Expected output:

```
Embedding shape: (2, 384)
Embedding sample: [ 0.0234 -0.0891  0.1123 -0.0456  0.0789]
```

Companion code: See `module-9/solution/src/embeddings_pipeline.py` for the sentence-transformers version that uses `all-MiniLM-L6-v2` instead of OpenAI. This runs entirely locally with no API key needed.

Model Comparison

MODEL	DIMENSIONS	COST	QUALITY	SPEED
OpenAI <code>text-embedding-3-small</code>	1536	\$0.02/1M tokens	Best	API call required
<code>all-MiniLM-L6-v2</code>	384	Free	Good	Fast, runs locally
<code>e5-large-v2</code>	1024	Free	Near-OpenAI	Slower, runs locally

Cost Estimation

OpenAI `text-embedding-3-small` at ~\$0.02 per 1M tokens:

- Average product description: ~50 tokens
- 100K products: ~5M tokens = ~\$0.10
- Even at 1M products, you're looking at \$1

Common Mistake - Embedding entire documents as one vector. Long documents should be chunked first (covered in Section 9.5). - Not storing the text alongside the embedding. You'll need it for debugging and returning results. - Re-embedding everything on every run. Use incremental logic – only embed new or changed items. - Using the wrong model for your use case. Multilingual data? Use a multilingual model.

Checkpoint

1. What is an embedding and what key property makes it useful?
2. Why should you combine product name + description before embedding?
3. When would you choose an open-source embedding model over OpenAI?

9.4 Vector Databases – Storage and Retrieval

TL;DR: A vector database stores and searches high-dimensional vectors efficiently using specialized indexes. Start with **pgvector** (Postgres extension) – no new infrastructure needed. It handles millions of vectors easily. Use HNSW indexes for production (faster queries) and IVFFlat for simpler cases. Cosine similarity is the standard metric.

You’ve got 100,000 product embeddings – each a list of 1,536 numbers. A customer searches for “comfortable shoes for running.” You generate an embedding for that query. Now you need to find the 10 most similar products. Vector databases solve this with special indexes that make similarity search fast.

You already know Postgres. Vector search is just another index type. That is the key insight here – pgvector turns your existing Postgres into a vector database without learning a new system.

Choosing a Vector Database

DATABASE	TYPE	BEST FOR
pgvector	Postgres extension	Teams already using Postgres – no new infrastructure
Pinecone	Managed cloud	Quick start, no ops (\$70+/month)
Weaviate	Open-source	Self-hosted, full-featured
Qdrant	Open-source	Performance-focused
ChromaDB	Open-source	Prototyping only (not production scale)

Recommendation: start with pgvector. If you’re already using Postgres, adding pgvector is one command. No new database to manage. It handles millions of vectors easily.

Setting Up pgvector

```
-- Enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Create the embeddings table
CREATE TABLE product_embeddings (
  product_id INTEGER PRIMARY KEY REFERENCES dim_products(product_id),
  embedding vector(1536), -- OpenAI text-embedding-3-small dimension
  updated_at TIMESTAMP DEFAULT NOW()
);

-- HNSW index (recommended for production -- faster queries, more accurate)
CREATE INDEX ON product_embeddings
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

Key Concept: Vector Index Types - IVFFlat – faster to build, slightly less accurate. Good for getting started. Set `lists = sqrt(num_rows)`. - **HNSW** – slower to build, more accurate, faster queries. Better for production. Without an index, every query does a full table scan. Fine for 1,000 rows, terrible for 100K+.

Companion code: See `module-9/starter/sql/init.sql` for the complete pgvector schema including both `product_embeddings` and `document_chunks` tables with HNSW indexes.

Similarity Search in SQL

```
-- Find 10 most similar products to a given embedding
-- The <=> operator computes cosine distance (1 - cosine_similarity)
SELECT
    p.product_id,
    p.product_name,
    p.description,
    1 - (pe.embedding <=> '[0.023, -0.156, 0.892, ...]':vector) AS similarity
FROM product_embeddings pe
JOIN dim_products p ON pe.product_id = p.product_id
ORDER BY pe.embedding <=> '[0.023, -0.156, 0.892, ...]':vector
LIMIT 10;
```

Python Integration

```

# vector_search/search.py
import psycopg2
from dataclasses import dataclass

@dataclass
class SearchResult:
    product_id: int
    product_name: str
    description: str
    similarity: float

class VectorSearch:
    def __init__(self, connection_string: str):
        self.conn = psycopg2.connect(connection_string)

    def search_similar_products(
        self,
        query_embedding: list[float],
        limit: int = 10,
        min_similarity: float = 0.5,
    ) -> list[SearchResult]:
        """Find products similar to the query embedding."""
        with self.conn.cursor() as cur:
            cur.execute("""
                SELECT
                    p.product_id,
                    p.product_name,
                    p.description,
                    1 - (pe.embedding <=> %s::vector) AS similarity
                FROM product_embeddings pe
                JOIN dim_products p ON pe.product_id = p.product_id
                WHERE 1 - (pe.embedding <=> %s::vector) > %s
                ORDER BY pe.embedding <=> %s::vector
                LIMIT %s
            """, (str(query_embedding), str(query_embedding),
                  min_similarity, str(query_embedding), limit))

            return [
                SearchResult(
                    product_id=row[0],
                    product_name=row[1],
                    description=row[2],
                    similarity=round(row[3], 4),
                )
                for row in cur.fetchall()
            ]

    def find_similar_to_product(
        self,
        product_id: int,
        limit: int = 10,
    ) -> list[SearchResult]:

```



```

"""Find products similar to a given product."""
with self.conn.cursor() as cur:
    cur.execute(
        "SELECT embedding FROM product_embeddings WHERE product_id = %s",
        (product_id,),
    )
    row = cur.fetchone()
    if row is None:
        return []

    embedding = row[0]

    cur.execute("""
        SELECT
            p.product_id,
            p.product_name,
            p.description,
            1 - (pe.embedding <=> %s::vector) AS similarity
        FROM product_embeddings pe
        JOIN dim_products p ON pe.product_id = p.product_id
        WHERE pe.product_id != %s
        ORDER BY pe.embedding <=> %s::vector
        LIMIT %s
    """, (embedding, product_id, embedding, limit))

    return [
        SearchResult(
            product_id=row[0],
            product_name=row[1],
            description=row[2],
            similarity=round(row[3], 4),
        )
        for row in cur.fetchall()
    ]

# Usage
search = VectorSearch("postgresql://postgres:postgres@localhost:5432/ecommerce")
results = search.find_similar_to_product(42, limit=5)
for r in results:
    print(f" {r.product_name} (similarity: {r.similarity})")

```

Expected output:

```

ProElite Running Max 72 (similarity: 0.9234)
ActiveGear Running Lite 18 (similarity: 0.9012)
NatureFit Training Pro 45 (similarity: 0.8756)
CoreBasics Running Essential 91 (similarity: 0.8543)
UrbanEdge Shoes Ultra 33 (similarity: 0.8201)

```

Companion code: See `module-9/solution/src/vector_search.py` for the sentence-transformers version that accepts natural language queries directly (embedding the query text internally).

Performance at Scale

ROWS	IVFFLAT QUERY TIME	HNSW QUERY TIME
100K	~5ms	~2ms
1M	~20ms	~5ms
10M	~100ms	~15ms

For most applications, this is more than fast enough. Sub-millisecond at billions of vectors is when you'd look at Pinecone or a dedicated vector database.

Common Mistake - Forgetting to create an index. Without one, queries do full table scans. - Not tuning index parameters. Adjust `lists` (IVFFlat) or `m` / `ef_construction` (HNSW) based on data size. - Mixing embedding dimensions. If you switch models (1536-dim to 384-dim), you need new columns or tables.

At Your Job: Natural Language Product Search

Your ML team says “we need a product search that understands natural language.” Here's how you build the data pipeline for that:

1. **Extract** product data from the catalog database (name, description, category, attributes)
2. **Transform** the text: combine name + category + description into a single embedding input
3. **Embed** each product using your chosen model (OpenAI API or sentence-transformers)
4. **Store** the embeddings in pgvector alongside the `product_id`
5. **Index** with HNSW for fast retrieval
6. **Serve** through a search API that embeds the user's query and runs a similarity search

The ML team handles the model selection. You handle steps 1-6 – the entire data pipeline.

Checkpoint

1. What's the difference between IVFFlat and HNSW indexes?
2. What does the `<=>` operator compute in pgvector?
3. Why should you start with pgvector instead of a dedicated vector database?

9.5 RAG Pipeline Architecture – Building the Data Layer

TL;DR: RAG (Retrieval-Augmented Generation) is ~20% LLM and ~80% data engineering. The data engineer builds the indexing pipeline (documents -> chunk -> embed -> store) and the retrieval layer (query -> embed -> vector search -> top K chunks). Chunking strategy has the biggest impact on quality. Use 300-500 token chunks with overlap.

RAG – Retrieval-Augmented Generation – is the pattern behind every “chat with your documents” application. And while the ML engineers get credit for the chatbot, guess who builds the entire data layer? Us.

Instead of fine-tuning an LLM (expensive, slow, requires ML expertise), you give it relevant context at query time. The DE builds the retrieval part. The LLM just reads the context you provide and generates an answer. Think of it like giving a smart intern the relevant pages from a manual before asking them a question – they don’t need to memorize the whole manual, they just need the right pages.

The Two Phases of RAG

Phase 1: Indexing (offline – your job)

Documents -> Chunk -> Embed -> Store in Vector DB

Phase 2: Query (online – real-time)

User Question -> Embed -> Search Vector DB -> Top K Chunks -> LLM -> Answer

You build Phase 1 and the retrieval part of Phase 2.

Step 1: Document Ingestion

```

# rag/ingest.py
from pathlib import Path
import fitz  # PyMuPDF for PDF parsing
from dataclasses import dataclass

@dataclass
class Document:
    content: str
    metadata: dict  # source file, page number, etc.

def load_pdf(path: str) -> list[Document]:
    """Extract text from a PDF file."""
    doc = fitz.open(path)
    documents = []
    for page_num, page in enumerate(doc):
        text = page.get_text().strip()
        if text:
            documents.append(Document(
                content=text,
                metadata={
                    "source": Path(path).name,
                    "page": page_num + 1,
                    "type": "pdf",
                },
            ))
    return documents

def load_text(path: str) -> list[Document]:
    """Load a plain text or markdown file."""
    content = Path(path).read_text()
    return [Document(
        content=content,
        metadata={"source": Path(path).name, "type": "text"},
    )]

def load_documents(directory: str) -> list[Document]:
    """Load all supported documents from a directory."""
    docs = []
    dir_path = Path(directory)

    for pdf in dir_path.glob("**/*.pdf"):
        docs.extend(load_pdf(str(pdf)))
    for txt in dir_path.glob("**/*.txt"):
        docs.extend(load_text(str(txt)))
    for md in dir_path.glob("**/*.md"):
        docs.extend(load_text(str(md)))

    print(f"Loaded {len(docs)} document sections from {directory}")
    return docs

```

Step 2: Chunking

This is the most underrated step in RAG. How you chunk documents has a massive impact on retrieval quality.

Why does chunking matter? If your chunks are too large, the embedding becomes a blurry average of many topics – the vector doesn't represent any single idea well. If too small, you lose context – a sentence fragment doesn't carry enough meaning on its own. The sweet spot is 300-500 tokens, where each chunk captures a coherent idea with enough surrounding context.

```

# rag/chunker.py
from dataclasses import dataclass

@dataclass
class Chunk:
    content: str
    metadata: dict
    chunk_index: int

def chunk_by_tokens(
    text: str,
    chunk_size: int = 500,      # tokens per chunk
    chunk_overlap: int = 50,    # overlap between chunks
) -> list[str]:
    """Split text into overlapping chunks by approximate token count.

    Rule of thumb: 1 token ~ 4 characters for English text.
    """
    words = text.split()
    tokens_per_word = 1.3 # average
    words_per_chunk = int(chunk_size / tokens_per_word)
    overlap_words = int(chunk_overlap / tokens_per_word)

    chunks = []
    start = 0
    while start < len(words):
        end = start + words_per_chunk
        chunk = " ".join(words[start:end])
        if chunk.strip():
            chunks.append(chunk)
            start = end - overlap_words

    return chunks

def chunk_documents(
    documents: list[Document],
    chunk_size: int = 500,
    chunk_overlap: int = 50,
) -> list[Chunk]:
    """Chunk all documents with metadata preservation."""
    all_chunks = []

    for doc in documents:
        text_chunks = chunk_by_tokens(doc.content, chunk_size, chunk_overlap)

        for i, text in enumerate(text_chunks):
            chunk = Chunk(
                content=text,
                metadata={
                    **doc.metadata,
                    "chunk_index": i,

```

```

        "chunk_count": len(text_chunks),
    },
    chunk_index=i,
)
all_chunks.append(chunk)

print(f"Created {len(all_chunks)} chunks from {len(documents)} documents")
return all_chunks

```

Companion code: See `module-9/solution/src/chunker.py` for a sentence-boundary-aware chunker that splits on `.!?` boundaries with overlap.

Chunking strategies:

STRATEGY	WHEN TO USE
Fixed-size token chunks	Simple, works for most cases
Sentence-based	Better for Q&A use cases
Paragraph-based (split on <code>\n\n</code>)	Good for well-structured documents
Semantic chunking	Best quality, slowest (uses embedding model to find breakpoints)

Pro Tip The overlap between chunks is critical – it prevents information from being split across chunk boundaries. 300-500 tokens per chunk with 50-100 token overlap is the sweet spot for most use cases.

Step 3: Embed and Store

```

# rag/indexer.py
import openai
import psycopg2
import json

class RAGIndexer:
    def __init__(self, openai_api_key: str, db_connection_string: str):
        self.client = openai.OpenAI(api_key=openai_api_key)
        self.conn = psycopg2.connect(db_connection_string)
        self._setup_table()

    def _setup_table(self):
        with self.conn.cursor() as cur:
            cur.execute("CREATE EXTENSION IF NOT EXISTS vector")
            cur.execute("""
                CREATE TABLE IF NOT EXISTS rag_chunks (
                    id SERIAL PRIMARY KEY,
                    content TEXT NOT NULL,
                    metadata JSONB,
                    embedding vector(1536),
                    created_at TIMESTAMP DEFAULT NOW()
                )
            """)
            cur.execute("""
                CREATE INDEX IF NOT EXISTS rag_chunks_embedding_idx
                ON rag_chunks USING hnsw (embedding vector_cosine_ops)
            """)
            self.conn.commit()

    def index_chunks(self, chunks: list[Chunk]) -> int:
        """Embed and store chunks. Returns count indexed."""
        batch_size = 100
        total = 0

        for i in range(0, len(chunks), batch_size):
            batch = chunks[i:i + batch_size]
            texts = [c.content for c in batch]

            response = self.client.embeddings.create(
                model="text-embedding-3-small",
                input=texts,
            )
            embeddings = [item.embedding for item in response.data]

            with self.conn.cursor() as cur:
                for chunk, embedding in zip(batch, embeddings):
                    cur.execute(
                        "INSERT INTO rag_chunks (content, metadata, embedding) VALUES
                        (%s, %s, %s)",
                        (chunk.content, json.dumps(chunk.metadata), str(embedding)),
                    )
            self.conn.commit()
            total += len(batch)

```

```
return total
```

Companion code: See `module-9/solution/src/indexer.py` for the sentence-transformers version that uses the local `all-MiniLM-L6-v2` model and indexes documents from files on disk.

Step 4: Retrieval

```

# rag/retriever.py
import openai
import psycopg2
import os

class RAGRetriever:
    def __init__(self, openai_api_key: str, db_connection_string: str):
        self.client = openai.OpenAI(api_key=openai_api_key)
        self.conn = psycopg2.connect(db_connection_string)

    def retrieve(self, query: str, top_k: int = 5) -> list[dict]:
        """Retrieve the most relevant chunks for a query."""
        response = self.client.embeddings.create(
            model="text-embedding-3-small",
            input=[query],
        )
        query_embedding = response.data[0].embedding

        with self.conn.cursor() as cur:
            cur.execute("""
                SELECT
                    content,
                    metadata,
                    1 - (embedding <=> %s::vector) AS similarity
                FROM rag_chunks
                ORDER BY embedding <=> %s::vector
                LIMIT %s
            """, (str(query_embedding), str(query_embedding), top_k))

        return [
            {
                "content": row[0],
                "metadata": row[1],
                "similarity": round(row[2], 4),
            }
            for row in cur.fetchall()
        ]

    def query_with_context(self, question: str, top_k: int = 5) -> str:
        """Full RAG: retrieve context, then ask the LLM."""
        chunks = self.retrieve(question, top_k=top_k)

        context = "\n\n--\n\n".join([
            f"[Source: {c['metadata'].get('source', 'unknown')}] ",
            f"Similarity: {c['similarity']}]\n{c['content']}"
            for c in chunks
        ])

        response = self.client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[
                {
                    "role": "system",

```

```

        "content": (
            "You are a helpful assistant. Answer the user's question "
            "based ONLY on the provided context. If the context doesn't "
            "contain enough information, say so. Cite your sources."
        ),
    },
    {
        "role": "user",
        "content": f"Context:\n{context}\n\nQuestion: {question}",
    },
],
temperature=0,
)

return response.choices[0].message.content

# Usage
retriever = RAGRetriever(
    openai_api_key=os.environ["OPENAI_API_KEY"],
    db_connection_string=os.environ["DATABASE_URL"],
)
answer = retriever.query_with_context("What is our return policy for electronics?")
print(answer)

```

Companion code: See [module-9/solution/src/retriever.py](https://github.com/DataEngineering/sentence-transformers-rag/blob/main/module-9/solution/src/retriever.py) for the sentence-transformers version that builds augmented prompts without calling the LLM (returning the prompt string for you to send to any LLM of your choice).

Common Mistake - Chunks too large (lose specificity) or too small (lose context). 300-500 tokens is the sweet spot. - No overlap between chunks – you’ll miss information at boundaries. - Not including metadata. Without source/page info, you can’t tell users WHERE the answer came from. - Embedding the query with a different model than the documents. Must use the same model for both.

Checkpoint

1. What are the two phases of RAG and who typically builds each?
2. Why is overlap between chunks important?
3. What system prompt instruction ensures the LLM only answers from the provided context?

9.6 LLM Data Preparation – Cleaning, Chunking, Metadata

TL;DR: Real-world documents are messy – PDFs with headers/footers, HTML with navigation menus, broken formatting. Clean text before embedding by removing artifacts, normalizing whitespace, and stripping boilerplate. Enrich metadata for filtering (content type, source, language). Good metadata enables filtered retrieval, which dramatically improves search quality.

The RAG pipeline works great with clean text files. Real-world data is messy. If you feed garbage into your embeddings, you get garbage search results.

Cleaning Techniques


```

# rag/cleaner.py
import re

def clean_text(text: str) -> str:
    """Clean text for embedding. Order matters."""
    # Remove excessive whitespace
    text = re.sub(r'\s+', ' ', text)

    # Remove common PDF artifacts
    text = re.sub(r'Page \d+ of \d+', '', text)
    text = re.sub(r'(?i)confidential|draft|internal use only', '', text)

    # Remove URLs (usually not useful for semantic meaning)
    text = re.sub(r'https?://\S+', '', text)

    # Remove email addresses
    text = re.sub(r'\S+@\S+\.\S+', '', text)

    # Normalize unicode
    text = text.encode('ascii', 'ignore').decode('ascii')

    # Remove very short lines (likely headers/footers)
    lines = text.split('\n')
    lines = [line for line in lines if len(line.strip()) > 20]
    text = '\n'.join(lines)

    return text.strip()

def clean_html(html: str) -> str:
    """Extract meaningful text from HTML."""
    from bs4 import BeautifulSoup

    soup = BeautifulSoup(html, 'html.parser')

    # Remove non-content elements
    for tag in soup(['script', 'style', 'nav', 'header', 'footer',
                    'aside', 'form', 'iframe']):
        tag.decompose()

    text = soup.get_text(separator='\n')
    return clean_text(text)

def enrich_metadata(content: str, source_metadata: dict) -> dict:
    """Add computed metadata useful for retrieval filtering."""
    metadata = {**source_metadata}

    # Approximate token count
    metadata['token_count'] = len(content.split()) * 1.3

    # Language detection (simple heuristic)
    common_english = {'the', 'is', 'at', 'which', 'on', 'a', 'an'}

```

```

words = set(content.lower().split()[:100])
metadata['likely_english'] = len(words & common_english) > 3

# Content type heuristic
if any(kw in content.lower() for kw in ['def ', 'class ', 'import ', 'function']):
    metadata['content_type'] = 'code'
elif any(kw in content.lower() for kw in ['table', 'figure', 'chart']):
    metadata['content_type'] = 'structured'
else:
    metadata['content_type'] = 'prose'

return metadata

```

Companion code: See `module-9/solution/src/cleaner.py` for additional utilities including `clean_markdown()` which strips markdown formatting while preserving the underlying text content.

Metadata-Filtered Retrieval

Good metadata enables filtering, which dramatically improves retrieval quality:

```

-- Instead of searching ALL chunks, filter first
SELECT content, metadata, 1 - (embedding <=> %s::vector) AS similarity
FROM rag_chunks
WHERE metadata->>'content_type' = 'prose'
      AND metadata->>'department' = 'customer_service'
ORDER BY embedding <=> %s::vector
LIMIT 5;

```

Common Mistake - Over-cleaning. Don't remove formatting that carries meaning (bullet points, numbered lists). - Not deduplicating. Duplicate content produces duplicate chunks and skewed search results. - Ignoring tables. Tables in PDFs are hard to parse but often contain the most valuable information.

Checkpoint

1. Why should you remove URLs and email addresses before embedding?
2. How does metadata filtering improve RAG retrieval quality?
3. What's the risk of over-cleaning document text?

9.7 Building an End-to-End AI Data Pipeline

TL;DR: All the AI data components – embeddings, vector search, RAG, feature stores – connect in a production system orchestrated by Airflow. The pipeline ingests new documents, cleans/chunks them, generates embeddings, indexes them in pgvector, computes ML features, and runs quality checks.

Pipeline Architecture

The Airflow DAG has two parallel branches: - **Branch 1:** Ingest Docs -> Clean & Chunk -> Embed & Index - **Branch 2:** Feature Computation -> Materialize to Store

Both branches converge at Quality Checks. Below the DAG: Postgres + pgvector serves both a FastAPI API (/search, /features, /ask) and direct analytics queries.

```

from airflow.decorators import dag, task
from datetime import datetime

@dag(
    schedule="@daily",
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["ai", "embeddings", "rag"],
)
def ai_data_pipeline():

    @task()
    def ingest_new_documents():
        """Check S3 for new documents, download them."""
        pass

    @task()
    def clean_and_chunk(documents):
        """Clean documents and split into chunks."""
        pass

    @task()
    def generate_embeddings(chunks):
        """Generate embeddings for new chunks."""
        pass

    @task()
    def index_in_vector_db(embeddings):
        """Store embeddings in pgvector."""
        pass

    @task()
    def compute_features():
        """Compute ML features from latest data."""
        pass

    @task()
    def run_quality_checks():
        """Validate embedding quality and feature freshness."""
        pass

    docs = ingest_new_documents()
    chunks = clean_and_chunk(docs)
    embeddings = generate_embeddings(chunks)
    indexed = index_in_vector_db(embeddings)
    features = compute_features()

    [indexed, features] >> run_quality_checks()

ai_data_pipeline()

```

Production Patterns: Batch vs Real-Time Embedding Updates

In production, you need to decide how to handle updates to your document corpus and product catalog:

Batch updates (most common, simplest): - Run the embedding pipeline on a schedule (daily, hourly) - New documents accumulate, then get processed in bulk - Good for: product catalogs, documentation, knowledge bases - Use Airflow or similar orchestrator

Real-time updates (when freshness matters): - Trigger embedding generation when a document is created/updated - Use a message queue (Kafka, SQS) to decouple ingestion from embedding - Good for: support tickets, live chat logs, breaking news - Higher operational complexity

Handling document updates: - Track document versions with a hash or `updated_at` timestamp - When a document changes, re-chunk and re-embed only that document - Delete old chunks for that document before inserting new ones - The companion code's `get_products_without_embeddings()` method shows this incremental pattern

Embedding model versioning: - When you switch embedding models, ALL existing embeddings become incompatible - You cannot compare embeddings from different models – the vector spaces are different - Strategy: create a new table/column, backfill all embeddings with the new model, then swap - Store the model name and version in metadata so you can track what generated each embedding

The Search and Retrieval API

```
# src/api.py
from fastapi import FastAPI, Query
from pydantic import BaseModel

app = FastAPI(title="RAG Search API")

class SearchResult(BaseModel):
    content: str
    source: str
    similarity: float

class RAGResponse(BaseModel):
    answer: str
    sources: list[SearchResult]

@app.get("/search", response_model=list[SearchResult])
async def search(
    q: str = Query(..., description="Search query"),
    top_k: int = Query(5, ge=1, le=20),
):
    """Semantic search over indexed documents."""
    # Use your retriever here
    pass

@app.get("/ask", response_model=RAGResponse)
async def ask(
    question: str = Query(..., description="Question to answer"),
):
    """RAG: retrieve context and generate an answer."""
    # Use your retriever.query_with_context() here
    pass

@app.get("/health")
async def health():
    return {"status": "healthy", "indexed_chunks": 0} # TODO: actual count
```

Companion code: See `module-9/solution/src/api.py` for the complete FastAPI application with `/search`, `/similar/{product_id}`, `/ask`, and `/health` endpoints, including lazy initialization of the search and retriever classes.

9.8 Module 9 Project – RAG-Ready Data Pipeline

TL;DR: Build a complete RAG pipeline: ingest documents from a local directory, clean and chunk them, generate embeddings, store in pgvector, and serve a retrieval/search API via FastAPI. Everything runs in Docker Compose.

Overview

Build a RAG-ready data pipeline that:

1. Ingests documents from a local directory (simulating S3)
2. Cleans and chunks them
3. Generates embeddings (OpenAI or sentence-transformers)
4. Stores in Postgres + pgvector
5. Serves a retrieval/search API via FastAPI
6. Orchestrated by Airflow

Project Structure

```
module-9-project/
|-- dags/
|   |-- rag_pipeline.py
|-- src/
|   |-- __init__.py
|   |-- ingest.py
|   |-- cleaner.py
|   |-- chunker.py
|   |-- embedder.py
|   |-- indexer.py
|   |-- retriever.py
|   |-- api.py
|-- data/
|   |-- documents/
|       |-- product-catalog.md
|       |-- return-policy.pdf
|       |-- faq.txt
|-- tests/
|   |-- test_chunker.py
|   |-- test_cleaner.py
|   |-- test_retriever.py
|-- docker-compose.yml
|-- Dockerfile
|-- pyproject.toml
|-- README.md
```

Step 1: Prepare Sample Documents

Create 3-5 sample documents in `data/documents/`. Use at least 2 different formats (txt, md, pdf).

Companion code: See `module-9/starter/data/documents/` for example documents including `faq.txt` (ShopFast FAQ with 15 Q&A pairs) and `product-catalog.md` (product listings across multiple categories).

Step 2: Build the Pipeline Components

Use the code from Sections 9.3-9.6 as your starting point.

Step 3: Build the API

See the FastAPI starter above, or use the companion code in `module-9/solution/src/api.py` as a reference.

Step 4: Docker Compose

```
version: "3.8"

services:
  api:
    build: .
    command: uvicorn src.api:app --host 0.0.0.0 --port 8000 --reload
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/ragdb
      - OPENAI_API_KEY=${OPENAI_API_KEY}
    depends_on:
      db:
        condition: service_healthy
    volumes:
      - ./src:/app/src
      - ./data:/app/data

  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: ragdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

Companion code: See `module-9/starter/docker-compose.yml` for a minimal setup that maps port 5434 on the host to 5432 in the container and auto-runs `sql/init.sql` on first start.

Step 5: Write Tests

```
# tests/test_chunker.py
from src.chunker import chunk_by_tokens

def test_chunk_size():
    text = " ".join(["word"] * 1000)
    chunks = chunk_by_tokens(text, chunk_size=100, chunk_overlap=10)
    for chunk in chunks:
        word_count = len(chunk.split())
        assert word_count <= 120 # Allow some variance

def test_chunk_overlap():
    text = " ".join([f"word{i}" for i in range(200)])
    chunks = chunk_by_tokens(text, chunk_size=50, chunk_overlap=10)
    assert len(chunks) > 1
```

Companion code: See `module-9/solution/tests/test_chunker.py` and `module-9/solution/tests/test_cleaner.py` for additional test examples including metadata preservation tests and HTML/markdown cleaning tests.

Try It Yourself

```
# Start the stack
docker-compose up -d

# Index documents
python -m src.indexer

# Test the API
curl "http://localhost:8000/search?q=return+policy&top_k=3"
curl "http://localhost:8000/ask?question=What+is+the+return+policy+for+electronics"
```

Expected output:

```
# /search returns top 3 matching chunks with similarity scores
# /ask returns an AI-generated answer with source citations
```

Completion Checklist

- ☐ Document ingestion handles at least 2 file formats
- ☐ Text cleaning removes common artifacts
- ☐ Chunking uses overlap and preserves metadata
- ☐ Embeddings generated and stored in pgvector
- ☐ HNSW or IVFFlat index created on embedding column

- [] `/search` endpoint returns ranked results with similarity scores
- [] `/ask` endpoint returns RAG-generated answers with source citations
- [] Docker Compose starts full stack with one command
- [] At least 3 unit tests
- [] README with setup instructions and example queries

Bonus Challenges

- Add a `/index` endpoint that accepts new documents via upload
 - Implement metadata filtering (filter by source, content type)
 - Add a simple web UI with Streamlit or Gradio
 - Use sentence-transformers instead of OpenAI (no API key needed)
 - Add an Airflow DAG that re-indexes documents on a schedule
-

The DE's Role in ML/AI Teams

At your job, the lines between data engineering and ML engineering are blurring. But here's how the responsibilities typically break down:

What the ML/AI engineer does: - Chooses the model architecture - Trains and fine-tunes models - Evaluates model performance - Designs the prompt templates

What you (the data engineer) do: - Builds the data pipelines that feed the models - Manages the vector database and embedding infrastructure - Handles document ingestion, cleaning, and chunking - Builds and maintains the feature store - Sets up the retrieval API - Monitors data quality and pipeline health - Handles scale – batch processing, rate limiting, incremental updates

The ML engineer might prototype with 100 documents loaded into ChromaDB in a Jupyter notebook. You're the one who makes it work with 10 million documents in production, with automatic updates, monitoring, and a 99.9% uptime API.

That's the value you bring. The AI is only as good as the data infrastructure behind it.

What's Next

Everything you've learned across 9 modules comes together in Module 10: the Capstone Project. You'll build a complete, production-grade Real-Time E-Commerce Analytics Platform with batch

and streaming pipelines, data quality checks, and an AI component powered by embeddings and pgvector. This is the project you put on your resume and walk through in interviews.

End of Module 9

Module 10: Capstone Project – Real-Time E-Commerce Analytics Platform

Why a Capstone Matters

This capstone is designed to be THE project on your resume. Hiring managers want to see a complete, end-to-end data platform – not a collection of disconnected tutorials. They want proof that you can take messy, multi-source data and turn it into something an analyst, a machine learning model, or a business stakeholder can actually use.

Everything you built in Modules 1 through 9 converges here: Docker containerization, SQL modeling, Python pipelines, Kafka streaming, Airflow orchestration, star schema design, data quality with Great Expectations, and AI-powered embeddings. This is not a toy example. This is essentially a miniature version of what you would build at a mid-size company running an e-commerce platform.

Key Takeaway: A single well-built, fully documented project is worth more than ten half-finished repos. This capstone is your chance to show depth, not breadth.

Architecture Overview

The platform follows a layered architecture that mirrors real production data systems. Every layer maps directly to a module you have already completed:



How Modules 1-9 Connect

LAYER	MODULE	WHAT YOU BUILT
Infrastructure	Module 1 (Docker, Git)	<code>docker-compose.yml</code> that spins up Postgres, Airflow, Kafka, MinIO
Storage	Module 2 (SQL)	Raw tables, dimension tables, fact tables – all the DDL in <code>sql/</code>
Data Modeling	Module 3 (Modeling)	Star schema with SCD Type 2 dimensions and grain-level fact tables
Python Pipelines	Module 4 (Python)	The <code>src/</code> package: API clients, file loaders, Kafka producer/consumer
Ingestion	Module 5 (Ingestion)	Three ingestion patterns: REST API, streaming (Kafka), batch file loads
Orchestration	Module 6 (Airflow)	Four DAGs that run ingestion, transformation, quality checks, and embeddings
Transformation	Module 7 (Transforms)	<code>StarSchemaBuilder</code> with SCD logic, dimension key lookups, derived facts
Data Quality	Module 8 (Quality)	Great Expectations integration + custom quality checks with alerting
AI / Serving	Module 9 (AI)	Product embeddings via pgvector, similarity search API via FastAPI

At your job: This is the same layered approach used at companies like Shopify, Instacart, and Wayfair. The specific tools differ (Snowflake instead of Postgres, Flink instead of Kafka consumer, dbt instead of raw SQL), but the architecture pattern is universal. If you can explain each layer and why it exists, you can adapt to any company's stack.

Project Structure

The companion code lives in `de-fast-track/modules/module-10/`. The `starter/` directory has TODO stubs for you to implement. The `solution/` directory has complete, working code.

```

ecommerce-data-platform/
├── docker-compose.yml      # Full stack: Postgres+pgvector, Airflow, Kafka, MinIO
├── pyproject.toml          # Python dependencies
├── .env.example            # Environment variable template
├── .github/workflows/ci.yml # GitHub Actions CI pipeline
├── sql/
│   ├── 001_create_raw_tables.sql
│   ├── 002_create_dim_tables.sql
│   ├── 003_create_fact_tables.sql
│   ├── 004_create_quality_tables.sql
│   └── 005_create_vector_tables.sql
├── data/
│   └── generate_seed_data.py # Creates realistic test data
├── dags/
│   ├── ingest_products.py   # Ingestion DAG
│   ├── transform_star_schema.py # Transformation DAG
│   ├── quality_checks.py    # Data quality DAG
│   └── embeddings_pipeline.py # AI embeddings DAG
├── src/
│   ├── common/
│   │   ├── config.py        # Centralized configuration
│   │   └── db.py            # Database connection utilities
│   ├── ingestion/
│   │   ├── api_client.py    # REST API product ingestion
│   │   ├── file_loader.py   # CSV/JSON file loading
│   │   ├── kafka_producer.py # Event stream producer
│   │   └── kafka_consumer.py # Event stream consumer
│   ├── transformation/
│   │   └── star_schema.py    # Star schema builder with SCD
│   ├── quality/
│   │   ├── expectations.py  # Quality check definitions
│   │   └── alerts.py        # Alerting on failures
│   └── ai/
│       ├── embeddings.py    # Product embedding generation
│       ├── vector_search.py # Similarity search logic
│       └── api.py           # FastAPI serving layer
└── tests/
    ├── test_ingestion.py
    ├── test_transformation.py
    └── test_quality.py

```

At your job: This is not an arbitrary file layout. Separating `src/` into domain-oriented subpackages (`ingestion/`, `transformation/`, `quality/`, `ai/`) is standard practice. It means different team members can work on different layers without stepping on each other. When a hiring manager opens your repo, this structure signals “this person has worked on real teams.”

Phase 1: Infrastructure Setup

TL;DR: Get Docker Compose running with Postgres (+ pgvector), Airflow, Kafka, and MinIO. Create all database schemas. Generate seed data.

Docker Compose Stack

The `docker-compose.yml` spins up seven services. This is your entire data platform in one command.

See companion code: `starter/docker-compose.yml`


```

services:
  warehouse-db:
    image: pgvector/pgvector:pg16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dataeng
      POSTGRES_PASSWORD: dataeng
      POSTGRES_DB: warehouse
    volumes:
      - ./sql:/docker-entrypoint-initdb.d
      - warehouse_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dataeng"]
      interval: 10s
      timeout: 5s
      retries: 5

  airflow-db:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow

  airflow-init:
    image: apache/airflow:2.8.1-python3.11
    depends_on:
      - airflow-db
    environment:
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
airflow:airflow@airflow-db:5432/airflow
    command: >
      bash -c "airflow db init &&
        airflow users create --username admin --password admin
        --firstname Admin --lastname User --role Admin --email admin@example.com ||
true"

  airflow-webserver:
    image: apache/airflow:2.8.1-python3.11
    depends_on:
      airflow-init:
        condition: service_completed_successfully
      warehouse-db:
        condition: service_healthy
    ports:
      - "8080:8080"
    environment:
      AIRFLOW__CORE__EXECUTOR: LocalExecutor
      AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
airflow:airflow@airflow-db:5432/airflow
      AIRFLOW__CORE__LOAD_EXAMPLES: "false"
    volumes:
      - ./dags:/opt/airflow/dags

```

```

- ./src:/opt/airflow/src
command: webserver

airflow-scheduler:
  image: apache/airflow:2.8.1-python3.11
  depends_on:
    airflow-init:
      condition: service_completed_successfully
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://
airflow:airflow@airflow-db:5432/airflow
    AIRFLOW__CORE__LOAD_EXAMPLES: "false"
  volumes:
    - ./dags:/opt/airflow/dags
    - ./src:/opt/airflow/src
  command: scheduler

zookeeper:
  image: confluentinc/cp-zookeeper:7.6.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181

kafka:
  image: confluentinc/cp-kafka:7.6.0
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://
localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

minio:
  image: minio/minio:latest
  ports:
    - "9000:9000"
    - "9001:9001"
  environment:
    MINIO_ROOT_USER: minioadmin
    MINIO_ROOT_PASSWORD: minioadmin
  command: server /data --console-address ":9001"
  volumes:
    - minio_data:/data

volumes:
  warehouse_data:
  minio_data:

```

Notice the key design decisions here. The `warehouse-db` uses `pgvector/pgvector:pg16` instead of plain Postgres because we need vector similarity search later. The SQL files in `sql/` are mounted to `/docker-entrypoint-initdb.d` so schemas are created automatically on first boot. The Airflow webserver and scheduler both mount `./dags` and `./src` so they can find your DAG definitions and source code.

At your job: Production environments would separate these into individual services on Kubernetes, each with its own scaling policy. But for development and portfolio purposes, Docker Compose gives you the entire stack with `docker compose up -d`. This is exactly how most teams develop locally.

Database Schema

The SQL files execute in alphabetical order when the database starts. They create three schemas: `raw` (landing zone), `analytics` (star schema), and `quality` (check results and quarantine).

See companion code: `starter/sql/001_create_raw_tables.sql`

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE SCHEMA IF NOT EXISTS raw;
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE SCHEMA IF NOT EXISTS quality;

-- Raw tables
CREATE TABLE raw.products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(300) NOT NULL,
  category VARCHAR(100),
  subcategory VARCHAR(100),
  brand VARCHAR(100),
  unit_cost NUMERIC(10,2),
  unit_price NUMERIC(10,2),
  description TEXT,
  loaded_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE raw.events (
  event_id VARCHAR(50) PRIMARY KEY,
  event_type VARCHAR(50) NOT NULL,
  customer_id INT,
  product_id INT,
  event_data JSONB,
  event_timestamp TIMESTAMP NOT NULL,
  loaded_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE raw.inventory (
  product_id INT,
  warehouse VARCHAR(50),
  quantity_on_hand INT,
  reorder_point INT,
  loaded_at TIMESTAMP DEFAULT NOW(),
  PRIMARY KEY (product_id, warehouse)
);

CREATE TABLE raw.customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(200) NOT NULL,
  email VARCHAR(200),
  city VARCHAR(100),
  state VARCHAR(50),
  segment VARCHAR(50),
  signup_date DATE,
  loaded_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_raw_events_type ON raw.events(event_type);
CREATE INDEX idx_raw_events_ts ON raw.events(event_timestamp);

```

Every raw table has a `loaded_at` timestamp. This is not optional – it is how you debug “when did this bad data arrive?” in production. The `event_data JSONB` column on `raw.events` stores the full event payload so you never lose data during ingestion, even if your schema does not cover every field.

See companion code: `starter/sql/002_create_dim_tables.sql`

```

-- Dimension tables
CREATE TABLE analytics.dim_products (
    product_key SERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    product_name VARCHAR(300),
    category VARCHAR(100),
    subcategory VARCHAR(100),
    brand VARCHAR(100),
    unit_cost NUMERIC(10,2),
    unit_price NUMERIC(10,2),
    effective_date DATE NOT NULL,
    expiry_date DATE NOT NULL DEFAULT '9999-12-31',
    is_current BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE analytics.dim_customers (
    customer_key SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    name VARCHAR(200),
    email VARCHAR(200),
    city VARCHAR(100),
    state VARCHAR(50),
    segment VARCHAR(50),
    signup_date DATE,
    effective_date DATE NOT NULL,
    expiry_date DATE NOT NULL DEFAULT '9999-12-31',
    is_current BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE analytics.dim_time (
    time_key INT PRIMARY KEY,
    calendar_date DATE NOT NULL UNIQUE,
    day_of_week VARCHAR(10),
    month INT,
    month_name VARCHAR(10),
    quarter INT,
    year INT,
    is_weekend BOOLEAN
);

-- Populate dim_time for 2024-2027
INSERT INTO analytics.dim_time (time_key, calendar_date, day_of_week,
    month, month_name, quarter, year, is_weekend)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d,
    TRIM(TO_CHAR(d, 'Day')),
    EXTRACT(MONTH FROM d),
    TRIM(TO_CHAR(d, 'Month')),
    EXTRACT(QUARTER FROM d),
    EXTRACT(YEAR FROM d),
    EXTRACT(ISODOW FROM d) IN (6, 7)
FROM generate_series('2024-01-01'::date, '2027-12-31'::date, '1 day') d;

```

```
CREATE INDEX idx_dim_products_natural ON analytics.dim_products(product_id, is_current)
;
CREATE INDEX idx_dim_customers_natural ON analytics.dim_customers(customer_id, is_current);
```

The `dim_products` and `dim_customers` tables use SCD Type 2 columns: `effective_date`, `expiry_date`, and `is_current`. This means when a customer moves from New York to Boston, we do not overwrite the old row. We expire it and insert a new current row. Historical orders still join to the version of the customer that existed when the order was placed. This is one of the most important concepts in dimensional modeling, and having it in your capstone is a strong signal to interviewers.

The `dim_time` table is pre-populated with every date from 2024 to 2027. This is standard practice – analysts can join on `time_key` (an integer in `YYYYMMDD` format) to get day-of-week, quarter, and other calendar attributes without computing them in every query.

See companion code: `starter/sql/003_create_fact_tables.sql`

```
CREATE TABLE analytics.fact_orders (
  order_key SERIAL PRIMARY KEY,
  order_id VARCHAR(50) NOT NULL,
  customer_key INT REFERENCES analytics.dim_customers(customer_key),
  product_key INT REFERENCES analytics.dim_products(product_key),
  time_key INT REFERENCES analytics.dim_time(time_key),
  quantity INT NOT NULL,
  unit_price NUMERIC(10,2),
  revenue NUMERIC(12,2),
  cost NUMERIC(12,2),
  profit NUMERIC(12,2)
);

CREATE TABLE analytics.fact_events (
  event_key SERIAL PRIMARY KEY,
  event_id VARCHAR(50) NOT NULL,
  event_type VARCHAR(50) NOT NULL,
  customer_key INT REFERENCES analytics.dim_customers(customer_key),
  product_key INT REFERENCES analytics.dim_products(product_key),
  time_key INT REFERENCES analytics.dim_time(time_key),
  event_data JSONB,
  event_timestamp TIMESTAMP NOT NULL
);

CREATE INDEX idx_fact_orders_time ON analytics.fact_orders(time_key);
CREATE INDEX idx_fact_orders_customer ON analytics.fact_orders(customer_key);
CREATE INDEX idx_fact_orders_product ON analytics.fact_orders(product_key);
CREATE INDEX idx_fact_events_type ON analytics.fact_events(event_type);
CREATE INDEX idx_fact_events_time ON analytics.fact_events(time_key);
```

Two fact tables, two different grains. `fact_orders` has one row per purchase line item. `fact_events` has one row per user interaction (page view, add to cart, search, etc.). The foreign

keys to dimension tables enforce referential integrity – you cannot load an order for a customer that does not exist in the dimension.

See companion code: [starter/sql/004_create_quality_tables.sql](#)

```
CREATE TABLE quality.check_results (  
    check_id SERIAL PRIMARY KEY,  
    check_name VARCHAR(200) NOT NULL,  
    table_name VARCHAR(200),  
    status VARCHAR(20) NOT NULL,  
    details JSONB,  
    checked_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE quality.quarantine (  
    quarantine_id SERIAL PRIMARY KEY,  
    source_table VARCHAR(200),  
    record_data JSONB,  
    violation_reason VARCHAR(500),  
    quarantined_at TIMESTAMP DEFAULT NOW()  
);
```

The quality schema gives you an audit trail. Every time quality checks run, the results are logged. The quarantine table is for records that fail validation – they get isolated instead of silently corrupting your analytics tables.

See companion code: [starter/sql/005_create_vector_tables.sql](#)

```
CREATE TABLE analytics.product_embeddings (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(300),  
    category VARCHAR(100),  
    description TEXT,  
    embedding vector(384),  
    updated_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE INDEX idx_product_embeddings_hnsw  
ON analytics.product_embeddings USING hnsw (embedding vector_cosine_ops);
```

The `vector(384)` column stores embeddings from the `all-MiniLM-L6-v2` model (which produces 384-dimensional vectors). The HNSW index enables approximate nearest neighbor search – this is what makes similarity queries fast even with thousands of products.

Common Utilities

See companion code: [starter/src/common/config.py](#)


```

"""Capstone project configuration."""
import os

class Config:
    DATABASE_URL = os.getenv("DATABASE_URL", "postgresql://
dataeng:dataeng@localhost:5432/warehouse")
    DB_HOST = os.getenv("DB_HOST", "localhost")
    DB_PORT = int(os.getenv("DB_PORT", "5432"))
    DB_USER = os.getenv("DB_USER", "dataeng")
    DB_PASSWORD = os.getenv("DB_PASSWORD", "dataeng")
    DB_NAME = os.getenv("DB_NAME", "warehouse")

    KAFKA_BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")

    S3_ENDPOINT = os.getenv("S3_ENDPOINT", "http://localhost:9000")
    S3_ACCESS_KEY = os.getenv("S3_ACCESS_KEY", "minioadmin")
    S3_SECRET_KEY = os.getenv("S3_SECRET_KEY", "minioadmin")
    S3_BUCKET = os.getenv("S3_BUCKET", "raw-data")

    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "")
    LOG_LEVEL = os.getenv("LOG_LEVEL", "INFO")

```

Every secret and connection string comes from environment variables with sensible defaults for local development. This pattern means the same code runs locally (hitting `localhost:5432`) and in CI (hitting a test database) without any code changes. The `.env.example` file documents which variables exist; developers copy it to `.env` and fill in their values.

See companion code: `starter/src/common/db.py`

```

"""Database utilities."""
import psycopg2
from contextlib import contextmanager
from .config import Config

@contextmanager
def get_db_connection():
    conn = psycopg2.connect(Config.DATABASE_URL)
    try:
        yield conn
    finally:
        conn.close()

@contextmanager
def get_cursor(commit=True):
    with get_db_connection() as conn:
        cur = conn.cursor()
        try:
            yield cur
            if commit:
                conn.commit()
        except Exception:
            conn.rollback()
            raise

```

The context managers handle connection lifecycle automatically. `get_cursor(commit=True)` means you never forget to commit or close a connection. The `rollback()` in the exception handler ensures partial writes do not corrupt the database.

Seed Data Generator

See companion code: `starter/data/generate_seed_data.py`

```

"""Generate seed data for the capstone project."""
import json
import csv
import random
import uuid
from datetime import datetime, timedelta
from pathlib import Path

random.seed(42)
OUTPUT_DIR = Path(__file__).parent / "seed"
OUTPUT_DIR.mkdir(exist_ok=True)

CATEGORIES = {
    "Electronics": ["Phones", "Laptops", "Audio", "Peripherals", "Accessories"],
    "Clothing": ["Shirts", "Pants", "Outerwear", "Shoes", "Accessories"],
    "Home & Garden": ["Furniture", "Kitchen", "Decor", "Tools", "Lighting"],
    "Sports": ["Running", "Yoga", "Cycling", "Outdoor", "Equipment"],
    "Books": ["Technical", "Fiction", "Business", "Science", "Self-Help"],
}

BRANDS = ["TechBrand", "ComfyWear", "ErgoLife", "FastFeet", "FlexFit",
          "SoundWave", "OReilly", "HomeStyle", "ActiveGear", "ReadMore"]

CITIES = [
    ("New York", "NY"), ("Los Angeles", "CA"), ("Chicago", "IL"),
    ("Houston", "TX"), ("Phoenix", "AZ"), ("Philadelphia", "PA"),
    ("San Antonio", "TX"), ("San Diego", "CA"), ("Dallas", "TX"),
    ("Austin", "TX"),
]

FIRST_NAMES = ["Alice", "Bob", "Carol", "David", "Eve", "Frank", "Grace", "Hank", "Ivy", "Jack",
               "Karen", "Leo", "Mia", "Noah", "Olivia", "Pete", "Quinn", "Rose", "Sam", "Tina"]
LAST_NAMES = ["Smith", "Johnson", "Brown", "Davis", "Wilson", "Moore", "Taylor", "Anderson", "Lee", "Martinez"]

def generate_products(count=500):
    products = []
    pid = 1
    for category, subcats in CATEGORIES.items():
        for subcat in subcats:
            for i in range(count // (len(CATEGORIES) * 5)):
                cost = round(5 + random.random() * 195, 2)
                markup = 1.5 + random.random() * 2
                products.append({
                    "product_id": pid,
                    "product_name": f"{subcat} {random.choice(['Pro', 'Plus', 'Lite', 'Max', 'Ultra', 'Basic'])} {pid}",
                    "category": category,
                    "subcategory": subcat,
                    "brand": random.choice(BRANDS),
                    "unit_cost": cost,
                    "unit_price": round(cost * markup, 2),
                    "description": f"High-quality {subcat.lower()} product in the {cate

```

```

gory.lower()} category.",
    })
    pid += 1
with open(OUTPUT_DIR / "products.json", "w") as f:
    json.dump(products[:count], f, indent=2)
print(f"Generated {min(len(products), count)} products")
return products[:count]

def generate_customers(count=1000):
    customers = []
    for i in range(1, count + 1):
        city, state = random.choice(CITIES)
        customers.append({
            "customer_id": i,
            "name": f"{random.choice(FIRST_NAMES)} {random.choice(LAST_NAMES)}",
            "email": f"customer{i}@example.com",
            "city": city,
            "state": state,
            "segment": random.choice(["Consumer", "Consumer", "Consumer", "SMB",
"SMB", "Enterprise"]),
            "signup_date": (datetime(2024, 1, 1) + timedelta(days=random.randint(0,
730))).strftime("%Y-%m-%d"),
        })
    with open(OUTPUT_DIR / "customers.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=customers[0].keys())
        writer.writeheader()
        writer.writerows(customers)
    print(f"Generated {count} customers")
    return customers

def generate_inventory(products, warehouses=None):
    if warehouses is None:
        warehouses = ["warehouse-east", "warehouse-west", "warehouse-central"]
    rows = []
    for p in products:
        for wh in warehouses:
            rows.append({
                "product_id": p["product_id"],
                "warehouse": wh,
                "quantity_on_hand": random.randint(0, 500),
                "reorder_point": random.randint(10, 50),
            })
    with open(OUTPUT_DIR / "inventory.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=rows[0].keys())
        writer.writeheader()
        writer.writerows(rows)
    print(f"Generated {len(rows)} inventory records")

def generate_events(products, customers, count=10000):
    event_types = ["page_view", "add_to_cart", "remove_from_cart", "purchase",
"search"]
    events = []

```

```

for _ in range(count):
    etype = random.choices(event_types, weights=[40, 25, 5, 20, 10], k=1)[0]
    cust = random.choice(customers)
    prod = random.choice(products)
    ts = datetime(2025, 1, 1) + timedelta(
        days=random.randint(0, 365),
        hours=random.randint(0, 23),
        minutes=random.randint(0, 59),
    )
    event = {
        "event_id": str(uuid.uuid4()),
        "event_type": etype,
        "customer_id": cust["customer_id"],
        "product_id": prod["product_id"],
        "timestamp": ts.isoformat(),
    }
    if etype == "purchase":
        qty = random.randint(1, 4)
        event["quantity"] = qty
        event["unit_price"] = prod["unit_price"]
        event["total_amount"] = round(qty * prod["unit_price"], 2)
    elif etype == "search":
        event["search_term"] = random.choice(
            ["laptop", "shoes", "headphones", "desk", "book", "camera"]
        )
    events.append(event)

with open(OUTPUT_DIR / ".." / "sample_events.json", "w") as f:
    json.dump(events, f, indent=2)
print(f"Generated {count} events")

if __name__ == "__main__":
    products = generate_products(500)
    customers = generate_customers(1000)
    generate_inventory(products)
    generate_events(products, customers, 10000)
    print("\nSeed data generation complete!")

```

Expected output:

```

Generated 500 products
Generated 1000 customers
Generated 1500 inventory records
Generated 10000 events

Seed data generation complete!

```

The seed data is realistic: 5 categories with 5 subcategories each, 500 products with cost/markup pricing, 1,000 customers across 10 cities, 1,500 inventory records across 3 warehouses, and 10,000 events with weighted probabilities (page views are most common, purchases less so). Using `random.seed(42)` makes it deterministic – every team member generates the same data.

Verify Phase 1

```
# Generate seed data
mkdir -p data/seed
python data/generate_seed_data.py

# Start the stack
docker compose up -d

# Wait 1-2 minutes for everything to initialize, then verify:

# Database schemas exist
docker compose exec warehouse-db psql -U dataeng -d warehouse -c "\dt raw.*"
docker compose exec warehouse-db psql -U dataeng -d warehouse -c "\dt analytics.*"

# MinIO is running -- open http://localhost:9001 (minioadmin/minioadmin)
# Airflow is running -- open http://localhost:8080 (admin/admin)
```

Phase 1 Checkpoint: All services running, schemas created, seed data generated.

Phase 2: Data Ingestion

TL;DR: Build three ingestion pipelines: (1) JSON file loader for products, (2) Kafka producer/consumer for real-time user events, (3) CSV file loader for customer and inventory data. Each is wrapped in an Airflow DAG.

File Loader (Products, Customers, Inventory)

See companion code: `solution/src/ingestion/file_loader.py`

```

"""File loader – loads CSV/JSON seed data into Postgres raw tables."""
import csv
import json
import logging
import psycopg2
from ..common.config import Config

logger = logging.getLogger(__name__)

class FileLoader:
    def __init__(self, db_url: str = None):
        self.conn = psycopg2.connect(db_url or Config.DATABASE_URL)

    def load_customers(self, filepath: str):
        with open(filepath) as f:
            reader = csv.DictReader(f)
            cur = self.conn.cursor()
            count = 0
            for row in reader:
                cur.execute("""
INSERT INTO raw.customers (customer_id, name, email, city, state,
segment, signup_date)
VALUES (%s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (customer_id) DO UPDATE SET
    name=EXCLUDED.name, email=EXCLUDED.email, city=EXCLUDED.city,
    state=EXCLUDED.state, segment=EXCLUDED.segment, loaded_at=NOW()
""", (row["customer_id"], row["name"], row["email"],
      row["city"], row["state"], row["segment"], row["signup_date"]))
                count += 1
            self.conn.commit()
            logger.info(f"Loaded {count} customers")

    def load_inventory(self, filepath: str):
        with open(filepath) as f:
            reader = csv.DictReader(f)
            cur = self.conn.cursor()
            count = 0
            for row in reader:
                cur.execute("""
INSERT INTO raw.inventory (product_id, warehouse, quantity_on_hand,
reorder_point)
VALUES (%s, %s, %s, %s)
ON CONFLICT (product_id, warehouse) DO UPDATE SET
    quantity_on_hand=EXCLUDED.quantity_on_hand, loaded_at=NOW()
""", (row["product_id"], row["warehouse"],
      row["quantity_on_hand"], row["reorder_point"]))
                count += 1
            self.conn.commit()
            logger.info(f"Loaded {count} inventory records")

    def load_products(self, filepath: str):
        with open(filepath) as f:
            products = json.load(f)

```

```

cur = self.conn.cursor()
for p in products:
    cur.execute("""
        INSERT INTO raw.products (product_id, product_name, category,
subcategory,
        brand, unit_cost, unit_price, description)
        VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
        ON CONFLICT (product_id) DO UPDATE SET
            product_name=EXCLUDED.product_name, category=EXCLUDED.category,
            unit_price=EXCLUDED.unit_price, loaded_at=NOW()
    """, (p["product_id"], p["product_name"], p["category"],
        p["subcategory"], p["brand"], p["unit_cost"],
        p["unit_price"], p.get("description", "")))
self.conn.commit()
logger.info(f"Loaded {len(products)} products")

def close(self):
    self.conn.close()

```

Every INSERT uses `ON CONFLICT ... DO UPDATE`. This makes the pipeline idempotent – you can run it twice and get the same result. This is critical for production pipelines where retries are common.

Kafka Streaming (Events)

See companion code: [solution/src/ingestion/kafka_producer.py](#)


```

"""Event producer – simulates e-commerce traffic."""
import json
import time
import random
import uuid
import logging
from datetime import datetime
from confluent_kafka import Producer
from ..common.config import Config

logger = logging.getLogger(__name__)

EVENT_TYPES = ["page_view", "add_to_cart", "remove_from_cart", "purchase", "search"]

class EventProducer:
    def __init__(self, bootstrap_servers: str = None, topic: str = "ecommerce-events"):
        self.topic = topic
        self.producer = Producer({
            "bootstrap.servers": bootstrap_servers or Config.KAFKA_BOOTSTRAP_SERVERS,
        })

    def generate_event(self, customer_id: int) -> dict:
        etype = random.choices(EVENT_TYPES, weights=[40, 25, 5, 20, 10], k=1)[0]
        product_id = random.randint(1, 500)
        event = {
            "event_id": str(uuid.uuid4()),
            "event_type": etype,
            "customer_id": customer_id,
            "product_id": product_id,
            "timestamp": datetime.now().isoformat(),
        }
        if etype == "purchase":
            qty = random.randint(1, 4)
            price = round(10 + random.random() * 190, 2)
            event["quantity"] = qty
            event["unit_price"] = price
            event["total_amount"] = round(qty * price, 2)
        elif etype == "search":
            event["search_term"] = random.choice(
                ["laptop", "shoes", "headphones", "desk", "book", "camera"]
            )
        return event

    def run(self, events_per_second: float = 10.0, max_events: int = 0):
        logger.info(f"Producing to '{self.topic}' at ~{events_per_second} events/sec")
        count = 0
        try:
            while True:
                event = self.generate_event(random.randint(1, 1000))
                self.producer.produce(
                    self.topic,
                    key=str(event["customer_id"]),
                    value=json.dumps(event),

```

```

    )
    self.producer.poll(0)
    count += 1
    if max_events and count >= max_events:
        break
    time.sleep(1.0 / events_per_second)
except KeyboardInterrupt:
    pass
finally:
    self.producer.flush()
    logger.info(f"Produced {count} events")

```

The producer uses `customer_id` as the Kafka message key. This ensures all events for the same customer land on the same partition, preserving per-customer ordering. The weighted `random.choices` creates realistic traffic patterns: 40% page views, 25% add-to-cart, 20% purchases, 10% search, 5% remove-from-cart.

See companion code: [solution/src/ingestion/kafka_consumer.py](#)

```

"""Kafka event consumer - reads events and inserts to raw.events."""
import json
import logging
import psycopg2
from confluent_kafka import Consumer, KafkaError
from ..common.config import Config

logger = logging.getLogger(__name__)

class EventConsumer:
    def __init__(self, bootstrap_servers: str = None, topic: str = "ecommerce-events",
                 group_id: str = "capstone-consumer"):
        self.topic = topic
        self.consumer = Consumer({
            "bootstrap.servers": bootstrap_servers or Config.KAFKA_BOOTSTRAP_SERVERS,
            "group.id": group_id,
            "auto.offset.reset": "earliest",
        })
        self.consumer.subscribe([topic])
        self.conn = psycopg2.connect(Config.DATABASE_URL)

    def consume(self, max_messages: int = 1000) -> int:
        cur = self.conn.cursor()
        count = 0
        while count < max_messages:
            msg = self.consumer.poll(1.0)
            if msg is None:
                break
            if msg.error():
                if msg.error().code() == KafkaError._PARTITION_EOF:
                    break
                logger.error(f"Consumer error: {msg.error()}")
                continue

            event = json.loads(msg.value().decode("utf-8"))
            cur.execute("""
                INSERT INTO raw.events (event_id, event_type, customer_id, product_id,
                                     event_data, event_timestamp)
                VALUES (%s, %s, %s, %s, %s, %s)
                ON CONFLICT (event_id) DO NOTHING
            """, (
                event["event_id"], event["event_type"],
                event.get("customer_id"), event.get("product_id"),
                json.dumps(event), event["timestamp"],
            ))
            count += 1
            if count % 100 == 0:
                self.conn.commit()

        self.conn.commit()
        logger.info(f"Consumed {count} events")
        return count

```

```
def close(self):  
    self.consumer.close()  
    self.conn.close()
```

The consumer commits to Postgres every 100 messages for throughput, and stores the full event JSON in `event_data` alongside the parsed fields. The `ON CONFLICT DO NOTHING` handles duplicate events gracefully – exactly-once semantics at the database level.

At your job: In production, you would likely use a framework like Flink or Spark Structured Streaming instead of a raw consumer loop. But the pattern is identical: consume from a topic, deserialize, insert to a raw table. Understanding the fundamentals makes the frameworks easy to learn.

Airflow DAG for Ingestion

See companion code: `solution/dags/ingest_products.py`

```

"""DAG: Ingest products from seed data."""
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    dag_id="ingest_products",
    schedule="@daily",
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=["capstone", "ingestion"],
)
def ingest_products():

    @task()
    def load_products():
        from src.ingestion.file_loader import FileLoader
        loader = FileLoader()
        loader.load_products("data/seed/products.json")
        loader.close()

    @task()
    def load_customers():
        from src.ingestion.file_loader import FileLoader
        loader = FileLoader()
        loader.load_customers("data/seed/customers.csv")
        loader.close()

    @task()
    def load_inventory():
        from src.ingestion.file_loader import FileLoader
        loader = FileLoader()
        loader.load_inventory("data/seed/inventory.csv")
        loader.close()

    load_products()
    load_customers()
    load_inventory()

    ingest_products()

```

Three independent tasks – products, customers, and inventory can load in parallel since they write to different tables. The imports happen inside the task functions (not at module level) so Airflow's DAG parser stays fast. This is an Airflow best practice that trips up many beginners.

Pro Tip: Create similar DAGs for events (`ingest_events.py`) and inventory. The events DAG should trigger the Kafka consumer; the inventory DAG should use the FileLoader.

Phase 2 Checkpoint: All three ingestion pipelines load data into raw tables. Products, customers, and inventory in the database.

Phase 3: Transformation (Star Schema Population)

TL;DR: Transform raw data into a star schema with SCD Type 2 for customers, SCD Type 1 for products, and fact tables populated via dimension key lookups. Orders are derived from purchase events.

See companion code: `solution/src/transformation/star_schema.py`

```

"""Star schema builder - transforms raw data into analytics dimensions and facts."""
import logging
import psycopg2
from ..common.config import Config

logger = logging.getLogger(__name__)

class StarSchemaBuilder:
    def __init__(self, db_url: str = None):
        self.conn = psycopg2.connect(db_url or Config.DATABASE_URL)

    def build_dim_products(self):
        cur = self.conn.cursor()
        # Expire changed products
        cur.execute("""
            UPDATE analytics.dim_products dp
            SET expiry_date = CURRENT_DATE - 1, is_current = FALSE
            FROM raw.products rp
            WHERE dp.product_id = rp.product_id AND dp.is_current = TRUE
            AND (dp.category != rp.category OR dp.unit_price != rp.unit_price)
        """)
        # Insert new/changed products
        cur.execute("""
            INSERT INTO analytics.dim_products (product_id, product_name, category,
            subcategory,
                brand, unit_cost, unit_price, effective_date)
            SELECT product_id, product_name, category, subcategory, brand,
                unit_cost, unit_price, CURRENT_DATE
            FROM raw.products rp
            WHERE NOT EXISTS (
                SELECT 1 FROM analytics.dim_products dp
                WHERE dp.product_id = rp.product_id AND dp.is_current = TRUE
            )
        """)
        self.conn.commit()
        logger.info("dim_products built")

    def build_dim_customers(self):
        cur = self.conn.cursor()
        # SCD Type 2 for customer changes
        cur.execute("""
            UPDATE analytics.dim_customers dc
            SET expiry_date = CURRENT_DATE - 1, is_current = FALSE
            FROM raw.customers rc
            WHERE dc.customer_id = rc.customer_id AND dc.is_current = TRUE
            AND (dc.city != rc.city OR dc.state != rc.state OR dc.segment !=
            rc.segment)
        """)
        cur.execute("""
            INSERT INTO analytics.dim_customers (customer_id, name, email, city, state,
            segment, signup_date, effective_date)
            SELECT customer_id, name, email, city, state, segment, signup_date::date,
            CURRENT_DATE

```

```

        FROM raw.customers rc
        WHERE NOT EXISTS (
            SELECT 1 FROM analytics.dim_customers dc
            WHERE dc.customer_id = rc.customer_id AND dc.is_current = TRUE
        )
    """
    self.conn.commit()
    logger.info("dim_customers built")

def build_fact_events(self):
    cur = self.conn.cursor()
    cur.execute("""
        INSERT INTO analytics.fact_events (event_id, event_type, customer_key,
product_key,
            time_key, event_data, event_timestamp)
        SELECT re.event_id, re.event_type, dc.customer_key, dp.product_key,
            TO_CHAR(re.event_timestamp, 'YYYYMMDD')::INT,
            re.event_data, re.event_timestamp
        FROM raw.events re
        LEFT JOIN analytics.dim_customers dc
            ON re.customer_id = dc.customer_id AND dc.is_current = TRUE
        LEFT JOIN analytics.dim_products dp
            ON re.product_id = dp.product_id AND dp.is_current = TRUE
        WHERE NOT EXISTS (
            SELECT 1 FROM analytics.fact_events fe WHERE fe.event_id = re.event_id
        )
    """)
    self.conn.commit()
    cur.execute("SELECT COUNT(*) FROM analytics.fact_events")
    logger.info(f"fact_events: {cur.fetchone()[0]} total rows")

def build_fact_orders(self):
    cur = self.conn.cursor()
    cur.execute("""
        INSERT INTO analytics.fact_orders (order_id, customer_key, product_key,
time_key,
            quantity, unit_price, revenue, cost, profit)
        SELECT re.event_id, dc.customer_key, dp.product_key,
            TO_CHAR(re.event_timestamp, 'YYYYMMDD')::INT,
            (re.event_data->'quantity')::INT,
            (re.event_data->'unit_price')::NUMERIC,
            (re.event_data->'total_amount')::NUMERIC,
            (re.event_data->'quantity')::INT * COALESCE(dp.unit_cost, 0),
            (re.event_data->'total_amount')::NUMERIC - (re.event_data-
>>'quantity')::INT * COALESCE(dp.unit_cost, 0)
        FROM raw.events re
        LEFT JOIN analytics.dim_customers dc
            ON re.customer_id = dc.customer_id AND dc.is_current = TRUE
        LEFT JOIN analytics.dim_products dp
            ON re.product_id = dp.product_id AND dp.is_current = TRUE
        WHERE re.event_type = 'purchase'
        AND NOT EXISTS (
            SELECT 1 FROM analytics.fact_orders fo WHERE fo.order_id =
re.event_id
        )
    """)
    self.conn.commit()
    logger.info("fact_orders built")

```



```

    """
    self.conn.commit()
    cur.execute("SELECT COUNT(*) FROM analytics.fact_orders")
    logger.info(f"fact_orders: {cur.fetchone()[0]} total rows")

    def run(self):
        self.build_dim_products()
        self.build_dim_customers()
        self.build_fact_events()
        self.build_fact_orders()
        logger.info("Star schema build complete")

    def close(self):
        self.conn.close()

```

This is the heart of the project. Study it carefully:

- `build_dim_products`: First expires any product rows where the category or price changed (SCD Type 2 expiration), then inserts new rows for products that do not have a current dimension record. The `NOT EXISTS` pattern ensures idempotency.
- `build_dim_customers`: Same SCD Type 2 pattern for customers. Tracks changes in city, state, and segment. When a customer moves from New York to Boston, the old row gets `is_current = FALSE` and a new row is inserted.
- `build_fact_events`: Joins raw events to dimension tables to get surrogate keys (`customer_key`, `product_key`), converts the timestamp to `time_key` format (`YYYYMMDD` integer), and deduplicates via `NOT EXISTS`.
- `build_fact_orders`: Filters for `event_type = 'purchase'` only, extracts quantity/price/amount from the JSONB `event_data` column, and calculates cost and profit using the dimension table's `unit_cost`.

At your job: The pattern of “expire old dimension rows, insert new ones, then populate facts with dimension key lookups” is the backbone of every Kimball-style data warehouse. dbt automates much of this with snapshot models, but understanding the raw SQL gives you the ability to debug when dbt does something unexpected.

Airflow DAG for Transformation

See companion code: `solution/dags/transform_star_schema.py`

```

"""DAG: Build star schema from raw data."""
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    dag_id="transform_star_schema",
    schedule="0 7 * * *",
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=["capstone", "transformation"],
)
def transform_star_schema():

    @task()
    def build_dimensions():
        from src.transformation.star_schema import StarSchemaBuilder
        builder = StarSchemaBuilder()
        builder.build_dim_products()
        builder.build_dim_customers()
        builder.close()

    @task()
    def build_facts():
        from src.transformation.star_schema import StarSchemaBuilder
        builder = StarSchemaBuilder()
        builder.build_fact_events()
        builder.build_fact_orders()
        builder.close()

    dims = build_dimensions()
    facts = build_facts()
    dims >> facts

transform_star_schema()

```

Dimensions must be built before facts (the `dims >> facts` dependency). Facts need surrogate keys from the dimension tables, so this ordering is not optional. The schedule is `0 7 * * *` (7 AM daily), giving the ingestion DAG time to complete first.

Phase 3 Checkpoint: Star schema populated. Dimensions have correct SCD behavior. Fact tables have proper foreign key lookups.

Phase 4: Data Quality and AI Component

TL;DR: Add quality checks on raw event data (null checks, uniqueness, valid event types, row count). Build a product embeddings pipeline that generates vectors and stores them in pgvector. Serve similarity search and semantic product search via FastAPI.

Data Quality Checks

See companion code: `solution/src/quality/expectations.py`

```

"""Data quality checks for the capstone pipeline."""
import logging
import psycopg2
import json
from ..common.config import Config

logger = logging.getLogger(__name__)

def run_quality_checks(db_url: str = None) -> list[dict]:
    conn = psycopg2.connect(db_url or Config.DATABASE_URL)
    cur = conn.cursor()
    results = []

    checks = [
        ("raw_events_not_empty", "SELECT COUNT(*) FROM raw.events", lambda v: v > 0),
        ("raw_events_no_null_ids", "SELECT COUNT(*) FROM raw.events WHERE event_id IS
NULL", lambda v: v == 0),
        ("raw_events_valid_types",
         "SELECT COUNT(*) FROM raw.events WHERE event_type NOT IN
('page_view', 'add_to_cart', 'remove_from_cart', 'purchase', 'search')",
         lambda v: v == 0),
        ("dim_products_no_duplicates",
         "SELECT COUNT(*) - COUNT(DISTINCT product_id) FROM analytics.dim_products
WHERE is_current",
         lambda v: v == 0),
        ("dim_customers_no_duplicates",
         "SELECT COUNT(*) - COUNT(DISTINCT customer_id) FROM analytics.dim_customers
WHERE is_current",
         lambda v: v == 0),
        ("fact_orders_positive_revenue",
         "SELECT COUNT(*) FROM analytics.fact_orders WHERE revenue < 0",
         lambda v: v == 0),
        ("fact_orders_no_orphans",
         "SELECT COUNT(*) FROM analytics.fact_orders fo LEFT JOIN
analytics.dim_customers dc ON fo.customer_key = dc.customer_key WHERE dc.customer_key
IS NULL",
         lambda v: v == 0),
    ]

    for name, sql, check_fn in checks:
        cur.execute(sql)
        value = cur.fetchone()[0]
        passed = check_fn(value)
        status = "PASS" if passed else "FAIL"
        results.append({"check": name, "value": value, "status": status})
        logger.info(f" {status}: {name} (value={value})")

    # Log to quality.check_results
    cur.execute("""
        INSERT INTO quality.check_results (check_name, table_name, status, details)
        VALUES (%s, %s, %s, %s)
    """, (name, "", status, json.dumps({"value": value})))

```

```

conn.commit()
conn.close()
return results

```

Seven checks covering four quality dimensions: - **Completeness:** `raw_events_not_empty` – the table should have data - **Validity:** `raw_events_no_null_ids` – primary keys must not be null - **Consistency:** `raw_events_valid_types` – only known event types allowed - **Uniqueness:** `dim_products_no_duplicates`, `dim_customers_no_duplicates` – no duplicate current rows - **Accuracy:** `fact_orders_positive_revenue` – revenue should not be negative - **Referential integrity:** `fact_orders_no_orphans` – every order should link to a real customer

Every check result is logged to `quality.check_results` for auditing. This means you can query historical check results: “When did the orphan check first start failing?”

See companion code: `solution/src/quality/alerts.py`

```

"""Alerting functions for quality check failures."""
import logging

logger = logging.getLogger(__name__)

def send_alert(check_name: str, details: dict):
    """Send an alert for a failed quality check.
    In production, this would integrate with Slack, PagerDuty, email, etc.
    """
    logger.warning(f"QUALITY ALERT: {check_name} -- {details}")

def evaluate_results(results: list[dict]) -> bool:
    """Evaluate quality check results and alert on failures. Returns True if all
    passed."""
    failures = [r for r in results if r["status"] == "FAIL"]
    if failures:
        for f in failures:
            send_alert(f["check"], f)
        logger.error(f"{len(failures)} quality checks FAILED")
        return False
    logger.info("All quality checks passed")
    return True

```

The alerting layer is intentionally simple – a logging call that you can extend to Slack, PagerDuty, or email. The `evaluate_results` function returns a boolean so the Airflow DAG can fail the task if any check does not pass, preventing downstream consumers from using bad data.

Airflow DAG for Quality Checks

See companion code: `solution/dags/quality_checks.py`

```

"""DAG: Run data quality checks."""
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    dag_id="quality_checks",
    schedule="0 8 * * *",
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=["capstone", "quality"],
)
def quality_checks():

    @task()
    def run_checks():
        from src.quality.expectations import run_quality_checks
        from src.quality.alerts import evaluate_results
        results = run_quality_checks()
        all_passed = evaluate_results(results)
        if not all_passed:
            raise Exception("Quality checks failed -- see alerts")

    run_checks()

quality_checks()

```

Scheduled at 8 AM, one hour after the transformation DAG. If checks fail, the task raises an exception, which Airflow marks as a failure and triggers any configured alerting (email, Slack webhook, etc.).

AI Component – Product Embeddings

See companion code: `solution/src/ai/embeddings.py`

```

"""Product embedder – generates and stores product embeddings."""
import logging
import psycopg2
from sentence_transformers import SentenceTransformer
from ..common.config import Config

logger = logging.getLogger(__name__)

class ProductEmbedder:
    def __init__(self, db_url: str = None, model_name: str = "all-MiniLM-L6-v2"):
        self.conn = psycopg2.connect(db_url or Config.DATABASE_URL)
        self.model = SentenceTransformer(model_name)

    def embed_products(self):
        cur = self.conn.cursor()
        cur.execute("""
            SELECT product_id, product_name, category, description
            FROM raw.products
            WHERE product_id NOT IN (SELECT product_id FROM
analytics.product_embeddings)
        """)
        products = cur.fetchall()
        if not products:
            logger.info("No new products to embed")
            return

        texts = [f"{r[1]} - {r[2]} - {r[3] or ''}" for r in products]
        embeddings = self.model.encode(texts, show_progress_bar=True)

        for product, embedding in zip(products, embeddings):
            cur.execute("""
                INSERT INTO analytics.product_embeddings (product_id, product_name,
category, description, embedding)
                VALUES (%s, %s, %s, %s, %s)
                ON CONFLICT (product_id) DO UPDATE SET
                    embedding = EXCLUDED.embedding, updated_at = NOW()
            """, (product[0], product[1], product[2], product[3], embedding.tolist()))
            self.conn.commit()
            logger.info(f"Embedded {len(products)} products")

    def close(self):
        self.conn.close()

```

The embedder only processes products that do not already have embeddings (`NOT IN` subquery), making it incremental. The text fed to the model combines name, category, and description: `"Running Pro 42 - Sports - High-quality running product"`. This gives the embedding enough context for meaningful similarity.

Similarity Search API

See companion code: `solution/src/ai/vector_search.py`

```

"""Vector similarity search for products."""
import psycpg2
from sentence_transformers import SentenceTransformer
from ..common.config import Config

class ProductSearch:
    def __init__(self, db_url: str = None, model_name: str = "all-MiniLM-L6-v2"):
        self.conn = psycpg2.connect(db_url or Config.DATABASE_URL)
        self.model = SentenceTransformer(model_name)

    def search(self, query: str, limit: int = 5) -> list[dict]:
        embedding = self.model.encode(query).tolist()
        cur = self.conn.cursor()
        cur.execute("""
            SELECT product_id, product_name, category, description,
                   1 - (embedding <=> %s::vector) AS similarity
            FROM analytics.product_embeddings
            ORDER BY embedding <=> %s::vector
            LIMIT %s
            """, (embedding, embedding, limit))
        return [
            {"product_id": r[0], "product_name": r[1], "category": r[2],
             "description": r[3], "similarity": float(r[4])}
            for r in cur.fetchall()
        ]

    def similar_to(self, product_id: int, limit: int = 5) -> list[dict]:
        cur = self.conn.cursor()
        cur.execute("""
            SELECT p2.product_id, p2.product_name, p2.category,
                   1 - (p1.embedding <=> p2.embedding) AS similarity
            FROM analytics.product_embeddings p1, analytics.product_embeddings p2
            WHERE p1.product_id = %s AND p2.product_id != %s
            ORDER BY p1.embedding <=> p2.embedding LIMIT %s
            """, (product_id, product_id, limit))
        return [{"product_id": r[0], "product_name": r[1], "category": r[2], "similarity": float(r[3])}
                for r in cur.fetchall()]

    def close(self):
        self.conn.close()

```

Two search modes: `search()` takes a natural language query (“noise cancelling headphones”), embeds it, and finds the closest products. `similar_to()` takes a product ID and finds its nearest neighbors. The `<=>` operator is pgvector’s cosine distance – the HNSW index makes this fast.

See companion code: [solution/src/ai/api.py](https://github.com/DataEngineering/solution/src/ai/api.py)


```

"""FastAPI application for product search."""
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="ShopFast Product Search", version="1.0")

_search = None

def get_search():
    global _search
    if _search is None:
        from .vector_search import ProductSearch
        _search = ProductSearch()
    return _search

class SearchRequest(BaseModel):
    query: str
    limit: int = 5

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/search")
def search_products(req: SearchRequest):
    return {"results": get_search().search(req.query, limit=req.limit)}

@app.get("/similar/{product_id}")
def similar_products(product_id: int, limit: int = 5):
    results = get_search().similar_to(product_id, limit=limit)
    if not results:
        raise HTTPException(404, "Product not found or no embeddings available")
    return {"results": results}

```

Try the API:

```

# Similar products
curl http://localhost:8000/similar/42?top_k=3

# Semantic search
curl "http://localhost:8000/search?q=noise+cancelling+headphones&top_k=5"

# Health check
curl http://localhost:8000/health

```

Airflow DAG for Embeddings

See companion code: [solution/dags/embeddings_pipeline.py](#)

```
"""DAG: Generate product embeddings."""
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    dag_id="embeddings_pipeline",
    schedule="0 9 * * *",
    start_date=datetime(2025, 1, 1),
    catchup=False,
    tags=["capstone", "ai"],
)
def embeddings_pipeline():

    @task()
    def generate_embeddings():
        from src.ai.embeddings import ProductEmbedder
        embedder = ProductEmbedder()
        embedder.embed_products()
        embedder.close()

    generate_embeddings()

embeddings_pipeline()
```

Scheduled at 9 AM – after quality checks pass at 8 AM. This creates a natural pipeline: ingest (daily) -> transform (7 AM) -> quality (8 AM) -> embeddings (9 AM). Each stage depends on the previous one completing successfully.

Phase 4 Checkpoint: Quality checks run and pass. Product embeddings stored in pgvector. Similarity API returns results.

Phase 5: Testing, CI/CD, and Documentation

TL;DR: Write unit tests, set up GitHub Actions CI, create a comprehensive README, and draw an architecture diagram. This is what turns a project into a portfolio piece.

Tests

See companion code: [solution/tests/test_ingestion.py](#)

```

"""Tests for ingestion components."""
import pytest
import json
import tempfile
import csv

def test_generate_seed_data():
    """Verify seed data generator produces expected output."""
    import random
    random.seed(42)

    categories = {"Electronics": ["Phones"], "Clothing": ["Shirts"]}
    products = []
    pid = 1
    for cat, subcats in categories.items():
        for sub in subcats:
            for i in range(5):
                products.append({"product_id": pid, "product_name": f"{sub} {pid}", "category": cat})
                pid += 1
    assert len(products) == 10
    assert all("product_id" in p for p in products)

def test_file_loader_customers_format():
    """Verify CSV format is correct for customer loading."""
    with tempfile.NamedTemporaryFile(mode="w", suffix=".csv", delete=False) as f:
        writer = csv.DictWriter(f, fieldnames=["customer_id", "name", "email", "city", "state", "segment", "signup_date"])
        writer.writeheader()
        writer.writerow({
            "customer_id": 1, "name": "Test User", "email": "test@test.com",
            "city": "NYC", "state": "NY", "segment": "Consumer", "signup_date": "2025-01-01",
        })
        path = f.name

    with open(path) as f:
        reader = csv.DictReader(f)
        rows = list(reader)
    assert len(rows) == 1
    assert rows[0]["customer_id"] == "1"

```

See companion code: [solution/tests/test_transformation.py](https://github.com/DataCamp/data-engineering-fundamentals/blob/main/solution/tests/test_transformation.py)

```

"""Tests for transformation logic."""
import pytest

def test_star_schema_builder_instantiation():
    """Verify StarSchemaBuilder can be imported."""
    from src.transformation.star_schema import StarSchemaBuilder
    assert StarSchemaBuilder is not None

def test_scd_type_2_logic():
    """Test SCD Type 2 logic conceptually."""
    existing = {"customer_id": 1, "city": "New York", "is_current": True}
    incoming = {"customer_id": 1, "city": "Boston"}

    changed = existing["city"] != incoming["city"]
    assert changed is True

    if changed:
        existing["is_current"] = False
        new_row = {**incoming, "is_current": True}
        assert new_row["city"] == "Boston"
        assert new_row["is_current"] is True
        assert existing["is_current"] is False

```

See companion code: solution/tests/test_quality.py

```

"""Tests for quality check logic."""
import pytest
from src.quality.alerts import evaluate_results

def test_evaluate_all_pass():
    results = [
        {"check": "test1", "value": 0, "status": "PASS"},
        {"check": "test2", "value": 100, "status": "PASS"},
    ]
    assert evaluate_results(results) is True

def test_evaluate_with_failure():
    results = [
        {"check": "test1", "value": 0, "status": "PASS"},
        {"check": "test2", "value": 5, "status": "FAIL"},
    ]
    assert evaluate_results(results) is False

def test_evaluate_empty():
    assert evaluate_results([]) is True

```

The testing strategy covers three layers: ingestion (data format validation), transformation (SCD logic), and quality (alert evaluation). These tests run without a database connection, making them fast and suitable for CI.

At your job: Teams with good test coverage catch bugs before they reach production. Even simple tests like “can I import this module?” and “does SCD logic correctly expire old rows?” prevent the most common failures.

GitHub Actions CI

See companion code: `solution/.github/workflows/ci.yml`

```
name: Capstone CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env:
          POSTGRES_USER: dataeng
          POSTGRES_PASSWORD: dataeng
          POSTGRES_DB: warehouse
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -e ".[dev]"
      - run: pytest tests/ -v --tb=short
```

The CI pipeline spins up a real Postgres instance with pgvector. This means your tests run against the same database engine as production – no SQLite mocking that hides Postgres-specific issues. The `pip install -e ".[dev]"` installs both the main dependencies and the dev dependencies (pytest, ruff, pytest-cov) from `pyproject.toml`.

README Template

Your README should look something like this (from the course PDF):

```
# Real-Time E-Commerce Analytics Platform

A production-grade data platform built as the capstone project for
**The Data Engineering Fast Track** course.

## Architecture

[Include your architecture diagram here]

**Data Sources:** REST API (products), Kafka stream (user events), CSV (inventory)
**Storage:** S3/MinIO data lake + Postgres analytics warehouse
**Processing:** Python + SQL transformations, Airflow orchestration
**Data Model:** Star schema (fact_orders, fact_events, dim_products, dim_customers,
dim_time)
**Data Quality:** Great Expectations with automated alerting
**AI Component:** Product embeddings via pgvector for similarity search

## Quick Start

```bash
Clone and setup
git clone https://github.com/gyatesofficial/de-fast-track.git
cd de-fast-track/module-10/starter
cp .env.example .env

Generate seed data
python data/generate_seed_data.py

Start all services
docker-compose up -d

Wait 1-2 minutes, then:
Airflow UI: http://localhost:8080 (admin/admin)
MinIO UI: http://localhost:9001 (minioadmin/minioadmin)
Similarity API: http://localhost:8000/docs
```

## Tech Stack

COMPONENT	TECHNOLOGY
Orchestration	Apache Airflow 2.8
Warehouse	PostgreSQL 16 + pgvector
Data Lake	MinIO (S3-compatible)
Streaming	Apache Kafka
Data Quality	Great Expectations
AI/Embeddings	OpenAI API / sentence-transformers
API	FastAPI
Infrastructure	Docker Compose, Terraform
CI/CD	GitHub Actions
Language	Python 3.11

## Data Model

### Star Schema

**Fact Tables:** - `fact_orders` – completed purchases (grain: one row per order line) - `fact_events`  
– all user events (grain: one row per event)

**Dimension Tables:** - `dim_products` – product catalog (SCD Type 1) - `dim_customers` – customer profiles (SCD Type 2) - `dim_time` – date dimension (2024-2027)

## Running Tests

```
pytest tests/ -v
```

## License

MIT ""

## Architecture Diagram Guide

For your draw.io or Excalidraw diagram, include these components:

**Left column (Data Sources):** - REST API icon -> “Product Catalog API” - Kafka icon -> “User Event Stream” - File icon -> “CSV Inventory Upload”

**Center-top (Ingestion):** - Airflow icon -> three sub-boxes: “API Ingester”, “Kafka Consumer”, “File Loader”

**Center (Storage):** - S3 bucket -> “MinIO / S3 Data Lake” with layers: raw/ staging/ prod/ - Database -> “PostgreSQL + pgvector” with: Raw Schema, Analytics Star Schema, Vector Embeddings

**Center-bottom (Processing):** - “SQL Transformations” (CTEs, Window Functions, SCD Type 2) - “Great Expectations” (Schema, Completeness, Freshness, Volume) - “Embedding Pipeline” (OpenAI / sentence-transformers)

**Right column (Serving):** - “Analytics Queries” - “FastAPI” with /similar/{id} and /search endpoints - “Quality Alerts”

**Bottom:** - Docker Compose, Terraform, GitHub Actions icons

---

## Completion Checklist

---

### Core Requirements (Must Have)

- [ ] **Repository structure** – clean, organized, follows the template
- [ ] **Docker Compose** – full stack starts with `docker compose up -d`
- [ ] **Data ingestion** – at least 2 of 3 sources working (API, Kafka, CSV)
- [ ] **Star schema** – fact and dimension tables populated with correct relationships
- [ ] **Airflow DAGs** – at least 2 DAGs (ingestion + transformation)
- [ ] **Data quality** – at least 5 Great Expectations checks that run and pass
- [ ] **AI component** – product embeddings stored in pgvector
- [ ] **Similarity API** – FastAPI endpoint returns similar products
- [ ] **Tests** – at least 5 passing tests
- [ ] **README** – includes architecture, setup instructions, tech stack

### Above and Beyond (Bonus)

- [ ] All 3 data sources working
- [ ] SCD Type 2 implemented in customer dimension
- [ ] Volume anomaly detection in quality checks



- [ ] Slack/webhook alerting on quality failures
- [ ] Terraform configs for AWS deployment
- [ ] GitHub Actions CI pipeline
- [ ] Data dictionary / column-level documentation
- [ ] Architecture diagram (draw.io / Excalidraw)
- [ ] Semantic search endpoint (query text -> similar products)
- [ ] 5-minute video walkthrough

## Scoring Guide

SCORE	LEVEL	DESCRIPTION
10/10 Core	<b>Complete</b>	All core requirements met
8-9/10 Core	<b>Strong</b>	Minor gaps but demonstrates full understanding
6-7/10 Core	<b>Acceptable</b>	Key components work but some are incomplete
< 6/10 Core	<b>Needs Work</b>	Major components missing – revisit relevant modules
5+ Bonus	<b>Outstanding</b>	Portfolio-ready project, go get that job!

## Making It Portfolio-Ready

### What Hiring Managers Actually Look For

When a hiring manager opens your GitHub profile and clicks on this repo, they spend about 30 seconds deciding whether to look deeper. Here is what they check, in order:

1. **README quality.** Is there a clear description, architecture diagram, and “how to run” section? If the README is empty or says “TODO”, they close the tab.
2. **Repository structure.** Does it look like a real project? Separate directories for source code, tests, configs, and infrastructure? Or is everything dumped in the root?
3. **Docker Compose / infrastructure as code.** Can they actually run it? `docker compose up -d` that works is worth more than 10 pages of documentation.
4. **Tests.** Even a few tests signal professionalism. No tests signals “this person writes code and hopes it works.”
5. **Commit history.** Clean, descriptive commits show discipline. A single “initial commit” with 50 files suggests the project was built in one sitting and never iterated on.

**At your job:** These same criteria apply to code review. Senior engineers notice the same things: clean structure, meaningful tests, good documentation, and clear commit messages.

## How to Present the Project

**GitHub Profile Optimization:** - Pin this repository to your profile - Write a profile README that mentions “Real-Time E-Commerce Analytics Platform” with a link - Add relevant topics to the repo:

`data-engineering`, `airflow`, `kafka`, `postgres`, `python`, `docker`, `star-schema`, `pgvector`

### Resume Line Item:

Built end-to-end e-commerce data platform ingesting from 3 sources (REST API, Kafka, CSV), transforming to star schema with SCD Type 2 dimensions, automated quality checks, and AI-powered product similarity search via pgvector – orchestrated by Airflow, containerized with Docker Compose, tested with CI/CD.

**LinkedIn Post:** Share the architecture diagram with a post explaining one interesting technical decision (e.g., “Why I chose SCD Type 2 for customers” or “How pgvector makes product similarity search fast”). Technical posts with visuals get engagement from hiring managers.

---

## Extension Ideas

Once the core capstone is working, here are ways to expand it and deepen your skills:

### Add More Data Sources

- **Webhook ingestion:** Build a FastAPI endpoint that receives real-time order notifications and writes to Kafka
- **Web scraping:** Add a competitor price scraper that runs daily and feeds into a `raw.competitor_prices` table
- **Third-party APIs:** Integrate a weather API and correlate sales with weather patterns

### Scale with Kubernetes

- Convert `docker-compose.yml` to Kubernetes manifests or Helm charts
- Deploy Airflow on Kubernetes using the KubernetesExecutor
- Add horizontal pod autoscaling for the FastAPI service

## Add Monitoring and Observability

- Add Prometheus metrics to the FastAPI service (request latency, error rates)
- Set up Grafana dashboards for pipeline health
- Implement structured logging with JSON output to a centralized log system
- Add OpenTelemetry tracing across the pipeline stages

## Enhance the AI Layer

- Add a RAG (Retrieval-Augmented Generation) endpoint that answers product questions using embeddings + an LLM
- Implement user-based collaborative filtering: “customers who bought X also bought Y”
- Build a recommendation API that combines vector similarity with purchase history

## Production Hardening

- Add Terraform configs for deploying to AWS (RDS, MSK, ECS)
  - Implement blue-green deployments for the API layer
  - Add data retention policies and automated cleanup of old raw data
  - Build a data catalog with column-level documentation
- 

## Interview Prep: How to Talk About This Project

---

When an interviewer asks “Tell me about a data engineering project you have built,” this is what you walk them through. Here is how to structure your answer and what questions to expect.

### The 2-Minute Walkthrough

Follow this structure:

1. **Problem statement** (10 seconds): “I built an end-to-end analytics platform for an e-commerce company that needed to unify product, customer, and event data from three different sources.”
2. **Architecture overview** (30 seconds): “Data flows from a REST API, Kafka stream, and CSV files through ingestion pipelines into a Postgres raw layer. Airflow orchestrates daily transformations into a star schema with SCD Type 2 customer dimensions. Quality checks gate every load. Product embeddings stored in pgvector power a similarity search API.”
3. **Technical depth** (60 seconds): Pick ONE interesting challenge and go deep. For example: “The trickiest part was getting SCD Type 2 right for customers. When a customer moves cities, I expire the old dimension row and insert a new one, so historical orders still join to the address that was current when the order was placed.”

4. **Impact / results** (20 seconds): “The platform processes 10,000 events daily, maintains 7 automated quality checks, and serves product similarity search with sub-100ms latency.”

## Questions You Should Expect

**“Why did you choose Postgres over Snowflake/BigQuery?”** “For a portfolio project, Postgres gives me full control and zero cloud costs. The concepts transfer directly – star schemas, SCD, and quality checks work the same way. I chose pgvector specifically because it lets me show AI integration without a separate vector database.”

**“How would you scale this?”** “The Kafka consumer is the bottleneck – I would add consumer group parallelism. For the warehouse, I would move to a columnar engine like DuckDB or Snowflake. The embeddings pipeline would benefit from batch processing with GPU acceleration. Airflow would move to KubernetesExecutor for per-task scaling.”

**“What happens when the pipeline fails?”** “Every stage is idempotent – I can rerun safely because of ON CONFLICT clauses. Quality checks gate downstream processing, so bad data does not reach the analytics layer. Airflow retries failed tasks automatically. The quality.check\_results table provides an audit trail of every check.”

**“How do you handle schema changes?”** “The SCD Type 2 pattern naturally handles additive changes. For breaking changes, I would version the raw tables (raw\_v2.events) and run both versions in parallel during migration. The quality checks would catch any unexpected schema drift.”

**“Walk me through the data model.”** “It is a classic Kimball star schema. Two fact tables at different grains: fact\_orders for completed purchases (one row per line item) and fact\_events for all user interactions. Three dimensions: products (SCD Type 1), customers (SCD Type 2 tracking city/state/segment changes), and a pre-populated time dimension. The foreign keys from facts to dimensions use surrogate keys, not natural keys.”

---

## Deliverables Summary

When you are done, you should have:

1. **GitHub repository** – public, with clean commit history
2. **Working Docker Compose stack** – one command to run everything
3. **Architecture diagram** – visual overview of the system
4. **README.md** – comprehensive documentation
5. **(Optional) 5-minute video walkthrough** – record yourself explaining the project
6. **(Optional) Blog post** – write up your design decisions for LinkedIn

**This project IS your portfolio piece.** When an interviewer asks “tell me about a data engineering project you have built,” this is what you walk them through. Make it clean, make it work, make it yours.

You have got this. Go build something awesome.

---

*End of Module 10 – End of The Data Engineering Fast Track*

# Appendix A: Career Resources & Cost Optimization for Data Engineers

---

## Why This Appendix Exists

---

Two things determine your trajectory as a data engineer more than anything else: your ability to manage cloud costs (which makes you indispensable to any organization) and your ability to manage your own career (which determines whether you get paid what you're worth). This appendix covers both.

Cost optimization is not a “nice to have” skill. It is a career-defining capability. The data engineer who saves their company \$500K/year in Snowflake credits gets promoted. The one who lets costs balloon unchecked gets blamed when the CFO starts asking questions.

**Key Takeaway:** Cost optimization is a technical skill on par with data modeling or pipeline development. Treat it as a core competency, not an afterthought. It is also the fastest way to demonstrate measurable business impact on your resume.

---

## Part 1: Cloud Cost Optimization for Data Engineers

---

### The \$50K/Month Mistake (A True Story)

I have seen teams waste \$50K/month on Snowflake because they did not set auto-suspend. The warehouse was a Large, running 24/7, and the team used it for ad-hoc queries during business hours only. That is roughly 16 hours per day of idle compute burning credits. The fix took one line of SQL:

```
ALTER WAREHOUSE analytics_wh SET AUTO_SUSPEND = 60;
```

That single command saved \$35K/month. Nobody noticed for six months because the bill was split across a cost center with 40 other line items. This is the reality of cloud data engineering: money disappears silently unless someone is watching.

Here are the most common cost disasters I have seen across dozens of data teams:

SCENARIO	MONTHLY WASTE	ROOT CAUSE	FIX TIME
Snowflake warehouse never auto-suspends	\$20K-\$50K	Default config left unchanged	5 minutes
Full table scans on BigQuery (no partitioning)	\$10K-\$30K	No partition pruning in queries	1-2 days
Dev/staging Redshift clusters running 24/7	\$5K-\$15K	Nobody owns non-prod resources	1 hour
S3 data never lifecycle-managed	\$5K-\$20K	Raw data accumulates forever	2-3 hours
Spark jobs on on-demand instances	\$10K-\$40K	Nobody configured spot instances	1 day
Over-sized warehouses for simple queries	\$5K-\$15K	“Just use X-Large” culture	1-2 hours

## Snowflake Credit Management

Snowflake pricing revolves around credits. One credit costs \$2-\$4 depending on your edition and region. A warehouse consumes credits per second while running.

WAREHOUSE SIZE	CREDITS/HOUR	MONTHLY COST (24/7)	MONTHLY COST (8HR/DAY, AUTO-SUSPEND)
X-Small	1	~\$2,200	~\$730
Small	2	~\$4,400	~\$1,460
Medium	4	~\$8,800	~\$2,920
Large	8	~\$17,600	~\$5,840
X-Large	16	~\$35,200	~\$11,680
2X-Large	32	~\$70,400	~\$23,360

**The math is stark.** A Large warehouse running 24/7 costs \$17,600/month. The same warehouse with auto-suspend after 60 seconds, used 8 hours/day, costs roughly \$5,800. That is a \$140K annual savings from one configuration change.

## Snowflake-Specific Cost Controls

**Resource Monitors** are your first line of defense. Set them up on day one:

```

-- Create a resource monitor that alerts at 75% and suspends at 100%
CREATE RESOURCE MONITOR monthly_budget
 WITH CREDIT_QUOTA = 5000
 FREQUENCY = MONTHLY
 START_TIMESTAMP = IMMEDIATELY
 TRIGGERS
 ON 75 PERCENT DO NOTIFY
 ON 90 PERCENT DO NOTIFY
 ON 100 PERCENT DO SUSPEND;

-- Assign it to a warehouse
ALTER WAREHOUSE analytics_wh SET RESOURCE_MONITOR = monthly_budget;

```

**Separate warehouses by workload.** Do not run your dashboards, ETL, and ad-hoc queries on the same warehouse. Different workloads have different size and concurrency needs:

```

-- ETL warehouse: large but only runs during pipeline execution
CREATE WAREHOUSE etl_wh
 WAREHOUSE_SIZE = 'LARGE'
 AUTO_SUSPEND = 60
 AUTO_RESUME = TRUE
 INITIALLY_SUSPENDED = TRUE;

-- Dashboard warehouse: small, multi-cluster for concurrency
CREATE WAREHOUSE dashboard_wh
 WAREHOUSE_SIZE = 'SMALL'
 MIN_CLUSTER_COUNT = 1
 MAX_CLUSTER_COUNT = 3
 SCALING_POLICY = 'STANDARD'
 AUTO_SUSPEND = 120
 AUTO_RESUME = TRUE;

-- Ad-hoc warehouse: medium with strict cost limits
CREATE WAREHOUSE adhoc_wh
 WAREHOUSE_SIZE = 'MEDIUM'
 AUTO_SUSPEND = 60
 AUTO_RESUME = TRUE;

ALTER WAREHOUSE adhoc_wh SET RESOURCE_MONITOR = adhoc_budget;

```

**Query the ACCOUNT\_USAGE schema regularly** to find cost offenders:



```

-- Find the top 20 most expensive queries in the last 30 days
SELECT
 query_id,
 user_name,
 warehouse_name,
 execution_time / 1000 AS execution_seconds,
 credits_used_cloud_services,
 query_text
FROM snowflake.account_usage.query_history
WHERE start_time > DATEADD('day', -30, CURRENT_TIMESTAMP())
ORDER BY execution_time DESC
LIMIT 20;

-- Find warehouses with low utilization (wasted money)
SELECT
 warehouse_name,
 SUM(credits_used) AS total_credits,
 COUNT(DISTINCT query_id) AS total_queries,
 SUM(credits_used) / NULLIF(COUNT(DISTINCT query_id), 0) AS credits_per_query,
 AVG(DATEDIFF('second', start_time, end_time)) AS avg_query_seconds
FROM snowflake.account_usage.warehouse_metering_history wm
LEFT JOIN snowflake.account_usage.query_history qh
 ON wm.warehouse_name = qh.warehouse_name
 AND qh.start_time BETWEEN wm.start_time AND wm.end_time
WHERE wm.start_time > DATEADD('day', -30, CURRENT_TIMESTAMP())
GROUP BY warehouse_name
ORDER BY total_credits DESC;

```

**Snowflake cost optimization checklist** (Module 4 covers the warehouse concepts; this is the operational side):

1. Auto-suspend set to 60 seconds on all warehouses (unless you have a sustained workload)
2. Resource monitors on every warehouse with alerting
3. Separate warehouses for ETL, BI, ad-hoc, and data science
4. Multi-cluster warehouses for dashboard workloads (scale on concurrency, not size)
5. Weekly review of top-cost queries
6. Materialized views for repetitive dashboard queries (Module 2 covers the SQL patterns)
7. Clustering keys on large fact tables queried by date range
8. Time travel set to 1 day for non-critical tables (default is 1 day; enterprise default is 90 days at significant storage cost)

---

## S3 Storage Classes and Data Lake Cost Optimization

Your data lake on S3 (or GCS/Azure Blob) is often your largest storage cost. Module 5 covers data lake architecture; here is how to keep costs under control.

### S3 storage class pricing (us-east-1, approximate):

STORAGE CLASS	\$/GB/ MONTH	RETRIEVAL COST	USE CASE
S3 Standard	\$0.023	Free	Hot data, current month raw/staging
S3 Infrequent Access	\$0.0125	\$0.01/GB	Data 30-90 days old, occasional reprocessing
S3 Glacier Instant	\$0.004	\$0.03/GB	Data 90-365 days old, rare but fast access needed
S3 Glacier Flexible	\$0.0036	\$0.03/GB + minutes-hours	Compliance archives, rarely accessed
S3 Glacier Deep Archive	\$0.00099	\$0.02/GB + 12 hours	7-year regulatory retention

**Real-world example:** A team stores 50 TB of raw event data in S3 Standard. Monthly cost: \$1,150. After implementing lifecycle policies to move data older than 30 days to IA and older than 90 days to Glacier Instant:

- 10 TB in Standard (current month): \$230
- 20 TB in IA (1-3 months): \$250
- 20 TB in Glacier Instant (3+ months): \$80
- **New monthly cost: \$560** (51% savings)

```
S3 lifecycle policy for a data lake -- apply to your raw and staging prefixes
lifecycle_config = {
 "Rules": [
 {
 "ID": "raw-data-tiering",
 "Status": "Enabled",
 "Filter": {"Prefix": "raw/"},
 "Transitions": [
 {"Days": 30, "StorageClass": "STANDARD_IA"},
 {"Days": 90, "StorageClass": "GLACIER_INSTANT_RETRIEVAL"},
 {"Days": 365, "StorageClass": "DEEP_ARCHIVE"},
],
 "Expiration": {"Days": 2555}, # Delete after 7 years
 },
 {
 "ID": "staging-cleanup",
 "Status": "Enabled",
 "Filter": {"Prefix": "staging/"},
 "Expiration": {"Days": 14}, # Staging data is ephemeral
 },
 {
 "ID": "dev-cleanup",
 "Status": "Enabled",
 "Filter": {"Prefix": "dev/"},
 "Expiration": {"Days": 7}, # Dev data lives one week max
 },
],
}
```

**Format matters enormously.** Converting CSV to Parquet (covered in Module 5) can reduce storage by 75-90% and query costs by 70-90% on scan-based systems like BigQuery and Athena:

DATA FORMAT	1 TB STORAGE COST/ MONTH	BIGQUERY SCAN COST (FULL TABLE)	ATHENA SCAN COST (FULL TABLE)
CSV	\$23.00	\$5.00	\$5.00
JSON	\$23.00	\$5.00	\$5.00
Parquet	\$4.00-\$6.00	\$0.50-\$1.50	\$0.50-\$1.50

This is often the single highest-ROI optimization for a data lake. If your data is still in CSV or JSON, stop reading and go convert it to Parquet right now.

## Spot Instances for Spark and Batch Processing

Module 7 covers Spark fundamentals. Here is how to run Spark for 60-90% less money.

**Spot instances** (AWS) / **Preemptible VMs** (GCP) / **Spot VMs** (Azure) are unused cloud capacity sold at a steep discount with the caveat that they can be reclaimed with 2 minutes notice.

**The pattern that works:** On-demand for the driver node (must not be interrupted), spot for executor/worker nodes (can be replaced).

```
EMR cluster configuration with spot instances for Spark workers
emr_config = {
 "Name": "daily-etl-spark",
 "Instances": {
 "MasterInstanceGroup": {
 "InstanceType": "m5.xlarge",
 "InstanceCount": 1,
 "Market": "ON_DEMAND", # Driver must be reliable
 },
 "CoreInstanceGroup": {
 "InstanceType": "m5.2xlarge",
 "InstanceCount": 4,
 "Market": "SPOT", # Workers can be interrupted
 "BidPrice": "OnDemandPrice", # Pay up to on-demand price
 },
 "TaskInstanceGroups": [
 {
 "InstanceType": "m5.2xlarge",
 "InstanceCount": 8,
 "Market": "SPOT",
 "BidPrice": "OnDemandPrice",
 "AutoScalingPolicy": {
 "Rules": [{
 "Name": "ScaleOut",
 "Action": {"SimpleScalingPolicyConfiguration": {
 "AdjustmentType": "CHANGE_IN_CAPACITY",
 "ScalingAdjustment": 4,
 "CoolDown": 300,
 }},
 "Trigger": {"CloudWatchAlarmDefinition": {
 "MetricName": "YARNMemoryAvailablePercentage",
 "ComparisonOperator": "LESS_THAN",
 "Threshold": 20,
 "Period": 300,
 }},
 }],
 },
 },
],
 },
}
```

**Cost comparison for a daily Spark ETL job (4 hours/day, 12 workers):**

CONFIGURATION	MONTHLY COST	RISK
All on-demand (m5.2xlarge)	~\$4,800	None
Driver on-demand, workers spot	~\$1,600	Occasional worker loss (auto-replaced)
All spot (risky)	~\$1,200	Job can fail if driver is reclaimed

### Best practices for spot instances:

1. Use instance fleet with multiple instance types (if m5.2xlarge is unavailable, fall back to m5a.2xlarge, r5.2xlarge, etc.)
2. Enable graceful decommissioning in Spark so interrupted nodes finish their current tasks before shutdown
3. Write intermediate results to S3 (not HDFS) so work is not lost when a node dies
4. Use checkpointing for long-running Spark Streaming jobs
5. Spread across multiple availability zones for better spot capacity

## Right-Sizing Warehouses and Compute

**The most common mistake:** Teams default to a Large or X-Large warehouse/cluster and never revisit the decision. Right-sizing means matching compute to actual workload requirements.

### How to right-size a Snowflake warehouse:

1. Start with X-Small for any new workload
2. Run your actual queries and measure execution time
3. Scale up only if queries exceed your SLA (e.g., dashboard refresh must be under 10 seconds)
4. Double the size, re-test, and compare cost-per-query vs. time saved
5. Stop scaling when the marginal time improvement no longer justifies the 2x cost increase

```
-- Compare query performance across warehouse sizes
-- Run this for each size to find the sweet spot
ALTER WAREHOUSE test_wh SET WAREHOUSE_SIZE = 'XSMALL';
-- Run your benchmark queries, record times

ALTER WAREHOUSE test_wh SET WAREHOUSE_SIZE = 'SMALL';
-- Run same queries, compare

ALTER WAREHOUSE test_wh SET WAREHOUSE_SIZE = 'MEDIUM';
-- Run same queries, compare
```

**A typical finding:** For most analytical queries on tables under 100 GB, the difference between Medium and Large is minimal (both use enough compute to saturate the I/O). You pay 2x for a 10%

speedup. Save Large and X-Large for genuine heavy lifting: large joins, complex window functions across billions of rows, or high-concurrency dashboard serving.

#### **BigQuery cost optimization** (on-demand pricing):

- Partition tables by date. A query on a 10 TB table that scans only today's partition (10 GB) costs \$0.05 instead of \$50.
- Use `SELECT` only the columns you need. BigQuery is columnar; selecting `*` on a 50-column table scans 50x more data than selecting 2 columns.
- Set per-user and per-project query cost limits:

```
-- BigQuery: Set maximum bytes billed per query (prevents accidental $500 queries)
-- This goes in your BigQuery client configuration
-- 10 GB limit = max $0.05 per query
SET @@query_settings.maximum_bytes_billed = 10737418240; -- 10 GB in bytes
```

#### **Redshift right-sizing:**

- Use RA3 nodes (storage separated from compute) so you can scale each independently
- Monitor WLM queue wait times. If queries are queuing, you need more concurrency (more nodes or concurrency scaling), not bigger nodes.
- Use Reserved Instances for baseline capacity (1-year RI saves ~30%, 3-year saves ~60%) and on-demand or serverless for burst capacity

---

## **Cost Optimization by Data Pipeline Stage**

Every stage of your data pipeline (covered across Modules 1-10) has specific cost levers:

PIPELINE STAGE	MODULE	TOP COST DRIVER	OPTIMIZATION STRATEGY
Ingestion	Module 3	API call volume, data transfer	Batch micro-batches, compress in transit, use same-region storage
Raw Storage	Module 5	S3/GCS storage volume	Lifecycle policies, Parquet conversion, delete staging data
Transformation	Module 6	Compute time (Spark/dbt)	Incremental models, spot instances, right-size clusters
Warehouse	Module 4	Warehouse runtime/credits	Auto-suspend, separate workloads, materialized views
Serving/BI	Module 2	Query volume and scan size	Result caching, aggregation tables, partition pruning
Orchestration	Module 8	Always-on scheduler	Use managed services (Cloud Composer, MWAA) to avoid idle infra
Quality/Testing	Module 9	Compute for data tests	Run quality checks on samples for large tables, prioritize critical tests

## Capacity Planning for Data Engineers

**The conversation you need to have every quarter** (this connects to the stakeholder communication skills in Module 10):

1. **With Product:** “What new data sources or features are planned? Each new source adds roughly \$X/month in ingestion and storage.”
2. **With Analytics:** “Which dashboards are most critical? Can we reduce refresh frequency on low-priority dashboards from 5 minutes to 30 minutes?”
3. **With Finance:** “Our current run rate is \$X/month. Based on data growth trends, we will hit \$Y in 6 months. Here are three scenarios with cost projections.”
4. **With Engineering:** “The new real-time feature will require a Kafka cluster and Flink jobs. Estimated cost: \$Z/month. Here is the architecture and the alternative approaches I evaluated.”

**Growth modeling for data platform costs:**

SCENARIO	PROBABILITY	12-MONTH COST PROJECTION	TRIGGER
Stable growth (20% data volume increase)	50%	\$X * 1.2	Organic business growth
Moderate growth (2x data volume)	30%	\$X * 1.8	New product launch, new data sources
Rapid growth (5x data volume)	15%	\$X * 3.5	Major customer acquisition, M&A
Explosive (10x+)	5%	\$X * 7+	Viral moment, regulatory change requiring new data collection

The key insight: data platform costs do not scale linearly with data volume. Costs often scale sub-linearly if you architect well (columnar storage, partitioning, incremental processing) or super-linearly if you do not (full table scans, no lifecycle policies, over-provisioned compute).

## Part 2: Career Resources for Data Engineers

### The Data Engineering Job Market

Data engineering salaries in 2025-2026 range from roughly \$100K for entry-level positions to \$350K+ for staff/principal engineers at top-tier companies. The range is wide because the role varies enormously:

LEVEL	TYPICAL TOTAL COMP (US)	WHAT YOU ARE EXPECTED TO DO
Junior DE (0-2 years)	\$100K-\$150K	Build and maintain pipelines, write SQL/Python, learn the stack
Mid-Level DE (2-5 years)	\$140K-\$220K	Own pipeline domains, improve reliability, mentor juniors
Senior DE (5-8 years)	\$180K-\$280K	Design systems, lead projects, influence architecture decisions
Staff DE (8+ years)	\$250K-\$350K+	Define strategy, cross-team impact, build platforms
Principal DE (10+ years)	\$300K-\$400K+	Organization-wide technical leadership, industry influence



These numbers reflect total compensation (base + bonus + equity) at technology companies in major US markets. Adjust down 15-30% for non-tech companies and non-coastal locations. Adjust up for FAANG/top-tier companies.

---

## Resume Optimization for Data Engineers

Your resume is a technical document. Treat it like a data pipeline: clean inputs, clear transformations, measurable outputs.

**The formula for every bullet point:** [Action verb] + [what you built/did] + [technology stack] + [measurable impact]

**Bad bullet points:** - “Worked on data pipelines” - “Used Spark and Airflow” - “Responsible for data warehouse”

**Good bullet points:** - “Built an incremental ingestion pipeline (Airflow + dbt + Snowflake) that reduced daily ETL runtime from 4 hours to 22 minutes and cut Snowflake compute costs by 65%” - “Designed and implemented a real-time fraud detection pipeline using Kafka and Flink, processing 50K events/second with p99 latency under 100ms” - “Migrated 15 TB data warehouse from Redshift to Snowflake, reducing monthly costs from \$18K to \$7K while improving average query performance by 3x” - “Implemented data quality framework using Great Expectations across 200+ dbt models, reducing data incidents from 12/month to 1/month” - “Designed star schema data model for e-commerce analytics serving 40 analysts, enabling self-service reporting that eliminated 80% of ad-hoc data requests”

### What hiring managers look for in DE resumes:

1. **Scale numbers:** How much data? How many pipelines? How many users served?
2. **Cost impact:** Did you save money? How much? (This is the single most impressive metric you can include.)
3. **Reliability impact:** Did you reduce incidents? Improve uptime? Decrease time-to-resolution?
4. **Architecture decisions:** Did you choose technologies, design systems, evaluate trade-offs?
5. **Breadth of stack:** Do you know SQL, Python, a cloud platform, orchestration, and a warehouse?

### Map your resume to this course:

COURSE MODULE	RESUME-WORTHY SKILLS
Module 1: Foundations	Dimensional modeling, star schemas, data warehouse design
Module 2: SQL Mastery	Window functions, CTEs, performance optimization, complex analytical queries
Module 3: Python for DE	API clients, file processing, data validation, testing
Module 4: Warehousing	Snowflake, BigQuery, Redshift architecture and optimization
Module 5: Data Lakes	S3/GCS, Parquet, Iceberg, lakehouse architecture
Module 6: dbt & Transformation	dbt models, incremental processing, testing, documentation
Module 7: Spark	PySpark, distributed processing, batch and streaming
Module 8: Orchestration	Airflow, DAG design, dependency management, monitoring
Module 9: Data Quality	Great Expectations, data contracts, SLAs, anomaly detection
Module 10: Capstone	End-to-end system design, project management, stakeholder communication

## LinkedIn Optimization for Data Engineers

**Headline formula:** [Current Title] | [Key Specialty] | [Technologies]

Examples: - “Senior Data Engineer | Real-Time Pipelines & Analytics | Kafka, Spark, Snowflake, dbt”  
 - “Data Engineer | Building Reliable Data Platforms | AWS, Airflow, dbt, Python” - “Staff Data Engineer | Data Platform Architecture | Distributed Systems, Lakehouse, ML Infrastructure”

**About section structure:** 1. One sentence about what you do and the scale you operate at 2. Two to three sentences about your key technical strengths 3. One sentence about what you are looking for (if actively searching) 4. A short list of core technologies

**Post content that gets noticed:** - “Here is how we reduced our Snowflake bill by 60%” (cost optimization stories) - “Lessons learned from migrating 50 TB to Iceberg” (migration war stories) - “The data quality incident that cost us \$2M” (failure stories with lessons) - “How I designed our real-time analytics pipeline” (architecture deep dives)

Do not post generic motivational content. Data engineering hiring managers respond to specificity and technical substance.

## Salary Negotiation for Data Engineers

**Know your market value.** Use levels.fyi, Glassdoor, and Blind to research compensation for your level at your target companies. The range for a Senior DE at a large tech company is typically \$200K-\$300K total comp. At FAANG, it can be \$250K-\$380K.

### Negotiation principles specific to DE:

1. **Quantify your impact in dollar terms.** “In my current role, I reduced data platform costs by \$400K/year and built the pipeline that supports \$10M in annual analytics-driven revenue.” This anchors the conversation around your value, not your current salary.
2. **Competing offers are your strongest lever.** Apply broadly. Even if you prefer Company A, an offer from Company B gives you negotiating power. The typical increase from negotiation is 10-20% of the initial offer.
3. **Negotiate the whole package, not just base salary.** For data engineers, the equity component can be 30-50% of total comp at public tech companies. A \$10K increase in base may matter less than an additional \$50K in RSUs.
4. **Use the “I am excited about this role” framing.**
5. “I am very excited about this opportunity. Based on my research and my experience reducing data platform costs by \$400K/year, I was hoping we could discuss a total compensation closer to \$X.”
6. Never say “I need more money” or “My current salary is X.” Focus on your value and market data.
7. **Specific to DE roles, emphasize rare skills:**
  8. Real-time streaming (Kafka/Flink) commands a premium over batch-only experience
  9. Cloud cost optimization experience is increasingly valued
  10. Platform/infrastructure DE roles typically pay more than pipeline/ETL roles
  11. Experience at scale (petabyte-level, thousands of pipelines) is a significant differentiator

**The salary conversation arc** (discussed in the context of the course’s career outcomes):

CAREER STAGE	NEGOTIATION FOCUS	TYPICAL LEVERAGE
First DE role (\$100K-\$130K)	Get in the door; base salary matters most	Limited; focus on learning opportunity
Mid-level (\$140K-\$200K)	Base + bonus; start caring about equity if at a tech company	Moderate; 1-2 competing offers help
Senior (\$180K-\$280K)	Total comp including equity; sign-on bonus for first year	Strong; multiple offers, demonstrated impact
Staff+ (\$250K-\$350K+)	Equity is the dominant component; level matters more than dollars	Very strong; companies compete for staff-level talent

## Building Your Portfolio (Connects to Module 10 Capstone)

Your Module 10 capstone project is the foundation of your portfolio. Here is how to maximize its career impact:

1. **Deploy it publicly.** A GitHub repo with a README is good. A live dashboard or API endpoint is great. A write-up explaining your architecture decisions is exceptional.
2. **Include cost analysis.** “This pipeline runs on AWS for approximately \$45/month in production. Here is the cost breakdown by service.” This demonstrates the cost awareness covered in Part 1 of this appendix.
3. **Show monitoring and observability.** Include screenshots of your Airflow DAGs, your data quality dashboards, your alerting setup. This proves you think about operations, not just code.
4. **Write a design document.** A 2-3 page document covering: problem statement, architecture decisions with trade-offs, data model, what breaks at 10x scale, and what you would do differently. This is what staff engineers produce. Producing one as a mid-level candidate is a strong signal.
5. **Map it to the course.** When discussing your capstone in interviews, reference the specific modules: “I used the dimensional modeling approach from our data modeling module, the incremental dbt patterns from our transformation module, and the data quality framework from our quality engineering module.”

## Continuous Learning Path

After completing this course (Modules 0-10), here is a prioritized learning path based on where the data engineering market is heading:

**Immediate (next 3 months):** - Get certified: Snowflake SnowPro Core, AWS Data Engineer Associate, or dbt Analytics Engineering - Contribute to an open-source data tool (dbt packages, Airflow providers, Great Expectations plugins) - Build one more portfolio project that demonstrates streaming (Kafka + Flink/Spark Streaming)

**Medium-term (3-12 months):** - Learn a lakehouse format deeply: Apache Iceberg (most momentum) or Delta Lake - Study distributed systems fundamentals (Designing Data-Intensive Applications by Martin Kleppmann is the essential text) - Practice system design interviews weekly (see Appendix C: Interview Prep)

**Long-term (1-2 years):** - Develop expertise in a specialization: ML platform engineering, real-time systems, or data platform architecture - Present at a local meetup or conference (even a 5-minute lightning talk builds credibility) - Mentor junior data engineers (teaching is the fastest way to deepen your own understanding)

**Key Takeaway:** Your career as a data engineer is a pipeline too. The inputs are your skills and experience. The transformations are deliberate practice, learning, and career management. The outputs are the roles, compensation, and impact you achieve. Optimize it with the same rigor you apply to your data systems.

---

## Quick Reference: Cost Optimization Checklist

---

Use this checklist quarterly to audit your data platform costs. Each item maps to concepts covered in the course modules.

- ☐ **Snowflake:** Auto-suspend enabled on all warehouses (60-120 seconds)
- ☐ **Snowflake:** Resource monitors set with alerting thresholds
- ☐ **Snowflake:** Separate warehouses for ETL, BI, ad-hoc workloads
- ☐ **Snowflake:** Top 20 expensive queries reviewed and optimized
- ☐ **BigQuery:** Tables partitioned by date; queries include partition filters
- ☐ **BigQuery:** Per-user query cost limits configured
- ☐ **S3/GCS:** Lifecycle policies on all buckets (raw, staging, dev)
- ☐ **S3/GCS:** Staging and dev data auto-deleted after 7-14 days
- ☐ **Storage:** All analytical data in Parquet or Iceberg format (not CSV/JSON)
- ☐ **Spark:** Worker nodes on spot/preemptible instances
- ☐ **Spark:** Cluster auto-scaling configured with appropriate min/max
- ☐ **dbt:** Incremental models for large tables (not full refresh)
- ☐ **Airflow:** Pipelines staggered to avoid compute spikes

- [ ] **General:** Monthly cost review meeting with trend analysis
  - [ ] **General:** Per-team cost attribution and accountability
- 

## What's Next

---

Appendix B (Case Studies) shows how real-world data teams apply these cost and architecture principles at scale. Appendix C (Interview Prep) helps you articulate your cost optimization experience and technical skills in interviews.

# Appendix B: Real-World Case Studies for Data Engineers

---

## Why Case Studies Matter

---

Every module in this course teaches a concept in isolation: data modeling in Module 1, SQL in Module 2, orchestration in Module 8. But real data engineering problems do not arrive in neatly labeled modules. They arrive as messy business requirements that demand you combine multiple skills simultaneously.

These four case studies show how the concepts from Modules 0-10 come together in production systems. Each case study is structured the same way: the business context, the data architecture, the specific technical decisions (mapped to course modules), the startup vs. enterprise approach, and the lessons learned.

These are not hypothetical. They are composites based on real systems I have seen at real companies, simplified for clarity but faithful to the actual patterns and trade-offs.

**Key Takeaway:** Read these case studies with your capstone project in mind. The e-commerce case study directly mirrors your Module 10 capstone. The others show how the same foundational skills apply in different domains.

---

## Case Study 1: E-Commerce Data Platform

---

### This Is Your Capstone at Production Scale

If you completed the Module 10 capstone project (building an e-commerce analytics pipeline), this case study shows what that system looks like when a real company runs it in production with 10 million customers, 50 data team members, and \$2M/year in data platform costs.

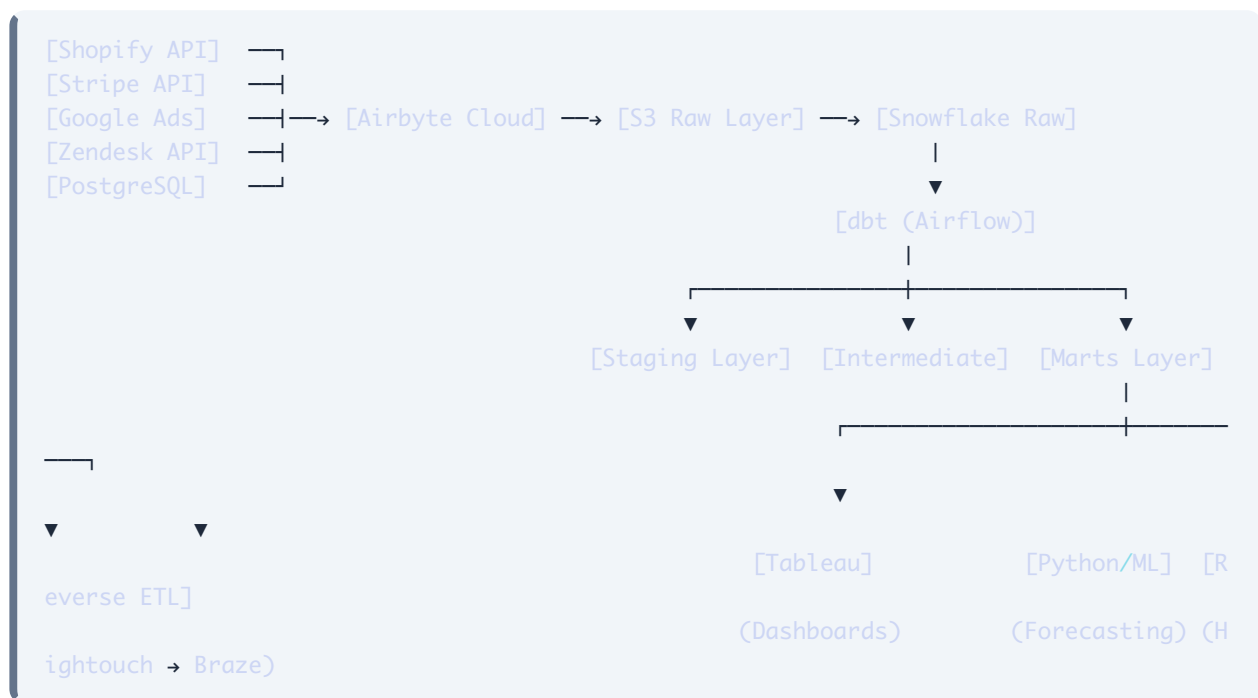
### Business Context

**Company:** Mid-market e-commerce retailer, \$500M annual revenue, 10M customers, 200K orders/day.

**Data team:** 8 data engineers, 15 analysts, 5 data scientists, 3 analytics engineers, and a head of data.

**Business requirements:** - Marketing needs customer segmentation and attribution reporting (which campaigns drive purchases?) - Operations needs inventory forecasting and supply chain analytics - Finance needs daily revenue reporting with 99.9% accuracy (this feeds SEC filings) - Product needs A/B test analysis for website experiments - Executives need a daily KPI dashboard available by 7 AM ET

## The Architecture



## Module-by-Module Breakdown

**Module 1 (Data Modeling):** The data model is a classic star schema with these core fact tables:



```

-- fact_orders: grain is one row per order line item
-- This is the most-queried table in the warehouse
CREATE TABLE marts.fact_orders (
 order_key BIGINT, -- Surrogate key
 order_id VARCHAR, -- Business key from Shopify
 order_line_id VARCHAR,
 order_date_key INT, -- FK to dim_date
 customer_key BIGINT, -- FK to dim_customer (SCD Type 2)
 product_key BIGINT, -- FK to dim_product
 channel_key INT, -- FK to dim_channel
 promotion_key INT, -- FK to dim_promotion
 quantity INT,
 unit_price DECIMAL(10,2),
 discount_amount DECIMAL(10,2),
 tax_amount DECIMAL(10,2),
 shipping_amount DECIMAL(10,2),
 gross_revenue DECIMAL(10,2), -- quantity * unit_price
 net_revenue DECIMAL(10,2), -- gross - discount
 cost_of_goods DECIMAL(10,2),
 gross_margin DECIMAL(10,2), -- net_revenue - cogs
 is_first_order BOOLEAN, -- Derived: is this the customer's first order?
 is_returned BOOLEAN,
 _loaded_at TIMESTAMP -- Pipeline metadata
);

-- dim_customer: SCD Type 2 tracks address changes (for regional analytics)
-- and loyalty tier changes (for retention analysis)
CREATE TABLE marts.dim_customer (
 customer_key BIGINT, -- Surrogate key
 customer_id VARCHAR, -- Business key
 email_hash VARCHAR, -- Hashed for privacy
 first_order_date DATE,
 customer_segment VARCHAR, -- 'new', 'active', 'at_risk', 'churned'
 lifetime_orders INT,
 lifetime_revenue DECIMAL(12,2),
 loyalty_tier VARCHAR, -- 'bronze', 'silver', 'gold', 'platinum'
 state VARCHAR,
 country VARCHAR,
 is_current BOOLEAN, -- SCD Type 2
 valid_from TIMESTAMP,
 valid_to TIMESTAMP
);

```

The customer segmentation uses a behavioral model: customers with no orders in 90 days are “at risk,” and 180 days means “churned.” This is recalculated daily by a dbt model.

**Module 2 (SQL):** The most complex queries in this system use window functions heavily. Here is the actual query that calculates customer cohort retention, which is the most requested report from marketing:

```

-- Customer cohort retention analysis
-- This query shows, for each monthly cohort, what % of customers
-- return to purchase in subsequent months
WITH customer_cohorts AS (
 SELECT
 customer_id,
 DATE_TRUNC('month', first_order_date) AS cohort_month
 FROM marts.dim_customer
 WHERE is_current = TRUE
),
monthly_activity AS (
 SELECT DISTINCT
 o.customer_id,
 DATE_TRUNC('month', o.order_date) AS activity_month
 FROM marts.fact_orders o
),
cohort_retention AS (
 SELECT
 c.cohort_month,
 DATEDIFF('month', c.cohort_month, a.activity_month) AS months_since_first,
 COUNT(DISTINCT a.customer_id) AS active_customers
 FROM customer_cohorts c
 INNER JOIN monthly_activity a ON c.customer_id = a.customer_id
 WHERE a.activity_month >= c.cohort_month
 GROUP BY 1, 2
)
SELECT
 cohort_month,
 months_since_first,
 active_customers,
 FIRST_VALUE(active_customers) OVER (
 PARTITION BY cohort_month ORDER BY months_since_first
) AS cohort_size,
 ROUND(100.0 * active_customers / FIRST_VALUE(active_customers) OVER (
 PARTITION BY cohort_month ORDER BY months_since_first
), 1) AS retention_pct
FROM cohort_retention
ORDER BY cohort_month, months_since_first;

```

**Module 3 (Python):** Python is used for custom Airbyte connectors where the off-the-shelf ones do not work (a common reality). The Shopify API connector needed custom logic for handling API rate limits and paginating through 200K daily orders.

**Module 4 (Warehousing):** Snowflake is the warehouse. Three separate warehouses handle different workloads: - **ETL\_WH** (Large, auto-suspend 60s): Runs dbt models overnight - **BI\_WH** (Small, multi-cluster 1-4, auto-suspend 120s): Serves Tableau dashboards - **ADHOC\_WH** (Medium, auto-suspend 60s, resource monitor at 500 credits/month): Analyst queries

**Module 5 (Data Lake):** S3 serves as the raw layer. All Airbyte output lands in S3 as Parquet files partitioned by date before being loaded into Snowflake raw tables. This means if Snowflake has an issue, the raw data is still safe in S3.

**Module 6 (dbt):** The dbt project has 180+ models organized into staging, intermediate, and marts layers. Incremental models handle the high-volume tables (fact\_orders, fact\_page\_views). The most critical dbt configuration:

```
dbt_project.yml (partial)
models:
 ecommerce:
 staging:
 +materialized: view # Staging is always views (no storage cost)
 intermediate:
 +materialized: table # Intermediate materializes for performance
 marts:
 +materialized: incremental # Marts are incremental where possible
 +on_schema_change: append_new_columns
```

**Module 7 (Spark):** Not used in the primary pipeline (Snowflake handles the transformation volume). Spark is used by the data science team for large-scale feature engineering and model training on customer behavior data, running on EMR with spot instances.

**Module 8 (Orchestration):** Airflow manages the entire pipeline with a carefully designed DAG dependency chain:

```
extract_shopify → load_raw_orders → dbt_staging → dbt_intermediate → dbt_marts →
dbt_test → notify_slack
extract_stripe
↗
extract_ga ↗
rt_on_failure ale
```

The pipeline runs at 2 AM ET and must complete by 6:30 AM so dashboards are fresh by 7 AM. SLA enforcement: if the DAG has not completed by 6 AM, PagerDuty alerts the on-call data engineer.

**Module 9 (Data Quality):** Every dbt model has tests. The most critical tests on fact\_orders:

```

models:
 - name: fact_orders
 tests:
 - dbt_utils.recently:
 datepart: hour
 field: _loaded_at
 interval: 6 # Fail if no data in last 6 hours
 columns:
 - name: net_revenue
 tests:
 - not_null
 - dbt_utils.accepted_range:
 min_value: -1000 # Allow reasonable refunds
 max_value: 50000 # Flag outliers
 - name: order_id
 tests:
 - unique
 - not_null

```

Additionally, a daily reconciliation query compares total revenue in fact\_orders against the Shopify API to ensure the pipeline has not dropped or duplicated orders. This check is non-negotiable because finance uses this data for SEC reporting.

**Module 10 (Capstone):** This entire system is what your capstone project aspires to become. The key difference between your capstone and this production system: operational maturity. This system has monitoring, alerting, on-call rotation, runbooks for common failures, cost controls, and a data catalog.

## Startup vs. Enterprise Approach

ASPECT	STARTUP (PRE-SERIES B)	ENTERPRISE (THIS CASE STUDY)
Team size	1-2 data engineers	8 DEs + 15 analysts + 5 DS
Warehouse	BigQuery (serverless, no admin)	Snowflake (more control, better for large teams)
Ingestion	Fivetran or manual scripts	Airbyte Cloud + custom connectors
Transformation	dbt Cloud (managed)	dbt Core on Airflow (self-managed for flexibility)
Data model	Single fact table, 3-4 dimensions	15+ fact tables, 30+ dimensions, SCD Type 2
Quality	dbt tests only	dbt tests + reconciliation + anomaly detection
Cost	\$500-\$2K/month	\$100K-\$200K/month
Time to build	2-4 weeks for MVP	6-12 months for full platform

**The startup version of this system** would be: Shopify data → Fivetran → BigQuery → dbt Cloud → Looker. One data engineer, \$1K/month, built in two weeks. It lacks the sophistication but delivers 80% of the value.

## Lessons Learned

1. **Data reconciliation against source systems is non-negotiable.** When your data feeds financial reporting, you cannot tolerate discrepancies. Build automated reconciliation from day one.
  2. **SCD Type 2 is worth the complexity for customer data.** Marketing needs to know that a customer was in the “gold” tier when they made a purchase, not just their current tier.
  3. **Separate your BI warehouse from your ETL warehouse.** The most common production incident was Tableau dashboards timing out because a large ad-hoc query was consuming all the compute.
  4. **Invest in incremental models early.** The team started with full-refresh dbt models. At 200K orders/day, the full refresh took 3 hours. Switching to incremental reduced it to 20 minutes.
- 

## Case Study 2: Fintech Real-Time Fraud Detection Pipeline

---

### When Latency Is Measured in Dollars

In e-commerce, a slow dashboard is an inconvenience. In fraud detection, a slow pipeline means money stolen from customers. This case study shows how a fintech company built a real-time fraud detection system where the data pipeline IS the product.

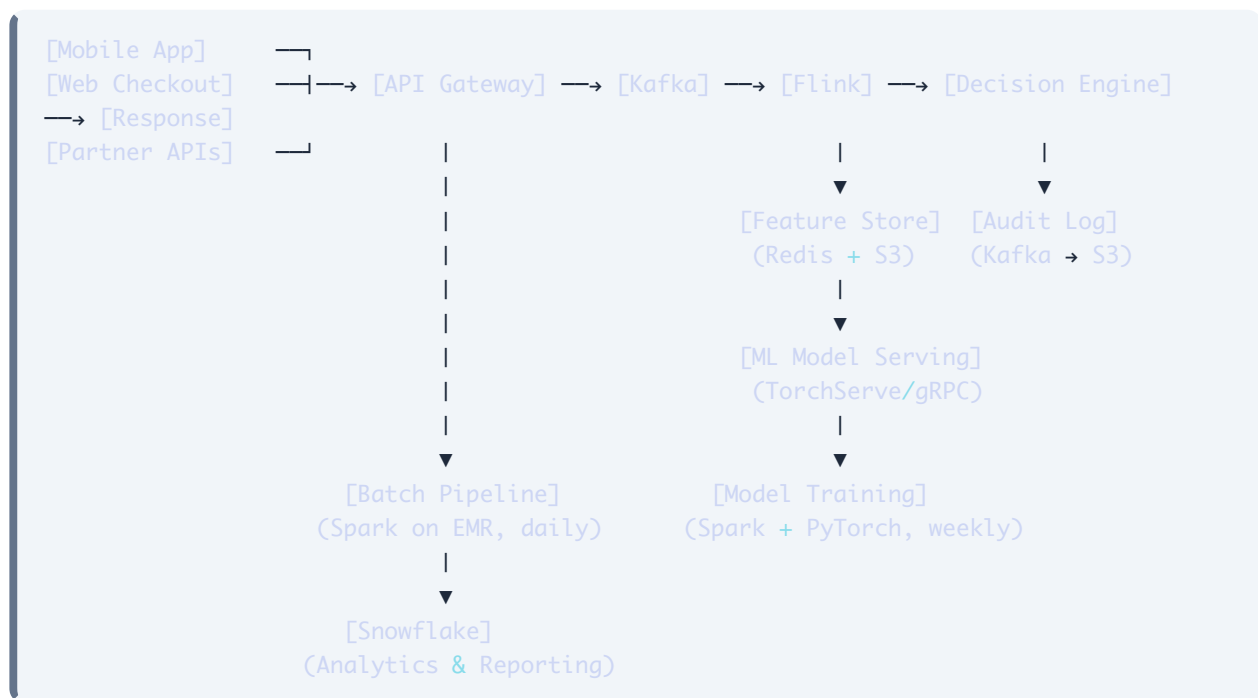
### Business Context

**Company:** Digital payments processor, handling 15K transactions/second at peak, \$80B annual transaction volume.

**Data team:** 6 data engineers, 4 ML engineers, 3 platform engineers.

**Requirements:** - Every transaction must receive a fraud/not-fraud decision in under 100ms - False positive rate must stay below 1.5% (blocking legitimate transactions costs revenue and customer trust) - Must detect new fraud patterns within hours, not days - Full audit trail required for regulatory compliance (PCI DSS, SOX) - System must handle 3x traffic spikes during holidays (Black Friday, Cyber Monday)

## The Architecture



## Module-by-Module Breakdown

**Module 1 (Data Modeling):** The transaction data model is unusual because it must serve both real-time scoring (denormalized for speed) and batch analytics (star schema for flexibility).

The real-time model in Redis is a flat, denormalized structure optimized for single-key lookups:

```
{
 "user_id": "u_12345",
 "features": {
 "txn_count_1h": 7,
 "txn_count_24h": 23,
 "txn_amount_1h": 459.32,
 "txn_amount_24h": 1847.50,
 "unique_merchants_1h": 4,
 "unique_merchants_24h": 12,
 "max_txn_amount_24h": 299.99,
 "avg_txn_amount_30d": 67.42,
 "is_new_device": false,
 "is_new_merchant": true,
 "distance_from_last_txn_km": 3.2,
 "time_since_last_txn_seconds": 847
 },
 "updated_at": "2026-04-03T14:22:07Z"
}
```

The batch analytics model in Snowflake is a star schema:

```

-- fact_transactions: one row per transaction, used for analytics
CREATE TABLE marts.fact_transactions (
 transaction_key BIGINT,
 transaction_id VARCHAR,
 transaction_ts TIMESTAMP,
 user_key BIGINT, -- FK to dim_user
 merchant_key BIGINT, -- FK to dim_merchant
 device_key BIGINT, -- FK to dim_device
 amount DECIMAL(12,2),
 currency VARCHAR(3),
 fraud_score FLOAT, -- ML model output (0-1)
 fraud_decision VARCHAR, -- 'approve', 'decline', 'review'
 is_fraud_confirmed BOOLEAN, -- Ground truth (labeled later)
 rule_triggers ARRAY, -- Which rules fired
 model_version VARCHAR, -- Which ML model version scored this
 latency_ms INT, -- End-to-end scoring latency
 _loaded_at TIMESTAMP
);

```

**Module 2 (SQL):** The analytics team runs complex queries to evaluate model performance and identify fraud patterns. Here is the query that calculates the precision/recall curve by score threshold, which the ML team uses to tune the decision boundary:

```

-- Model performance by score threshold
-- This helps the ML team decide: at what fraud_score should we decline?
WITH thresholds AS (
 SELECT 0.1 AS threshold UNION ALL SELECT 0.2 UNION ALL SELECT 0.3
 UNION ALL SELECT 0.4 UNION ALL SELECT 0.5 UNION ALL SELECT 0.6
 UNION ALL SELECT 0.7 UNION ALL SELECT 0.8 UNION ALL SELECT 0.9
),
scored AS (
 SELECT
 t.threshold,
 f.is_fraud_confirmed,
 CASE WHEN f.fraud_score >= t.threshold THEN 1 ELSE 0 END AS predicted_fraud
 FROM marts.fact_transactions f
 CROSS JOIN thresholds t
 WHERE f.transaction_ts > DATEADD('day', -30, CURRENT_TIMESTAMP())
 AND f.is_fraud_confirmed IS NOT NULL -- Only labeled transactions
)
SELECT
 threshold,
 SUM(CASE WHEN predicted_fraud = 1 AND is_fraud_confirmed THEN 1 ELSE 0 END) AS true_positives,
 SUM(CASE WHEN predicted_fraud = 1 AND NOT is_fraud_confirmed THEN 1 ELSE 0 END) AS false_positives,
 SUM(CASE WHEN predicted_fraud = 0 AND is_fraud_confirmed THEN 1 ELSE 0 END) AS false_negatives,
 -- Precision: of all we flagged, how many were actually fraud?
 ROUND(100.0 * SUM(CASE WHEN predicted_fraud = 1 AND is_fraud_confirmed THEN 1 ELSE 0 END)
 / NULLIF(SUM(predicted_fraud), 0), 2) AS precision_pct,
 -- Recall: of all actual fraud, how many did we catch?
 ROUND(100.0 * SUM(CASE WHEN predicted_fraud = 1 AND is_fraud_confirmed THEN 1 ELSE 0 END)
 / NULLIF(SUM(is_fraud_confirmed::INT), 0), 2) AS recall_pct
FROM scored
GROUP BY threshold
ORDER BY threshold;

```

**Module 3 (Python):** The Flink jobs are written in Java for performance, but the feature engineering pipeline, model training, and monitoring are all Python. The most critical Python component is the feature computation service that updates Redis:



```

Simplified feature computation -- runs inside Flink (via PyFlink)
or as a standalone service consuming from Kafka
def compute_user_features(transaction: dict, current_features: dict) -> dict:
 """Update a user's real-time features based on a new transaction.

 This runs for every transaction (15K/second at peak).
 Must complete in under 5ms to stay within the latency budget.
 """
 now = transaction["timestamp"]
 amount = transaction["amount"]

 # Sliding window counters (approximate -- use HyperLogLog in production)
 features = current_features.copy()
 features["txn_count_1h"] = features.get("txn_count_1h", 0) + 1
 features["txn_count_24h"] = features.get("txn_count_24h", 0) + 1
 features["txn_amount_1h"] = features.get("txn_amount_1h", 0) + amount
 features["txn_amount_24h"] = features.get("txn_amount_24h", 0) + amount

 # Velocity checks
 last_txn_time = features.get("last_txn_timestamp")
 if last_txn_time:
 features["time_since_last_txn_seconds"] = (now - last_txn_time).total_seconds()

 # Geographic anomaly
 last_lat = features.get("last_txn_lat")
 last_lon = features.get("last_txn_lon")
 if last_lat and last_lon and transaction.get("lat"):
 features["distance_from_last_txn_km"] = haversine(
 last_lat, last_lon, transaction["lat"], transaction["lon"]
)

 features["last_txn_timestamp"] = now
 features["last_txn_lat"] = transaction.get("lat")
 features["last_txn_lon"] = transaction.get("lon")

 return features

```

**Module 7 (Spark):** Spark runs the daily batch pipeline that: 1. Recomputes exact features for all users (the real-time approximate features drift over time) 2. Joins transactions with merchant data, device fingerprints, and chargeback reports 3. Creates training datasets for the ML models 4. Loads analytics-ready data into Snowflake

**Module 8 (Orchestration):** Airflow orchestrates the batch pipeline but NOT the real-time pipeline. The real-time pipeline runs continuously via Flink and is monitored by a separate system (Prometheus + Grafana). This is a critical architectural decision: batch orchestrators like Airflow are not designed for real-time systems.

**Module 9 (Data Quality):** Data quality in fraud detection is literally a matter of financial safety. The quality checks include:

- **Completeness:** Every transaction must have a fraud score. Missing scores mean unscored transactions are approved by default, creating a vulnerability.
- **Latency:** If scoring latency exceeds 100ms, transactions are routed to a fallback rules-only engine. The Flink job monitors its own p99 latency and emits alerts.
- **Model drift:** The batch pipeline compares the daily fraud rate against the model's predicted fraud rate. If they diverge by more than 2 percentage points, an alert fires and the ML team investigates.
- **Feature consistency:** The batch pipeline recomputes features exactly and compares them to the real-time approximations. Drift beyond 5% triggers a reconciliation job that corrects the real-time features.

## The Latency Budget

This is the single most important design artifact in this system:

COMPONENT	BUDGET	ACTUAL P99	NOTES
API Gateway → Kafka	5ms	3ms	Internal network, same AZ
Kafka → Flink	5ms	4ms	Consumer lag monitored
Feature lookup (Redis)	5ms	2ms	Redis cluster, local replica
ML inference (gRPC)	30ms	22ms	TorchServe with GPU, batched
Rules engine	10ms	7ms	In-memory rule evaluation
Decision → Response	5ms	3ms	Serialization + network
<b>Total</b>	<b>60ms</b>	<b>41ms</b>	<b>Budget: 100ms, headroom: 59ms</b>

The 59ms of headroom is not waste. It absorbs spikes during Black Friday and allows for future feature additions without breaching the SLA.

## Startup vs. Enterprise Approach

ASPECT	STARTUP (SEED TO SERIES A)	ENTERPRISE (THIS CASE STUDY)
Fraud detection	Rules-only engine (no ML)	ML ensemble + rules + graph analysis
Latency target	500ms (acceptable for most merchants)	100ms (required by card networks)
Feature store	PostgreSQL with materialized views	Redis cluster + S3 feature archive
Model training	Scikit-learn on a laptop, weekly	PyTorch on Spark + GPU, daily
Monitoring	Basic alerting on fraud rate	Full observability stack (Prometheus, Grafana, PagerDuty)
Compliance	Basic logging	Full audit trail, PCI DSS Level 1, SOX
Cost	\$2K-\$5K/month	\$150K-\$300K/month
Team	1-2 engineers	13 engineers across three specialties

**The startup version:** Transactions → PostgreSQL → Python rules engine → Decision. No ML, no streaming, no Redis. The rules engine checks: is the amount above \$500? Is the card used in a new country? Is the velocity above 10 transactions/hour? This catches 60-70% of fraud. Good enough to get started. You can add ML later.

## Lessons Learned

- 1. Start with rules, add ML later.** A simple rules engine catches the obvious fraud. ML catches the subtle fraud. But ML without rules is dangerous because ML models can be fooled by novel attack patterns.
- 2. The feature store is the hardest part.** Keeping real-time features consistent with batch features is an ongoing engineering challenge. Budget 30% of your engineering time for this.
- 3. Latency budgets prevent scope creep.** When someone says “let’s add one more feature to the model,” you can point to the latency budget and ask: “Which 5ms are you willing to give up?”
- 4. Compliance is not optional in fintech.** PCI DSS requires encryption at rest and in transit, access logging, vulnerability scanning, and annual audits. This adds 20-30% overhead to every engineering decision. Factor it in from the start.

## Case Study 3: SaaS Analytics Platform (Product Analytics)

### Understanding User Behavior at Scale

Product analytics is the use case that makes data engineering directly visible to the entire company. When the CEO asks “how many users are active this week?” or the product manager asks “what is the conversion rate of our new onboarding flow?”, the answer comes from the data platform you built.

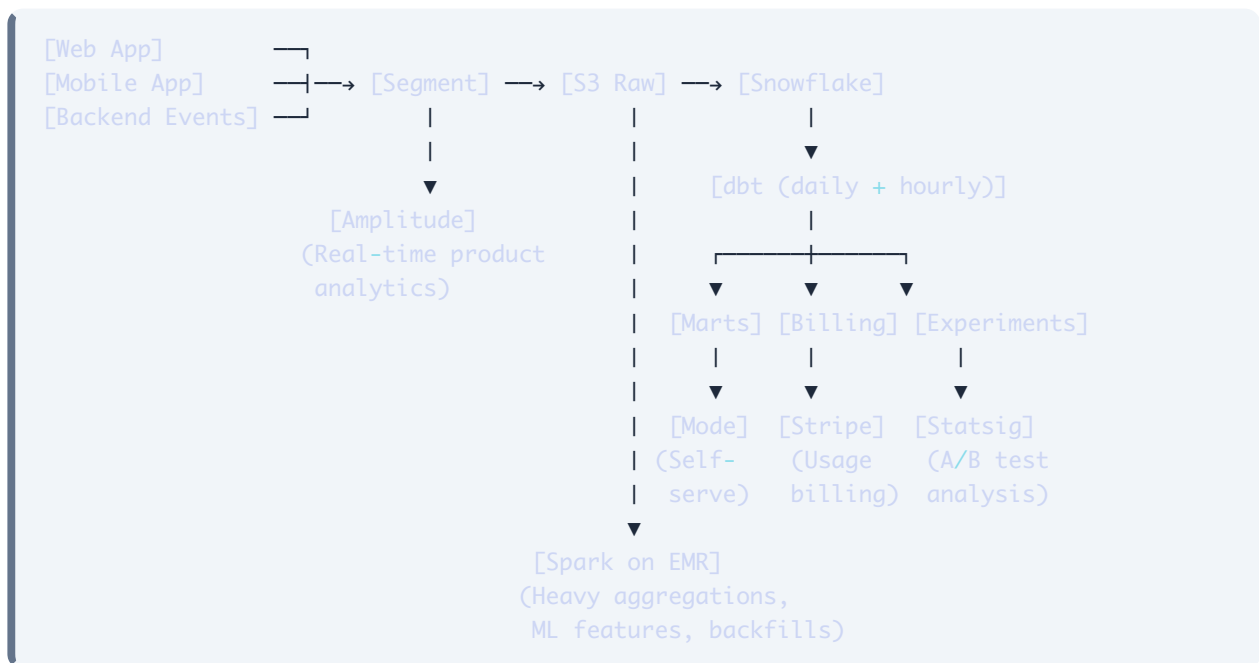
### Business Context

**Company:** B2B SaaS product (project management tool), 500K active users, 2M events/day.

**Data team:** 4 data engineers, 8 analysts, 2 analytics engineers.

**Requirements:** - Track every user interaction (page views, clicks, feature usage) via event tracking - Self-service analytics for product managers (they should not need to ask a data engineer for basic metrics) - Experimentation platform for A/B tests on product features - Funnel analysis: sign-up → onboarding → activation → retention → expansion - Usage-based billing: compute each customer’s monthly usage for invoicing

### The Architecture



### The Event Table: The Heart of Product Analytics

Every SaaS analytics platform revolves around an event table. Getting this right is the single most important data modeling decision:

```

-- The core event table: one row per user interaction
-- This table receives 2M+ rows/day and is the source of truth
-- for all product analytics
CREATE TABLE raw.events (
 event_id VARCHAR, -- UUID, globally unique
 event_name VARCHAR, -- 'page_viewed', 'button_clicked', 'feature_used'
 event_timestamp TIMESTAMP,
 user_id VARCHAR, -- Authenticated user (null if anonymous)
 anonymous_id VARCHAR, -- Device/session ID (always present)
 session_id VARCHAR,
 properties VARIANT, -- JSON blob of event-specific properties
 context VARIANT, -- Device, browser, IP, UTM params
 received_at TIMESTAMP, -- When Segment received the event
 _loaded_at TIMESTAMP -- When our pipeline loaded it
);

-- Example event:
-- event_name: 'project_created'
-- properties: {"project_name": "Q2 Launch", "template": "kanban", "team_size": 5}
-- context: {"browser": "Chrome", "os": "macOS", "utm_source": "google"}

```

**Why VARIANT/JSON for properties?** Because every event type has different properties. A “page\_viewed” event has `page_url` and `referrer`. A “project\_created” event has `project_name` and `template`. Enforcing a rigid schema across hundreds of event types is impractical. The VARIANT column lets you store structured data flexibly, and Snowflake can query inside it efficiently.

**Module 2 (SQL):** The most common product analytics queries are funnel analyses and retention calculations. Here is the activation funnel that the product team reviews weekly:

```

-- Activation funnel: what % of new signups complete each onboarding step?
-- This directly drives product decisions about onboarding UX
WITH signups AS (
 SELECT
 user_id,
 MIN(event_timestamp) AS signup_ts
 FROM raw.events
 WHERE event_name = 'account_created'
 AND event_timestamp > DATEADD('day', -30, CURRENT_TIMESTAMP())
 GROUP BY user_id
),
funnel_steps AS (
 SELECT
 s.user_id,
 s.signup_ts,
 -- Step 1: Created an account (100% by definition)
 TRUE AS step_1_signup,
 -- Step 2: Completed profile setup
 MAX(CASE WHEN e.event_name = 'profile_completed'
 AND e.event_timestamp <= DATEADD('day', 7, s.signup_ts)
 THEN TRUE ELSE FALSE END) AS step_2_profile,
 -- Step 3: Created first project
 MAX(CASE WHEN e.event_name = 'project_created'
 AND e.event_timestamp <= DATEADD('day', 7, s.signup_ts)
 THEN TRUE ELSE FALSE END) AS step_3_project,
 -- Step 4: Invited a team member
 MAX(CASE WHEN e.event_name = 'team_member_invited'
 AND e.event_timestamp <= DATEADD('day', 7, s.signup_ts)
 THEN TRUE ELSE FALSE END) AS step_4_invite,
 -- Step 5: Active for 3+ days in first week (activation!)
 CASE WHEN COUNT(DISTINCT CASE WHEN e.event_timestamp <= DATEADD('day', 7, s.sig
nup_ts)
 THEN DATE_TRUNC('day', e.event_timestamp) END) >= 3
 THEN TRUE ELSE FALSE END AS step_5_activated
 FROM signups s
 LEFT JOIN raw.events e ON s.user_id = e.user_id
 GROUP BY s.user_id, s.signup_ts
)
SELECT
 COUNT(*) AS total_signups,
 SUM(step_2_profile::INT) AS completed_profile,
 ROUND(100.0 * SUM(step_2_profile::INT) / COUNT(*), 1) AS pct_profile,
 SUM(step_3_project::INT) AS created_project,
 ROUND(100.0 * SUM(step_3_project::INT) / COUNT(*), 1) AS pct_project,
 SUM(step_4_invite::INT) AS invited_team,
 ROUND(100.0 * SUM(step_4_invite::INT) / COUNT(*), 1) AS pct_invite,
 SUM(step_5_activated::INT) AS activated,
 ROUND(100.0 * SUM(step_5_activated::INT) / COUNT(*), 1) AS pct_activated
FROM funnel_steps;

```

**Module 6 (dbt):** The dbt project has two run frequencies: - **Hourly:** Core event rollups and near-real-time metrics (active users, feature usage counts) - **Daily:** Full funnel calculations, retention cohorts, billing aggregations

```

-- dbt model: intermediate/int_daily_user_activity.sql
-- Incremental model that summarizes daily user activity
-- Used by retention, engagement, and billing models downstream
{{
 config(
 materialized='incremental',
 unique_key=['user_id', 'activity_date'],
 incremental_strategy='merge'
)
}}

SELECT
 user_id,
 DATE_TRUNC('day', event_timestamp) AS activity_date,
 COUNT(*) AS total_events,
 COUNT(DISTINCT event_name) AS distinct_events,
 COUNT(DISTINCT session_id) AS sessions,
 MIN(event_timestamp) AS first_event_at,
 MAX(event_timestamp) AS last_event_at,
 SUM(CASE WHEN event_name LIKE 'feature_%' THEN 1 ELSE 0 END) AS feature_events,
 ARRAY_AGG(DISTINCT event_name) AS event_types
FROM raw.events
WHERE user_id IS NOT NULL
{% if is_incremental() %}
 AND event_timestamp > (SELECT MAX(last_event_at) FROM {{ this }})
{% endif %}
GROUP BY 1, 2

```

**Usage-based billing** (a unique requirement for SaaS) uses this daily activity data to compute each customer's monthly invoice:

```

-- marts/mart_monthly_billing.sql
-- Computes usage metrics per customer account per month
-- Feeds directly into the Stripe billing integration
SELECT
 a.account_id,
 DATE_TRUNC('month', d.activity_date) AS billing_month,
 COUNT(DISTINCT d.user_id) AS active_users,
 SUM(d.total_events) AS total_events,
 SUM(d.sessions) AS total_sessions,
 -- Billing tier based on active users
 CASE
 WHEN COUNT(DISTINCT d.user_id) <= 5 THEN 'free'
 WHEN COUNT(DISTINCT d.user_id) <= 25 THEN 'team'
 WHEN COUNT(DISTINCT d.user_id) <= 100 THEN 'business'
 ELSE 'enterprise'
 END AS billing_tier,
 -- Calculated price
 CASE
 WHEN COUNT(DISTINCT d.user_id) <= 5 THEN 0
 WHEN COUNT(DISTINCT d.user_id) <= 25 THEN COUNT(DISTINCT d.user_id) * 12
 WHEN COUNT(DISTINCT d.user_id) <= 100 THEN COUNT(DISTINCT d.user_id) * 10
 ELSE COUNT(DISTINCT d.user_id) * 8 -- Volume discount
 END AS monthly_charge_usd
FROM {{ ref('int_daily_user_activity') }} d
JOIN {{ ref('dim_user_accounts') }} a ON d.user_id = a.user_id
GROUP BY 1, 2

```

**Module 9 (Data Quality):** When your event data drives billing, quality is financial. The critical quality checks:

1. **Event volume anomaly detection:** If daily event volume drops more than 30% from the 7-day average, alert immediately. A tracking code bug can silently stop sending events, leading to under-billing customers (revenue loss) or under-counting usage (wrong product decisions).
2. **Schema validation:** Every event must have `event_id`, `event_name`, `event_timestamp`, and `user_id` or `anonymous_id`. Events failing validation go to a dead-letter queue for investigation.
3. **Billing reconciliation:** Monthly billing totals are compared against Stripe records. Any discrepancy over \$100 triggers a manual review.



## Startup vs. Enterprise Approach

ASPECT	STARTUP (EARLY STAGE)	ENTERPRISE (THIS CASE STUDY)
Event tracking	Custom script, 5-10 event types	Segment with tracking plan, 200+ event types
Analytics tool	Amplitude or Mixpanel (managed)	Custom warehouse + Mode (self-serve SQL)
Billing	Manual CSV upload to Stripe	Automated pipeline: events → dbt → Stripe API
Experimentation	Feature flags (LaunchDarkly)	Full A/B testing platform with statistical rigor
Team	1 data-aware backend engineer	4 DEs + 8 analysts + 2 analytics engineers
Cost	\$500/month (Amplitude + Segment free tiers)	\$80K/month (Snowflake + tools + headcount)

## Lessons Learned

- 1. Define your tracking plan before writing any code.** A tracking plan is a spreadsheet listing every event, its properties, and when it fires. Without it, you get inconsistent event names (“button\_click” vs “buttonClick” vs “btn\_clicked”) and missing properties that make analysis impossible.
- 2. Event data is append-only; schema changes are painful.** If you add a new property to an event, historical events do not have it. Design your event properties to be stable. Use the VARIANT/JSON column to handle evolving properties gracefully.
- 3. Usage-based billing is a data engineering problem.** The billing pipeline must be as reliable as the payment processing system. Treat it with the same rigor.
- 4. Self-serve analytics requires investment in data modeling.** Analysts cannot write queries against raw event data efficiently. The dbt marts layer (clean, documented, tested) is what makes self-serve possible.

---

## Case Study 4: Healthcare Data Pipeline

---

### When Data Quality Is Life-or-Death

Healthcare data engineering is unique because the consequences of errors are not financial or reputational – they are clinical. A duplicate patient record can lead to a wrong medication being admin-

istered. A missing lab result can delay a diagnosis. A data quality failure in healthcare is categorically different from a data quality failure in e-commerce.

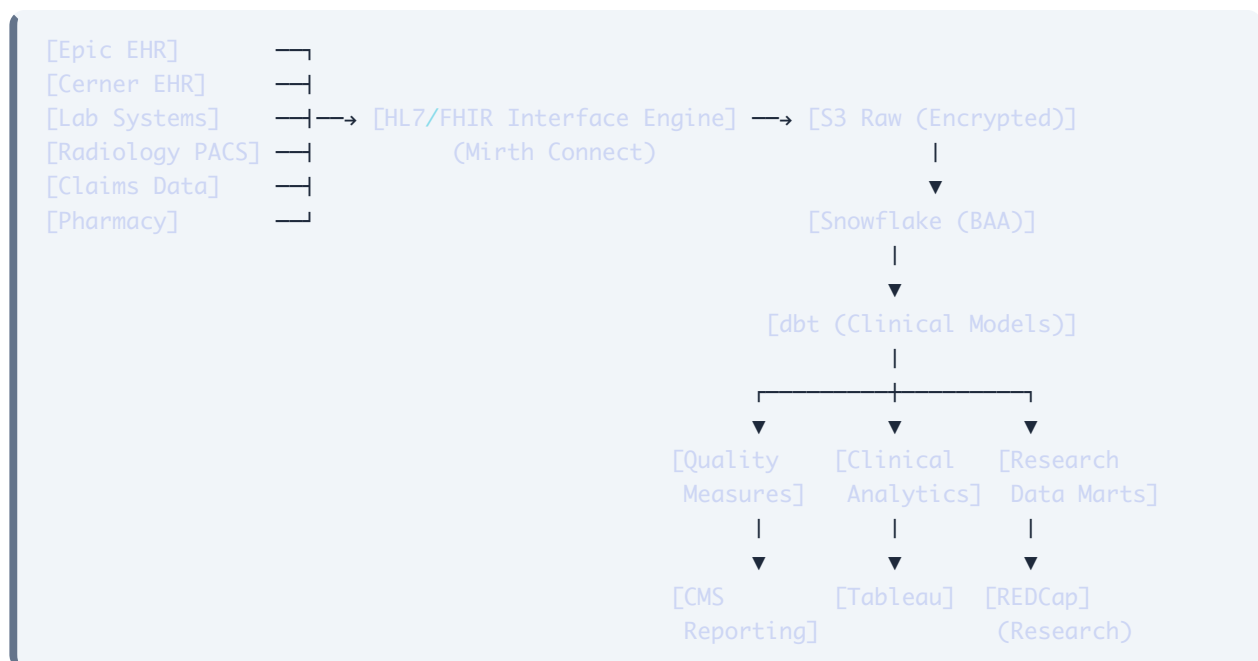
## Business Context

**Company:** Regional health system, 12 hospitals, 3M patient encounters/year, 200 clinics.

**Data team:** 10 data engineers, 6 analysts, 4 clinical informaticists (domain experts who bridge clinical and technical).

**Requirements:** - Consolidate patient data from 15 different EHR (Electronic Health Record) systems into a single analytics platform - Population health analytics: identify high-risk patients for preventive care programs - Regulatory reporting: CMS quality measures, state health department reports - Clinical research data marts for academic hospital partners - HIPAA compliance on every single component - Data must be accurate. Period. A wrong lab value or duplicate patient record is a patient safety issue.

## The Architecture



## HIPAA: The Constraint That Shapes Everything

HIPAA (Health Insurance Portability and Accountability Act) is not a box-checking exercise. It fundamentally constrains your architecture:

**What HIPAA requires for data engineering:**

1. **Encryption at rest and in transit.** Every S3 bucket uses AES-256 encryption. Every database connection uses TLS. Every Snowflake connection uses encrypted channels.

2. **Access controls with audit logging.** Every query against patient data is logged with the user, timestamp, and data accessed. Role-based access control (RBAC) ensures analysts can only see data for their facility.
3. **Business Associate Agreements (BAAs).** Every vendor that touches patient data must sign a BAA. This means Snowflake (has BAA option), AWS (has BAA), dbt Cloud (has BAA). Your favorite random SaaS tool? Probably does not have a BAA. You cannot use it.
4. **Minimum necessary access.** Users should only see the patient data they need for their job function. An analyst studying diabetes outcomes should not have access to psychiatric records.
5. **De-identification for research.** Research data marts must strip or hash all 18 HIPAA identifiers (name, DOB, SSN, address, etc.).

```
-- De-identification view for research data marts
-- Strips all 18 HIPAA identifiers while preserving analytical value
CREATE VIEW research.encounters_deidentified AS
SELECT
 -- Replace patient_id with a one-way hash (cannot be reversed)
 SHA2(patient_id || 'research_salt_2026') AS research_patient_id,
 -- Shift dates by a random offset per patient (preserves intervals)
 encounter_date + patient_date_offset AS encounter_date_shifted,
 -- Age in years (age > 89 → '90+' per HIPAA Safe Harbor)
 CASE WHEN age > 89 THEN 90 ELSE age END AS age_category,
 -- Keep clinical data but remove identifying context
 diagnosis_code,
 procedure_code,
 lab_results,
 -- Remove: name, DOB, SSN, address, phone, email, MRN, etc.
 -- State-level geography only (ZIP codes with < 20K population → NULL)
 CASE WHEN zip_population >= 20000 THEN LEFT(zip_code, 3) ELSE NULL END AS zip_3digit
FROM clinical.encounters e
JOIN clinical.patient_date_offsets o ON e.patient_id = o.patient_id;
```

## Module-by-Module Breakdown

**Module 1 (Data Modeling):** Healthcare data modeling uses domain-specific standards. The OMOP (Observational Medical Outcomes Partnership) Common Data Model is widely adopted:

```

-- Simplified OMOP-inspired patient encounter model
-- The key challenge: patient matching across 15 different EHR systems
CREATE TABLE clinical.dim_patient (
 patient_key BIGINT,
 master_patient_id VARCHAR, -- Enterprise Master Patient Index (EMPI)
 -- A single patient may have different IDs across systems:
 epic_mrn VARCHAR, -- Medical Record Number in Epic
 cerner_mrn VARCHAR, -- MRN in Cerner
 -- Demographics
 birth_date DATE,
 gender VARCHAR,
 race VARCHAR,
 ethnicity VARCHAR,
 -- Derived risk scores (updated daily)
 hcc_risk_score FLOAT, -- CMS Hierarchical Condition Category
 readmission_risk FLOAT, -- 30-day readmission probability
 -- SCD Type 2 for address changes (affects care coordination)
 address_line_1 VARCHAR,
 city VARCHAR,
 state VARCHAR,
 zip_code VARCHAR,
 is_current BOOLEAN,
 valid_from DATE,
 valid_to DATE
);

```

**The Master Patient Index (EMPI) problem:** The hardest data engineering challenge in healthcare is patient matching. Patient “John Smith, DOB 1985-03-15” in the Epic system is the same person as “Jon Smith, DOB 03/15/1985” in the Cerner system. But “John Smith, DOB 1985-03-15” in one hospital may be a completely different person than “John Smith, DOB 1985-03-15” in another hospital.

Duplicate patient records in a clinical setting can result in: - Duplicate medication orders (overdose risk) - Missing allergies from the other record (adverse reaction risk) - Fragmented clinical history (misdiagnosis risk)

The EMPI uses probabilistic matching with weighted scores across name, DOB, SSN, address, and phone number. This is typically handled by dedicated EMPI software (IBM Initiate, Verato), not built from scratch.

**Module 2 (SQL):** Clinical quality measures are complex SQL queries mandated by CMS. Here is a simplified version of the diabetes HbA1c control measure:

```

-- CMS Quality Measure: Diabetes HbA1c Control
-- Denominator: Patients 18-75 with diabetes diagnosis
-- Numerator: Those patients with HbA1c < 8.0 in the measurement period
-- This feeds regulatory reporting submitted to CMS quarterly

WITH diabetic_patients AS (
 SELECT DISTINCT p.patient_key, p.master_patient_id
 FROM clinical.dim_patient p
 JOIN clinical.fact_diagnoses d ON p.patient_key = d.patient_key
 WHERE d.diagnosis_code LIKE 'E11%' -- ICD-10 codes for Type 2 diabetes
 AND p.is_current = TRUE
 AND DATEDIFF('year', p.birth_date, CURRENT_DATE()) BETWEEN 18 AND 75
),
latest_hba1c AS (
 SELECT
 l.patient_key,
 l.result_value,
 l.result_date,
 ROW_NUMBER() OVER (
 PARTITION BY l.patient_key
 ORDER BY l.result_date DESC
) AS rn
 FROM clinical.fact_lab_results l
 WHERE l.lab_code = '4548-4' -- LOINC code for HbA1c
 AND l.result_date >= DATEADD('year', -1, CURRENT_DATE())
)
SELECT
 COUNT(dp.patient_key) AS denominator,
 SUM(CASE WHEN h.result_value < 8.0 THEN 1 ELSE 0 END) AS numerator,
 ROUND(100.0 * SUM(CASE WHEN h.result_value < 8.0 THEN 1 ELSE 0 END)
 / COUNT(dp.patient_key), 1) AS performance_rate_pct
FROM diabetic_patients dp
LEFT JOIN latest_hba1c h ON dp.patient_key = h.patient_key AND h.rn = 1;

```

**Module 8 (Orchestration):** The orchestration is conservative by design. Healthcare pipelines favor reliability over speed:

- **No auto-retry on failure.** If a pipeline fails, it alerts and waits for human review. An auto-retry that inserts duplicate records is worse than a delayed pipeline.
- **Reconciliation at every stage.** Row counts are compared at ingestion, staging, and marts layers. Any discrepancy halts the pipeline.
- **Change data capture (CDC) from EHR systems.** EHR data changes retroactively (a doctor amends a note, a lab result is corrected). The pipeline must handle late-arriving updates, not just new inserts.

**Module 9 (Data Quality):** Data quality in healthcare is the most rigorous of any industry:

```

dbt tests for clinical data -- these are non-negotiable
models:
 - name: fact_lab_results
 tests:
 # A patient cannot have a lab result before their birth date
 - dbt_utils.expression_is_true:
 expression: "result_date >= patient_birth_date"
 name: lab_result_after_birth
 # HbA1c values outside 2-20 are likely data entry errors
 - dbt_utils.expression_is_true:
 expression: "CASE WHEN lab_code = '4548-4' THEN result_value BETWEEN 2.0 AND
20.0 ELSE TRUE END"
 name: hba1c_reasonable_range
 columns:
 - name: patient_key
 tests:
 - not_null
 - relationships:
 to: ref('dim_patient')
 field: patient_key
 - name: result_value
 tests:
 - not_null
 - name: lab_code
 tests:
 - not_null
 - accepted_values:
 values: ['4548-4', '2345-7', '718-7'] # Known LOINC codes only

```

## Startup vs. Enterprise Approach

ASPECT	HEALTH TECH STARTUP	ENTERPRISE HEALTH SYSTEM (THIS CASE)
Data sources	1-2 EHR integrations via FHIR API	15 EHR systems, HL7v2 + FHIR + flat files
Patient matching	Simple deterministic matching	Probabilistic EMPI with manual review
Compliance	HIPAA basics + SOC 2	HIPAA + HITRUST + state regulations + IRB for research
Quality checks	Standard dbt tests	Clinical validation rules + reconciliation + manual chart review
Infrastructure	All-managed (BigQuery + dbt Cloud)	Hybrid (on-prem interface engine + cloud warehouse)
Team	2-3 engineers with clinical advisor	10 DEs + 4 clinical informaticists
Cost	\$3K-\$10K/month	\$200K-\$500K/month
Time to build	3-6 months MVP	18-24 months for full platform

## Lessons Learned

- Domain expertise is non-negotiable.** You cannot build healthcare data pipelines without clinical informaticists on the team. ICD-10 codes, LOINC codes, HL7 message parsing, clinical workflow understanding – these require deep domain knowledge.
- Data quality checks must be clinically meaningful.** A test that checks “not null” is not enough. You need tests like “a patient’s discharge date cannot be before their admission date” and “a HbA1c value above 20 is almost certainly a data error.”
- Patient matching is the hardest problem.** Plan to spend 30-40% of your engineering effort on the EMPI and data reconciliation. Getting this wrong has patient safety consequences.
- HIPAA adds cost and complexity to everything.** Budget 20-30% more time and cost for HIPAA compliance. Every vendor evaluation, every architecture decision, every data access request goes through a compliance review.
- Late-arriving data is the norm, not the exception.** Clinical data gets amended, corrected, and back-dated constantly. Your pipeline must handle updates gracefully, not just inserts.

## Cross-Case-Study Module Map

Here is how each course module appears across all four case studies:

MODULE	E-COMMERCE	FRAUD DETECTION	SAAS ANALYTICS	HEALTHCARE
1: Data Modeling	Star schema, SCD2 customers	Denormalized (real-time) + star (batch)	Event table + VARIANT/JSON	OMOP standard, EMPI
2: SQL	Cohort retention, window functions	Precision/recall by threshold	Funnel analysis, activation metrics	Clinical quality measures
3: Python	Custom API connectors	Feature computation service	Event processing scripts	HL7 message parsing
4: Warehousing	Snowflake multi-warehouse	Snowflake for batch analytics	Snowflake with hourly refreshes	Snowflake with BAA, RBAC
5: Data Lake	S3 raw layer (safety net)	S3 feature archive + audit logs	S3 raw events from Segment	S3 encrypted, lifecycle managed
6: dbt	180+ models, incremental	Batch feature recomputation	Hourly + daily models, billing	Clinical models with strict tests
7: Spark	ML feature engineering	Daily batch recomputation	Heavy aggregations, backfills	Not used (volume is moderate)
8: Orchestration	Airflow with SLA monitoring	Airflow for batch only; Flink is self-managed	Airflow with hourly + daily DAGs	Conservative: no auto-retry
9: Data Quality	Revenue reconciliation	Latency monitoring, drift detection	Volume anomaly detection, billing reconciliation	Clinical validation, patient safety checks
10: Capstone	This IS the production capstone	Extends capstone with real-time	Extends capstone with event tracking	Extends capstone with compliance

## Patterns Across All Four Case Studies

**1. Every system has a “source of truth” problem.** E-commerce reconciles against Shopify. Fraud detection reconciles features between real-time and batch. SaaS analytics reconciles billing against



Stripe. Healthcare reconciles patient records across 15 systems. The reconciliation pipeline is as important as the primary pipeline.

**2. Data modeling is the foundation.** In every case study, the data model was designed first and drove all downstream decisions. Changing the data model after the system is built is the most expensive refactoring you can do.

**3. Incremental processing is essential at scale.** Every case study that handles more than a few thousand rows/day uses incremental models. Full-refresh dbt models do not scale.

**4. Data quality investment scales with consequence severity.** E-commerce checks revenue accuracy. Fintech checks latency and model drift. SaaS checks billing accuracy. Healthcare checks clinical validity. The investment in quality is proportional to the cost of getting it wrong.

**5. The startup version of every system is radically simpler.** You do not need Kafka, Flink, Redis, and a 13-person team to solve these problems at startup scale. You need PostgreSQL, Python, dbt, and one capable engineer. Scale the architecture when the business demands it, not before.

**Key Takeaway:** These case studies are not meant to intimidate. They are meant to show that the skills you are learning in Modules 0-10 are the same skills used at production scale. The difference is operational maturity, not fundamentally different technology. Build your capstone well, and you have demonstrated 80% of what these production systems require.

---

## What's Next

---

Appendix C (Interview Prep) shows you how to discuss these architectures in interviews. Practice describing the trade-offs, drawing the architecture diagrams from memory, and explaining why you would choose one approach over another.

# Appendix C: Interview Preparation for Data Engineers

---

## The Data Engineering Interview Landscape

---

Data engineering interviews are different from software engineering interviews. You will not be asked to implement a red-black tree or solve dynamic programming puzzles. Instead, you will face a combination of: SQL coding challenges, Python coding for data tasks, system design for data pipelines, data modeling questions, and behavioral questions about working with data at scale.

After conducting 150+ system design interviews at three companies, I can tell you: the difference between candidates who get offers and those who do not is rarely technical knowledge. Most senior candidates know Kafka, know SQL, know the CAP theorem. The difference is **how they think and communicate**.

The best candidates treat the interview as a collaborative design session – like a meeting with a colleague to whiteboard an architecture. They ask clarifying questions, state assumptions explicitly, explain trade-offs, and invite the interviewer into the decision-making process.

**Key Takeaway:** Data engineering interviews test five things: (1) Can you write production-quality SQL? (2) Can you design data systems? (3) Can you model data correctly? (4) Can you write Python for data tasks? (5) Can you communicate technical decisions clearly? This appendix prepares you for all five.

---

## Part 1: Data Modeling Interview Questions

---

Data modeling questions test whether you understand how to structure data for analytical workloads. These tie directly to Module 1 (Foundations) and Module 6 (dbt).

### Question 1: Design a Star Schema for a Ride-Sharing Company

**Interviewer:** “Design the data model for a ride-sharing analytics platform. The business wants to analyze ride volume, revenue, driver performance, and rider behavior.”

**Strong answer:**

“I would start with a star schema centered on a `fact_rides` table. The grain is one row per completed ride.

#### Fact table:

```
fact_rides (
 ride_key BIGINT, -- Surrogate key
 ride_id VARCHAR, -- Business key
 rider_key BIGINT, -- FK to dim_rider
 driver_key BIGINT, -- FK to dim_driver
 pickup_location_key BIGINT, -- FK to dim_location
 dropoff_location_key BIGINT, -- FK to dim_location
 pickup_date_key INT, -- FK to dim_date
 pickup_time_key INT, -- FK to dim_time
 vehicle_key BIGINT, -- FK to dim_vehicle
 -- Measures
 ride_distance_miles DECIMAL(8,2),
 ride_duration_minutes DECIMAL(8,2),
 base_fare DECIMAL(8,2),
 surge_multiplier DECIMAL(4,2),
 total_fare DECIMAL(8,2),
 driver_payout DECIMAL(8,2),
 platform_fee DECIMAL(8,2),
 tip_amount DECIMAL(8,2),
 rating_by_rider INT, -- 1-5
 rating_by_driver INT, -- 1-5
 wait_time_minutes DECIMAL(6,2),
 ride_status VARCHAR -- 'completed', 'cancelled_rider',
 'cancelled_driver'
)
```

#### Dimension tables:

- `dim_rider`: rider demographics, signup date, lifetime rides, rider tier (SCD Type 2 for tier changes)
- `dim_driver`: driver demographics, signup date, vehicle type, average rating, driver tier (SCD Type 2)
- `dim_location`: city, neighborhood, latitude, longitude, H3 hex cell (for geospatial aggregation)
- `dim_date`: standard date dimension with day of week, holiday flags, fiscal period
- `dim_time`: hour, minute, time-of-day bucket (morning rush, midday, evening rush, late night)
- `dim_vehicle`: vehicle type, year, model, capacity

I would use SCD Type 2 on `dim_rider` and `dim_driver` because tier changes affect analysis. If a driver was ‘gold’ tier when they gave a ride, we want to know that, not just their current tier.”

#### Follow-up: “Why not a single timestamp column instead of separate date and time dimension keys?”

“Separate date and time dimensions make aggregation much faster. If I want ‘total rides on Saturdays during evening rush,’ I can filter on `dim_date.day_of_week = 'Saturday'` and

`dim_time.time_bucket = 'evening_rush'` using simple equality joins. With a single timestamp, every query needs date functions like `EXTRACT(DOW FROM timestamp)` and `EXTRACT(HOUR FROM timestamp)`, which cannot use sort keys or clustering as effectively.”

**Follow-up: “How would you handle cancelled rides?”**

“I would include them in `fact_rides` with a `ride_status` column. Cancelled rides have measures of zero for fare and distance but still have valid data for wait time, location, and time. Analyzing cancellation patterns is a key business need: where are cancellations happening? Which drivers cancel most? What time of day has the highest cancellation rate? Excluding them would lose this analytical capability.”

---

## Question 2: Explain SCD Types and When to Use Each

**Interviewer:** “What are Slowly Changing Dimensions and when would you use Type 1 vs Type 2 vs Type 3?”

**Strong answer:**

“Slowly Changing Dimensions handle the problem of attribute changes in dimension tables over time.

**Type 1 (Overwrite):** Replace the old value with the new value. No history is preserved. Use this when history does not matter – for example, fixing a typo in a product name or updating a customer’s email address.

**Type 2 (Add New Row):** Create a new row with the new value, marking the old row as inactive with `valid_from` and `valid_to` dates. This preserves full history. Use this when the change is analytically meaningful – for example, when a customer changes their address (affects regional sales analysis) or when a customer’s loyalty tier changes (affects behavior analysis).

**Type 3 (Add New Column):** Add a `previous_value` and `current_value` column. Only tracks one change. I almost never recommend this because it is limited to exactly one historical value, which is rarely sufficient.

In practice, I default to Type 1 for most attributes and use Type 2 selectively for attributes that drive analytical decisions. Over-using Type 2 causes dimension table bloat and query complexity. Under-using it means you lose important history.

A common mistake is using Type 2 for everything. If your `dim_customer` table has 1 million customers and you track 10 attributes with Type 2, you might end up with 15 million rows, most of which are historical and rarely queried. Be selective.”

---

## Question 3: Normalize vs. Denormalize

**Interviewer:** “When would you normalize your data model and when would you denormalize it?”

**Strong answer:**

“The choice depends on the workload pattern.

**Normalize (3NF) when:** - The system is write-heavy (OLTP) and you need to minimize data redundancy - You need referential integrity enforced at the database level - The data is the operational source of truth (transactional databases) - Example: the production PostgreSQL database behind a web application

**Denormalize (star schema, wide tables) when:** - The system is read-heavy (OLAP/analytics) and you need fast query performance - Users are running aggregations, joins, and filters across large datasets - The data is loaded in batch and queried frequently - Example: a Snowflake data warehouse serving Tableau dashboards

**The trade-off in one sentence:** Normalization optimizes for write performance and data integrity. Denormalization optimizes for read performance and query simplicity.

In modern data engineering, the common pattern is: normalized in the source system, denormalized in the warehouse. The dbt transformation layer (Module 6) handles the conversion.”

---

## Part 2: SQL Coding Challenges

---

SQL is the most tested skill in data engineering interviews. You will be asked to write queries on a whiteboard or in a shared coding environment. These questions map directly to Module 2 (SQL Mastery).

### Challenge 1: Deduplicate Records

**Problem:** “Given a table `raw_events` with duplicate records (same `event_id` appearing multiple times due to at-least-once delivery), write a query to deduplicate it, keeping only the first occurrence based on `received_at`.”

```

-- Method 1: ROW_NUMBER (most common approach)
WITH ranked AS (
 SELECT
 *,
 ROW_NUMBER() OVER (
 PARTITION BY event_id
 ORDER BY received_at ASC
) AS rn
 FROM raw_events
)
SELECT * FROM ranked WHERE rn = 1;

-- Method 2: QUALIFY (Snowflake/BigQuery -- more concise)
SELECT *
FROM raw_events
QUALIFY ROW_NUMBER() OVER (
 PARTITION BY event_id
 ORDER BY received_at ASC
) = 1;

```

**What interviewers look for:** Do you know ROW\_NUMBER vs RANK vs DENSE\_RANK? (ROW\_NUMBER assigns unique numbers even for ties; RANK leaves gaps; DENSE\_RANK does not.) Can you explain why you chose `ORDER BY received_at ASC`? (We want the first occurrence, so ascending order puts it at row number 1.)

---

## Challenge 2: Running Totals and Moving Averages

**Problem:** “Write a query that shows daily revenue with a 7-day moving average and a running total for the month.”

```

SELECT
 order_date,
 daily_revenue,
 -- 7-day moving average (current day + 6 preceding days)
 AVG(daily_revenue) OVER (
 ORDER BY order_date
 ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
) AS moving_avg_7d,
 -- Running total for the month (resets each month)
 SUM(daily_revenue) OVER (
 PARTITION BY DATE_TRUNC('month', order_date)
 ORDER BY order_date
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS monthly_running_total
FROM (
 SELECT
 DATE_TRUNC('day', order_timestamp) AS order_date,
 SUM(total_amount) AS daily_revenue
 FROM orders
 GROUP BY 1
) daily
ORDER BY order_date;

```

**What interviewers look for:** Do you understand the difference between `ROWS BETWEEN` and `RANGE BETWEEN`? Can you explain why `PARTITION BY DATE_TRUNC('month', order_date)` resets the running total each month?

---

### Challenge 3: Gap and Island Detection

**Problem:** “Given a table of user login dates, find consecutive login streaks (islands) for each user.”

```

-- Classic gaps-and-islands using the ROW_NUMBER trick
WITH numbered AS (
 SELECT
 user_id,
 login_date,
 -- Subtracting a sequential number from the date
 -- creates a constant value for consecutive dates
 login_date - INTERVAL '1 day' * ROW_NUMBER() OVER (
 PARTITION BY user_id
 ORDER BY login_date
) AS group_key
 FROM (
 SELECT DISTINCT user_id, DATE_TRUNC('day', login_timestamp) AS login_date
 FROM user_logins
) daily_logins
)
SELECT
 user_id,
 MIN(login_date) AS streak_start,
 MAX(login_date) AS streak_end,
 COUNT(*) AS streak_days
FROM numbered
GROUP BY user_id, group_key
HAVING COUNT(*) >= 3 -- Only streaks of 3+ days
ORDER BY user_id, streak_start;

```

**Explain the trick:** If a user logs in on Jan 1, 2, 3, 5, 6: subtracting row numbers (1,2,3,4,5) from the dates gives (Dec 31, Dec 31, Dec 31, Jan 1, Jan 1). The consecutive days map to the same `group_key`, while the gap creates a new group.

## Challenge 4: Sessionization

**Problem:** “Given a table of page view events with timestamps, define a session as a group of events where no two consecutive events are more than 30 minutes apart. Assign session IDs.”



```

WITH time_gaps AS (
 SELECT
 user_id,
 event_timestamp,
 page_url,
 LAG(event_timestamp) OVER (
 PARTITION BY user_id ORDER BY event_timestamp
) AS prev_event_ts,
 DATEDIFF('minute',
 LAG(event_timestamp) OVER (
 PARTITION BY user_id ORDER BY event_timestamp
),
 event_timestamp
) AS minutes_since_last
 FROM page_views
),
session_starts AS (
 SELECT
 *,
 CASE
 WHEN minutes_since_last IS NULL THEN 1 -- First event
 WHEN minutes_since_last > 30 THEN 1 -- New session
 ELSE 0
 END AS is_new_session
 FROM time_gaps
)
SELECT
 user_id,
 event_timestamp,
 page_url,
 SUM(is_new_session) OVER (
 PARTITION BY user_id
 ORDER BY event_timestamp
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS session_id
FROM session_starts;

```

**What interviewers look for:** Can you use LAG to compute inter-event time? Can you use a cumulative SUM of a flag to assign group IDs? This is a pattern that appears constantly in product analytics (Module 6, Case Study 3 in Appendix B).

## Challenge 5: Pivot / Unpivot

**Problem:** “Given a table with one row per student per subject with their score, pivot it to show one row per student with columns for each subject.”

```

-- Pivot (rows to columns)
SELECT
 student_id,
 MAX(CASE WHEN subject = 'math' THEN score END) AS math_score,
 MAX(CASE WHEN subject = 'science' THEN score END) AS science_score,
 MAX(CASE WHEN subject = 'english' THEN score END) AS english_score
FROM student_scores
GROUP BY student_id;

-- Unpivot (columns to rows) -- Snowflake syntax
SELECT student_id, subject, score
FROM student_scores_wide
UNPIVOT (score FOR subject IN (math_score, science_score, english_score));

```

## Challenge 6: Cumulative Distinct Count

**Problem:** “For each day, show the cumulative number of distinct users who have ever made a purchase up to that day.”

```

-- This is tricky because COUNT(DISTINCT) does not work as a window function
WITH first_purchases AS (
 SELECT
 user_id,
 MIN(DATE_TRUNC('day', purchase_timestamp)) AS first_purchase_date
 FROM purchases
 GROUP BY user_id
),
daily_new_users AS (
 SELECT
 first_purchase_date AS purchase_date,
 COUNT(*) AS new_users
 FROM first_purchases
 GROUP BY first_purchase_date
)
SELECT
 purchase_date,
 new_users,
 SUM(new_users) OVER (
 ORDER BY purchase_date
 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS cumulative_distinct_users
FROM daily_new_users
ORDER BY purchase_date;

```

**The key insight:** You cannot use `COUNT(DISTINCT user_id)` as a window function over expanding frames. Instead, find each user’s first purchase date, count new users per day, then use a cumulative SUM.

## Part 3: System Design for Data Pipelines

System design interviews for data engineers focus on designing data systems, not web applications. You will be asked to design ETL pipelines, real-time analytics systems, and data platforms.

### The 45-Minute Structure

#### Minutes 1-8: Requirements and Scale Estimation

Ask clarifying questions. The interviewer deliberately leaves the problem ambiguous.

Sample dialogue:

**You:** “Before I start designing, I’d like to understand the requirements. When you say ‘design a data warehouse for analytics,’ who are the primary users?”

**Interviewer:** “We have about 20 analysts, a few executives, and a data science team.”

**You:** “Got it. And what’s the data volume? Are we talking gigabytes or petabytes?”

**Interviewer:** “We process about 50,000 orders per day, so maybe a few terabytes total.”

**You:** “And for freshness – do stakeholders need real-time data, or is daily sufficient?”

Always estimate scale: “50,000 orders/day times 500 bytes is about 25 MB/day raw. Over 5 years, that is 45 GB. With dimension tables and 3x overhead: roughly 150 GB. This is well within a single Snowflake warehouse.”

#### Minutes 8-20: High-Level Architecture

Draw 5-7 boxes connected by arrows. State technology choices with one-sentence justifications.

```
[Data Sources] --> [Ingestion (Airbyte)] --> [Raw Storage (S3)]
--> [Transform (dbt on Snowflake)] --> [Analytics Tables]
--> [BI (Tableau)]
--> [ML (Python)]
```

#### Minutes 20-35: Component Deep Dive

The interviewer steers you to the most interesting component. Go deep: data models, schema design, query patterns, error handling.

#### Minutes 35-40: Scale and Optimize

Identify bottlenecks: “At 10x scale, the transformation step becomes the bottleneck. I would switch from full-refresh to incremental dbt models and parallelize independent transformations.”

#### Minutes 40-45: Trade-offs and Alternatives

Acknowledge what you sacrificed: “I chose Snowflake over BigQuery because the team is multi-cloud, but BigQuery would be cheaper for sporadic query patterns.”

---

## Design Problem 1: Real-Time Analytics Dashboard

**Prompt:** “Design a system that shows real-time metrics (orders per minute, revenue, active users) on a dashboard that updates every 10 seconds.”

**Strong answer structure:**



“I would use a lambda architecture with two paths:

**Real-time path:** Events flow through Kafka, processed by Flink for windowed aggregations (orders per minute, running revenue totals), and written to Redis. The dashboard reads from Redis via WebSocket for sub-second updates.

**Batch path:** The same Kafka events are also written to S3 as Parquet files (hourly partitions), loaded into Snowflake by dbt for historical analytics. This gives us the full history for trend analysis, cohort analysis, and ad-hoc queries.

**Why two paths?** Real-time aggregations in Redis are approximate and ephemeral (last 24 hours). Batch processing in Snowflake is exact and permanent. The real-time dashboard answers ‘what is happening right now?’ while the historical dashboards answer ‘what happened last quarter?’

**Key design decisions:** - Kafka partitioned by event type (orders, page views, user actions) for parallel processing - Flink tumbling windows of 60 seconds for per-minute metrics, with late event tolerance of 5 minutes - Redis TTL of 24 hours on real-time aggregations (do not keep stale data) - S3 lifecycle policy: Standard for 30 days, IA for 90 days, Glacier for 1 year

**What breaks first at 10x scale?** Redis becomes the bottleneck for writes. I would shard Redis by metric type and add read replicas for the dashboard. Kafka scales horizontally by adding partitions. Flink scales by adding task slots.”

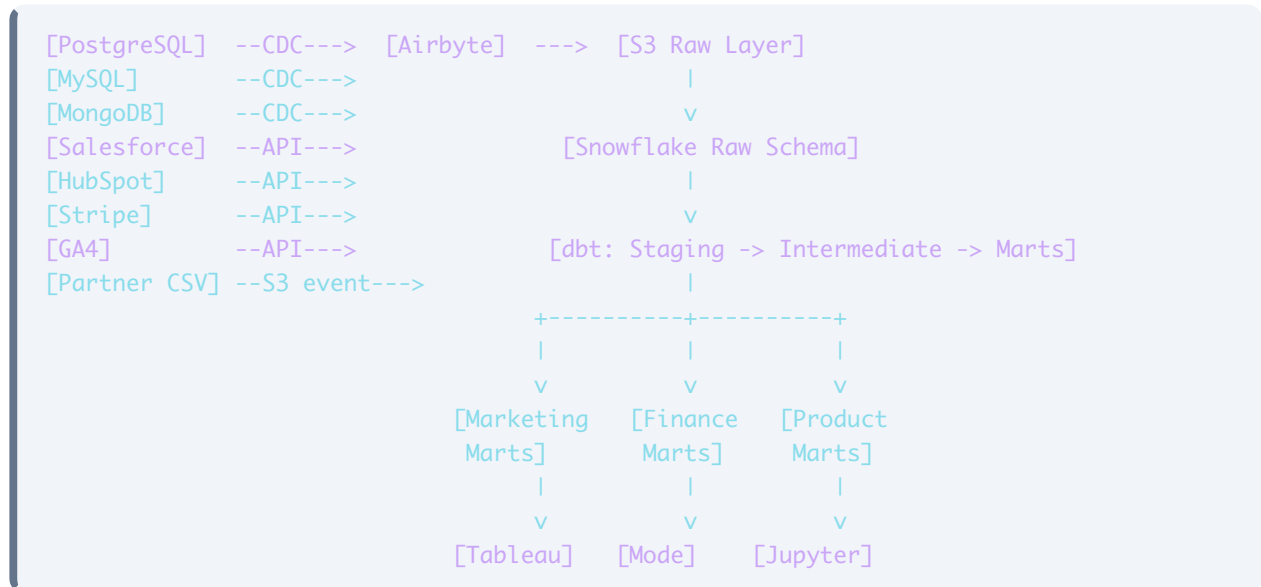
---

## Design Problem 2: ETL Pipeline for a Data Warehouse

**Prompt:** “Design an ETL pipeline that ingests data from 10 different sources (3 databases, 4 APIs, 3 file drops) into a data warehouse for a team of 30 analysts.”

**Strong answer structure:**

“First, let me understand the sources: - 3 databases (PostgreSQL, MySQL, MongoDB): CDC for real-time, full extract for initial load - 4 APIs (Salesforce, HubSpot, Stripe, Google Analytics): scheduled extraction, rate-limited - 3 file drops (CSV files from partners, landing in S3): event-driven trigger



**Orchestration:** Airflow with a dependency chain: 1. Extract all sources in parallel (10 Airbyte sync jobs) 2. Wait for all extracts to complete (using Airflow sensor or trigger rules) 3. Run dbt staging models (clean each source independently) 4. Run dbt intermediate models (join across sources) 5. Run dbt mart models (business-specific aggregations) 6. Run dbt tests (data quality) 7. Notify Slack on success, PagerDuty on failure

**Data modeling:** Star schema in the marts layer. Key fact tables: `fact_orders`, `fact_leads`, `fact_website_sessions`. Shared dimensions: `dim_customer`, `dim_product`, `dim_date`. Customer dimension uses SCD Type 2 for segment changes.

**Error handling:** - Airbyte retries each source 3 times with exponential backoff - If a source fails, the pipeline continues with other sources (partial load is better than no load) - dbt tests run after transformation; failures alert but do not block dashboard access to yesterday’s data - Idempotent loads: every run can be safely re-executed without creating duplicates

**Cost estimate:** Airbyte Cloud (\$500/month for 10 connectors), Snowflake (\$3K-\$5K/month for this volume), Airflow on Astronomer (\$400/month). Total: roughly \$5K/month for 30 analysts.”

## Design Problem 3: Data Lake to Lakehouse Migration

**Prompt:** “Your company has a 500 TB data lake on S3 with Parquet files, queried by Spark and Athena. Leadership wants to migrate to a lakehouse architecture. Design the migration.”

**Strong answer:**

“I would migrate to Apache Iceberg on top of the existing S3 data. Iceberg gives us ACID transactions, schema evolution, time travel, and partition evolution without moving the data to a new system.

**Phase 1 (weeks 1-4): Foundation** - Deploy an Iceberg catalog (AWS Glue Catalog or Nessie for git-like branching) - Convert the 10 most-queried tables from Parquet to Iceberg format using in-place migration (Iceberg can register existing Parquet files without rewriting them) - Validate that existing Spark jobs and Athena queries work against Iceberg tables

**Phase 2 (weeks 5-8): Pipeline migration** - Update Spark write paths to use Iceberg’s merge-on-read for incremental updates - Implement partition evolution: migrate from static `year/month/day` partitioning to Iceberg’s hidden partitioning (queries do not need to know about partition structure) - Add schema evolution: enable column additions and renames without breaking existing queries

**Phase 3 (weeks 9-12): Advanced features** - Enable time travel for debugging and auditing (query data as of any point in time) - Implement table maintenance: compaction (merge small files), expire snapshots (clean up old metadata), orphan file cleanup - Connect Snowflake as a query engine for analyst access (Snowflake supports Iceberg external tables)

**Key trade-off:** Iceberg vs Delta Lake vs Hudi. I chose Iceberg because of its vendor neutrality (works with Spark, Flink, Trino, Snowflake, BigQuery), its hidden partitioning (the strongest partition evolution story), and its momentum in the open-source community. Delta Lake is strong if you are committed to the Databricks ecosystem.”

---

## Part 4: Python Coding for Data Engineering Interviews

---

Python questions in DE interviews focus on practical data tasks, not algorithms. You will be asked to write API clients, file processors, and data validators.

### Challenge 1: API Client with Pagination and Rate Limiting

**Problem:** “Write a Python function that fetches all records from a paginated REST API, handling rate limits (429 responses) with exponential backoff.”

```

import requests
import time
from typing import Generator

def fetch_all_records(base_url: str, api_key: str,
 max_retries: int = 5) -> Generator[dict, None, None]:
 """Fetch all records from a paginated API with rate limit handling.

 Yields individual records. Handles pagination via cursor-based
 pagination and rate limits via exponential backoff.
 """
 cursor = None
 retry_count = 0

 while True:
 params = {"limit": 100}
 if cursor:
 params["cursor"] = cursor

 headers = {"Authorization": f"Bearer {api_key}"}

 try:
 response = requests.get(base_url, params=params,
 headers=headers, timeout=30)

 if response.status_code == 429:
 # Rate limited: exponential backoff
 retry_count += 1
 if retry_count > max_retries:
 raise Exception(f"Rate limited {max_retries} times, giving up")
 wait_time = min(2 ** retry_count, 60) # Max 60 seconds
 retry_after = response.headers.get("Retry-After")
 if retry_after:
 wait_time = int(retry_after)
 time.sleep(wait_time)
 continue

 response.raise_for_status()
 retry_count = 0 # Reset on success

 data = response.json()
 for record in data["results"]:
 yield record

 # Check for next page
 cursor = data.get("next_cursor")
 if not cursor:
 break # No more pages

 except requests.exceptions.Timeout:
 retry_count += 1
 if retry_count > max_retries:
 raise
 time.sleep(2 ** retry_count)

```

```
Usage:
for record in fetch_all_records("https://api.example.com/users", "key123"):
process(record)
```

**What interviewers look for:** Generator pattern (memory efficient for large datasets), exponential backoff, proper error handling, respecting Retry-After headers, timeout on requests.

---

## Challenge 2: File Processing with Schema Validation

**Problem:** “Write a function that reads a CSV file, validates each row against a schema, writes valid rows to a Parquet file, and logs invalid rows to a separate error file.”



```

import csv
import json
import pyarrow as pa
import pyarrow.parquet as pq
from datetime import datetime
from pathlib import Path
from dataclasses import dataclass
from typing import Optional

@dataclass
class ValidationResult:
 is_valid: bool
 errors: list[str]

def validate_row(row: dict, schema: dict) -> ValidationResult:
 """Validate a single row against a schema definition.

 Schema format:
 {
 "columns": {
 "user_id": {"type": "int", "required": True},
 "email": {"type": "str", "required": True, "pattern": "@"},
 "age": {"type": "int", "required": False, "min": 0, "max": 150},
 }
 }
 """
 errors = []

 for col_name, rules in schema["columns"].items():
 value = row.get(col_name)

 # Required check
 if rules.get("required") and (value is None or value == ""):
 errors.append(f"{col_name}: required but missing")
 continue

 if value is None or value == "":
 continue

 # Type check
 expected_type = rules.get("type")
 if expected_type == "int":
 try:
 int(value)
 except ValueError:
 errors.append(f"{col_name}: expected int, got '{value}'")
 continue
 elif expected_type == "float":
 try:
 float(value)
 except ValueError:
 errors.append(f"{col_name}: expected float, got '{value}'")
 continue

```

```

Range check
if "min" in rules and float(value) < rules["min"]:
 errors.append(f"{col_name}: {value} below minimum {rules['min']}")
if "max" in rules and float(value) > rules["max"]:
 errors.append(f"{col_name}: {value} above maximum {rules['max']}")

Pattern check
if "pattern" in rules and rules["pattern"] not in str(value):
 errors.append(f"{col_name}: does not match pattern '{rules['pattern']}'")

return ValidationResult(is_valid=len(errors) == 0, errors=errors)

def process_csv_to_parquet(
 input_path: str,
 output_path: str,
 error_path: str,
 schema: dict
) -> dict:
 """Read CSV, validate, write valid rows to Parquet, errors to JSON lines."""
 valid_rows = []
 error_count = 0
 total_count = 0

 with open(input_path, "r") as csv_file, \
 open(error_path, "w") as error_file:

 reader = csv.DictReader(csv_file)

 for row in reader:
 total_count += 1
 result = validate_row(row, schema)

 if result.is_valid:
 valid_rows.append(row)
 else:
 error_count += 1
 error_record = {
 "row_number": total_count,
 "row_data": row,
 "errors": result.errors,
 "timestamp": datetime.utcnow().isoformat(),
 }
 error_file.write(json.dumps(error_record) + "\n")

Write valid rows to Parquet
if valid_rows:
 table = pa.Table.from_pylist(valid_rows)
 pq.write_table(table, output_path, compression="snappy")

return {
 "total_rows": total_count,
 "valid_rows": len(valid_rows),
 "error_rows": error_count,

```

```
 "error_rate": round(error_count / max(total_count, 1), 4),
}
```

**What interviewers look for:** Clean separation of validation and I/O, use of Parquet (not CSV) for output, error logging with context (row number, specific error), summary statistics returned.

---

### Challenge 3: Data Deduplication in Python

**Problem:** “Write a function that deduplicates records from a large file that does not fit in memory. Records are JSON lines with an `id` field.”

```

import json
import hashlib
from pathlib import Path
from typing import Iterator

def deduplicate_large_file(
 input_path: str,
 output_path: str,
 key_field: str = "id"
) -> dict:
 """Deduplicate a JSON lines file that may not fit in memory.

 Uses a set of hashed keys (much smaller than full records)
 to track seen records. For a file with 100M records and
 UUID keys, the set uses roughly 4 GB of memory.

 For truly massive files, use an external sort or Bloom filter.
 """
 seen_keys = set()
 total = 0
 duplicates = 0

 with open(input_path, "r") as infile, \
 open(output_path, "w") as outfile:

 for line in infile:
 total += 1
 record = json.loads(line)
 key = record.get(key_field)

 if key is None:
 # No key -- write it (or skip it, depending on requirements)
 outfile.write(line)
 continue

 # Hash the key to save memory (16 bytes vs variable-length string)
 key_hash = hashlib.md5(str(key).encode()).digest()

 if key_hash not in seen_keys:
 seen_keys.add(key_hash)
 outfile.write(line)
 else:
 duplicates += 1

 return {
 "total_records": total,
 "unique_records": total - duplicates,
 "duplicates_removed": duplicates,
 }

```

### Follow-up: “What if even the key hashes do not fit in memory?”

“I would use a probabilistic approach with a Bloom filter. It gives a small false positive rate (maybe 0.1%), meaning some duplicates might slip through, but it uses dramatically less memory. For a

0.1% false positive rate with 1 billion records, a Bloom filter uses about 1.2 GB. Alternatively, I would sort the file externally (using Unix `sort` or a merge sort on disk) and then deduplicate the sorted output in a single pass.”

---

## Part 5: Behavioral Questions for Data Engineers

---

Behavioral questions in DE interviews are specific to the challenges of working with data: quality issues, pipeline failures, stakeholder communication, and technical trade-offs.

### Question 1: “Tell me about a time data quality failed.”

**Framework:** Use STAR (Situation, Task, Action, Result) but lead with the impact.

**Example answer:**

“At my previous company, we had a data quality incident that went undetected for three weeks. Our marketing team was running campaigns based on a customer segmentation model that relied on purchase frequency data. A schema change in our source PostgreSQL database renamed a column from `order_total` to `total_amount`. Our extraction pipeline did not break – it just started loading NULLs for that column.

**Impact:** The segmentation model classified high-value customers as inactive because their purchase amounts appeared to be zero. Marketing sent win-back campaigns to our best customers, which felt insulting and generated complaints.

**What I did:** First, I fixed the extraction to handle the renamed column. Second, I backfilled three weeks of data. Third – and this is the important part – I implemented three preventive measures: 1. Schema drift detection in our ingestion layer that alerts when source columns change 2. A dbt test on the orders model: `accepted_range` on `order_total` with a minimum of \$0.01 (null or zero values fail the test) 3. A daily anomaly check that compares today’s average order value against the 30-day rolling average and alerts if it deviates by more than 20%

**Result:** We have not had a similar incident in the 18 months since. The schema drift detection has caught 4 upstream changes before they affected downstream models.”

---

### Question 2: “Tell me about a time you disagreed with a stakeholder about technical approach.”

**Example answer:**

“Our VP of Analytics wanted to switch from Snowflake to BigQuery because BigQuery’s per-query pricing seemed cheaper for our usage pattern. I disagreed because the migration would take 3-4

months, our 20 analysts would need retraining on BigQuery SQL dialects, and our dbt models had Snowflake-specific SQL that would need rewriting.

I did not just say 'no.' I ran the numbers. I estimated the total cost of migration: 3 months of engineering time (roughly \$120K in loaded salary), analyst productivity loss during transition (estimated 30% reduction for 2 months), and the risk of pipeline incidents during migration.

Then I showed that we could achieve the same cost savings (\$40K/year) by optimizing our Snowflake usage: auto-suspend on idle warehouses, materialized views for the top 10 dashboard queries, and right-sizing our ETL warehouse from Large to Medium.

The VP agreed to the optimization approach. We achieved the cost savings in 2 weeks instead of 3 months. The lesson: when you disagree, come with data and an alternative solution, not just objections.”

---

### Question 3: “How do you prioritize when everything is urgent?”

#### Example answer:

“I use a simple framework: revenue impact times time sensitivity. A broken billing pipeline that affects invoicing this week is higher priority than a slow dashboard that annoys analysts.

Concretely, I categorize issues into four tiers: - **P0**: Data loss or corruption affecting production systems or financial reporting. Drop everything. - **P1**: Pipeline failures affecting stakeholder SLAs (dashboards not refreshed by agreed time). Fix today. - **P2**: Performance degradation or non-critical quality issues. Fix this week. - **P3**: Improvements, optimizations, technical debt. Scheduled in sprint planning.

I communicate this prioritization to stakeholders immediately. If an analyst asks me to investigate a dashboard discrepancy and I am in the middle of fixing a pipeline that feeds financial reporting, I tell them: ‘I am fixing a P0 issue affecting finance. I will look at your dashboard issue this afternoon. If you need it sooner, here is the raw data query you can run directly.’”

---

### Question 4: “Describe a pipeline you built from scratch.”

**Framework:** Walk through the pipeline end-to-end, but focus on decisions and trade-offs, not just technology.

“I built the customer analytics pipeline at [Company]. The source was our PostgreSQL production database with customer, order, and product data. The requirement was daily refreshed dashboards for 20 analysts showing customer lifetime value, cohort retention, and product affinity.

**Decision 1: Ingestion method.** I chose CDC (Change Data Capture) via Airbyte rather than full extracts. With 5 million customer records, a full extract took 45 minutes. CDC captured only changes, reducing ingestion to under 3 minutes. The trade-off: CDC is more complex to set up and can miss changes if the WAL is purged.

**Decision 2: Transformation approach.** I used dbt with incremental models for the fact tables. The `fact_orders` table grows by 50K rows/day; full refresh would scan 20M rows every run. Incremental processing touches only new and updated rows.

**Decision 3: Data quality.** I implemented three layers of tests: dbt generic tests (not null, unique, relationships), dbt custom tests (order amounts within expected range, no future-dated orders), and a daily reconciliation against the source database (total orders and revenue must match).

The pipeline ran in Airflow with a 2 AM start time and a 6:30 AM SLA. It consistently completed in 18 minutes. Over 12 months, we had 2 incidents: one Snowflake outage (not our fault) and one schema change in the source database (caught by our schema drift detection within 15 minutes)."

---

## Part 6: Take-Home Project Tips

---

Many companies include a take-home project in the DE interview process. This is where your Module 10 capstone becomes your secret weapon.

### Your Capstone IS Your Take-Home

If a company asks you to "build a data pipeline that ingests, transforms, and serves data," you have already done this. Adapt your capstone:

1. **Read the requirements carefully.** Match your capstone architecture to their specific ask. Do they want real-time? Add a Kafka component. Do they want ML? Add a simple model. Do they want cost analysis? Include it (you covered this in Appendix A).
2. **Do not over-engineer.** A clean, well-documented pipeline with 5 components beats a sprawling system with 15 components and no documentation. Reviewers spend 30-60 minutes evaluating your project. Make it easy to understand.
3. **Include a README with these sections:**
  4. Architecture diagram (even ASCII art works)
  5. How to run it (Docker Compose up, ideally)
  6. Technology choices and why
  7. Data model with schema descriptions
  8. What you would do differently with more time
  9. What breaks at 10x scale

10. **Include tests.** Even 5-10 tests show that you think about quality. Test your dbt models, test your Python functions, test your API responses.
11. **Include monitoring.** Even a simple `print(f"Loaded {row_count} rows in {duration}s")` shows operational awareness. A Grafana dashboard is better. Airflow with alerting is best.

## Common Take-Home Mistakes

- **No README or poor documentation.** The reviewer cannot figure out how to run it. Automatic fail.
  - **Works on your machine only.** Not containerized, hard-coded paths, missing dependencies. Use Docker.
  - **No error handling.** The API returns a 500 and the pipeline crashes with an unhandled exception.
  - **No data quality.** Raw data flows straight to the output with no validation, deduplication, or null handling.
  - **Premature optimization.** Using Kafka, Flink, and Redis for a project that processes 1000 rows. Shows lack of judgment.
- 

## Part 7: Company-Specific Interview Preparation

---

### FAANG / Big Tech (Google, Meta, Amazon, Netflix, Apple)

**Format:** Typically 5-6 rounds over a full day (virtual or on-site): 1. SQL coding (45 min): Medium to hard LeetCode-style SQL problems. Window functions, CTEs, self-joins. 2. Python coding (45 min): Data processing tasks, not algorithmic puzzles. 3. System design (45 min): “Design a data pipeline for...” at massive scale. 4. Data modeling (45 min): Design a star schema, discuss trade-offs, handle edge cases. 5. Behavioral (45 min): Leadership principles (Amazon), collaboration stories, handling ambiguity. 6. Hiring manager (30 min): Culture fit, career goals, team dynamics.

**What differentiates FAANG interviews:** - Scale expectations are higher. “How does this work at 1 billion events per day?” is a normal follow-up. - They care about custom solutions. “Kafka does not meet our latency requirements at this scale. What would you build?” - They test fundamentals deeply. Do not just know that Parquet is columnar; explain how column pruning reduces I/O. - Behavioral questions are rigorous, especially at Amazon (every answer should map to a Leadership Principle).

**Preparation tips:** - Practice SQL daily for 2 weeks before the interview. Use LeetCode Database problems (medium and hard). - Prepare 8-10 behavioral stories using STAR format. Ensure each story demonstrates a different quality (technical depth, conflict resolution, working with ambiguity,



delivering under pressure). - Study the company's public engineering blog. Netflix Tech Blog, Meta Engineering, Google Cloud Blog all publish articles about their data infrastructure.

---

## Unicorn / Late-Stage Startup (Stripe, Databricks, Snowflake, Airbnb)

**Format:** Typically 4-5 rounds: 1. Technical screen (SQL + Python, 60 min) 2. System design (45-60 min) 3. Practical/take-home project (2-4 hours, done at home) 4. Behavioral + culture fit (45 min) 5. Hiring manager (30 min)

**What differentiates unicorn interviews:** - More practical and less theoretical than FAANG. "How would you actually build this?" vs "What is the theoretical optimal approach?" - Take-home projects are common. You may build a small pipeline from scratch. - They value speed of delivery. Can you build something good in 4 hours? - Domain knowledge matters. If interviewing at Stripe, know payment data. At Databricks, know Spark deeply.

---

## Mid-Market and Enterprise Companies

**Format:** Typically 3-4 rounds: 1. Technical screen (SQL focus, 45 min) 2. System design or architecture discussion (45 min) 3. Behavioral (30-45 min) 4. Hiring manager (30 min)

**What differentiates enterprise interviews:** - SQL is the primary technical assessment. Window functions, CTEs, and performance tuning. - Less emphasis on distributed systems and more on data modeling, warehouse optimization, and BI integration. - They care about communication with non-technical stakeholders. "How would you explain this data model to a product manager?" - Domain knowledge can be a significant advantage (healthcare, finance, retail).

---

## Early-Stage Startup (Seed to Series B)

**Format:** Typically 2-3 rounds: 1. Technical conversation (60 min, mix of SQL, Python, and architecture) 2. Take-home project or pair programming (2-3 hours) 3. Founder/CTO (30-45 min)

**What differentiates startup interviews:** - Breadth over depth. Can you do ingestion AND transformation AND analytics AND infra? - They want to see pragmatism. "I would use BigQuery and dbt Cloud to ship this in 2 weeks" beats "I would design a custom lakehouse architecture." - Speed and scrappiness matter. They need someone who can build the first version alone. - Culture fit with the founding team matters as much as technical ability.

---

## Part 8: Practice Plan

---

### 4-Week Interview Preparation Schedule

**Week 1: SQL Foundations** - Days 1-3: Practice 3 medium SQL problems per day (LeetCode Database section) - Days 4-5: Practice 2 hard SQL problems per day - Days 6-7: Write SQL solutions to the challenges in Part 2 of this appendix from memory

**Week 2: System Design + Data Modeling** - Days 1-2: Practice designing an ETL pipeline (Design Problem 2 above). Time yourself to 45 minutes. - Days 3-4: Practice designing a real-time system (Design Problem 1 above). Time yourself to 45 minutes. - Days 5-6: Practice data modeling questions (Part 1 above). Draw schemas on paper. - Day 7: Review the case studies in Appendix B. Practice describing one architecture in 5 minutes.

**Week 3: Python + Behavioral** - Days 1-3: Code the Python challenges in Part 4. Then write them again from memory. - Days 4-5: Write out your behavioral stories (Part 5). Practice delivering them out loud in 2 minutes each. - Days 6-7: Mock interview with a friend or practice partner. Full 45-minute system design.

**Week 4: Company-Specific Prep** - Days 1-2: Research the specific company. Read their engineering blog. Understand their data stack. - Days 3-4: Practice the interview format specific to that company type (Part 7 above). - Days 5-6: Review weak areas identified in mock interviews. - Day 7: Rest. You are prepared.

---

### Quick Reference: What to Review the Night Before

---

- [ ] Star schema design pattern: fact tables, dimension tables, grain, surrogate keys
- [ ] SCD Type 1 vs Type 2 (when to use each)
- [ ] SQL window functions: ROW\_NUMBER, RANK, LAG, LEAD, SUM OVER
- [ ] CTE syntax and when to use CTEs vs subqueries
- [ ] Deduplication pattern: ROW\_NUMBER OVER (PARTITION BY key ORDER BY timestamp)
- [ ] Python: requests library, error handling, generators
- [ ] System design: Lambda architecture (real-time + batch paths)
- [ ] Kafka basics: topics, partitions, consumer groups, at-least-once delivery
- [ ] dbt basics: staging → intermediate → marts, incremental models, tests
- [ ] Airflow basics: DAGs, operators, sensors, task dependencies
- [ ] Cost optimization: auto-suspend, partition pruning, lifecycle policies
- [ ] Your 3 best behavioral stories (data quality failure, technical disagreement, pipeline build)

- [ ] Your capstone project architecture (be ready to draw it from memory)

**Key Takeaway:** Interview preparation is not about memorizing answers. It is about internalizing patterns so deeply that you can apply them to any question. The SQL patterns, system design structures, and communication frameworks in this appendix are the same patterns used in every module of this course. You have been preparing for these interviews since Module 0. Trust your preparation, communicate clearly, and show your thinking process.